O-RAN.WG11.TS.STS-R004-v11.00

Technical Specification

O-RAN Work Group 11 (Security Work Group)

O-RAN Security Test Specifications

O-RAN ALLIANCE e.V., Buschkauler Weg 27, 53347 Alfter, Germany

Register of Associations, Bonn VR 11238, VAT ID DE321720189

Contents

List of figures

List of tables

Foreword

This Technical Specification (TS) has been produced by WG11 of the O-RAN ALLIANCE.

The content of the present document is subject to continuing work within O-RAN and may change following formal O-RAN approval. Should the O-RAN ALLIANCE modify the contents of the present document, it will be re-released by O-RAN with an identifying change of version date and an increase in version number as follows:

version xx.yy.zz

where:

- xx: the first digit-group is incremented for all changes of substance, i.e. technical enhancements, corrections, updates, etc. (the initial approved document will have xx=01). Always 2 digits with leading zero if needed.
- yy: the second digit-group is incremented when editorial only changes have been incorporated in the document. Always 2 digits with leading zero if needed.
- zz: the third digit-group included only in working versions of the document indicating incremental changes during the editing process. External versions never include the third digit-group. Always 2 digits with leading zero if needed.

Modal verbs terminology

In the present document "**shall**", "**shall not**", "**should**", "**should not**", "**may**", "**need not**", "**will**", "**will not**", "**can**" and "**cannot**" are to be interpreted as described in clause 3.2 of the O-RAN Drafting Rules (Verbal forms for the expression of provisions).

"**must**" and "**must not**" are **NOT** allowed in O-RAN deliverables except when used in direct citation.

1 Scope

The present document provides description of the Security Tests, which validate security functions and configurations per security and security protocols requirements and are based on the priority of the risk analysis for O-RAN systems.

2 References

2.1 Normative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies. In the case of a reference to a 3GPP document, a non-specific reference implicitly refers to the latest version of that document in Release 18, or the latest 3GPP release prior to Release 18 that includes that document.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, O-RAN cannot guarantee their long-term validity.

The following referenced documents are necessary for the application of the present document.

[1] O-RAN.WG1.TS.OAD: "O-RAN Architecture Description"

[2] O-RAN.WG11.TS.SPS: "O-RAN Security Protocols Specifications"

[3] void

[4] O-RAN.TIFG.TS.E2E-Test: "O-RAN End-to-End Test Specification"

[5] O-RAN.WG11.TS.SRCS: "O-RAN Security Requirements and Controls Specifications"

[6] 3GPP TR 21.905: "Vocabulary for 3GPP Specifications"

[7] 3GPP TS 33.117: "Catalogue of General Security Assurance Requirements"

[8] 3GPP TS 33.511: "Security Assurance Specification (SCAS) for the next generation Node B (gNodeB) network product class"

[9] 3GPP TS 33.216: "Security Assurance Specification (SCAS) for the Evolved Node B (eNodeB) network product class"

[10] ISO 8601: "Date and time"

[11] "IEEE Standard for Local and Metropolitan Area Networks--Port-Based Network Access Control," in IEEE Std 802.1X-2020 (Revision of IEEE Std 802.1X-2010 Incorporating IEEE Std 802.1Xbx-2014 and IEEE Std 802.1Xck-2018), vol., no., pp.1-289, 28 Feb. 2020, doi: 10.1109/IEEESTD.2020.9018454

[12] "Generating Software Bills of Materials (SBOMs) with SPDX at Microsoft", https://devblogs.microsoft.com/engineering-at-microsoft/generating-software-bills-of-materials-sboms-with-spdx-at-microsoft/

[13] NTIA: "The Minimum Elements For a Software Bill of Materials (SBOM)", https://www.ntia.gov/sites/default/files/publications/sbom_minimum_elements_report_0.pdf

[14] IETF RFC 8341: "Network Configuration Access Control Model", https://datatracker.ietf.org/doc/html/rfc8341

[15] IETF RFC 5905: "Network Time Protocol Version 4: Protocol and Algorithms Specification", https://datatracker.ietf.org/doc/html/rfc5905

[16] IETF RFC 5906: "Network Time Protocol Version 4: Autokey Specification", https://datatracker.ietf.org/doc/html/rfc5906

[17] IETF RFC 4493: "The AES-CMAC Algorithm", https://datatracker.ietf.org/doc/html/rfc4493

[18] IETF RFC 8446: "The Transport Layer Security (TLS) Protocol Version 1.3", https://datatracker.ietf.org/doc/html/rfc8446

[19] IETF RFC 7519: "JSON Web Token (JWT)", https://datatracker.ietf.org/doc/html/rfc7519

[20] IETF RFC 7515: "JSON Web Signature (JWS)", https://datatracker.ietf.org/doc/html/rfc7515

[21] O-RAN.WG4.TS.MP: "O-RAN WG4 Management Plane Specification"

[22] O-RAN.WG1.TS.Use-Cases-Detailed-Specification: "O-RAN Use Cases Detailed Specification"

[23] 3GPP TS 33.523: "5G Security Assurance Specification (SCAS); Split gNB product classes"

[24] void

[25] 3GPP TS 33.501: "Security architecture and procedures for 5G system"

[26] O-RAN.WG4.TS.CUS: "O-RAN WG4 Control, User and Synchronization Plane Specification"

[27] O-RAN.WG3.TS.RICARCH: "O-RAN Near-RT RIC Architecture"

[28] O-RAN.WG3.TS.E2AP: "O-RAN E2 Application Protocol (E2AP)"

[29] O-RAN.WG3.E2TS: "O-RAN E2 Interface Test Specification"

[30] IETF RFC 9562: "Universally Unique IDentifiers (UUIDs)", https://datatracker.ietf.org/doc/html/rfc9562

[31] O-RAN.WG10.TS.OnboardingSMOSGAP: "Onboarding SMOS General Aspects and Principles"

[32] IEEE Std 802.1AE-2018: "IEEE Standard for Local and metropolitan area networks — Media Access Control (MAC) Security".

2.2 Informative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies. In the case of a reference to a 3GPP document, a non-specific reference implicitly refers to the latest version of that document in Release 18, or the latest 3GPP release prior to Release 18 that includes that document.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, O-RAN cannot guarantee their long-term validity.

The following referenced documents are not necessary for the application of the present document, but they assist the user with regard to a particular subject area.

[i.1] Service Name and Transport Protocol Port Number Registry, https://www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.xhtml

[i.2] SPDX, https://spdx.dev/

[i.3] CycloneDX, https://cyclonedx.org/

[i.4] NIST IR 8060: "Guidelines for the Creation of Interoperable Software Identification (SWID) Tags", https://nvlpubs.nist.gov/nistpubs/ir/2016/NIST.IR.8060.pdf

[i.5] O-RAN.WG11.TR.Threat-Modeling: "O-RAN Security Threat Modeling and Risk Assessment"

3 Definition of terms, symbols and abbreviations

3.1 Terms

For definitions of the security terms Attack, Attack surface, Risk, Security control, Technical control, Threat, and Vulnerability used in this document, refer to [i.5].

For the purposes of the present document, the terms and definitions given in 3GPP TR 21.905 **Error! Reference source not found.**, O-RAN Architecture Description [1], and the following in this clause apply. A term defined in the present document takes precedence over the definition of the same term, if any, in 3GPP TR 21.905 **Error! Reference source not found.** and O-RAN Architecture Description [1].

A1: Interface between non-RT RIC and Near-RT RIC to enable policy-driven guidance of Near-RT RIC applications/functions, and support AI/ML workflow.

E2: Interface connecting the Near-RT RIC and one or more O-CU-CPs, one or more O-CU-UPs, and one or more O-DUs.

RAN: Generally referred as Radio Access Network. In terms of this document, any component below Near-RT RIC per O-RAN architecture, including O-CU/O-DU/O-RU.

Overcommitting resources: The practice of allocating or promising more resources than are physically available on a system. This concept is commonly used in virtualized and cloud environments. The idea behind overcommitment is to optimize resource utilization based on the observation that not all applications will use their allocated resources to the maximum at the same time. Here's a breakdown of overcommitment for different resources:

- CPU Overcommitment: More virtual CPUs (vCPUs) are allocated to VMs or Containers than there are physical CPU cores available on the host.
- Memory Overcommitment: The total memory allocated to VMs or Containers exceeds the physical RAM available on the host.
- Storage Overcommitment: More storage space is allocated to VMs or Containers than the actual available capacity on the storage device.
- Network Overcommitment: More bandwidth is promised to VMs or Containers than the physical network can provide.

Overcommit ratios: The extent to which resources can be overallocated compared to the actual available physical resources.

- CPU Overcommit Ratio: A CPU overcommit ratio of 2:1 indicates that twice the number of virtual CPUs (vCPUs) can be allocated compared to the physical CPU cores available on the host. For instance, if a server has 8 physical CPU cores, 16 vCPUs could be allocated across various VMs or Containers.
- Memory Overcommit Ratio: A memory overcommit ratio of 1.5:1 indicates that 1.5 times the amount of virtual RAM can be allocated compared to the physical RAM available on the host. For a server with 64GB of physical RAM, a total of 96GB of RAM could be allocated across various VMs or Containers.

Signed object: A signed unit of content that is subject to signature verification. This term encompasses application packages, software images, application artifacts, update bundles, AI/ML models, and any content requiring authenticity and integrity validation by a DUT during onboarding, deployment, or instantiation.

For definition of the terms Application, Application Artifact, and Application Package see the WG10 Onboarding SMOS General Aspects and Principles document **Error! Reference source not found.**, clause 3.1. Other terms defined in WG10 Onboarding SMO General Aspects and Principles **Error! Reference source not found.** also apply.

3.2 Symbols

Void

3.3 Abbreviations

For the purposes of the present document, the abbreviations given 3GPP TR21.905 **Error! Reference source not found.**, O-RAN Architecture Description [1], and the following in this clause apply. A abbreviation defined in the present document takes precedence over the definition of the same abbreviation, if any, in 3GPP TR21.905 **Error! Reference source not found.** and O-RAN Architecture Description [1].

AI/ML Artificial Intelligence / Machine Learning

CMS/PKCS#7/CAdES Cryptographic Message Syntax/Public-Key Cryptography Standards/CMS Advanced Electronic Signatures

CNF Cloud Native Function

COT Chain of Trust

CSI Channel State Information

CSR Certificate Signing Request

DoS Denial of Service

DTLS Datagram Transport Layer Security

DUT Device Under Test

eCPRI Enhanced Common Public Radio Interface

FTP File Transfer Protocol

FTPS File Transfer Protocol Secure

IPsec Internet Protocol Security

JSF JSON Signature Format

JSON JavaScript Object Notation

JWT JSON Web Token

JWS JSON Web Signature

KPI Key Performance Indicator

LLS Low Layer Split

mTLS Mutual Transport Layer Security

MITM Man-in-the-Middle

NACM Network Configuration Access Control Model

NETCONF Network Configuration Protocol

NFO Network Function Orchestration

NTIA National Telecommunications and Information Administration - United States Department of Commerce

OAuth Open Authentication

PDCP Packet Data Convergence Protocol

PNF Physical Network Function

PRTC Primary Reference Time Clock

PTP Precision Timing Protocol

RBAC Role-based Access Control

REST Representational state transfer

RDF Resource Description Format

RIC O-RAN RAN Intelligent Controller

RoT Root of Trust

SBOM Software Bill of Materials

SDLC Software Development Lifecycle

SSH Secure Shell

SUT System Under Test

TLS Transport Layer Security

VNF Virtualized Network Function

WAS Web Application Security

XML Extensible Markup Language

YAML YAML Ain’t Markup Language

4 Objectives and scope

This security test specification is focused on:

- Validating the implementation of security requirements and security protocols specified in [5] and [2].
- Emulating security attacks against the O-RAN architecture elements, interfaces, and the system to measure the robustness of the O-RAN system and the service impact(s).
- Validating the effectiveness of the security mitigation method(s) to protect the O-RAN system and the services it offers.
- Providing consistent test setups that ensure fair and comparable test results among various test campaigns.
- Describing the test conditions, methodologies, and procedures, so that the tests can be reproduced if needed, and the test results can be used for comparison or reference purposes.

This security test specification is based on the priority of the risk assessment of the O-RAN security threats and security requirements of the O-RAN system.

5 Testing methodology and configuration

5.1 DUT / SUT

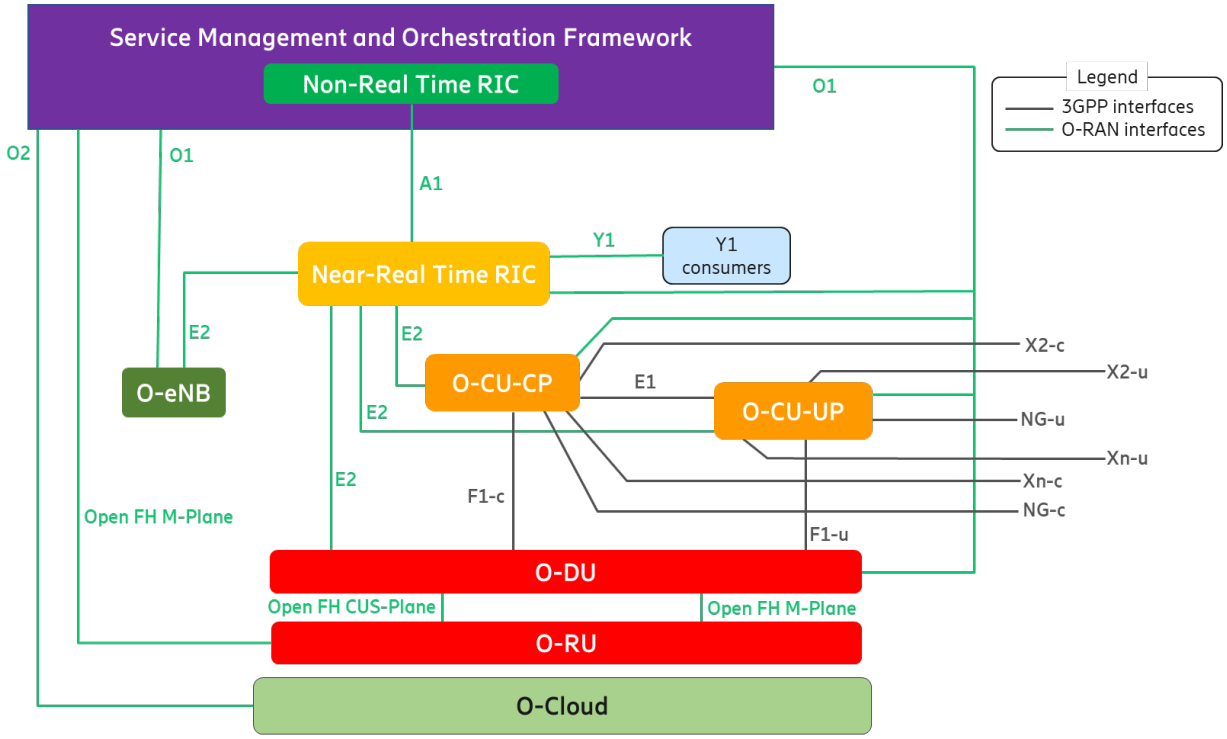

Figure 5-1: Logical Architecture of O-RAN system

Error! Reference source not found. illustrates the O-RAN architecture elements, interfaces, and overall system.

As specified in [1], the following O-RAN architecture elements and interfaces shall be the DUT or SUT addressed in this security test specification.

O-RAN architecture elements:

- Network functions and applications
  - Service Management and Orchestration (SMO)
  - Non-RT RIC and rApps
  - Near-RT RIC and xApps
  - O-CU-CP/UP
  - O-DU
  - O-RU
  - O-eNB
- Cloud computing platform
  - O-Cloud comprising a collection of physical infrastructure nodes that meet O-RAN requirements to host the relevant O-RAN functions (such as Near-RT RIC, O-CU-CP, O-CU-UP, and O-DU), the supporting software components (such as Operating System, Virtual Machine Monitor, Container Runtime, etc.) and the appropriate management and orchestration functions.

Maintained interfaces by O-RAN:

- A1 Interface between Non-RT RIC and Near-RT RIC to enable policy-driven guidance of Near-RT RIC applications/functions, and support AI/ML workflow.
- O1 Interface connecting the SMO to the Near-RT RIC, one or more O-CU-CPs, one or more O-CU-UPs, and one or more O-DUs.
- O2 Interface between the SMO and the O-Cloud.
- E2 Interface connecting the Near-RT RIC and one or more O-CU-CPs, one or more O-CU-UPs, one or more O-DUs, and one or more O-eNBs.
- Open Fronthaul CUS-Plane Interface between O-RU and O-DU.
- Open Fronthaul M-Plane Interface between O-RU and O-DU as well as between O-RU and SMO.

During the test execution, only one DUT or SUT shall be tested at the same time. The rest of elements involved in the test setup should be simulated or real, according to the test preconditions, but only the DUT or SUT shall be considered under evaluation.

5.2 Test Setup

Refer to the security test cases listed in the following clauses for their specific test setups.

5.3 Test and measurement equipment and tools

The following table lists test and measurement equipment required for the security tests in the present document.

Table 5-1: Test and measurement equipment list

Test tool	Description
Commercial UE and/or UE emulator	A commercial UE or UE emulator shall be used to establish stateful end-to-end connection and to generate or receive data traffic. The commercial UE used in this context as a test tool is typically a UE which is designed for commercial or testing applications with certain test and diagnostic functions enabled for test and measurements purposes. Such test and diagnostic functions should not affect the performance. This commercial UE requires an (emulated) SIM card which is pre-provisioned with subscriber profiles. A UE emulator or multiple commercial UEs can be used in multi-UE test scenarios requiring multiple UEs sessions. The UE shall connect to the SUT either via RF cables or via an over the air (OTA) connection. In a lab environment, the UE shall be placed inside an RF shielded box/room to avoid interference from external signals. A logging tool connected to the UE shall be used to capture measurements and KPI logs for test validation and reporting.
4G/5G Core or Core emulator	A 4G/5G core or core emulator shall be used to terminate 4G/5G NAS sessions, and to support core network procedures required for RAN (SUT) testing. 4G/5G core or core emulator shall support end-to-end connection and data transfer between Application server and commercial UE/UE emulator.
Application (traffic) server	An application (traffic) server shall be used as an endpoint for generation and/or termination of data traffic streams to/from commercial UE(s)/UE emulator. The application server shall be capable of generating data traffic for the services under test.
Network impairment emulator	A network impairment emulator shall be used for tests which require insertion of impairment (packet delay and/or jitter) at the network interface (e.g. OpenFH).
Packet generation tool / DoS emulator	A packet generation tool / Denial of Service (DoS) emulator shall be used for DoS traffic generation of security tests. The tool shall support crafting network traffic over the following network protocols: Ethernet, IP, UDP, TCP, PTP, eCPRI, TLS, HTTP/HTTPS.
High-volume packet generation/DDoS simulation tool	A high-volume packet generation / Distributed Denial of Service (DDoS) simulation tool shall be used for high-volume DDoS traffic generation from multiple sources. The tool shall support crafting high-volume network traffic over the following network protocols: Ethernet, IP, UDP, TCP, PTP, eCPRI, TLS, HTTP/HTTPS.
Packet capture tool	A packet capture tool shall be used to capture samples of data traffic for validation, analysis, and troubleshooting. It may be used to capture samples of legitimate traffic, which then may be used as templates for fuzzing attacks. The tool shall support capturing network traffic over the following network protocols: Ethernet, IP, UDP, TCP, PTP, eCPRI, TLS, QUIC, HTTP/HTTPS.
Network tap	A network tap shall be a hardware or software device which provides access and visibility to the data flowing across a computer network.
Port scanner	A protocol scanner shall be used for probing network protocols and services. It shall be able to detect open ports. It shall be able to detect what service is exposed as active on the open port. Port scanners commonly come with built-in database of services. Service detection can use numerous built-in probes for querying various services. In practice, port scanners are often used for service detection.
Fuzzing tool	A protocol fuzzing tool shall be used for unexpected protocol input generation of security tests. The tool shall support mutating and replaying of captured network traffic over the following network protocols: Ethernet, IP, UDP, TCP, PTP, eCPRI, TLS, HTTP/HTTPS.
Vulnerability scanning tool	A vulnerability scanning tool shall be used for blind exploitation of well-known vulnerabilities during security tests. The tool may rely on cyclically updated database of known vulnerabilities

	based on Common Vulnerabilities and Exposures (CVE) and should support scanning network services running on TCP/IP stack of protocols.
NFV benchmarking and resource exhaustion tool	A Network Function Virtualization (NFV) tool shall be used for O-Cloud system performance measurement and resource exhaustion type of DoS attack generation. This tool shall be capable of supporting any types of O-Cloud environment (public or private) with testing VNF(s) and/or CNF(s).
SSH audit tool	An SSH audit tool shall be used to verify the following properties: version of protocol, cipher suites, and known vulnerabilities in server and client SSH software.
TLS scanning tool	A TLS scanning tool shall be used to verify the following properties: version of protocol, cipher suites, and known vulnerabilities in server TLS software.
DTLS scanning tool	A DTLS scanning tool shall be used to verify the following properties: version of protocol, cipher suites, and known vulnerabilities in server DTLS software.
IKE scanning tool	An IKE scanning tool shall be used to verify the following properties: version of protocol, cipher suites, and known vulnerabilities in server IPsec software.
Software image signing tool	A Software image signing tool shall be used to digitally sign and verify the software image, e.g. xApps or O-RAN architecture element delivered by a software producer/provider.

5.4 Test report

Tests should be described in the test report with sufficient detail to allow the tests to be reproducible by different parties and to enable comparison. A template for a complete test report is found in Annex B and may be used. Photos and screenshots should also be taken as part of the test report to illustrate the test environment. Additional parameters are specified in the description of each test in the subsequent clauses.

5.5 Assumptions

All threat IDs in the present document are referenced from O-RAN Security Threat Modeling and Risk Assessment [i.5].

5.6 Testing tools

The tools outlined in this clause represent a selection of commonly used resources for testing processes. It is important to emphasize that this list is not exhaustive. Testers are encouraged to use additional tools as needed for comprehensive and effective testing, ensuring they meet the standards and requirements set forth in this test plan.

1) Packet capture and traffic analysis tools:

- Wireshark: Wireshark is a widely used open-source network protocol analyser that can capture and analyse network traffic. It allows for the inspection of packets to identify issues related to confidentiality, integrity, and replay. Additionally, it can be used to verify authentication mechanisms and analyse access control measures.
- tcpdump: tcpdump is a command-line packet analyser available on various operating systems. It captures network traffic and can save it to a file for later analysis. tcpdump offers powerful filtering capabilities to capture specific traffic based on criteria such as source/destination IP addresses, protocols, or ports.
- Netscout Sniffer: Netscout Sniffer is a commercial network analysis tool that offers real-time packet capture and analysis capabilities. It provides comprehensive visibility into network traffic and offers advanced features for troubleshooting and performance analysis.
- Colasoft Capsa: Colasoft Capsa is a network analyser designed for network monitoring and troubleshooting. It captures and analyses network traffic, providing insights into protocols, applications, and potential security issues. Capsa offers both real-time and post-capture analysis.

- Tcpreplay: Tcpreplay is an open-source tool used for replaying captured network traffic. It enables the replay of network packets from a previously captured pcap file, simulating real-world traffic scenarios. Although its primary purpose is not security testing, tcpreplay can be utilized as a tool in security testing efforts, particularly for testing the replay and handling of network packets.

2) Traffic Generation Tools:

- Scapy: Scapy is a powerful Python-based tool that can create, manipulate, and send custom network packets. It allows to generate and replay packets on an interface to test for replay vulnerabilities.
- Hping: Hping is a command-line tool that can send custom packets and perform various network-related activities. It can be used to generate replayed packets on an interface for testing purposes.

3) Scripting and Automation Tools to develop custom test scripts:

- Python: Python scripting language provides libraries (e.g., socket, scapy) that enables the creation of custom scripts to generate and replay packets on an interface.
- Bash scripting: Bash scripting can be utilized to automate the process of capturing packets and replaying them on an interface.

4) Network Emulation Tools:

- GNS3: GNS3 is a network emulation tool that enables the simulation of complex network topologies. It can be used to create a virtual environment with RAN E1 interfaces, generate traffic, and simulate replay attacks for testing purposes.

5) Network performance tools:

- iperf3: iPerf3 is an open-source tool for network performance testing and measurement. While it is primarily focused on network performance evaluation, it can also be utilized as a tool to indirectly assess certain aspects of security, such as bandwidth availability and network congestion.

6) Traffic Manipulation Tools:

- Burp Suite: Burp Suite is a web application security testing tool that can intercept, modify, and replay network traffic. While it is primarily designed for web applications, it allows to test the integrity, confidentiality, and authenticity of data transmitted over an interface.

7) Vulnerability assessment tools

- Nessus: Nessus is a popular vulnerability assessment tool that can scan an interface for known security vulnerabilities and misconfigurations. It can help identify potential weaknesses related to confidentiality, integrity, replay attacks, and access control.
- OpenVAS: OpenVAS (Open Vulnerability Assessment System) is an open-source vulnerability scanner that can perform security audits on security protocols implementations. It can detect vulnerabilities, misconfigurations, and compliance issues, helping ensure that an interface adheres to security best practices and standards.

8) Security Information and Event Management (SIEM) Tools:

- SIEM tools like Splunk or ELK (Elasticsearch, Logstash, Kibana) can help collect and analyse security events and logs related to the O-RAN architecture elements and interfaces. They can assist in identifying potential security incidents, monitoring access control, and detecting anomalies.

9) IPsec tool

- OpenSwan is an open-source implementation of the IPsec (Internet Protocol Security) protocols suite. It provides tools and libraries for setting up and managing IPsec connections, which can be used to test the confidentiality, integrity, replay, authenticity, and access control of an interface. Here's how OpenSwan can be used for testing:

- Confidentiality and Integrity:
  - OpenSwan allows to configure IPsec tunnels with encryption algorithms (e.g., AES) and integrity algorithms (e.g., HMAC-SHA256). By setting up IPsec connections using OpenSwan, it is possible to verify the confidentiality and integrity of data transmitted over an interface.
- Replay Attack:
  - OpenSwan supports replay protection mechanisms, which protect against replay attacks by assigning sequence numbers to IPsec packets. These mechanisms can be tested to ensure that replayed packets are detected and rejected.
- Authenticity:
  - OpenSwan supports authentication mechanisms such as pre-shared keys or digital certificates, which ensure the authenticity of IPsec connections. Testing can be performed to verify the proper authentication of an interface.
- Access Control:
  - OpenSwan allows to configure IPsec security policies, including source/destination IP address filtering, protocol filtering, and port filtering. These policies can be tested to ensure that only authorized traffic is allowed through an interface.

- StrongSwan: An open-source IPsec-based VPN solution that includes testing capabilities. It enables the configuration and simulation of IPsec connections, testing of authentication methods, and performance of security checks.

10) Cryptographic operations testing tools

- Hashing Tools: Hashing tools such as sha256sum, can be used to calculate hash values of transmitted data. By comparing the computed hash values at the source and destination, the integrity of the data can be verified.

Cryptographic Libraries: Cryptographic libraries, such as Bouncy Castle, provide APIs and tools for implementing and testing integrity protection mechanisms. These libraries offer functions to generate integrity checks (e.g., MAC) and validate the integrity of received data.

6 Security Protocol & APIs Validation

6.1 Overview

This clause contains test cases to validate implementation of security protocols against O-RAN security requirements in [2] and [5].

6.2 SSH Server & Client

Requirement Name: Network Security Protocol - SSH

Requirement Reference: Clause 4.1, O-RAN Security Protocols Specifications [2]

Requirement Description: Robust protocol implementation with adequately strong cipher suites is being required for SSH

Threat References: T-O-RAN-05

DUT/s: SMO, O-DU, O-RU

Test Name: TC_SSH_Server_and_Client_Protocol

Purpose: To verify implementation of the secure communication protocol SSH as specified in [2].

Procedure and execution steps

Preconditions

- Tool: SSH audit tool with capabilities as defined in clause **Error! Reference source not found.**
- Testing of server configuration (DUT in the role of server): Network access to SSH server
- Testing of client configuration (DUT in the role of client): Access to configuration of SSH client

Execution steps

Testing of server configuration (DUT in the role of server):

- Run SSH audit tool in server audit mode against target SSH server.
- Compare the tool's output with the list of approved SSH protocol versions and algorithms (for key agreement, symmetric encryption, key exchange, and MACs) as defined by clause 4.1, O-RAN Security Protocols Specifications [2].

Testing of client configuration (DUT in the role of client):

- Run SSH audit tool on target SSH client in client audit mode.
- Compare the tool's output with the list of approved SSH protocol versions and algorithms (for key agreement, symmetric encryption, key exchange, and MACs) as defined by clause 4.1, O-RAN Security Protocols Specifications [2].

Expected results

- All detected SSH protocol versions are allowed by [2], clause 4.1.
- All detected SSH algorithms (for key agreement, symmetric encryption, key exchange, and MACs) are allowed by [2], clause 4.1.

Expected format of evidence: Report files produced by SSH audit tool and/or screenshots

6.3 TLS

6.3.1 TLS Support

Requirement Name: Network Security Protocol - TLS

Requirement Reference: Clause 4.2, O-RAN Security Protocols Specifications [2]

Requirement Description: Support TLS/mTLS with protocol profiles

Threat References: T-O-RAN-05

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_TLS_Protocol

Purpose: To verify implementation of the secure communication protocol TLS as specified in [2], clause 4.2.

Procedure and execution steps

Preconditions

- Tool: TLS scanning tool with capabilities as defined in clause 5.3 with certificate(s) installed
- DUT equipped with CA certificate that is a trust anchor for the certificate(s)
- Network access to DUT with TLS or mTLS enabled

Execution steps

1) Configure DUT in client role
2) Protocol scanning

- Run TLS scanning tool against DUT for detection of:
  - TLS version
  - Cipher suites
  - Elliptic curves
  - Certificate type
  - Diffie-Hellman groups
  - Compression methods
- Compare the test result/report with the list of approved TLS versions and profiles as defined by O-RAN Security Protocols Specifications [2], clause 4.2.

3) For Mutual Authentication, also execute the following steps:

- Run TLS scanning tool with mTLS 1.2 and valid certificate against DUT with mutual authentication enabled to verify the establishment of the TLS session after successful authentication.
- Run TLS scanning tool with mTLS 1.2 and invalid certificate (including but not limited to expired certificate, missing field certificate, untrusted CA signed certificate, …) against DUT with mutual authentication enabled to verify the failed attempt of the TLS session establishment due to certificate validation.
- Run TLS scanning tool with mTLS 1.3 and valid certificate against DUT with mutual authentication enabled to verify the establishment of the TLS session after successful authentication.

- Run TLS scanning tool with mTLS 1.3 and invalid certificate (including but not limited to expired certificate, missing field certificate, untrusted CA signed certificate, …) against DUT with mutual authentication enabled to verify the failed attempt of the TLS session establishment due to certificate validation.

4) In case of mTLS, configure DUT in server role and repeat the steps 2 and 3.

Expected results

- All supported TLS protocol versions are explicitly allowed by [2], clause 4.2.
- All detected TLS cipher suites, elliptic curves, Diffie-Hellman groups and compression methods are explicitly allowed by [2], clause 4.2.
- Mutual authentication support works with certificates.

Expected format of evidence: Report files produced by TLS scanning tool and/or screenshots

6.3.2 TLS Version Negotiation

Requirement Name: Network Security Protocol – TLS Version Negotiation

Requirement Reference: Clause 4.2, O-RAN Security Protocols Specifications [2]

Requirement Description: Negotiate TLS versions 1.2 and 1.3

Threat References: T-O-RAN-05

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_TLS_Version

Purpose: To verify implementation of the secure communication protocol TLS as specified in [2], clause 4.2, is able to successfully negotiate the TLS version.

Procedure and execution steps

Preconditions

- Tool: TLS scanning tool with capabilities as defined in clause 5.3 with certificate(s) installed
- DUT equipped with CA certificate that is a trust anchor for the certificate(s)
- Network access to DUT with TLS or mTLS enabled

Execution steps

- Configure TLS scanning tool for only TLS 1.2
- Run TLS scanning tool with TLS/mTLS 1.2 and valid certificate against DUT to verify the establishment of the TLS session after successful authentication
- Configure TLS scanning tool for only TLS 1.3
- Run TLS scanning tool with TLS/mTLS 1.3 and valid certificate against DUT to verify the establishment of the TLS session after successful authentication
- Configure TLS scanning tool to support TLS 1.2 and TLS 1.3

NOTE: The "supported_versions" extension is used by the client to indicate which versions of TLS it supports and by the server to indicate which version it is using. The extension contains a list of supported versions in preference order, with the most preferred version first, as specified in section 4.2.1 from RFC 8446 [18].

- Run the TLS scanning tool with TLS/mTLS 1.2 and TLS/mTLS 1.3 and valid certificate against DUT to verify establishment of TLS session after successful authentication

Expected results

- DUT is able to successfully negotiate TLS/mTLS 1.2.
- DUT is able to successfully negotiate TLS/mTLS 1.3.

Expected format of evidence: Report files produced by TLS scanning tool and/or screenshots.

6.3.3 TLS Deprecated Versions

Requirement Name: Network Security Protocol – Deprecated TLS Versions

Requirement Reference: Clause 4.2, O-RAN Security Protocols Specifications [2]

Requirement Description: Reject TLS versions 1.0 and 1.1.

Threat References: T-O-RAN-05

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_TLS_DeprecatedVersions

Purpose: To verify implementation of the secure communication protocol TLS as specified in [2], clause 4.2, does not establish a TLS session with TLS 1.0 or 1.1, or SSL 2.0 or 3.0.

Procedure and execution steps

Preconditions

- Tool: TLS scanning tool with capabilities as defined in clause 5.3 with certificate(s) installed
- DUT equipped with CA certificate that is a trust anchor for the certificate(s)
- Network access to DUT with TLS or mTLS enabled

Execution steps

- Configure TLS scanning tool to support only TLS 1.0
- Run the TLS scanning with TLS 1.0 and valid certificate against the DUT
- Configure TLS scanning tool to support only TLS 1.1
- Run the TLS scanning with TLS 1.1 and valid certificate against the DUT
- Configure TLS scanning tool to support only SSL 2.0
- Run the TLS scanning with SSL 2.0 and valid certificate against the DUT
- Configure TLS scanning tool to support only SSL 3.0
- Run the TLS scanning with SSL 3.0 and valid certificate against the DUT

Expected results

- DUT rejects connection for TLS 1.0
- DUT rejects connection for TLS 1.1
- DUT rejects connection for SSL 2.0
- DUT rejects connection for SSL 3.0

Expected format of evidence: Report files produced by TLS scanning tool and/or screenshots

6.4 DTLS

Requirement Name: Network Security Protocol - DTLS

Requirement Reference: Clause 4.4, O-RAN Security Protocols Specifications [2]

Requirement Description: Support DTLS

Threat References: T-O-RAN-01

DUT/s: Near-RT RIC, O-CU-CP, O-CU-UP, O-DU

Test Name: TC_DTLS_Protocol

Purpose: To verify implementation of the secure communication protocol DTLS as specified in [2], clause 4.4.

Procedure and execution steps

Preconditions

- Tool: DTLS scanning tool with capabilities as defined in clause 5.3
- Network access to DUT

Execution steps

- Run DTLS scanning tool against DUT for detection of:
  - DTLS version
  - Cipher suites
  - Elliptic curves
  - Certificate type
  - Diffie-Hellman groups
  - Compression methods
- Compare the test result/report with the list of approved DTLS versions and profiles as defined by Security Protocols Specification [2], clause 4.4.

Expected results

- All supported DTLS protocol versions are explicitly allowed by [2], clause 4.4.
- All detected DTLS cipher suites, elliptic curves, Diffie-Hellman groups and compression methods are explicitly allowed by [2], clause 4.4.

Expected format of evidence: Report files produced by DTLS scanning tool and/or screenshots.

6.5 IPsec

6.5.1 IPsec security

Requirement Name: Network Security Protocol - IPsec

Requirement Reference: Clause 4.5, O-RAN Security Protocols Specifications [2]

Requirement Description: Support IPsec tunnel mode with confidentiality, integrity, authentication, and replay protection.

Threat References: T-O-RAN-05

DUT/s: Near-RT RIC, O-CU-CP, O-CU-UP, O-DU

Test Name: TC_IPsec_Security

Purpose: To verify implementation of the secure communication protocol IPsec.

Procedure and execution steps

Preconditions

- Tool: IKE scanning tool with capabilities as defined in clause 5.3
- Network access to DUT

Execution steps

- Run IKE scanning tool against DUT for detection of:
  - ESP Encryption Transforms
  - ESP Authentication Transforms
  - Diffie-Hellman groups
  - Certificate type
  - Pseudo-random function
- Compare the test result/report with the list of approved IPsec IKE versions, ESP Encryption Transforms, ESP Authentication Transforms, Diffie-Hellman groups and Pseudo-random function as defined by O-RAN Security Protocols Specification [2], clause 4.5.

Expected results

- All detected IPsec IKE versions are allowed by [2], clause 4.5.
- All detected ESP Encryption Transforms, ESP Authentication Transforms, Diffie-Hellman groups and Pseudo-random function are allowed by [2], clause 4.5. IKE version (v2) support with no older version(s) enabled.
- If certificates are used, their format is X.509v3.

Expected format of evidence:

- .pcap files capturing the IKE negotiations between the tool and the DUT.
- Report or output from the IKE scanning tool, specifically highlighting:
  - Detected ESP Encryption Transforms.
  - Detected ESP Authentication Transforms.
  - Detected Diffie-Hellman groups.
  - Detected Certificate type.
  - Detected Pseudo-random function.
- Screenshots from the IKE scanning tool showing scan results, especially the supported IKE version detected.
- If certificates are used, a sample or screenshot verifying the X.509v3 format.

6.5.2 IKE Header Flags Fuzzing

Requirement Name: Network Security Protocol - IPsec

Requirement Reference: Clause 4.5, O-RAN Security Protocols Specifications [2]

Requirement Description: Support IPsec tunnel mode with confidentiality, integrity, authentication, and replay protection.

Threat References: T-O-RAN-01

DUT/s: Near-RT RIC, O-CU-CP, O-CU-UP, O-DU

Test Name: TC_IKE_HEADER_FLAGS_FUZZING

Purpose: The purpose of this test is to verify the robustness of the IKEv2 server when faced with malformed IKE headers. Flags within the IKE header are intended to provide specific instructions or information about the message. By fuzzing these flags, we can identify potential vulnerabilities or flaws in the server's processing logic.

Procedure and execution steps

Preconditions

- A controlled environment with an IKEv2 server and a test client.
- Packet capture tool (e.g., Wireshark) for monitoring the traffic.
- Fuzzing tool or script to generate malformed IKE header flags.

Execution steps

1) Begin by starting the packet capture tool to record the test session.
2) Use the fuzzing tool or script to generate IKEv2 messages with the following malformed flags in the IKE header:

- Initiator flag: Flip this flag to see if the server can identify a message that isn’t from an initiator.
- Version flag: Introduce an unsupported version.
- Response flag: Send messages that have this flag inappropriately set.
- Combination of multiple flags: Mix flags to generate completely unexpected combinations.

3) Send each of these malformed messages to the IKEv2 server individually, waiting for a response before sending the next.
4) Observe server reactions, looking specifically for any unhandled exceptions, crashes, or irregular behaviours.

Expected Results

- The IKEv2 server handles the malformed flags gracefully, either by rejecting the message or by ignoring the unexpected flag values.
- There is no crashes, hangs, or undefined behaviours.

Expected format of evidence

- Packet capture files (.pcap) showing the malformed flags sent and the server's responses.
- Server logs indicating the handling (or rejection) of the malformed messages.

6.5.3 IKE Key Exchange Payload Fuzzing

Requirement Name: Network Security Protocol - IPsec

Requirement Reference: Clause 4.5, O-RAN Security Protocols Specifications [2]

Requirement Description: Support IPsec tunnel mode with confidentiality, integrity, authentication, and replay protection.

Threat References: T-O-RAN-01

DUT/s: Near-RT RIC, O-CU-CP, O-CU-UP, O-DU

Test Name: TC_IKE_KEY_EXCHANGE_PAYLOAD_FUZZING

Purpose: The purpose of this test is to examine the IKEv2 server's ability to manage corrupted or unexpected data within the Key Exchange (KE) payload. The KE payload carries the Diffie-Hellman public value. If the server is unable to handle malformed KE payloads, it might be susceptible to attacks or crashes.

Procedure and execution steps

Preconditions

- A controlled environment with an IKEv2 server and a test client.
- Packet capture tool (e.g., Wireshark) for monitoring the traffic.
- Fuzzing tool or script capable of generating malformed KE payloads.

Execution steps

1) Initiate the packet capture tool to ensure every detail of the test session is recorded.
2) Use your fuzzing tool or script to generate IKEv2 messages with the following specific manipulations in the KE payloads:

- Unexpected length: Prepare 10 distinct messages where the KE payload's declared length is longer or shorter than the actual payload.
- Corrupted data: Generate 10 messages introducing random bytes into the KE payload to see how the server handles non-standard values.
- Unsupported Diffie-Hellman groups: Create 5 messages attempting to initiate a key exchange using a DH group that is either deprecated or not supported by the server.
- Empty KE payload: Formulate 5 messages with an empty KE payload.

3) Sequentially send these 30 malformed messages to the IKEv2 server. After sending each message, wait for the server's response to avoid overloading it. Ensure the following sequence:

- Send the 10 "Unexpected Length" messages.
- Follow with the 10 "Corrupted Data" messages.
- Continue with the 5 "Unsupported Diffie-Hellman Groups" messages.
- Conclude with the 5 "Empty KE Payload" messages.

NOTE: Monitor the server's reactions closely. The server ideally handles errors gracefully, either ignoring them or responding with an appropriate error message, without any crashes or hangs.

Expected Results

- The IKEv2 server gracefully handles the malformed KE payloads, either by ignoring them, responding with an error, or requesting a valid KE payload.
- No crashes, hangs, or undefined behaviours occur.

Expected format of evidence

- Packet capture files (.pcap) highlighting the malformed KE payloads and the server's corresponding responses.
- Server logs detailing the handling (or rejection) of the malformed KE payloads.

6.5.4 IKE Malformed Certificate Payload

Requirement Name: Network Security Protocol - IPsec

Requirement Reference: Clause 4.5, O-RAN Security Protocols Specifications [2]

Requirement Description: Support IPsec tunnel mode with confidentiality, integrity, authentication, and replay protection.

Threat References: T-O-RAN-01

DUT/s: Near-RT RIC, O-CU-CP, O-CU-UP, O-DU

Test Name: TC_IKE_MALFORMED_CERTIFICATE_PAYLOAD

Purpose: This test aims to verify the IKEv2 server's capability to properly validate certificate payloads. Certificate payloads are essential in the IKEv2 authentication phase. A server vulnerable to malformed certificate payloads could be susceptible to impersonation or man-in-the-middle attacks.

Procedure and execution steps

Preconditions

- A controlled environment with an IKEv2 server and a test client.
- Packet capture tool (e.g., Wireshark) to monitor and capture traffic.
- A set of both valid and deliberately malformed certificates.

Execution steps

1) Valid Certificate Test:

- Initiate an IKEv2 session using a valid certificate to ensure baseline functionality.
- Confirm successful authentication and session establishment.

2) Expired Certificate:

- Use a previously valid certificate that has now expired.
- Attempt to initiate an IKEv2 session.
- Observe the server's rejection of this certificate.

3) Certificate with Invalid Signature:

- Modify a valid certificate's content slightly (e.g., change an attribute) without re-signing it. This will invalidate its signature.
- Attempt to initiate an IKEv2 session using this certificate.
- The server detects the invalid signature and rejects the connection.

4) Certificate from Untrusted Authority:

- Generate a new certificate signed by a Certificate Authority (CA) that the IKEv2 server doesn't trust or recognize.
- Attempt to initiate a connection using this certificate.

- Observe the server rejecting the certificate due to the untrusted CA.

5) Certificate with Modified Subject/Issuer Fields:

- Modify the subject or issuer fields of a certificate to contain irregular or unexpected values (e.g., overly long strings, special characters).
- Use this certificate to initiate an IKEv2 session.
- The server validates these fields, notices the irregularities, and potentially rejects the connection.

6) Certificate with Invalid Key Usage:

- Use a certificate that doesn't have "key encipherment" or "digital signature" as its key usage, which are typically needed for IKEv2 operations.
- Attempt to initiate a session.
- The server detects the inappropriate key usage and declines the connection.

Expected Results:

- For the valid certificate, the IKEv2 server authenticates successfully and establish a session.
- For all other scenarios, the IKEv2 server detects the certificate anomalies and rejects the connection attempts. Specific error messages or logs relating to certificate validation failure are generated.

Expected format of evidence:

- Packet capture files (.pcap) capturing the entire exchange, showing the certificate exchange and the server's response.
- Server logs detailing the acceptance or rejection of each certificate, with corresponding reasons or error messages for rejections.

6.6 OAuth 2.0

6.6.1 API Consumer

Requirement Name: Authorization based on OAuth 2.0

Requirement Reference: Clause 4.7, O-RAN Security Protocols Specifications [2]

Requirement Description: O-RAN OAuth 2.0 based authorization including resource registration, access token request and service access request based on token verification process as defined in [2]

Threat References: T-O-RAN-05

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_OAuth2.0_API_Consumer

Purpose: To verify implementation of the authorization for API consumer based on OAuth 2.0 as specified in [2]

Procedure and execution steps

Preconditions

- DUT is acting as a OAuth2.0 API Consumer
- For Test case A:
  - Certificates for OAuth2.0 authentication

- mTLS connection configured for all the components involved in the test case.
- DUT establishes a mTLS connection with Authorization Server and another mTLS connection with Resource Server.

- For Test case B:
  - Client ID and client secret for OAuth2.0 authentication
  - TLS connection configured for all the components involved in the test case.
  - DUT establishes a TLS connection with Authorization Server and another TLS connection with Resource Server.
- Resource Server, may be real or simulated
- Authorization server – OAuth 2.0 Authorization Server (real or simulated)
- Resource server registered with the Authorization server for its supported API service(s)
  - This process can be a manual or automatic process preceding with Resource server authentication
- API consumer with the role to have access to the resource to be requested
- Network access to Authorization Server, Resource Server and API consumer

Execution steps

Test case A :

1) DUT requests an access token to the Authorization Server.
2) With access token, the DUT sends an API service request towards Resource Server (API producer) using the access token obtained as a response to access token request

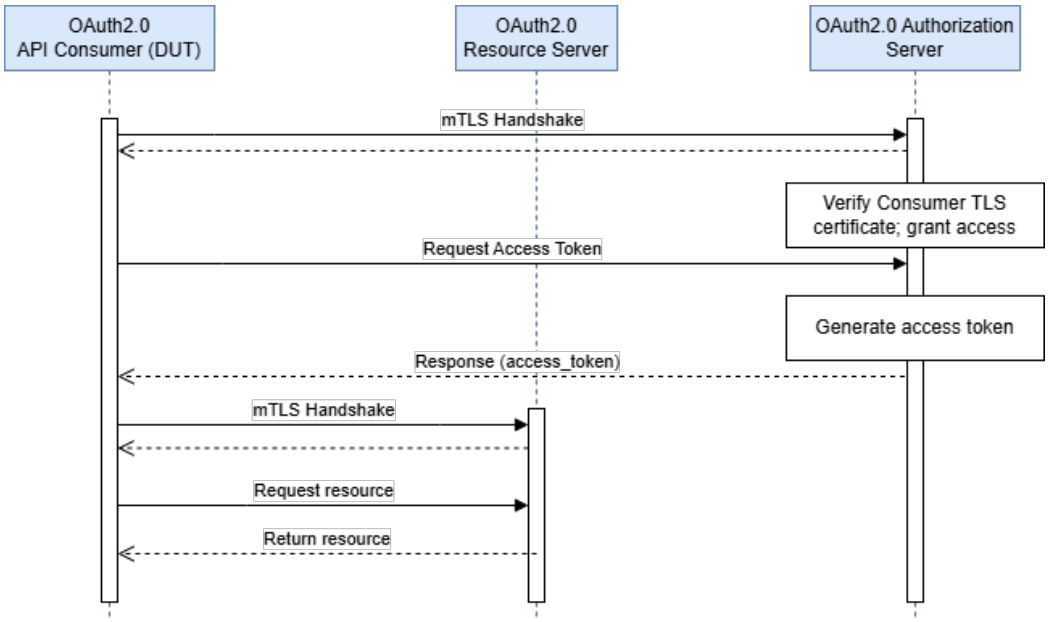

Figure 6-1: Token request using mTLS and Service request

Test case B:

1) DUT requests a token to the Authorization Server using client ID and the secret as authentication.
2) With access token, the DUT sends an API service request towards Resource Server (API producer) using the access token obtained as a response to access token request

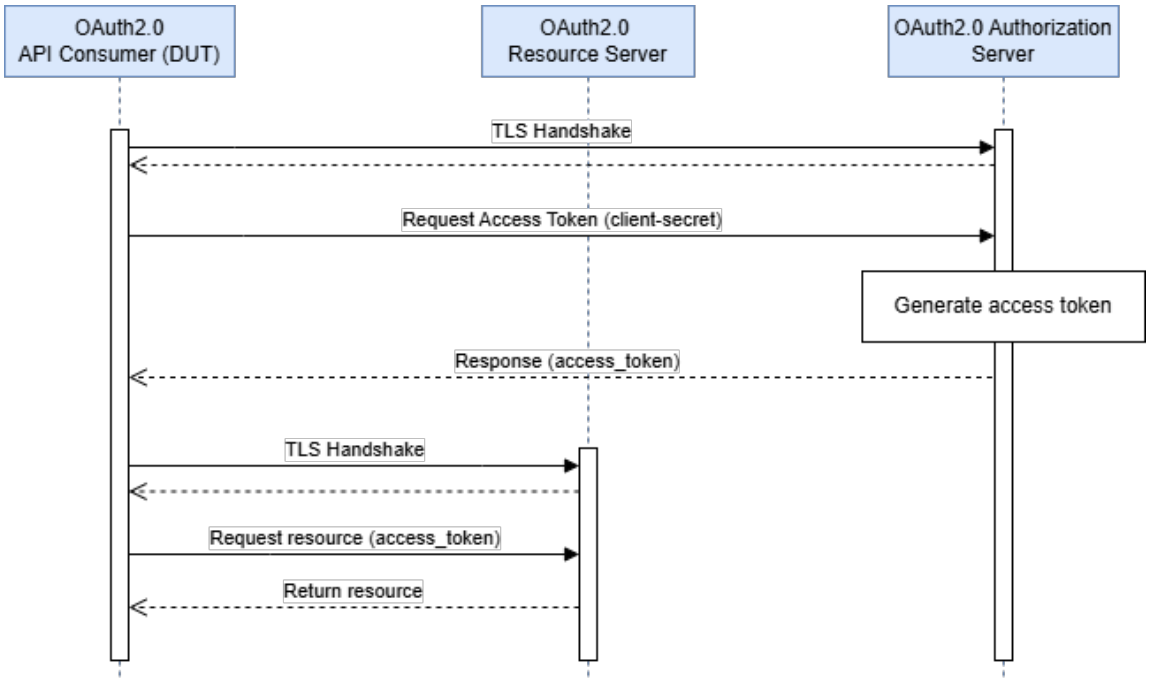

Figure 6-2: Token request with secret ID and Service request

Expected results

For Test case A:

The DUT establishes the mTLS connection with the Authorization Server and the Resource Server. The DUT obtains access to the requested resource onto the Resource Server.

For Test case B:

The DUT establishes the TLS connection with the Authorization Server and the Resource Server. The DUT obtains the access to the requested resource onto the Resource Server.

Expected format of evidence: Log files, traffic captures and/or screenshots.

6.6.2 Resource Server

Requirement Name: Authorization based on OAuth 2.0

Requirement Reference: Clause 4.7, O-RAN Security Protocols Specifications [2]

Requirement Description: O-RAN OAuth 2.0 based authorization including resource registration, access token request and service access request based on token verification process as defined in [2]

Threat References: T-O-RAN-05

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-Cloud

Test Name: TC_OAuth2.0_Resource_Server

Purpose: To verify implementation of the authorization for Resource Server based on OAuth 2.0 as specified in [2]

Procedure and execution steps

Preconditions

- DUT is acting as a OAuth 2.0 Resource Server
- For Test case A:

- Certificates for OAuth 2.0 authentication.
- mTLS connection configured for all the components involved in the test case.
- DUT establishes a mTLS connection with Authorization Server and another mTLS connection with the API consumer.

- For Test case B:
  - Share client ID and client secrets for OAuth 2.0 authentication
  - TLS connection configured for all the components involved in the test case.
  - DUT establishes a TLS connection with Authorization Server and another TLS connection with the API Consumer.
- API Consumer may be real or simulated
- Authorization Server – OAuth 2.0 Authorization Server (real or emulated)
- Resource Server registered with the Authorization Server for its supported API service(s)
  - This process can be a manual or automatic process preceding with Resource Server authentication
- API Consumer with the role to have access to the resource to be requested
- Network access to Authorization Server, Resource Server and API Consumer

Execution steps

Test case A:

1) API consumer requests a token from the Authorization server.
2) With the access token, the API consumer sends an API service request towards Resource Server (API producer) using the access token obtained in the previous step.

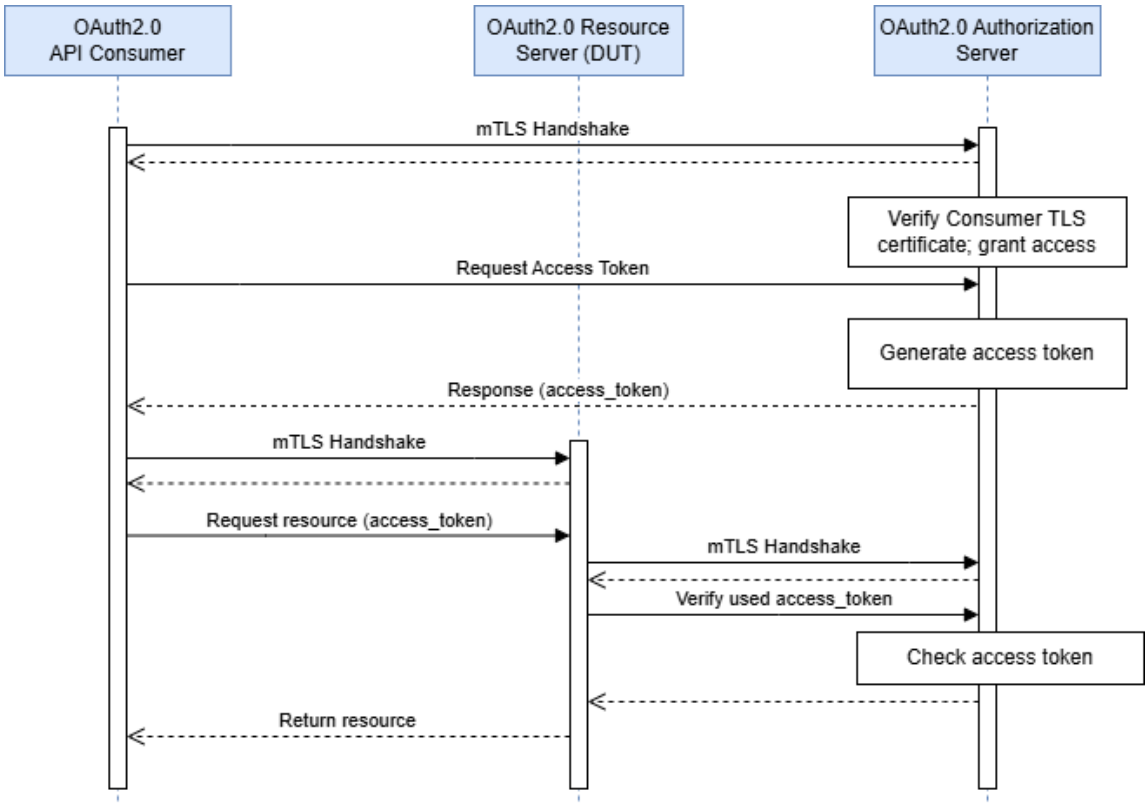

Figure 6-3: Token request using mTLS and Service Request

Test case B:

1) DUT requests a token from the Authorization Server using the client ID and secret as authentication.
2) With access token, the API Consumer sends an API service request to the DUT (Resource Server) using the access token obtained in the previous step.
3) The DUT verifies the access token is valid with the Authorization Server.
4) The DUT returns the service requested to the API Consumer.

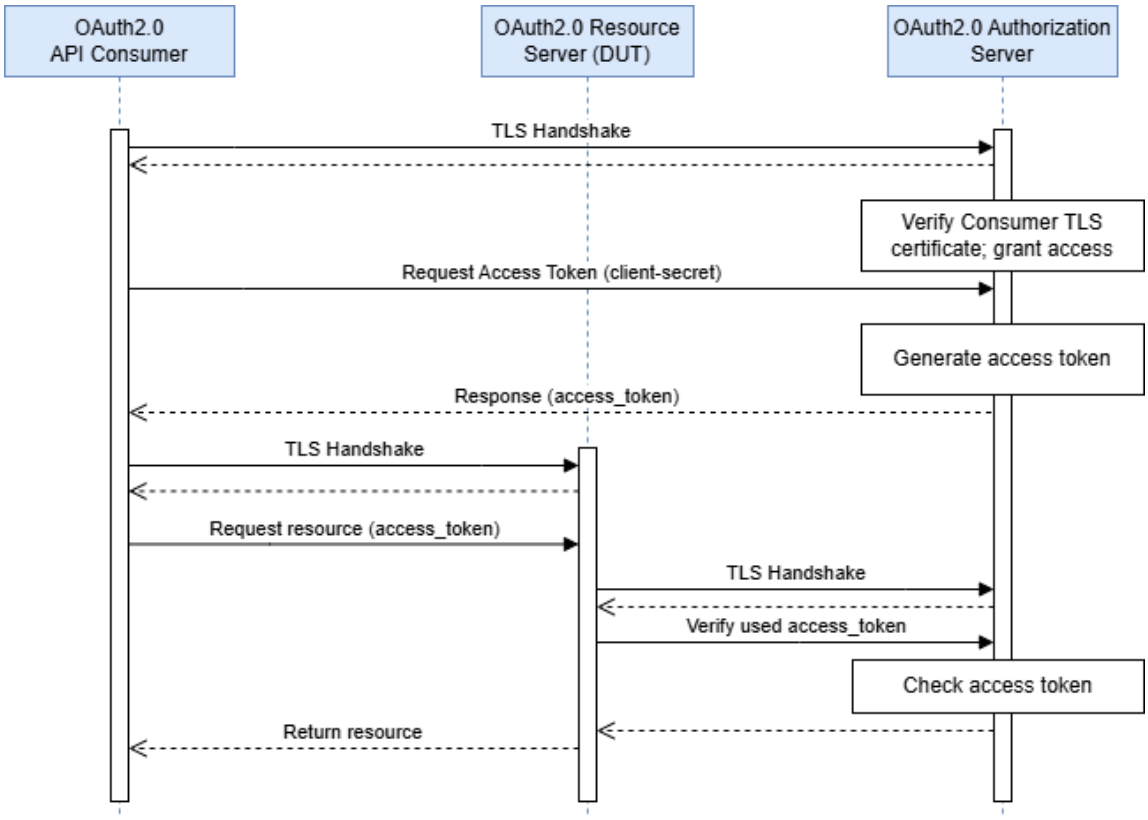

Figure 6-4: Token request with secret ID and Service request

Expected results

For Test case A: The DUT establishes the mTLS connection with the Authorization Server and with the API Consumer. The DUT sends the requested service to the API Consumer.

For Test case B: The DUT establishes the TLS connection with the Authorization Server and with the API Consumer. The DUT sends the requested resource to the API Consumer.

Expected format of evidence: Log files, traffic captures and/or screenshots.

6.7 NACM

6.7.1 NACM RBAC Configuration

Requirement Name: NACM security

Requirement Reference: REQ-NAC-FUN-1 to REQ-NAC-FUN-10, clause 5.2.2.3 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: NETCONF/NACM support

Threat References: T-O-RAN-03, T-O-RAN-05, T-O-RAN-06

DUT/s: Near-RT RIC, O-CU-CP, O-CU-UP, O-DU

Test Name: TC_NACM_RBAC_CONFIGURATION

Purpose: The purpose of this test is to verify the RBAC configuration for secure access control on the TLS-based NACM with NETCONF.

Procedure and execution steps

Preconditions

- The NACM and NETCONF services are properly configured and operational.
- The RBAC feature is supported and enabled in the NACM system.
- RBAC roles, access control rules, and denied resources or operations are properly defined.

Execution steps

1) Verify RBAC role definitions.

- Check that RBAC roles are properly defined for access control.
  - Review the RBAC role definitions.

EXAMPLE 1: Command "**show nacm rbac roles**"

  - Validate that the defined roles match the intended access control requirements.

2) Verify RBAC role assignment.

- Test the assignment of RBAC roles to users or user groups.
  - Assign roles to users or user groups.

EXAMPLE 2: "Command: configure nacm rbac role-assignment"

  - Verify that the assigned roles are reflected in the configuration.

3) Verify unauthorized access denial.

- Test access to resources or operations that are not permitted for a specific RBAC role.
  - Identify a resource or operation that is denied for a specific role.

EXAMPLE 3: "Command show nacm rbac role-permissions <role_name>"

  - Attempt to access the denied resource or operation with a user assigned to the role.

EXAMPLE 4: "Command: execute netconf operation <operation_name>"

Expected Results

For step 1), Roles are defined with their associated permissions and restrictions.

For step 2), Roles are assigned to the appropriate users or user groups.

For step 3)-a, The denied resource or operation is listed for the specified role.

For step 3)-b, Access to the denied resource or operation is denied, and an appropriate error message is displayed.

Expected format of evidence

For step 1), Log file with the output of the show nacm rbac roles command, showing the defined roles and their associated permissions and restrictions. In case the logs do no show the required information, screenshots are used.

For step 2), Log file with the confirmation that the roles have been successfully assigned to the appropriate users or user groups, as reflected in the configuration. In case the logs do not show the required information, screenshots are used.

For step 3), Log file showing the appropriate error message indicating access denial when attempting to access a denied resource or operation with a user assigned to a specific role. In case the logs do not show the required information, screenshots are used.

6.7.2 NACM Logging Monitoring

Requirement Name: NACM security

Requirement Reference: REQ-NAC-FUN-1 to REQ-NAC-FUN-10, clause 5.2.2.3, REQ-SEC-SLM-AAI-EVT-1 to REQ-SEC-SLM-AAI-EVT-10, clause 5.3.8.11.6 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: NETCONF/NACM support

Threat References: T-O-RAN-03, T-O-RAN-05, T-O-RAN-06

DUT/s: Near-RT RIC, O-CU-CP, O-CU-UP, O-DU

Test Name: TC_NACM_LOGGING_MONITORING

Purpose: The purpose of this test is to verify the logging and monitoring configuration for the TLS-based NACM with NETCONF.

Procedure and execution steps

Preconditions

- The NACM and NETCONF services are properly configured and operational.
- Logging and monitoring systems are in place, integrated and configured with the NACM system.

Execution steps

1) Verify logging configuration.

- Check that logging is properly configured to capture relevant security-related events.
  - Review the logging configuration settings.

EXAMPLE 1: "Command: show nacm logging configuration"

  - Trigger security-related events (e.g., access violations, failed authentication attempts) and validate that the events are logged.

2) Verify monitoring configuration.

- Test the monitoring configuration to ensure that security-related events and performance metrics are monitored.
  - Review the monitoring configuration settings.

EXAMPLE 2: "Command: show nacm monitoring configuration"

  - Trigger security-related events or exceed performance thresholds and verify that the monitoring system captures and reports these events or metrics.

3) Verify audit log review.

- Test the ability to review audit logs for security-related events.
  - Retrieve the audit logs.

EXAMPLE 3: "Command: **show nacm audit-logs**"

  - Review the audit logs to ensure that they contain the expected information and provide a detailed record of security-related activities.

Expected Results

For step 1), Logging is enabled with appropriate log levels, log destinations, and log retention policies.

For step 2), Monitoring is enabled with appropriate metrics, thresholds, and alerting mechanisms.

For step 3), Audit logs containing security-related events are available.

Expected format of evidence

Text file showing the confirmation that logging is enabled with the expected log levels, log destinations, and log retention policies. Additionally, evidence of captured security-related events in the logs. In case the logs do not show the required information, screenshots are used.

Text file showing the confirmation that monitoring is enabled with the configured metrics, thresholds, and alerting mechanisms. Evidence of captured security-related events or performance metrics exceeding thresholds. In case the logs do not show the required information, screenshots are used.

Audit logs containing the security-related events, demonstrating that they contain the expected information and provide a detailed record of security-related activities.

6.7.3 Void

6.8 802.1X

Void

6.9 X.509

6.9.1 X.509 Certificate Structure Verification for TLS

Requirement Name: X.509 security

Requirement Reference: SEC-CTL-O1-1, clause 5.2.2.2, SEC-CTL-O2-1, clause 5.2.3.1, REQ-SEC-OCLOUD-O2dms-1 to REQ-SEC-OCLOUD-O2dms-3, clause 5.1.8.9.1.1, REQ-SEC-OCLOUD-O2ims-1 to REQ-SEC-OCLOUD-O2ims-3, clause 5.1.8.9.1.2, REQ-SEC-O-CLOUD-NotifAPI-1 to REQ-SEC-O-CLOUD-NotifAPI-2, clause 5.1.8.9.1.3, SEC-CTL-A1-1, SEC-CTL-A1-2, clause 5.2.1.2, SEC-CTL-R1-1, SEC-CTL-R1-2, clause 5.2.6.2, REQ-SEC-Y1-1 to REQ-SEC-Y1-3, clause 5.2.7.2, O-RAN Security Requirements and Controls Specifications [5], clause 4.2.3 in O-RAN Security Protocols Specifications [2].

Requirement Description: To verify the X.509 certificate structure used by TLS components in all the ORAN systems.

Threat References: T-O-RAN-03, T-O-RAN-05, T-O-RAN-06

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_X509_CERT_STRUCTURE_VERIFICATION

Purpose: The purpose of this test is to ensure that the X.509 certificate follows the correct structure and format for TLS. This test is applicable to X.509 certificates used in TLS modules in the O-RAN system.

Procedure and execution steps

Preconditions:

- End entity X.509 certificate in ASN.1 format.
- CA certificate that signed the end entity certificate is available to the test suite.

Execution steps

Following properties of the certificate are verified.

- Certificate Version field, Subject field, Issuer, Validity of certificate, Key Usage.

As defined in clause 4.2.3 in O-RAN Security Protocols Specifications [2], the following certificate fields are present.

1) Certificate Fields Examination:

- Validate the certificate's version field. The version number is set to v3.
- Verify the Subject field is present in the certificate. Verify that the Subject field conforms to the format defined in clause 4.2.3 in O-RAN Security Protocols Specifications [2].
- Verify the Validity field to ensure the "Not Before" date is earlier than the "Not After" date, indicating a valid time range for the certificate's use.
- Validate that the certificate’s signature field is based on the algorithm used by the CA to sign the certificate.

2) Key Usage Extension:

- Validate the presence of the Key Usage Extension. Verify that it conforms with the mandatory critical Key Usage extension as specified in the clause 4.2.3 in O-RAN Security Protocols Specifications [2].

3) CRL Distribution Point:

- Validate that the certificate contains the cRLDistributionPoint extension.

4) Subject Alternative Name:

- Validate that the certificate contains the subjectAltName extension and conforms to the requirements as specified in the clause 4.2.3 in O-RAN Security Protocols Specifications [2].

Expected Results

The certificate adheres to the X.509 standard structure as defined in clause 4.2.3 in O-RAN Security Protocols Specifications [2].

Expected format of evidence

Report containing the output of the different steps executed. In case the report does not show the required information, logs are used.

6.9.2 X.509 Certificate Validity Period Verification

Requirement Name: X.509 security

Requirement Reference: SEC-CTL-O1-1, clause 5.2.2. 2, REQ-SEC-O2-1, clause 5.2.3.1, REQ-SEC-OCLOUD-O2dms-1 to REQ-SEC-OCLOUD-O2dms-3, clause 5.1.8.9.1.1, REQ-SEC-OCLOUD-O2ims-1 to REQ-SEC-OCLOUD-O2ims-3, clause 5.1.8.9.1.2, REQ-SEC-O-CLOUD-NotifAPI-1 to REQ-SEC-O-CLOUD-NotifAPI-2, clause 5.1.8.9.1.3, REQ-SEC-A1-1, REQ-SEC-A1-2, clause 5.2.1.1, REQ-SEC-E2-1, clause 5.2.4.1, REQ-SEC-R1-1, REQ-SEC-R1-2, clause 5.2.6.1, REQ-SEC-Y1-1 to REQ-SEC-Y1-3, clause 5.2.7.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Correctness of X.509 certificate

Threat References: T-O-RAN-03, T-O-RAN-05, T-O-RAN-06

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_X509_CERT_VALIDITY_PERIOD_VERIFICATION

Purpose: The purpose of this test is to ensure that the certificate's validity dates are accurate and within an acceptable range. This test is relevant for all X.509 certificates within the O-RAN system.

Procedure and execution steps

Preconditions

Prepare certificates with different validity periods (valid, expired, not yet valid).

Execution steps

1) Verify a Valid Certificate:

- Set up a valid certificate with appropriate "Not Before" and "Not After" dates.
- Verify that the certificate is accepted when used for its intended purpose.

2) Verify an Expired Certificate:

- Set up a certificate with a past expiration date.
- Attempt to use the expired certificate for its intended purpose.
- Verify that the certificate is rejected due to expiration.

3) Verify a Not Yet Valid Certificate:

- Set up a certificate with a "Not Before" date in the future.
- Attempt to use the certificate before the valid start date.
- Verify that the certificate is rejected due to being not yet valid.

Expected Results

Valid certificates are accepted, while expired and not-yet-valid certificates are rejected.

Expected format of evidence

Report containing the output of the different steps executed. In case the report does not show the required information, logs are used.

6.9.3 X.509 Certificate Key Usage Verification

Requirement Name: X.509 security

Requirement Reference: SEC-CTL-O1-1, clause 5.2.2. 2, REQ-SEC-O2-1, clause 5.2.3.1, REQ-SEC-OCLOUD-O2dms-1 to REQ-SEC-OCLOUD-O2dms-3, clause 5.1.8.9.1.1, REQ-SEC-OCLOUD-O2ims-1 to REQ-SEC-OCLOUD-O2ims-3, clause 5.1.8.9.1.2, REQ-SEC-O-CLOUD-NotifAPI-1 to REQ-SEC-O-CLOUD-NotifAPI-2, clause 5.1.8.9.1.3, REQ-SEC-A1-1, REQ-SEC-A1-2, clause 5.2.1.1, REQ-SEC-E2-1, clause 5.2.4.1, REQ-SEC-R1-1, REQ-SEC-R1-2, clause 5.2.6.1, REQ-SEC-Y1-1 to REQ-SEC-Y1-3, clause 5.2.7.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Correctness of X.509 certificate

Threat References: T-O-RAN-03, T-O-RAN-05, T-O-RAN-06

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_X509_CERT_KEY_USAGE_VERIFICATION

Purpose: The purpose of this test is to confirm that the certificate's key usage and extended key usage extensions are correctly defined.

Procedure and execution steps

Preconditions

Prepare certificates with different key usage and extended key usage extensions.

Execution steps

1) Verify a Certificate with Correct Usage Extensions:

- Set up a certificate with proper key usage and extended key usage extensions matching its intended purpose (e.g., server authentication).
- Attempt to use the certificate for its designated purpose.
- Verify that the certificate is accepted.

2) Verify a Certificate with Incorrect or Missing Usage Extensions:

- Set up a certificate with incorrect or missing key usage or extended key usage extensions.
- Attempt to use the certificate for its intended purpose.
- Verify that the certificate is rejected.

Expected Results

Certificates with correct usage extensions are accepted, while those with incorrect or missing extensions are rejected.

Expected format of evidence

Report containing the output of the different steps executed. In case the report does not show the required information, logs are used.

6.9.4 X.509 Certificate Chain Validation

Requirement Name: X.509 security

Requirement Reference: SEC-CTL-O1-1, clause 5.2.2. 2, REQ-SEC-O2-1, clause 5.2.3.1, REQ-SEC-OCLOUD-O2dms-1 to REQ-SEC-OCLOUD-O2dms-3, clause 5.1.8.9.1.1, REQ-SEC-OCLOUD-O2ims-1 to REQ-SEC-OCLOUD-O2ims-3, clause 5.1.8.9.1.2, REQ-SEC-O-CLOUD-NotifAPI-1 to REQ-SEC-O-CLOUD-NotifAPI-2, clause 5.1.8.9.1.3, REQ-SEC-A1-1, REQ-SEC-A1-2, clause 5.2.1.1, REQ-SEC-E2-1, clause 5.2.4.1, REQ-SEC-R1-1, REQ-SEC-R1-2, clause 5.2.6.1, REQ-SEC-Y1-1 to REQ-SEC-Y1-3, clause 5.2.7.2 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Correctness of X.509 certificate chain

Threat References: T-O-RAN-03, T-O-RAN-05, T-O-RAN-06

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_X509_CERT_CHAIN_VALIDATION

Purpose: The purpose of this test is to validate the certificate chain's integrity and trustworthiness. This test is applicable to scenarios where certificates are part of a chain (e.g., intermediate and root certificates).

Procedure and execution steps

Preconditions

Prepare a certificate chain with correct and incorrect configurations.

Execution steps

1) Verify a Certificate Chain with Correct Order and Valid Signatures:

- Set up a valid certificate chain with correct order and valid signatures.
- Attempt to use the certificate chain for its intended purpose.
- Verify that the certificate chain is accepted.

2) Verify a Certificate Chain with Incorrect Order:

- Set up a certificate chain with incorrect order.
- Attempt to use the certificate chain for its intended purpose.
- Verify that the certificate chain is rejected.

3) Verify a Certificate Chain with Invalid Signatures:

- Set up a certificate chain with certificates with invalid signatures.
- Attempt to use the certificate chain for its intended purpose.
- Verify that the certificate chain is rejected

4) Verify a Certificate Chain with Incorrect Order and Invalid Signatures:

- Set up certificate chain with incorrect order and certificates with invalid signatures.
- Attempt to used the certificate chain for its intended purpose.
- Verify that the certificate chain is rejected.

Expected Results

A valid certificate chain is accepted, while an invalid chain is rejected.

Expected format of evidence

Report containing the output of the different steps executed. In case the report does not show the required information, logs are used.

6.10 eCPRI

6.10.1 Void

6.10.2 eCPRI Input Validation

Requirement Name: eCPRI security

Requirement Reference: REQ-SEC-TRAN-1, clause 5.3.4.1 in O-RAN Security Requirements and Controls Specifications [5].

Requirement Description: eCPRI protocol robustness – ability to handle unexpected inputs (not in-line with protocol specification) without functional compromise.

Threat References: T-FRHAUL-01, T-FRHAUL-02

DUT/s: O-RU, O-DU

Test Name: TC_eCPRI_INPUT_VALIDATION

Purpose: The purpose of this test is to ensure that the DUT properly validates and sanitizes input through the eCPRI interface to prevent common security vulnerabilities such as injection attacks.

Procedure and execution steps

Preconditions

- eCPRI interface is accessible.
- Input fields requiring validation are identified.

Execution steps

1) Positive Case:

- Send requests through the eCPRI interface with valid and expected input values.
- Verify that the DUT processes the requests successfully and provides the expected responses.

2) Negative Case:

- Generate requests through the eCPRI interface by systematically applying fuzzing techniques to introduce deliberately malicious input values containing potential security threats.
- Verify that the DUT detects and rejects the malicious input, responding with appropriate error messages or status codes.

Expected Results: The DUT validates and sanitizes input through the eCPRI interface to prevent security vulnerabilities related to improper input handling.

Expected format of evidence

- A log file documenting the requests sent to the DUT through the eCPRI interface, including valid and malicious inputs.
- Screenshots of the DUT responses through the eCPRI interface showing the handling of valid inputs and appropriate error messages for malicious inputs.

6.10.3 eCPRI Error and Timeout Handling

Requirement Name: eCPRI security

Requirement Reference: REQ-SEC-TRAN-1, clause 5.3.4.1 in O-RAN Security Requirements and Controls Specifications [5].

Requirement Description: eCPRI protocol robustness - ability to handle unexpected inputs (not in-line with protocol specification) without functional compromise.

Threat References: T-FRHAUL-01, T-FRHAUL-02

DUT/s: O-RU, O-DU

Test Name: TC_eCPRI_ERROR_TIMEOUT_HANDLING

Purpose: The purpose of this test is to ensure that the DUT securely handles errors on the eCPRI interface, including malformed packets, unexpected messages, and timeout scenarios, without disclosing sensitive information or compromising DUT stability.

Procedure and execution steps

Preconditions

- eCPRI interface is accessible.
- Various error scenarios (malformed, unexpected, or delayed eCPRI packets) are identified.

Execution steps

1) Attempt to force error conditions on the eCPRI interface:

- Transmit eCPRI packets with anomalies such as invalid headers, incorrect protocol versions, unsupported message types, or corrupted payloads.
- Introduce significant delays or slow down the network connection on the eCPRI interface.

2) Verify that the DUT detects and handles the errors.

3) Restore normal connectivity.

4) Resend a normal request to the DUT through the eCPRI interface.

5) Verify that the DUT processes the request successfully and provides the expected response.

Expected Results: The DUT handles errors securely, providing meaningful error messages without disclosing sensitive information and recovering seamlessly when the connection is restored.

Expected format of evidence

- Screenshots of the error messages or status codes received from the DUT through the eCPRI interface in response to triggered errors.
- A log file documenting the requests and responses during error scenarios.

6.10.4 Void

6.10.5 eCPRI Logging and Auditing

Requirement Name: eCPRI security

Requirement Reference: REQ-SEC-SLM-APP-EVT-1, clause 5.3.8.11.4 in O-RAN Security Requirements and Controls Specifications [5].

Requirement Description:

Threat References: T-FRHAUL-01, T-FRHAUL-02

DUT/s: O-RU, O-DU

Test Name: TC_eCPRI_LOGGING_AUDITING

Purpose: The purpose of this test is to validate that the DUT logs relevant security events and activities on the eCPRI interface and supports auditing capabilities.

Procedure and execution steps

Preconditions

- eCPRI interface is accessible.
- Logging and auditing mechanisms are enabled and configured.

Execution steps

1) Perform various eCPRI security events. Key eCPRI security events include [26]:

- Total count of received messages (valid and erroneous) for user and control planes
- Messages received within the expected time window
- Early and late arrivals of messages

- Sequence ID errors for on-time messages
- Messages dropped due to protocol violations or corruption
- Timing and sequence errors in control plane messages
- Total outbound messages for user and control planes
- Messages discarded due to errors, resource constraints, or policy violations
- Identified duplicated packets.

2) Verify that the DUT generates appropriate log entries for each security event, capturing relevant security-related information.

3) Access and review the generated logs to ensure they contain the necessary details for security auditing purposes.

Expected Results: The DUT generates accurate logs, recording security-related events and activities for auditing and forensic analysis.

Expected format of evidence

- The generated log files containing recorded security events and activities during the testing process.
- Screenshots of log entries highlighting relevant security events and timestamps.

6.10.6 Void

6.11 SCTP

6.11.1 Void

6.11.2 Void

6.11.3 Void

6.11.4 Void

6.11.5 SCTP DoS Prevention Rate Limiting

Requirement Name: SCTP security

Requirement Reference: REQ-SEC-TRAN-1, clause 5.3.4.1 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Ability to handle unexpected input

Threat References: T-E2-01, T-E2-02, T-E2-03

DUT/s: Near-RT RIC, O-CU-CP, O-CU-UP, O-DU

Test Name: TC_SCTP_ DOS_PREVENTION_RATE_LIMITING

Purpose: The purpose of this test is to verify that the SCTP protocol effectively handles DoS attacks and prevents resource exhaustion.

Procedure and execution steps

Preconditions

- Enable DoS prevention mechanisms.
- The rate limiting parameters, such as the maximum number of connections or allowed data transfer rate, are properly defined.
- Use SCTP library.

EXAMPLE 1: the sctplib library in the C programming language

Execution steps

1) Simulate a DoS attack by overwhelming the SCTP protocol with a large number of connection requests (send data at a rate that exceeds the defined rate limiting parameters).

EXAMPLE 2: Sample SCTP commands:

for (int i = 0; i < num_connections; i++) {sctp_socket = sctp_socket(AF_INET, SOCK_STREAM, IPPROTO_SCTP);
// Establish connections rapidly beyond system limits
}

2) Monitor the SCTP protocol's response and behaviour during the excessive connection and data transfer attempts.

Expected Results

- The SCTP protocol detects the excessive usage and applies rate limiting measures to restrict or reject connections or data transfers that exceed the defined limits.
- The system handles the rate limiting effectively, ensuring that resources are not exhausted or overwhelmed.

Expected format of evidence

- Test logs showing successful handling of the DoS attack, such as connection limits or rejection messages.
- System performance metrics or logs indicating the proper handling of excessive connection requests.

6.11.6 SCTP Input Validation

Requirement Name: SCTP security

Requirement Reference: REQ-SEC-TRAN-1, clause 5.3.4.1 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Ability to handle unexpected input

Threat References: T-E2-01, T-E2-02, T-E2-03

DUT/s: Near-RT RIC, O-CU-CP, O-CU-UP, O-DU

Test Name: TC_SCTP_INPUT_VALIDATION

Purpose: To verify that the SCTP protocol performs proper input validation to prevent security vulnerabilities such as buffer overflows or injection attacks.

Procedure and execution steps

Preconditions

- The SCTP protocol is configured with input validation enabled.
- Use SCTP library.

EXAMPLE 1: the sctplib library in the C programming language

Execution steps

1) Attempt to establish a connection using the SCTP protocol and provide invalid or malicious input.

EXAMPLE 2: Sample SCTP command: sctp_socket = sctp_socket(AF_INET, SOCK_STREAM, IPPROTO_SCTP);

2) Send data containing invalid or malicious content over the connection.

EXAMPLE 3: Sample SCTP command: sctp_sendmsg(sctp_socket, malicious_data_buffer, data_length, NULL, 0, 0, 0, stream_id, 0, 0);

Expected Results

- The SCTP protocol performs input validation and rejects or sanitizes the invalid or malicious input.
- The connection is not established, or the malicious data is handled safely.

Expected format of evidence

- Test logs showing the rejection or sanitization of invalid or malicious input.
- Output from the application indicating the successful validation and rejection of malicious data.

6.11.7 Void

6.11.8 Void

6.12 Transactional APIs

6.12.1 Transactional API Authentication – service producer role

Requirement Name: RESTful API protection

Requirement Reference: REQ-SEC-O-CLOUD-NotifAPI-1, REQ-SEC-O-CLOUD-NotifAPI-2, clause 5.1.8.9.1.3, REQ-SEC-API-1, REQ-SEC-API-2, REQ-SEC-API-3, REQ-SEC-API-4, REQ-SEC-API-5, REQ-SEC-API-6, REQ-SEC-API-8, REQ-SEC-API-9, REQ-SEC-API-10, REQ-SEC-API-13, REQ-SEC-API-15, clause 5.3.10.2 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: API robustness

Threat References: T-O-RAN-01, T-O-RAN-02, T-O-RAN-03, T-O-RAN-05, T-O-RAN-06

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_REST_API_AUTHENTICATION

Purpose: The purpose of this test is to verify the authentication mechanism of an O-RAN NF supporting RESTful API, service producer role.

Procedure and execution steps

Preconditions

- An O-RAN NF supporting the RESTful API is provisioned and running.
- Access to the O-RAN NF management system or command-line interface.

Execution steps

1) Positive Case:

- Authenticate using valid credentials or API tokens:

EXAMPLE 1: **curl -X POST -H "Content-Type: application/json" -d '{"username":"<username>", "password":"<password>"}' http://<ORAN_IP>/auth**

- Capture the authentication token from the response.
- Execute an authenticated request against a RESTful API present in an O-RAN NF acting as a service producer, using a signed token (e.g., get cell status).
- Verify that the request is successful and returns the expected response.

2) Negative Case:

- Attempt to access the RESTful API present in an O-RAN NF acting as a service producer, without providing valid authentication credentials:

EXAMPLE 2: curl http://<ORAN_IP>/cell-status

- Verify that the request fails and returns an unauthorized response.

3) Negative Case:

- Attempt to access the RESTful API present in an ORAN NF acting as service producer, using unsigned token (the “alg” field in header section of the JWT token should be set to “none" and empty string for signature value).
- Verify that the request fails and returns failure from API end point.

Expected Results

1) Positive Case:

- Authentication using valid credentials or API tokens is successful.
- Authorized requests to O-RAN NF resources return the expected responses.

2) Negative Case:

- Requests without valid authentication credentials are rejected with an unauthorized response.

3) Negative Case:

- Requests with token which are not signed are rejected.

Expected format of evidence

- Screenshots or logs showing the successful authentication and authorized requests.
- Screenshots or logs showing the failed authentication attempts.

6.12.2 Transactional API Authorization and Access Control

Requirement Name: RESTful API protection

Requirement Reference: REQ-SEC-O-CLOUD-NotifAPI-1, REQ-SEC-O-CLOUD-NotifAPI-2, clause 5.1.8.9.1.3, REQ-SEC-API-1, REQ-SEC-API-2, REQ-SEC-API-3, REQ-SEC-API-4, REQ-SEC-API-5, REQ-SEC-API-6, REQ-SEC-API-8, REQ-SEC-API-9, REQ-SEC-API-10, REQ-SEC-API-13, REQ-SEC-API-15, clause 5.3.10.2 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: API robustness

Threat References: T-O-RAN-01, T-O-RAN-02, T-O-RAN-03, T-O-RAN-05, T-O-RAN-06

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_REST_AUTHORIZATION_ACCESS_CONTROL

Purpose: The purpose of this test is to ensure that the RESTful API enforces proper authorization and access control mechanisms.

Procedure and execution steps

Preconditions

- An O-RAN NF supporting the RESTful API is provisioned and running.
- Access to the O-RAN NF management system or command-line interface.
- User roles and permissions are defined and configured.

Execution steps

1) Positive Case:

- Authenticate using credentials associated with a user assigned to a role with necessary permissions:

EXAMPLE 1: **curl -X POST -H "Content-Type: application/json" -d '{"username":"<username>", "password":"<password>"}' http://<ORAN_IP>/auth**

- Capture the authentication token from the response.
- Execute a request that requires the permissions granted by the user's role (e.g., update configuration).
- Verify that the request is successful and returns the expected response.

2) Negative Case:

- Authenticate using credentials associated with a user not assigned to a role with necessary permissions:

EXAMPLE 2: **curl -X POST -H "Content-Type: application/json" -d '{"username":"<username>", "password":"<password>"}' http://<ORAN_IP>/auth**

- Capture the authentication token from the response.
- Execute a request that requires the permissions beyond the user's role (e.g., perform a restricted operation).
- Verify that the request fails and returns a forbidden response.

Expected Results

1) Positive Case:

- Users with appropriate roles and permissions can perform authorized actions.
- Requests requiring specific permissions return the expected responses.

2) Negative Case:

- Users without necessary roles or permissions are restricted from performing unauthorized actions.
- Requests requiring permissions beyond the user's role return a forbidden response.

Expected format of evidence

- Screenshots or logs showing the successful authorization and access control enforcement.
- Screenshots or logs showing the failed authorization attempts.

6.12.3 Transactional API Input Validation and Sanitization

Requirement Name: RESTful API protection

Requirement Reference: REQ-SEC-API-12, REQ-SEC-API-13, REQ-SEC-API-15, clause 5.3.10.2 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: API robustness

Threat References: T-O-RAN-04

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_REST_INPUT_VALIDATION_SANITIZATION

Purpose: The purpose of this test is to validate that the RESTful API properly validates and sanitizes input data to prevent common security vulnerabilities.

Procedure and execution steps

Preconditions

- An O-RAN NF supporting the RESTful API is provisioned and running.
- Access to the O-RAN NF management system or command-line interface.

Execution steps

1) Positive Case:

- Construct a valid request with appropriate input data:

EXAMPLE 1: The input parameters can be within the valid range for the parameters supported for the API. This includes the supported (valid) HTTP method(s) for the API.
curl -X POST -H "Content-Type: application/json" -d '{"parameter1":"value1", "parameter2":"value2"}' http://<ORAN_IP>/api-endpoint

- Verify that the request is successful and returns the expected response.

2) Negative Case (1):

- Construct a request with invalid input data:

EXAMPLE 2: The input parameters can be outside the valid range for the parameters supported for the API.
curl -X POST -H "Content-Type: application/json" -d '{"parameter1":"<script>alert(1)</script>", "parameter2":"value2"}' http://<ORAN_IP>/api-endpoint

- Verify that the request fails and returns an error response.
- Verify that log is generated for the input validation errors.

3) Negative Case (2):

- Construct a request with invalid/not supported HTTP method:

EXAMPLE 3: HTTP method being invoked is invalid, i.e. not a supported method for the API.

- Verify that the request fails and returns an error response.
- Verify that log is generated indicating HTTP method is not supported.

Expected Results

1) Positive Case:

- Requests with valid and appropriate input data are successfully processed.

- Responses from the O-RAN NF RESTful API are as expected.

2) Negative Case (1):

- Requests with invalid input data are rejected and handled properly to prevent security vulnerabilities.
- Log generated for the invalid inputs after unsuccessful validation.

3) Negative Case (2):

- Requests with invalid or unsupported HTTP methods are rejected.
- Log generated for the invalid HTTP method.

Expected format of evidence

- Screenshots or logs showing the successful input validation and sanitization.
- Screenshots and logs showing failed input validation or sanitization attempts.

6.12.4 Transactional API Security Logging and Monitoring

Requirement Name: RESTful API protection

Requirement Reference: REQ-SEC-O-CLOUD-NotifAPI-1, REQ-SEC-O-CLOUD-NotifAPI-2, clause 5.1.8.9.1.3, REQ-SEC-API-1, REQ-SEC-API-2, REQ-SEC-API-3, REQ-SEC-API-4, REQ-SEC-API-5, REQ-SEC-API-6, REQ-SEC-API-8, REQ-SEC-API-9, REQ-SEC-API-10, REQ-SEC-API-13, REQ-SEC-API-15, clause 5.3.10.2 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: API robustness

Threat References: T-O-RAN-01, T-O-RAN-02, T-O-RAN-03, T-O-RAN-05, T-O-RAN-06

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_REST_SECURITY_LOGGING_MONITORING

Purpose: The purpose of this test is to verify that the O-RAN NF logs and monitors API activities for security and compliance purposes.

Procedure and execution steps

Preconditions

- An O-RAN NF supporting the RESTful API is provisioned and running.
- Access to the O-RAN NF management system or command-line interface.

Execution steps

1) Positive Case:

- Enable API logging and monitoring for the O-RAN NF.
- Generate a series of API requests and actions.
- Review the logs or monitoring system for the recorded activities.

2) Negative Case:

- Attempt unauthorized API actions or exploit security vulnerabilities.
- Verify that the logs or monitoring system captures and raises alerts for these activities.

Expected Results

1) Positive Case:

- API activities are logged and monitored by the O-RAN NF.
- Logs or monitoring system records the expected API requests and actions.

2) Negative Case:

- Unauthorized or malicious API actions trigger alerts in the logs or monitoring system.
- Logs or monitoring system captures and records failed security attempts.

Expected format of evidence

Logs from the O-RAN NF management system showing the successful or failed API logging and monitoring settings. In case the logs do not show the required information, screenshots are used.

6.13 MACsec

Requirement Name: MACsec Cipher Suite Support Validation

Requirement Reference: SEC-CTL-OFHMECC-5, SEC-CTL-OFHWMM-2, SEC-CTL-OFHLMM-3, clause 5.2.5.6 in O-RAN Security Requirements and Controls Specifications **Error! Reference source not found.**

Requirement Description: Open FH elements supporting MACsec—whether in EDE-CC WAN, or LAN mode—implement the cipher suites specified in clause 4.9 of the O-RAN Security Protocols Specification **Error! Reference source not found.**

NOTE 1: This test case is conditionally applicable only if MACsec is supported by the DUT.

NOTE 2: The cipher suite GCM-AES-128 is mandatory according to IEEE 802.1AE-2018 [32]. Other cipher suites, such as GCM-AES-256, are optional. If an optional cipher suite is supported by the DUT, this test case applies for each supported cipher.

Threat References: T-O-RAN-08

DUT/s: O-RU, O-DU

Test Name: TC_MACsec_Cipher_Suite_Support

Purpose: To verify that the DUT supports and correctly negotiates the required MACsec cipher suite(s) in all supported MACsec modes: EDE-CC, WAN and LAN.

Procedure and Execution Steps

Preconditions

- The DUT supports MACsec (IEEE 802.1AE) and operates in one of the following modes: EDE-CC, WAN, or LAN.
- The test setup includes a MACsec-capable test tool.
- The cipher suite under test, at minimum GCM-AES-128, is supported by both the DUT and the test tool.

Execution steps

For each MACsec mode supported by the DUT (EDE-CC, WAN, LAN):

- Configure the DUT to operate in the target MACsec mode.
- Initiate MACsec handshake using a supported key agreement protocol.
- For each supported cipher suite (starting with GCM-AES-128):

- Negotiate a specific cipher suite.
- Transmit traffic over the MACsec-secured interface.
- Verify that the secure channel is established using the selected cipher.

Expected Results

For each tested MACsec mode and cipher suite:

- The DUT successfully negotiates the selected cipher suite.
- MACsec encryption is correctly applied.

Expected Format of Evidence: Logs from the testing tool for each tested mode and cipher suite, showing the accepted cipher suite, and confirming that no captured packets are in plaintext.

7 Common Network Security Tests for O-RAN architecture elements

7.1 Overview

This clause contains a set of security evaluations that are performed from outside and inside of the network function in a network capacity. It is used to measure the external exposure and risk(s) of the function in place and leverages common techniques used in cyber security to evaluate the risk(s) device under test faces or has.

The objects in scope of these network-based security tests are SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU and O-Cloud.

7.2 Network Protocol and Service Enumeration

7.2.1 Network Protocol and Service Enumeration

Requirement Name: Network protocol and service enumeration

Requirement Reference: REQ-SEC-NET-1, clause 5.3.3.1, O-RAN Security Requirements and Controls Specifications [6]

Requirement Description: Protocols and services are documented by vendor. No undocumented protocols and services are offered.

Threat References: T-O-RAN-01, T-O-RAN-02

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test name: TC_Network_Procotol_And_Enumeration

Purpose: To verify that the list of active network protocols and services on DUT is in line with vendor-provided list of network protocols and services supported. Probing of network protocols and services on DUT provides the information whether the service is active or not. This test case probes all possible TCP and SCTP ports in range 0-65535 using port scanner for presence of the active services.

This test case probes all documented UDP ports from vendor-provided list using port scanner for presence of the active services. Optionally, additional UDP ports may be scanned as well.

NOTE 1: In practice, such probing is often referred to as network scanning or port scanning.

NOTE 2: In practice, services may also run on ports different from ports defined in [i.1] .

Procedure and execution steps

Preconditions

- Port scanner with capabilities as defined in clause 5.3 of present document.
- Network access to DUT
- Vendor-provided list of network protocols and services supported by DUT

Execution steps

1) List of open ports is determined as follows:

- Port scanner scans all TCP ports in range 0-65535 on the IP interface of DUT. TCP SYN/ACK response by DUT are interpreted as open port.

- Port scanner scans all SCTP ports in range 0-65535 on the IP interface of DUT. SCTP INIT-ACK response by DUT are interpreted as open port.
- All UDP ports documented in vendor-provided list are interpreted as open ports. Other UDP ports may be considered as open for the purpose of service detection.

NOTE 3: Due to the nature of UDP protocol, there is no simple method of open port detection similar to TCP/SCTP methods based on analysis of response message type (TCP: SYN/ACK, SCTP: INIT-ACK). In case of UDP, open port detection inevitably relies on service detection which is discussed in step 2 of this test procedure. In practice, port scans of entire UDP port range 0-65535 are impractical and time consuming. Typically, service detection is performed only for subset of UDP ports. UDP port subset selection is arbitrary and not standardized. Service detection in this test procedure is required for UDP ports from vendor-provided list and is optional for other UDP ports.

2) For each open port from previous step, port scanner performs service detection by sending service probe(s) as follows:

- If open port is listed in vendor-provided list, port scanner uses service probe from its built-in database that exactly matches service documented in vendor-provided list.
- If open port is not listed in vendor-provided list, port scanner uses service probe from its built-in database that exactly matches service defined in [i.1] for the that open port. If such service is not defined in [i.1] , port scanner may report service as "unknown". Alternatively, port scanner may perform further service detection attempts based on other service probes from its built-in database.

NOTE 4: Service detection for open ports that are also listed in vendor-provided list requires only one probe. However, service information can be helpful in discussion with DUT vendor. This test procedure therefore accommodates optional service detection based on one probe or multiple probes.

3) Port scanner produces list of detected active network protocols, ports and services on DUT.

Expected results

Comparison between the vendor-provided list of all supported network protocols and services and the list or active network protocols and services found by port scanner is performed.

All services found by port scanner are documented in vendor-provided list. This test case ends with success if:

- both lists match exactly
- list of network protocols and services found by port scanner has fewer items than vendor-provided list; all items found by port scanner exactly match items from vendor-provided list.

If any service is found by port scanner and it is not documented in vendor-provided list, this test case shall fail. It means that vendor-provided list is incorrect and undocumented attack surface exists.

Expected format of evidence: Result of probing the DUT is a list of active network protocols and services. Each item contains network protocol (TCP, UDP, SCTP), port number (from range 0-65535) and service name. If service type cannot be determined during probing, service name is "unknown".

Service name is in line with Service Name and Transport Protocol Port Number Registry defined by IANA [i.1] . If service name is not defined in [i.1] , vendor provided service name is used.

7.3 Password-Based Authentication

7.3.1 Password guessing

Requirement Name: Password-Based Authentication

Requirement Reference: SEC-CTL-PASS-2, clause 5.5.7.2, REQ-SEC-PASS-1, clause 5.3.7.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Password guessing protection mechanism is present on the DUT

Threat References: T-O-RAN-02, T-O-RAN-05, T-O-RAN-06

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_Password_Guessing

Purpose: To verify that DUT has protection mechanism(s) implemented to prevent password guessing attacks against services using password-based authentication. . Simulation of password guessing attacks against services on DUT provides the information whether any protection mechanism is present.

This test case is run against all services on DUT that use password-based authentication. Vendor-provided list of all supported network protocols and services are used as a source.

NOTE 1: Vendor-provided list of all supported network protocols and services may not include the specific information about presence of password-based authentication as it is including network protocol, port and service name. In practice, only subset of services from vendor-provided list will use password-based authentication.

Procedure and execution steps

Preconditions

- Valid username for each tested service. This test case does not mandate any specific list of passwords to be used for testing.
- Network access to DUT
- Physical access to DUT (applicable if the DUT is in physical form)
- Vendor-provided list of network protocols and services supported by DUT
- Number of authentication attempts configured in account lock-out policy

Execution steps

1) List of services using password-based authentication is determined by analysing the vendor-provided list as well as by analysing local services that are not remotely accessible.

2) For services identified in the previous step, presence of protection mechanism is tested as follows:

- combination of valid username and invalid password (or various invalid passwords) are used for authentication repeatedly.
- after certain number of authentication attempts, protection mechanism of DUT are detected.
- minimum number of authentication attempts is the calculated as (number configured in the lock-out policy + 1) attempts.

Expected results

In context of each of the services using password-based authentication, protection mechanism(s) is present. Applicable to local services and to remotely accessible services.

NOTE 2: In practice, brute-forcing and dictionary attacks are the most common classes of password guessing attacks. Traditional approach to brute-forcing and dictionary attacks uses fixed username with various candidate passwords. Password spraying is another approach that can be combined with brute-forcing and dictionary attacks; fixed password is tested with various candidate usernames. Example of protection mechanism is enforcing delay before next authentication attempt(s) by the same client. This test case cannot list all possible techniques that protection mechanisms can use. However, following list provides overview of the most common approaches:

- Increase the delay after each unsuccessful authentication attempt.

- Implement challenge-response authentication (example of such measure: CAPTCHA)
- In order to prevent more attempts, impose temporary lock out on the client when threshold of consecutive failed authentication attempts is reached. During defined period of time all authentication attempts by locked-out client are rejected.

EXAMPLE: Lock-out policy is configured for 10 invalid authentication attempts. If DUT uses protection mechanism based on delaying authentication attempts, such delay is observed when DUT receives 11th consecutive invalid authentication attempt.

This test case fails if one or more services using password-based authentication have no protection mechanism present.

Expected format of evidence: Report file, log files and/or screenshots.

7.3.2 Unauthorized Password Reset

Requirement Name: Password-Based Authentication

Requirement Reference: REQ-SEC-PASS-1, clause 5.3.7.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Out-of-band password recovery mechanism absent or deactivated on DUT

Threat References: T-O-RAN-02, T-O-RAN-05, T-O-RAN-06

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_Unauthorized_Password_Reset

Purpose: To verify that password reset mechanism DUT cannot be circumvented, disabled, or misused to gain access to DUT, its configuration, and data. Test covers services using password-based authentication and out-of-band mechanisms of password reset present in DUT in physical form.

Procedure and execution steps

Preconditions

- Network access to DUT
- Physical access to DUT (applicable if the DUT is in physical form)
- Vendor-provided list of network protocols and services supported by DUT

Execution steps

List of services using password-based authentication are determined by analysing the vendor-provided list as well as by analysing local services that are not remotely accessible.

1) For services identified in the previous step, presence of password reset is tested.
2) For DUT that has physical form, it verifies that use of hardware factory reset switch or switches results in factory reset. Using any out-of-band mechanism, it is not possible to reset password only. If password reset is required, factory reset of O-RAN architecture element is performed. Factory reset wipes O-RAN architecture element, its configuration and data.

Expected results

In context of each of the services using password-based authentication, no password change mechanism is present. Applicable to local services and to remotely accessible services.

This test case fails if one or more services using password-based authentication have password reset mechanism exposed.

This test case fails if DUT in physical form has hardware switch or switches that can be used to reset password without triggering factory reset of DUT.

Expected format of evidence: Report file, log files and/or screenshots.

7.3.3 Password Policy Enforcement

Requirement Name: Password-Based Authentication

Requirement Reference: REQ-SEC-PASS-1, clause 5.3.7.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Secure password policy is supported and enforced on the DUT

Threat References: T-O-RAN-02, T-O-RAN-03, T-O-RAN-05, T-O-RAN-06

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_Password_Policy_Enforcement

Purpose: To verify that password policy applied for services using password-based authentication is effectively enforced by DUT.

Procedure and execution steps

Preconditions

- Set of valid username and valid password for each tested service
- Network access to DUT
- Physical access to DUT (applicable if the DUT is in physical form)
- Vendor-provided list of network protocols and services supported by DUT
- Password policy that is actually applied on the DUT

Execution steps

1) List of services using password-based authentication is determined by analyzing the vendor-provided list as well as by analyzing local services that are not remotely accessible.

2) For services identified in the previous step, effectiveness of password policy enforcement is verified as follows:

- combination of valid username and valid password are used to authenticate
- password change is performed using password that does not conform to applied password policy

EXAMPLE: DUT uses password policy to set rules for password length, type of characters used (allowed and disallowed characters), complexity (character groups), and denied passwords (deny-list of passwords that cannot be set). Candidate password that does not conform to rules are chosen for this test. As password policy may be complex set of rules, multiple candidate passwords are tested to fully cover possible password policy violations.

Expected results

In context of each of the services using password-based authentication, applied password policy are effectively enforced and non-compliant passwords are rejected by DUT during password change. Applicable to local services and to remotely accessible services.

Expected format of evidence: Report file, log files and/or screenshots.

7.4 Network Protocol Fuzzing

Fuzzing is an automated process of sending invalid or random inputs to a SUT to cause it to malfunction or crash.

Fuzzing is effective for finding vulnerabilities because while most modern programs have extensive input fields, the test coverage of these areas is relatively small. Even though this process can be a powerful capability to ensure robustness, it needs to be sufficiently defined and implemented throughout the system development lifecycle to be helpful and achieve the required results in a multi-vendor environment.

While traditional fuzzing techniques involve fuzzing piece(s) of software and generating inputs through command line or input files, fuzzing telecommunication network protocols tends to be different, requiring sending information via network ports. Furthermore, the complex nature of network protocols in the SUT resulting from how they are layered over each other adds to the challenges of fuzzing such SUTs.

The following are examples of the protocols that fuzzing will cover;

General Transport Protocols
SCTP
IP
TCP
UDP
SSH
HTTP
HTTP/2

and

O-RAN Specific Protocols
NETCONF
E1AP
E2AP
A1
CTI
eCPRI
PTP

It is anticipated that many O-RAN architecture elements utilize common software frameworks used for the lower-level general communication. In this case it should be evaluated if these General Transport Protocols are being tested in extensive Fuzzing tests in other activities and can therefore be considered to have lower risk profiles compared to the O-RAN Specific Protocols with less testing in the general industry.

Many of the O-RAN specific protocols are state- machine based protocols that can have multiple end points served at the same time, e.g. the protocol needs to be tested in scale to understand if possible memory leaks or other similar aspects is available that could lead to buffer overflows (opening up for possible code execution) or software crashes of the O-RAN specific software.

Fuzzing on the M-Plane protocol inside the Configuration of the O-RAN Fronthaul can be a possible significant area as this is combining multiple technologies from many domains into a single solution. In order for the Fuzzing to be time and resource efficient, it is important that this Fuzzing is protocol and state machine aware so that the Fuzzing can focus on the relevant aspects of the SUT representing the most significant risk exposure. Further effectiveness can be achieved if the Fuzzing capability is able to intelligently respond to the SUT behaviour. The Fuzzing tool should be able to both perform test with and without access to relevant credentials. Many possible vulnerabilities would be present on the inside of the authenticated session of the management protocols and would lead to escalation of privileges.

In order to identify the possible risk for memory leaks, Denial of Service (DoS) or other similar aspects a robust logging of the underlying platform (hardware and software), the virtualization or container platform and the O-RAN function, the logging needs to be detailed enough to evaluate the trends early but not intrusive to degrade the performance of the platform and lead to inaccurate results.

As general guidance, vendors and operators running fuzzing tests aim to document the list of all of the protocols of the SUTs reachable externally on an IP-based interface, together with indications of whether adequate available robustness and fuzz testing tools have been used against them. The tool's name, their unambiguous version (also for plug-ins if applicable), user settings, and the relevant output evidenced and should be documented. Additionally, any input causing unspecified, undocumented, or unexpected behaviour and a description of this behaviour should be highlighted in the testing documentation.

Since fuzzing test cases are not exhaustive and difficult to define and replicate, it's likely that test results even from testing the same set of protocols by different vendors may end up resulting in different outputs. So further effort and time needs to be invested in fuzzing activities until a satisfactory approach based on the vendor's or/and operators adopted risk-based model is satisfied.

7.5 Denial of Service/Message Flooding

7.5.1 Protocol, Application and Volumetric Based DDoS Attacks

Requirement Name: Robustness against DoS/Volumetric DDoS Attacks

Requirement Reference: REQ-SEC-DOS-1, clause 5.3.5.1, REQ-SEC-NEAR-RT-6, REQ-SEC-NEAR-RT-7, REQ-SEC-NEAR-RT-8, REQ-SEC-NEAR-RT-9, clause 5.1.3.1, REQ-SEC-NonRTRIC-4, REQ-SEC-NonRTRIC-5, REQ-SEC-NonRTRIC-6, clause 5.1.2.1, REQ-SEC-SMO-5, REQ-SEC-SMO-6, REQ-SEC-SMO-7, 5.1.1.1.1, REQ-SEC-SharedORU-7, clause 5.1.9.1, REQ-SEC-NFO-FOCOM-6, REQ-SEC-NFO-FOCOM-7, clause 5.1.1.1.5, O-RAN Security Requirements and Controls Specifications [6]

Requirement Description: An O-RAN architecture element with a network interface has ability to withstand network transport protocol based volumetric DoS/DDoS attacks without system crash and return to normal service level after the attack

Threat References: T-O-RAN-03, T-O-RAN-04, T-O-RAN-09, T-SMO-03

DUT/s: Architecture elements/applications implementing O-RAN interfaces defined in clause 5.1 of present document.

Test Name: TC_Robustness_DDoS

Purpose: To evaluate the resilience of the DUT against DoS/DDoS attacks and its recovery capabilities.

Each DUT interface is tested to validate its handling of large numbers of requests, similar to what is seen from denial of service (DoS) or/and distributed denial of service (DDoS) attempts. DoS/DDoS scenario can occur as a result of malicious attack or because of network/operator error. DoS/DDoS attacks can be in these forms: protocol layer attacks (e.g. SYN Floods, UDP Floods, TCP Floods), volume based attacks (e.g. ICMP floods, Smurf DDoS) and application layer attacks (e.g. GET/POST floods, low-and-slow attacks, attacks that target specific software/application with exposed network services or operating system network services).

Verify that the O-RAN Architecture element/Application:

- - has a detailed technical description of the overload control mechanisms used to deal with overload scenarios.
- - has a test report verifying the operation of the overload control mechanisms.

Procedure and execution steps

Preconditions

- A document that provides a detailed technical description of the overload control mechanisms.
- A test report from the test execution phase of overload control mechanism testing.

Execution steps

1) The tester verifies that the technical description document contains:

- Various overload scenarios that the DUT's (or the O-RAN Architecture element/Application) are expected to handle. This includes for example:
  - Traffic overload
  - Resource overload
  - Service overload
- Various overload control thresholds that the DUT uses to trigger overload control mechanisms. This includes for example:
  - CPU usage limit

- Memory usage limit
- Request rate limit

- Description of the types of attacks that can cause overload conditions. This includes for example:
  - Denial of Service (DoS)
  - Distributed Denial of Service (DDoS)
  - Resource Exhaustion
- A description of how the DUT's security functions operate and perform corrective actions under excessive overload conditions. i.e. when the overload is significantly greater than the defined overload thresholds.

2) The tester verifies that a detailed test report:

- Contain details of the test setup including the mechanisms for creating the overload scenarios that are consistent with the technical description document, where simulators and/or scripts are used to artificially create a load, then details of these should also be included.
- Describe test procedures used to verify the overload control mechanisms.
- Contain logs capturing all relevant events, which demonstrate/indicate that the detection of overload conditions and the triggering of overload control mechanisms described in the technical description document have been implemented.

Expected results

The technical description document and test report contain:

- An overview of the types of overload scenarios that are considered.
- An overview of the determined specific thresholds that triggered overload control mechanisms.
- A description of the types of attacks that cause overload conditions to the system and how these are handled.
- A description of how the DUT demonstrated the ability to discard excess requests gracefully while continuing to process valid request during overload situations.
- A description of how the DUT security functions remain operational and perform corrective actions under excessive overload situations. This includes system shutdowns, abatements and other corrective actions

NOTE: The vendor provides an explanation for any of the items listed above that they consider not applicable to their architecture element/application.

Expected format of evidence: Documentation showing each of the points in the results sections.

7.5.2 Void

7.5.3 Void

7.5.4 Void

7.5.5 Near-RT RIC A1 interface DoS/DDoS protection and recovery

Refer to test case 7.5.1 for the test verification

7.6 Input validation and error handling

7.6.1 O-CU input validation and error handling

Requirement Name: Input validation and error handling on data provided through E2 and O1 interfaces.

Requirement Reference: REQ-SEC-OCU-1, clause 5.1.4.1 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: O-CU performs input validation on data provided via E2 and O1 interfaces and rejects invalid or malicious inputs.

Threat References: T-O-RAN-05

DUT/s: O-CU

Test Name: TC_INPUT_VALIDATION_ERR_HANDL_OCU

Purpose: The purpose of this test is to verify that the O-CU performs proper input validation on provided data via E2 and O1 interfaces and rejects invalid or malicious inputs. It verifies that the O-CU correctly handles errors and responds appropriately.

Procedure and execution steps

Preconditions:

- The O-CU is powered on and operational.
- Test environment is set up with E2 and O1 interfaces configured.
- Input validation mechanisms are implemented on O-CU.
- Error handling mechanisms (e.g., error codes, error messages) are implemented by O-CU.

Execution steps:

1) Case of malformed input data

- The tester provides invalid or malformed input data to the O-CU via E2 and O1 interfaces, violating the specified format or containing unexpected values.
- The tester captures and analyses the response from the E2 and O1 interfaces.
- The tester verifies that the O-CU detects the invalid input and rejects it appropriately, returning an error message or taking necessary actions to mitigate the impact.

EXAMPLE: Actions could be rejecting the message, sending an error indication, etc.

2) Case of malicious input data

- The tester provides malicious input data to the O-CU via E2 and O1 interfaces, aiming to exploit known vulnerabilities (e.g., CVE database, OWASP Top Ten, NIST National Vulnerability Database (NVD), vendor-specific vulnerability database) . If exploitation is successful, the tester performs unauthorized actions.
- The tester verifies that the O-CU identifies the malicious input and implements security measures to prevent exploitation, such as input sanitization, access controls, or anomaly detection.

3) Boundary case

- Provide input data at the boundaries of the allowed range or limits defined for specific inputs.
- Verify that the O-CU handles the boundary cases correctly, without encountering any unexpected behaviour or errors due to boundary conditions.

Expected Results:

For case 'malformed input data;, the O-CU properly validates incoming inputs from E2 and O1 interfaces and rejects those with invalid or malformed data, returning an appropriate error response and preventing any potential security risks or system failures.

For case 'malicious input data', the O-CU detects and mitigates the malicious input, preventing any potential security breaches or unauthorized operations.

For case 'boundary', the O-CU properly handles the boundary cases, ensuring that inputs at the limits are processed accurately without causing any system instability or vulnerabilities.

Expected format of evidence:

Logs detailing the invalid or malformed input data provided to the O-CU via E2 and O1 interfaces, alongside system logs capturing the O-CU's error messages or indications in response to the invalid input.

Logs documenting the malicious input data sent to the O-CU and the targeted vulnerabilities, complemented by system logs highlighting the O-CU 's detection and mitigation actions upon receiving the malicious input.

Logs of the boundary input data values provided to the O-CU, paired with system logs capturing the O-CU's messages or behaviours in response to the boundary inputs.

7.6.2 O-DU input validation and error handling

Requirement Name: Input validation and error handling on data provided through E2, Open FH and O1 interfaces.

Requirement Reference: REQ-SEC-ODU-1, clause 5.1.5.1 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: O-DU performs input validation on data provided via E2, Open FH and O1 interfaces and rejects invalid or malicious inputs.

Threat References: T-O-RAN-05

DUT/s: O-DU

Test Name: TC_INPUT_VALIDATION_ERR_HANDL_ODU

Purpose: The purpose of this test is to verify that the O-DU performs proper input validation on provided data via E2, Open FH and O1 interfaces and rejects invalid or malicious inputs. It verifies that the O-DU correctly handles errors and responds appropriately.

Procedure and execution steps

Preconditions:

- The O-DU is powered on and operational.
- Test environment is set up with E2, Open FH and O1 interfaces configured.
- Input validation mechanisms are implemented on O-DU.
- Error handling mechanisms (e.g., error codes, error messages) are implemented by O-DU.

Execution steps:

1) Case of malformed input data

- The tester provides invalid or malformed input data to the O-DU via E2, Open FH and O1 interfaces, violating the specified format or containing unexpected values.
- The tester captures and analyses the response from the E2, Open FH and O1 interfaces.

- The tester verifies that the O-DU detects the invalid input and rejects it appropriately, returning an error message or taking necessary actions to mitigate the impact.

EXAMPLE: Actions could be rejecting the message, sending an error indication, etc.

2) Case of malicious input data

- The tester provides malicious input data to the O-DU via E2, Open FH and O1 interfaces, aiming to exploit known vulnerabilities (e.g., CVE database, OWASP Top Ten, NIST National Vulnerability Database (NVD), vendor-specific vulnerability database). If exploitation is successful, the tester performs unauthorized actions.
- The tester verifies that the O-DU identifies the malicious input and implements security measures to prevent exploitation, such as input sanitization, access controls, or anomaly detection.

3) Boundary case

- Provide input data at the boundaries of the allowed range or limits defined for specific inputs.
- Verify that the O-DU handles the boundary cases correctly, without encountering any unexpected behaviour or errors due to boundary conditions.

Expected Results:

For case 'malformed input data', the O-DU properly validates incoming inputs form E2, Open FH and O1 interfaces and rejects those with invalid or malformed data, returning an appropriate error response and preventing any potential security risks or system failures.

For case 'malicious input data', the O-DU detects and mitigates the malicious input, preventing any potential security breaches or unauthorized operations.

For case 'boundary', the O-DU properly handles the boundary cases, ensuring that inputs at the limits are processed accurately without causing any system instability or vulnerabilities.

Expected format of evidence:

Logs detailing the invalid or malformed input data provided to the O-DU via E2, Open FH and O1 interfaces, alongside system logs capturing the O-DU's error messages or indications in response to the invalid input.

Logs documenting the malicious input data sent to the O-DU and the targeted vulnerabilities, complemented by system logs highlighting the O-DU's detection and mitigation actions upon receiving the malicious input.

Logs of the boundary input data values provided to the O-DU, paired with system logs capturing the O-DU's messages or behaviours in response to the boundary inputs.

7.6.3 Near-RT RIC input validation and error handling

Requirement Name: Error handling by Near-RT RIC

Requirement Reference: REQ-SEC-NEAR-RT-7, REQ-SEC-NEAR-RT-8, REQ-SEC-NEAR-RT-9, clause 5.1.3.1 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Near-RT RIC performs input validation on data provided via A1, E2 and Y1 interfaces and rejects invalid or malicious inputs.

Threat References: T-NEAR-RT-03, T-NEAR-RT-04

DUT/s: NEAR-RT RIC

Test Name: TC_INPUT_VALIDATION_ERR_HANDL_NEAR_RT_RIC

Purpose: The purpose of this test is to verify that the Near-RT RIC performs proper input validation on provided data via A1, E2 and Y1 interfaces and rejects invalid or malicious inputs. It verifies that the Near-RT RIC correctly handles errors and responds appropriately.

Procedure and execution steps

Preconditions

- Near-RT RIC is powered and operational.
- Test environment is set up with A1, E2 and Y1 interfaces configured.
- Input validation mechanisms are implemented on Near-RT RIC.
- Error handling mechanisms (e.g., error codes, error messages) are implemented by Near-RT RIC.

Execution steps:

1) Case of malformed input data

- The tester provides invalid or malformed input data to the Near-RT RIC via A1, E2 and Y1 interfaces, violating the specified format or containing unexpected values.
- The tester captures and analyses the response from the A1, E2 and Y1 interfaces.
- The tester verifies that the Near-RT RIC detects the invalid input and rejects it appropriately, returning an error message or taking necessary actions to mitigate the impact.

EXAMPLE: Actions could be rejecting the message, sending an error indication, etc.

2) Case of malicious input data

- The tester provides malicious input data to the Near-RT RIC via A1, E2 and Y1 interfaces, aiming to exploit known vulnerabilities (e.g., CVE database, OWASP Top Ten, NIST National Vulnerability Database (NVD), vendor-specific vulnerability database). If exploitation is successful, the tester performs unauthorized actions.
- The tester verifies that the Near-RT RIC identifies the malicious input and implements security measures to prevent exploitation, such as input sanitization, access controls, or anomaly detection.

3) Boundary case

- Provide input data at the boundaries of the allowed range or limits defined for specific inputs.
- Verify that the Near-RT RIC handles the boundary cases correctly, without encountering any unexpected behaviour or errors due to boundary conditions.

Expected Results

For case 'malformed input data', the Near-RT RIC properly validates incoming inputs from A1, E2 and Y1 interfaces and rejects those with invalid or malformed data, returning an appropriate error response and preventing any potential security risks or system failures.

For case 'malicious input data', the Near-RT RIC detects and mitigates the malicious input, preventing any potential security breaches or unauthorized operations.

For case 'boundary', the Near-RT RIC properly handles the boundary cases, ensuring that inputs at the limits are processed accurately without causing any system instability or vulnerabilities.

Expected format of evidence:

Logs detailing the invalid or malformed input data provided to the Near-RT RIC via A1, E2 and Y1 interfaces, alongside system logs capturing the Near-RT RIC's error messages or indications in response to the invalid input.

Logs documenting the malicious input data sent to the Near-RT RIC and the targeted vulnerabilities, complemented by system logs highlighting the Near-RT RIC 's detection and mitigation actions upon receiving the malicious input.

Logs of the boundary input data values provided to the Near-RT RIC, paired with system logs capturing the Near-RT RIC 's messages or behaviours in response to the boundary inputs.

7.6.4 Near-RT RIC input validation and error handling of data received from xApp

Requirement Name: Error handling by Near-RT RIC

Requirement Reference: SEC-CTL-NEAR-RT-18, clause 5.1.3.2.5 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The Near-RT RIC shall verify data received through E2 related APIs initiated by xApp as follows: the data values are valid; the Near-RT RIC shall log security event(s) if any of the verification steps fail.

Threat References: T-NEAR-RT-03, T-NEAR-RT-04

DUT/s: NEAR-RT RIC

Test Name: TC_E2API_INPUT_VALIDATION_ERR_HANDL_NEAR_RT_RIC_XAPP

Purpose: The purpose of this test is to verify that the Near-RT RIC performs proper input validation on received data from xApp through E2 related APIs and rejects malformed inputs. It verifies that the Near-RT RIC correctly handles errors and responds appropriately.

Procedure and execution steps

Preconditions:

- Near-RT RIC is deployed.
- A testing software that supports communication with Near-RT RIC over E2 related APIs [27].
- Error handling mechanisms are implemented by Near-RT RIC.

EXAMPLE 1: E2 Error or Failure message with Cause as defined in clause 9.2.1 of [28].

Execution Steps:

1) Malformed input data

- Provide malformed input data to the Near-RT RIC via E2 related APIs, violating the specified format.

EXAMPLE 2: Input data containing unknown IEs, incorrect format and mandatory IEs not present.

- Capture and analyse the response from Near-RT RIC via E2 related APIs.
- Verify that the Near-RT RIC detects the malformed input and rejects it appropriately, returning an error message or taking necessary actions to mitigate the impact.

EXAMPLE 3: Actions could be rejecting the message, sending an error rejection or failure indication, etc.

2) Valid and Invalid boundary cases

- Provide invalid (wrong value) data outside of the boundaries or limits of the acceptable values or ranges defined for specific parameters in clause 5.2.2 & 5.2.3, O-RAN E2 Interface Test Specification **Error! Reference source not found.** for E2 related APIs.
- Provide valid data at the boundaries or inside the allowed ranges defined for specific parameters in clause 5.2.2 & 5.2.3, O-RAN E2 Interface Test Specification **Error! Reference source not found.** for E2 related APIs.
- Verify that the Near-RT RIC handles these boundary cases correctly, without encountering any unexpected behaviour or errors due to boundary conditions.

Expected Results:

1) Malformed input data

- The Near-RT RIC properly validates incoming inputs via E2 related APIs from xApp and rejects those with malformed data, returning an appropriate error response.

2) Valid and Invalid boundary cases

- The Near-RT RIC properly handles the valid and invalid boundary cases.

Expected format of evidence:

1) Malformed input data

- Logs detailing the malformed input data provided to the Near-RT RIC via E2 related APIs, alongside system logs capturing the Near-RT RIC's error messages.

2) Valid and Invalid boundary cases

- Logs of the input data values provided to the Near-RT RIC, paired with system logs capturing the Near-RT RIC 's messages or behaviours in response to the boundary inputs.

7.7 Secure configuration enforcement

Requirement Name: O-CU configuration security enforcement

Requirement Reference: SEC-CTL-OCU-1, clause 5.1.4.2, SEC-CTL-ODU-1, clause 5.1.5.2, ,REQ-SEC-ORU-1, clause 5.1.6.1 in O-RAN Security Requirements and Controls Specifications [5], 3GPP TS 33.501 clause 5.3.4 [25]

Requirement Description: The DUT ensures that settings and software configurations are protected from unauthorized modifications.

Threat Reference: T-O-RAN-02

DUT/s: O-CU, O-DU, O-RU

Test Name: TC_CONF_ENFORCEMENT

Purpose: Ensure the DUT prevents unauthorized settings and software configurations changes, as required by 3GPP TS 33.501 clause 5.3.4 [25].

Procedure and execution steps

Preconditions:

- The DUT is powered on and operational.
- Settings and software configurations are defined and applied on the DUT.
- Successful access to the DUT is established.

Execution steps:

1) Attempt to modify the DUT settings and software configurations with proper authorization.
2) Verify that the DUT detects and accepts any authorized modification attempts.
3) Attempt to modify the DUT settings and software configurations without proper authorization.
4) Verify that the DUT detects and rejects any unauthorized modification attempts.

Expected results:

- DUT accepts authorized modifications.
- DUT detects and rejects any unauthorized modifications.

Expected format of evidence:

- Logs indicating the detection and acceptance of authorized modifications.
- Logs indicating the detection and rejection of unauthorized modifications.

EXAMPLE: examples of settings and software configurations include:

- Security configuration: e.g., protocols and keys used for authentication, authorization, and secure communication.
- Software management: e.g., current software version
- Interface settings

7.8 Logging and monitoring

7.8.1 O-CU logging and monitoring

Requirement Name: O-CU logging and monitoring

Requirement Reference: REQ-SEC-OCU-1, clause 5.1.4.1 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-O-RAN-07

DUT/s: O-CU

Test Name: TC_LOG_OCU

Purpose: The purpose of this test is to verify that the O-CU correctly logs and monitors security-related events effectively.

Procedure and execution steps

Preconditions

- The O-CU is powered on and operational.
- Logging and monitoring configurations are properly set up on the O-CU.

Execution steps

1) Logging

- The tester triggers an error or failure condition in the O-CU, such as connection attempts with invalid credentials, unauthorized access and a dropped connection.
- The tester verifies that the O-CU logs the error by capturing the relevant log entry.

2) Monitoring

- The tester monitors the key performance indicators (KPIs) of the O-CU, such as throughput, latency, or signal quality.
- The tester verifies that the monitoring system accurately collects and displays the KPI values in real-time.
- The tester introduces a simulated degradation or overload scenario on the O-CU, such as increasing network traffic or reducing available resources.
- The tester monitors the O-CU performance under the simulated scenario.

- The tester verifies that the monitoring system detects and raises alerts for the degraded performance or overload condition.

Expected Results

- O-CU logs and generates alerts for security-related events, providing necessary information and timestamps for incident investigation and analysis.
- The monitoring system provides accurate and real-time KPI values for the O-CU. The monitoring system detects and raises appropriate alerts for the degraded performance or overload condition.

Expected format of evidence:

- Capture and analyse the logged error in the O-CU logs or logging system and document the presence of the log entry.
- Document the monitored KPI values and the raised alerts, validate them against the expected values, and ensure they are triggered accurately in the monitoring system.

7.8.2 O-DU logging and monitoring

Requirement Name: O-DU logging and monitoring

Requirement Reference: REQ-SEC-ODU-1, clause 5.1.5.1 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-O-RAN-07

DUT/s: O-DU

Test Name: TC_LOG_ODU

Purpose: The purpose of this test is to ensure that the O-DU correctly logs and monitors security-related events effectively.

Procedure and execution steps

Preconditions

- The O-DU is powered on and operational.
- Logging and monitoring configurations are properly set up on the O-DU.

Execution steps

1) Logging

- The tester triggers an error or failure condition in the O-DU, such as connection attempts with invalid credentials, unauthorized access and a dropped connection.
- The tester verifies that the O-DU logs the error by capturing the relevant log entry.

2) Monitoring

- The tester monitors the key performance indicators (KPIs) of the O-DU, such as throughput, latency, or signal quality.
- The tester verifies that the monitoring system accurately collects and displays the KPI values in real-time.
- The tester introduces a simulated degradation or overload scenario on the O-DU, such as increasing network traffic or reducing available resources.

- Th tester monitors the O-DU performance under the simulated scenario.
- The tester verifies that the monitoring system detects and raises alerts for the degraded performance or overload condition.

Expected Results

- O-DU logs and generates alerts for security-related events, providing necessary information and timestamps for incident investigation and analysis.
- The monitoring system provides accurate and real-time KPI values for the O-DU. The monitoring system detects and raises appropriate alerts for the degraded performance or overload condition.

Expected format of evidence:

- Capture and analyse the logged error in the O-DU logs or logging system and document the presence of the log entry.
- Document the monitored KPI values and the raised alerts, validate them against the expected values, and ensure they are triggered accurately in the monitoring system.

7.8.3 O-RU logging and monitoring

Requirement Name: O-RU logging and monitoring

Requirement Reference: REQ-SEC-ORU-1, REQ-SEC-ORU-2, clause 5.1.6.1 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-O-RAN-07

DUT/s: O-RU

Test Name: TC_LOG_ORU

Purpose: The purpose of this test is to ensure that the O-RU correctly logs and monitors security-related events effectively.

Procedure and execution steps

Preconditions

- The O-RU is powered on and operational.
- Logging and monitoring configurations are properly set up on the O-RU.

Execution steps

1) Logging

- The tester triggers an error or failure condition in the O-RU, such as connection attempts with invalid credentials, unauthorized access and a dropped connection.
- The tester verifies that the O-RU logs the error by capturing the relevant log entry.

2) Monitoring

- The tester monitors the key performance indicators (KPIs) of the O-RU, such as throughput, latency, or signal quality.
- The tester verifies that the monitoring system accurately collects and displays the KPI values in real-time.

- The tester introduces a simulated degradation or overload scenario on the O-RU, such as increasing network traffic or reducing available resources.
- The tester monitors the O-RU performance under the simulated scenario.
- The tester verifies that the monitoring system detects and raises alerts for the degraded performance or overload condition.

Expected Results

- O-RU logs and generates alerts for security-related events, providing necessary information and timestamps for incident investigation and analysis.
- The monitoring system provides accurate and real-time KPI values for the O-RU. The monitoring system detects and raises appropriate alerts for the degraded performance or overload condition.

Expected format of evidence:

- Capture and analyse the logged error in the O-RU logs or logging system and document the presence of the log entry.
- Document the monitored KPI values and the raised alerts, validate them against the expected values, and ensure they are triggered accurately in the monitoring system.

7.8.4 Near-RT RIC logging and monitoring

Requirement Name: Near-RT RIC logging and monitoring

Requirement Reference: REQ-SEC-NEAR-RT-4, clause 5.1.3.1 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-O-RAN-07

DUT/s: NEAR-RT RIC

Test Name: TC_LOG_NEAR_RT_RIC

Purpose: The purpose of this test is to ensure that the Near-RT RIC correctly logs and monitors security-related events effectively.

Procedure and execution steps

Preconditions

- The Near-RT RIC is powered on and operational.
- Logging and monitoring configurations are properly set up on the Near-RT RIC.

Execution steps

1) Logging

- The tester triggers an error or failure condition in the Near-RT RIC, such as connection attempts with invalid credentials, unauthorized access, or a dropped connection.
- The tester verifies that the Near-RT RIC logs the error by capturing the relevant log entry.

2) Monitoring

- The tester monitors the key performance indicators (KPIs) of the Near-RT RIC, such as throughput, latency, or signal quality.

- The tester verifies that the monitoring system accurately collects and displays the KPI values in real-time.
- The tester introduces a simulated degradation or overload scenario on the Near-RT RIC, such as increasing network traffic or reducing available resources.
- Th tester monitors the Near-RT RIC performance under the simulated scenario.
- The tester verifies that the monitoring system detects and raises alerts for the degraded performance or overload condition.

Expected Results

- Near-RT RIC logs and generates alerts for security-related events, providing necessary information and timestamps for incident investigation and analysis.
- The monitoring system provides accurate and real-time KPI values for the Near-RT RIC. The monitoring system detects and raises appropriate alerts for degraded performance or overload conditions.

Expected format of evidence:

- Capture and analyse the logged error in the Near-RT RIC logs or logging system and document the presence of the log entry.
- Document the monitored KPI values and the raised alerts, validate them against the expected values, and ensure they are triggered accurately in the monitoring system.

8 System security evaluation for O-RAN architecture elements

8.1 Overview

This clause contains security evaluations to be performed at the system level of an O-RAN architecture element, covering vulnerability scanning, data and information protection and system logging.

The objects in scope of these system security evaluation are SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU and O-Cloud.

8.2 System Vulnerability Scanning

8.2.1 System Vulnerability Scanning

Requirement Name: Robustness of OS and Applications

Requirement Reference: REQ-SEC-SYS-1, clause 5.3.6.1, REQ-SEC-ALM-PKG-1, clause 5.3.2.1.1, O-RAN Security Requirements and Controls Specifications [6]

Requirement Description: Operating System (OS) and applications vulnerability scan of DUT

Threat References: T-O-RAN-01

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_Vulnerability_Scanning

Purpose: To verify the DUT does not contain known vulnerabilities in the OS and applications. Perform vulnerability scanning to ensure that there are no known vulnerabilities on DUT, both in the Operating System(OS) and the applications installed, that can be detected by means of automatic testing tools via the IP enabled network interfaces, or to identify the known vulnerabilities on DUT and have a clear mitigation plan for the ones of high severity.

Known vulnerabilities are considered those which are publicly disclosed, found by users or reported by security researchers. Those vulnerabilities are widely detected by commercial, or open-source tools designed for this purpose.

Procedure and execution steps

Preconditions

DUT is the O-RAN architecture element with IP enabled network interfaces.

Execution steps

1) Run the vulnerability scanning tool and check the potential known vulnerabilities existing on OS and applications levels.
2) Evaluate the severity level of the existing vulnerabilities.

Expected results

The DUT is free from known vulnerabilities or there are security controls in place to mitigate the exploits associated with the vulnerabilities of high severity.

Expected format of evidence: Report files, log files and/or screenshots.

8.3 Data and Information Protection

Void

8.4 System logging

8.4.1 Introduction

This clause contains test cases related to security log management.

8.4.2 Security log format and related log fields

Requirement Name: Security logs check for date, time and location field IP address.

Requirement Reference: SEC-CTL-SLM-FLD-1, SEC-CTL-SLM-FLD-2, clause 5.3.8.8.3, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Support for security logs containing date, time and location field IP address.

Threat References: T-O-RAN-07

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_Logs_Datetime_Fields_Validation

Purpose: To verify the log fields of security log data from DUT as per clause 5.3.8.8 of Security Requirements and Controls specifications [5]. The security log has the recommended date and time in ISO 8601 [10] format and mandatorily log the location field IP address (IP address of the host from which security events are generated).

Procedure and execution steps

Preconditions

DUT is any O-RAN architecture element that creates/generates security event logs.

Client is the test system equipped to communicate securely with O-RAN architecture element and able to perform security related operations on DUT.

Vendor documents operation(s) which generate security logs.

Execution steps

Execute one valid operation on the DUT which triggers/generates the security logs.

Expected results

- Date and time format as per ISO 8601 [10] as recommended by clause 5.3.8.8.3 of [5]
- Location field IP address (IP address of the DUT) as mandated by clause 5.3.8.8.3 of [5]

Expected format of evidence: Log files containing the output of the steps executed. In case the logs do not show the required information, report of a tool is used.

8.4.3 Authenticated Time Stamping

Requirement Name: Authenticated Time-Stamping

Requirement Reference: SEC-CTL-SLM-ATS-1, SEC-CTL-SLM-ATS-2, SEC-CTL-SLM-ATS-3, clause 5.3.8.9.2.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Optional support NTPv4

Threat References: T-O-RAN-07

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_Logs_Authenticated_Time_Stamping

Purpose: To verify that the DUT fulfills the optional requirement of supporting Network Time Protocol (NTP) version 4 as specified by RFC 5905 [15] for authenticated time stamping in the client role only.

Procedure and execution steps

Preconditions

- The DUT is powered on and operational.
- The NTP server specified for testing is reachable and configured to support authenticated time stamping.

Execution steps

Verify NTP Client Version:

- Access the DUT configuration settings related to NTP.
- Confirm that the DUT specifies NTP version 4 as the selected protocol.

Authentication Setup

- Configure the DUT to use the necessary authentication methods and -credentials (AES-CMAC/RFC 4493 [17], certificates for Autokey/RFC 5906 [16]) required by RFC 5905 for authenticated time stamping.
- Provide valid authentication credentials (certificates) for NTP communication.

Time Synchronization

- Initiate an NTP time synchronization process from the DUT to the specified NTP server.
- Monitor the communication between the DUT and the NTP server to ensure that the NTP packets are properly constructed with the required authentication parameters.
- Verify that the DUT successfully receives the authenticated time stamps from the NTP server.

Time Accuracy Check

- After synchronization, record the DUT internal clock time.
- Obtain the time from the NTP server's authenticated time stamp.
- Calculate the time difference between the DUT internal clock time and the received authenticated time stamp.
- Ensure that the time difference is within an acceptable tolerance, considering network latency and authentication processing.

Expected results

The DUT fulfils the requirement of supporting Network Time Protocol (NTP) version 4 for authenticated time stamping, as specified by RFC 5905. The NTP communication successfully employs the configured authentication methods, and the time synchronization process ensures accurate timekeeping within the specified tolerance. An accuracy below 1 second should be measured to pass.

Expected format of evidence: Log files, traffic captures and/or screenshots.

8.4.4 Network Security and System Security Events

Requirement Name: Network Security Events to be Logged and System Security Events to be Logged.

Requirement Reference: REQ-SEC-SLM-NET-EVT-1, clause 5.3.8.11.2, REQ-SEC-SLM-GEN-EVT-1, REQ-SEC-SLM-GEN-EVT-2, REQ-SEC-SLM-GEN-EVT-3, clause 5.3.8.11.3.1.1, REQ-SEC-SLM-HYP-EVT-1, REQ-SEC-SLM-HYP-EVT-2, REQ-SEC-SLM-HYP-EVT-3, clause 5.3.8.11.3.2, REQ-SEC-SLM-CON-EVT-1, REQ-SEC-SLM-CON-EVT-2, REQ-SEC-SLM-CON-EVT-3, clause 5.3.8.11.3.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Logging of network and system security events in O-Cloud

Threat References: T-O-RAN-01, T-O-RAN-02, T-O-RAN-03, T-O-RAN-09, T-VM-C-01, T-VM-C-02, T-VM-C-03, T-VM-C-04, T-VM-C-05, T-VM-C-06, T-IMG-01, T-IMG-02, T-ADMIN-02

DUT/s: O-Cloud

Test Name: TC_Logs_Network_System_Security_Events

Purpose: The purpose of the test is to verify the logging of security events from O-Cloud as per the Security Requirements and Controls Specifications [5].

Procedure and execution steps

Preconditions

A tester has access to testing equipment that can connect to the O-Cloud with administrative privileges to the operating system, hypervisor, and container engine.

Execution steps

1) Login to the DUT via testing equipment with administrative credentials.
2) Execute the following operations on the DUT.
    a) Create a new network configuration.
    b) Modify an existing network configuration.
    c) Disable a port.
    d) Enable a port.
    e) Generate packets that exceed configured firewall limits.
    f) Generate at least one network connection.

g) Reboot a virtual machine and then reboot the host operating system.

h) Shutdown a virtual machine then shutdown the host operating system.

i) Create a scheduled job within the host operating systems, hypervisor, and container engine.

j) Make a configuration change to the host operating system and hypervisor.

k) Attach and detach a virtual disk to a virtual machine.

l) Create a virtual machine.

m) Start a virtual machine.

n) Stop a virtual machine.

o) Delete a virtual machine.

p) Add an image to the container repository.

q) Modify an image to the container repository.

r) Remove an image to the container repository.

s) Create a container.

t) Start a container.

u) Stop a container.

v) Restart a container.

w) Delete a container.

x) Create a container volume.

y) Mount a container volume.

z) Delete a container volume.

Expected results

All the security logs produced by O-Cloud contain log messages that describe the actions taken in the execution steps.

- For execution step 2.a the log message indicates the creation of a new network configuration.
- For execution step 2.b the log message indicates the modification of an existing network configuration.
- For execution step 2.c the log message indicates the disabling of a port.
- For execution step 2.d the log message indicates the enabling a port.
- For execution step 2.e the log message indicates that packets have exceeded configured firewall limits.
- For execution step 2.f the log message indicates a network connection has been attempted along with details about that network connection including source and destination IP addresses.
- For execution step 2.g the log message indicates that a virtual machine was rebooted, and a subsequent log message indicates that a host operating system has been rebooted.
- For execution step 2.h the log message indicates that a virtual machine has been shut down and a subsequent log message indicates that the host operating system has been shut down.
- For execution step 2.i the log message indicates that a scheduled job was created within the host operating system, a subsequent log message indicates that a scheduled job was created in the hypervisor, and a subsequent log message indicates that a scheduled job was created in the container engine.

- For execution step 2.j the log message indicates that a configuration change was made to the host operating system and a subsequent log message indicates that a configuration change was made to the hypervisor.
- For execution step 2.k the log message indicates that a virtual disk was attached to a virtual machine, and a subsequent log message indicates that a virtual disk was detached from a virtual machine.
- For execution step 2.l the log message indicates that a virtual machine was created.
- For execution step 2.m the log message indicates that a virtual machine was started.
- For execution step 2.n the log message indicates that a virtual machine was stopped.
- For execution step 2.o the log message indicates that a virtual machine was deleted.
- For execution step 2.p the log message indicates that an image was added to the container repository.
- For execution step 2.q the log message indicates that an image was modified in the container repository.
- For execution steps 2.r the log message indicates that an image was removed from the container repository.
- For execution step 2.s the log message indicates a container was created.
- For execution step 2.t the log message indicates that a container was started.
- For execution step 2.u the log message indicates that a container was stopped.
- For execution step 2.v the log message indicated that a container was restarted.
- For execution step 2.w the log message indicates that a container was deleted.
- For execution step 2.x the log message indicates that a container volume was created.
- For execution step 2.y the log message indicates that a container volume was mounted.
- For execution step 2.z the log message indicates that a container volume was deleted.

Expected format of evidence: Generated Log Files from DUT/s.

8.4.5 Application Security Events

Requirement Name: Application Security Events to be Logged.

Requirement Reference: REQ-SEC-SLM-APP-EVT-1, REQ-SEC-SLM-APP-EVT-2, clause 5.3.8.11.4, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Support for the logging of security events in network functions

Threat References: T-OPENSRC-01, T-xAPP-01, T-xAPP-02, T-xAPP-03, T-xAPP-04, T-rAPP-01, T-rAPP-02, T-rAPP-03, T-rAPP-04, T-rAPP-05, T-rAPP-06, T-rAPP-07, T-PNF-01.

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_Logs_Application_Security_Events

Purpose: The purpose of the test is to verify the logging of security event data from O-RAN Network.

Procedure and execution steps

Preconditions

A tester has access to testing equipment that can connect to any O-RAN network function.

Execution steps

NOTE: Execution steps not applicable to the DUT may be skipped.

1) Login to the DUT via test equipment with authorized credentials.
2) Conduct an operation on the DUT that is known to generate an error.
3) Conduct an operation on the DUT that is known to load a dynamic library.

Expected results

All the security logs produced by O-RAN Network Functions contain log messages that pertain to the actions taken in the execution steps.

- For execution step 2 the log message contains an error message.
- For execution step 3 the log message contains a message indicating that a dynamic library loaded and details about that library.

Expected format of evidence: Generated Log Files from DUT/s.

8.4.6 Data Access Security Events

Requirement Name: Data Access Security Events to be Logged.

Requirement Reference: REQ-SEC-SLM-DAT-EVT-1, REQ-SEC-SLM-DAT-EVT-2, REQ-SEC-SLM-DAT-EVT-3, REQ-SEC-SLM-DAT-EVT-4, REQ-SEC-SLM-DAT-EVT-5, REQ-SEC-SLM-DAT-EVT-6, REQ-SEC-SLM-DAT-EVT-7, REQ-SEC-SLM-DAT-EVT-8, clause 5.3.8.11.5, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Logging of data access security events.

Threat References: T-VM-C-01, T-NEAR-RT-03, T-O-RAN-07, T-O-RAN-08, T-GEN-05

DUT/s: SMO, Non-RT RIC, Near-RT RIC, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_Logs_Data_Access_Security_Events

Purpose: The purpose of the test is to verify the logging of data access security.

Procedure and execution steps

Preconditions

A tester has access to testing equipment that can communicate securely with the DUT and is able to perform security and administrative related operations.

Execution steps

NOTE: Execution steps not applicable to the DUT may be skipped.

1) Login to the DUT via testing equipment with authorized credentials.
2) Execute the following operations on the DUT.
    a) Add a new file.
    b) Delete an existing file.
    c) Attempt to add a file in an unauthorized location.
    d) Attempt to delete a file from an unauthorized location.
    e) Read an existing file.
    f) Write to an existing file.
    g) Attempt to read to a file in an unauthorized location.
    h) Attempt to write to a file to an unauthorized location.

i) Create a new directory.

j) Delete an existing directory.

k) Attempt to create a directory in an unauthorized location.

l) Attempt to delete a directory from an unauthorized location.

m) Add data to a datastore or database.

n) Delete data from a datastore or database.

o) Attempt to add data to a datastore or database in an unauthorized location.

p) Attempt to delete data from a datastore or database from an unauthorized location.

q) Read data from a datastore or database.

r) Write data from a datastore or database.

s) Attempt to read data from a datastore or database from an unauthorized location.

t) Attempt to write data to a datastore or database in an unauthorized location.

u) Make a permissions change to a file.

v) Make a permissions change to a directory.

w) Make a permissions change to a datastore or database.

Expected results

All the security logs produced by O-RAN architecture elements contain log messages that document appropriately the actions taken in the execution steps.

- For execution step 2.a the log message indicates that a new file was added.
- For execution step 2.b the log message indicates an existing file was deleted.
- For execution step 2.c the log message indicates an unauthorized attempt to add a file.
- For execution step 2.d the log message indicates an unauthorized attempt to delete a file.
- For execution step 2.e the log message indicates an existing file was read.
- For execution step 2.f the log message indicates an existing file was written.
- For execution step 2.g the log message indicates an unauthorized attempt to read to a file.
- For execution step 2.h the log message indicates an unauthorized attempt to write to a file.
- For execution step 2.i the log message indicates a new directory was created.
- For execution step 2.j the log message indicates an existing directory was deleted.
- For execution step 2.k the log message indicates an unauthorized attempt to create a directory.
- For execution step 2.l the log message indicates an unauthorized attempt to delete a directory.
- For execution step 2.m the log message indicates data was added to a datastore or database.
- For execution step 2.n the log message indicates data was deleted from a datastore or database.
- For execution step 2.o the log message indicates an unauthorized attempt to add data to a datastore or database.
- For execution step 2.p the log message indicates an unauthorized attempt to delete data from a datastore or database.

- For execution step 2.q the log message indicates that data was read from a datastore or database.
- For execution step 2.r the log message indicates that data was written to a datastore or database.
- For execution step 2.s the log message indicates an unauthorized attempt to read data from a datastore or database.
- For execution step 2.t the log message indicates an unauthorized attempt to write data to a datastore or database.
- For execution step 2.u the log message indicates a permissions change to a file.
- For execution step 2.v the log message indicates a permissions change to a directory.
- For execution step 2.w the log message indicates a permissions change to a datastore or database.

Expected format of evidence: Generated Log Files from DUT.

8.4.7 Account and Identity Security Events

Requirement Name: Account and Identity Security Events to be Logged.

Requirement Reference: REQ-SEC-SLM-AAI-EVT-1, REQ-SEC-SLM-AAI-EVT-2, REQ-SEC-SLM-AAI-EVT-3, REQ-SEC-SLM-AAI-EVT-4, REQ-SEC-SLM-AAI-EVT-5, REQ-SEC-SLM-AAI-EVT-6, REQ-SEC-SLM-AAI-EVT-7, REQ-SEC-SLM-AAI-EVT-9, REQ-SEC-SLM-AAI-EVT-10, clause 5.3.8.11.6, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Logging of account and identity security events.

Threat References: T-GEN-02, T-O-RAN-02, T-O-RAN-06, T-O-RAN-07, T-ProtocolStack-02, T-SMO-02, T-SMO-05, T-SMO-08, T-SMO-25, T-SMO-30, T-NEAR-RT-03

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_Logs_Account_and_Identity_Security_Events

Purpose: The purpose of the test is to verify the logging of account and identity access security.

Procedure and execution steps

Preconditions

A tester will have access to testing equipment that can communicate securely with the DUT and is able to perform security and administrative related operations.

Execution steps

NOTE: Execution steps not applicable to the DUT may be skipped.

1) Login to the DUT via testing equipment with authorized credentials.
2) Execute the following operations on the DUT.
    a) Create an account.
    b) Modify an existing account.
    c) Delete an existing account.
    d) Attempt to create an account in an unauthorized location.
    e) Change the privilege level of an existing account from a lower privilege to a higher privilege.
    f) Attempt to change the privilege level of an existing account in an unauthorized location.

g) Change the group membership of an existing account.

h) Attempt to change the group membership of an existing account in an unauthorized location.

i) Use a function in the DUT that requires a specific assigned authorization.

j) Attempt to use a function in the DUT that requires a specific unassigned authorization.

k) Authenticate an account to the DUT that has been configured to access that DUT.

l) Attempt to authenticate an account to the DUT that has not been configured to access that DUT.

m) Change the privilege level of an existing account from a higher privilege to a lower privilege.

n) Access the DUT with an account the does not require authentication.

o) End a session with the DUT.

Expected results

All the security logs produced by O-RAN architecture elements contain log messages that document appropriately the actions taken in the execution steps.

- For execution step 2.a the log message indicates that an account was created.
- For execution step 2.b the log message indicates that an existing account was modified.
- For execution step 2.c the log message indicates that an existing account was deleted.
- For execution step 2.d the log message indicates an unauthorized attempt to create an account.
- For execution step 2.e the log message indicates a privilege level change of an existing account from a lower privilege to a higher privilege.
- For execution step 2.f the log message indicates an unauthorized attempt to change the privilege level of an existing account.
- For execution step 2.g the log message indicates that the group membership had changed for an existing account.
- For execution step 2.h the log message indicates an unauthorized attempt to change the group membership of an existing account.
- For execution step 2.i the log message indicates the use of a restricted function.
- For execution step 2.j the log message indicates an unauthorized attempt to use a restricted function.
- For execution step 2.k the log message indicates the successful authentication of an account.
- For execution step 2.l the log message indicates the unsuccessful attempt to authenticate an account.
- For execution step 2.m the log message indicates a privilege level change of an existing account from a higher privilege to a lower privilege.
- For execution step 2.n the log message indicates access with an account the does not require authentication.
- For execution step 2.o the log message indicates the end of a session.

Expected format of evidence: Generated Log Files from DUT.

8.4.8 General Security Events

Requirement Name: General Security Events to be logged.

Requirement Reference: REQ-SEC-SLM-GSE-1, REQ-SEC-SLM-GSE-2, REQ-SEC-SLM-GSE-3, REQ-SEC-SLM-GSE-4, REQ-SEC-SLM-GSE-5, REQ-SEC-SLM-GSE-6, clause 5.3.8.11.7, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Logging of general security events.

Threat References: T-ORAN-01, T-O-RAN-02, T-O-RAN-03, T-O-RAN-08, T-GEN-02, T-VM-C-01, T-VM-C-04, T-VM-C-06, T-IMG-01, T-IMG-04, T-VL-01, T-VL-02, T-xAPP-01, T-rAPP-03, T-HW-02

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_General_Security_Events_Logged

Purpose: The purpose of the test is to verify the logging of general security events.

Procedure and execution steps

Preconditions

A tester will have access to testing equipment that can communicate securely with the DUT and is able to perform security and administrative related operations.

Execution steps

NOTE: Execution steps not applicable to the DUT may be skipped.

1) Login to the DUT via testing equipment with authorized credentials.
2) Execute the following operations on the DUT.
    a) Enable security software such as firewalls, malware protection, data loss prevention or intrusion detection systems.
    b) Disable security software such as firewalls, malware protection, data loss prevention or intrusion detection systems.
    c) Log into DUT using an account with administrative privileges and perform a function that requires those privileges.
    d) Make a change to the security configuration of the DUT.
    e) View a certificate or key on the DUT.
    f) Export a certificate or key from the DUT.
    g) Renew a certificate or key on the DUT.
    h) Import a certificate or key from the DUT.
    i) Modify a certificate or key on the DUT.
    j) Delete a certificate or key from the DUT.
    k) Perform a cryptographic operation on the DUT that involves signatures, encryption, hashing, key generation or key destruction.
    l) Submit a security patch to the DUT but do not apply it.

Expected results

All the security logs produced by O-RAN architecture elements contain log messages that document appropriately the actions taken in the execution steps.

- For execution steps 2.a and 2.b the log message indicates the security software has been enabled or disabled.
- For execution step 2.c the log message indicates the use of administrative privileges.

- For execution step 2.d the log message indicates a change to the security configuration has occurred and the nature of the change.
- For executions 2.e through 2.k the log message is absent of any sensitive information related to the certificate or key.
- For execution 2.l the log message indicates that a security patch was submitted but not applied.

Expected format of evidence: Generated Log Files from DUT.

8.4.9 Void

9 Software security evaluation for O-RAN architecture elements

9.1 Overview

This clause contains a set of software security evaluations of an O-RAN architecture element, covering Software Lifecycle Management.

9.2 Open-Source Software Component Analysis

Void

9.3 Binary Static Analysis

Void

9.4 Software Bill of Materials (SBOM)

9.4.1 SBOM Signature

Requirement Name: A digital signature is provided for the SBOM.

Requirement Reference: SEC-CTL-SBOM-001, clause 6.3.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: SBOM is authenticity, integrity protected and provided in a standard format.

Threat References: T-O-RAN-09

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test name: TC_SBOM_Signature

Purpose: To verify the SBOM is provided with a digital signature

Procedure and execution steps

Preconditions

SBOM is provided. Tools to verify the digital signature are available. Solution Provider key or certificate is provided.

Execution steps

Ensure the SBOM is provided with a digital signature in the format as described below. Verify SBOM digital signature is valid using the software provider's public key or certificate. Depending on the format of the SBOM, there are various ways how to include and verify the digital signature of the SBOM. Below, the digital signature methods are detailed.

SPDX

YAML, RDF and tag data: The signature is in a separate file from the SPDX file. Digital signature format is CMS/PKCS#7/CAdES.

EXAMPLE: foo.spdx is accompanied by foo.spdx.sig containing its signature

XML: XML Signature 2.0

JSON: JSON Web Signature (JWS), and JSON Signature Format (JSF).

CycloneDX

XML: XML Signature 2.0

JSON: JSON Web Signature (JWS), and JSON Signature Format (JSF).

SWID

XML: XML Signature 2.0

Expected results

Digital signature of the SBOM is valid.

Expected format of evidence: Report file, screenshot, or log file from tool used for verification of digital signature.

9.4.2 SBOM Data Fields

Requirement Name: Data fields are according to NTIA guidance [13]

Requirement Reference: REQ-SBOM-002, REQ-SBOM-011, clause 6.3.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: A minimum set of data fields are included in the SBOM and it is in an standard format.

Threat References: T-O-RAN-09

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_SBOM_Data_Fields

Purpose: To verify the minimum set of data fields are included in the SBOM

Procedure and execution steps

Preconditions

SBOM file is provided. Tools to verify the data fields are available.

Execution steps

Run the SBOM check tool and verify that there is minimum set of data fields present in SBOM depending on the SBOM format used.

Table 9-1: Minimum set of data fields for SPDX [12]

NTIA field	NTIA description	SPDX 2.2.1 field
Supplier Name	The name of an entity that creates, defines, and identifies components	PackageSupplier
Component Name	Designation assigned to a unit of software defined by the original supplier	PackageName
Version of the Component	Identifier used by the supplier to specify a change in software from a previously identified version	PackageVersion
Other Unique Identifiers	Other identifiers that are used to identify a component, or serve as a look-up key for relevant databases	SPDXID (Package SPDX Identifier)
Dependency Relationship	Characterizing the relationship that an upstream component X is included in software Y	Relationship: CONTAINS
Author of SBOM Data	The name of the entity that creates the SBOM data for this component	Creator
Timestamp	Record of the date and time of the SBOM data assembly	Created

Table 9-2: Minimum set of data fields for CycloneDX [13]

NTIA field	NTIA description	CycloneDX field
Supplier Name	The name of an entity that creates, defines, and identifies components	publisher
Component Name	Designation assigned to a unit of software defined by the original supplier	name
Version of the Component	Identifier used by the supplier to specify a change in software from a previously identified version	version
Other Unique Identifiers	Other identifiers that are used to identify a component, or serve as a look-up key for relevant databases	bom/serialNumber and component/bom-ref
Dependency Relationship	Characterizing the relationship that an upstream component X is included in software Y	(Nested assembly/subassembly and/or dependency graphs)
Author of SBOM Data	The name of the entity that creates the SBOM data for this component	bom-descriptor:metadata/ manufacture/contact
Timestamp	Record of the date and time of the SBOM data assembly	timestamp

Table 9-3: Minimum set of data fields for SWID [13]

NTIA field	NTIA description	SWID tag
Supplier Name	The name of an entity that creates, defines, and identifies components	<Entity> @role (softwareCreator/publisher), @name
Component Name	Designation assigned to a unit of software defined by the original supplier	<softwareIdentity> @name
Version of the Component	Identifier used by the supplier to specify a change in software from a previously identified version	<softwareIdentity> @version
Other Unique Identifiers	Other identifiers that are used to identify a component, or serve as a look-up key for relevant databases	<softwareIdentity> @tagID
Dependency Relationship	Characterizing the relationship that an upstream component X is included in software Y	<Link> @rel, @href
Author of SBOM Data	The name of the entity that creates the SBOM data for this component	<Entity> @role (tagCreator), @name
Timestamp	Record of the date and time of the SBOM data assembly	-

This test is part of the O-RAN software producer's Software Development Lifecycle (SDLC).

Expected results

Minimum set of data fields are present.

Expected format of evidence: Report file, screenshot, or log file from SBOM check tool.

9.4.3 SBOM Format

Requirement Name: SBOM is provided in one of the accepted formats: SPDX, CycloneDX, or SWID.

Requirement Reference: REQ-SBOM-011, clause 6.3.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: SBOM is provided in a standard format.

Threat References: T-O-RAN-09

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_SBOM_Format

Purpose: To verify that the SBOM is provided in one of standard formats.

Procedure and execution steps

Preconditions

SBOM is provided. Tools to verify the SBOM are available.

Execution steps

Run the SBOM check tool to verify the SBOM format.

Expected results

SBOM format is SPDX, CycloneDX, or SWID.

Expected format of evidence: Report file.

9.4.4 SBOM Depth

Requirement Name: SBOM Depth is in the required level.

Requirement Reference: REQ-SBOM-004, clause 6.3.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The SBOM Depth is the required for the different types of software.

Threat References: T-O-RAN-09

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_SBOM_Depth

Purpose: To verify that the SBOM depth is provided to the level specified.

Procedure and execution steps

Preconditions

SBOM is provided. Tools to verify the SBOM are available.

Execution steps

Run the SBOM check tool to verify the SBOM depth provided.

At a minimum, all top-level dependencies are listed.

Expected results

SBOM depth is as specified in the requirements:

- top-level for every O-RAN software delivery

Expected format of evidence: Report file, log file from SBOM check tool.

9.4.5 Void

9.4.6 SBOM Version Verification

Requirement Name: The version in the SBOM is accurately and matches the actual O-RAN software package version.

Requirement Reference: REQ-SBOM-002, clause 6.3.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Version of the software component included in the SBOM is matching the actual software component version.

Threat References: T-O-RAN-08, T-O-RAN-09

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_SBOM_VERSION_VERIFICATION

Purpose: The purpose of this test is to ensure the SBOM reflects the current version of software package.

Procedure and execution steps

Preconditions

SBOM is provided. Tools to verify the SBOM are available. Access to version information regarding O-RAN software package.

Execution steps:

1) For each software component in the package, compare the version listed in the SBOM with the actual software component version.
2) Ensure that the SBOM's version matches the component's version.
3) Document any discrepancies.

Expected Results:

The version specified in the SBOM aligns with the actual version of the software component.

Expected format of evidence:

A report detailing:

- name of the software package component(s), SBOM indicated version, actual version, notes on any discrepancies or issues found.

9.4.7 Void

9.4.8 SBOM Presence

Requirement Name: SBOM provided with all O-RAN Software.

Requirement Reference: REQ-SBOM-001, clause 6.3.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: For every O-RAN software delivery, an SBOM is available.

Threat References: T-O-RAN-09

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_SBOM_Presence

Purpose: The purpose of this test is to ensure that for every O-RAN software delivery an SBOM is available. This is applicable to all software components within the O-RAN system.

Procedure and execution steps

Preconditions

SBOM is provided. Tools to verify the SBOM are available.

Execution steps

1) List O-RAN software delivery.
2) Look for associated files or documentation indicating the presence of an SBOM.
3) Validate the SBOM's content to ensure it's not just a placeholder.
4) Document any software delivery that lacks a genuine SBOM.

Expected Results

Every O-RAN software delivery has a genuine SBOM associated with it.

Expected Format of Evidence:

A report detailing:

- names of the software package components;
- status of its SBOM (Present/Absent);
- notes on any discrepancies or issues found.

9.4.9 SBOM Vulnerabilities Field

Requirement Name: Vulnerabilities field omission in SBOMs.

Requirement Reference: REQ-SBOM-003, clause 6.3.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Vulnerabilities are not included as an additional data field because it would represent a static view from a specific point in time, while vulnerabilities are constantly evolving. Therefore, the vulnerabilities field in SBOM for O-RAN software cannot be relied upon to determine the SBOM vulnerabilities by the Service provider. Service providers need to perform their own vulnerability assessment, at least at the moment of the SBOM release.

Threat References: T-O-RAN-09

DUT/s: SMO, Non-RT RIC and rApps, Near-RT RIC and xApps, O-CU-CP, O-CU-UP, O-DU, O-RU, O-Cloud

Test Name: TC_SBOM_Vulnerabilities_Fields

Purpose: Verify that vulnerabilities fields is not included as an additional field to the SBOM.

Procedure and execution steps

Preconditions

SBOM is provided. Tools to verify the SBOM are available.

Execution steps

Verify that no vulnerabilities fields exist within the SBOM.

Expected Results

There are no vulnerabilities field(s) present in the SBOM.

Expected Format of Evidence: screenshot(s)

9.4.10 Void

9.5 Signature Verification

9.5.1 Signature Verification Matrix per DUT and Phase

This clause defines the roles and responsibilities of each DUT in verifying digital signatures on signed objects.

The following table summarizes:

- the type of signed object each DUT tests,
- the phase where signature validation is expected (e.g., onboarding, deployment),
- and the DUT's corresponding responsibility in verifying digital signatures.

All positive and negative signature verification test cases (as defined in clause 9.5.2) apply accordingly based on this table.

Table 9-4: Signature verification per DUT, signed object and phase

DUT	Signed object	Phase	Signature(s) to Verify
SMO	Application Packages (xApps/rApps images, O-RAN Architecture Element images, PNF images) Application artifacts O-Cloud software images AI/ML models rApp images	Onboarding, Deployment, Instantiation	Verifies Service Provider (if present) and Solution Provider signatures
O-Cloud	O-RAN Architecture Element Images Application artifacts O-Cloud software images xApp images AI/ML models	Deployment, Instantiation	Verifies Service Provider (if present) and Solution Provider signatures

NOTE: For all test cases defined in this clause, when both Service Provider and Solution Provider signatures are present on a signed object, the DUT performs signature verification in the following order:
1. Service Provider signature,
2. Solution Provider signature.

For each DUT, it is sufficient to perform the test case on one representative signed object type, since the signature verification mechanism is common across all object types listed for that DUT. The intent is to verify the implementation of the cryptographic validation logic, not to exhaustively test all signed object formats.

9.5.2 Signature Verification Test Cases

Requirement Name: Signature verification of signed objects.

Requirement Reference: SEC-CTL-ALM-PKG-1, SEC-CTL-ALM-PKG-1B, SEC-CTL-ALM-PKG-9, clause 5.3.2.1.2, SEC-CTL-OCLOUD-SW-1, clause 5.1.8.3.2 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Each signed object is verified for integrity and authenticity protection.

Threat References: T-IMG-01, T-IMG-04, T-AppLCM-01

DUT/s: SMO, O-Cloud

Test Case 1: Positive signature verification

Test Name: TC_Sig_Verification_Positive

Purpose: To verify that the DUT correctly validates the digital signature of the signed object, as per Table 9-4.

Procedure and execution steps

Preconditions

Digitally signed object using a trusted certificate and approved algorithm as defined by the O-RAN Security Protocols Specification **Error! Reference source not found.**, clause 5, is available.

The DUT has access to the relevant trust anchors.

Execution steps

1) Submit the signed object to the DUT via the appropriate interface.

EXAMPLE: The interface may include the onboarding API for the SMO or a deployment automation pipeline in the O-Cloud, depending on the phase in which signature verification occurs.

2) Trigger the DUT to initiate signature verification of the signed object according to Table 9-4.

3) The DUT performs digital signature verification:

   a) Validates the certificate chain of the signature.

   b) Verifies the cryptographic integrity of the object.

Expected results

Signature verification succeeds, and the signed object is accepted.

Expected Format of Evidence: logs showing successful signature verification.

Test Case 2: Hash Mismatch Signature Verification Failure

Test Name: TC_SIG_Validation_HashMismatch

Purpose: To verify that the DUT correctly detects and rejects a signed object whose hash does not match the signed hash.

Procedure and execution steps

Preconditions

- Valid digitally signed object
- Signed object content is modified after signing (introducing a hash mismatch)
- Trusted CA certificates are installed on the DUT

Execution steps

1) Submit the altered signed object to the DUT via the appropriate interface.

2) Trigger the DUT to initiate signature verification of the altered signed object according to Table 9-4.

3) The DUT computes the hash of the altered signed object and compares it against the signed hash.

Expected Results

- Signature verification fails due to hash mismatch
- Signed object is rejected

Expected format of evidence: Logs indicating hash mismatch detection

Test Case 3: Non-Compliant Algorithm Signature Verification Failure

Test Name: TC_SIG_Validation_NonCompliantAlgorithm

Purpose: To verify that the DUT correctly rejects signed objects that are signed using cryptographic algorithms not listed in the accepted algorithm set defined by the O-RAN Security Protocols Specification **Error! Reference source not found.**, clause 5.

Procedure and execution steps

Preconditions

- A signed object is available that was signed using a cryptographic algorithm not listed in the Security Protocols Specification **Error! Reference source not found.**, clause 5.
- The DUT has access to the relevant trust anchors.

Execution steps

1) Submit the signed object to the DUT via the appropriate interface.
2) Trigger the DUT to initiate signature verification of the signed object according to Table 9-4.
3) The DUT detects that the signature uses a non-compliant algorithm.

Expected Results

- Signature verification fails due to the use of a non-compliant algorithm not listed in the O-RAN Security Protocols Specification **Error! Reference source not found.**, clause 5.
- Signed object is rejected

Expected format of evidence: Logs showing failure of signature verification and indicating the non-compliant algorithm used.

Test Case 4: Certificate Public Key Mismatch Verification Failure

Test Name: TC_SIG_Validation_CertPublicKeyMismatch

Purpose: To verify that the DUT correctly detects and rejects a public/private key mismatch during signature verification.

Procedure and execution steps

Preconditions

- Signed object with a valid private key, but certificate used during validation contains an unrelated public key.
- The DUT has access to the relevant trust anchors.

Execution steps

1) Submit the signed object to the DUT via the appropriate interface.
2) Trigger the DUT to initiate signature verification of the signed object according to Table 9-4.
3) The DUT detects a mismatch between the public key in the certificate and the signature.

Expected Results

- Signature verification fails due to public/private key mismatch
- Signed object is rejected

Expected format of evidence: Logs indicate public key mismatch error.

Test Case 5: Untrusted CA Signature Verification Failure

Test Name: TC_SIG_Validation_UntrustedCA

Purpose: To verify that the DUT correctly rejects signed objects using certificates from untrusted CAs.

Procedure and execution steps

Preconditions

- Signed object using a certificate from an untrusted CA
- The DUT has access to the relevant trust anchors.

Execution steps

1) Submit the signed object to the DUT via the appropriate interface.
2) Trigger the DUT to initiate signature verification of the signed object according to Table 9-4.
3) The DUT checks the issuer of the certificate and compares it with the entries in its trust store.

Expected Results

- Signature verification fails due to untrusted CA
- Signed object is rejected

Expected format of evidence: Logs indicate untrusted CA.

Test Case 6: Expired Certificate Signature Verification Failure

Test Name: TC_SIG_Validation_ExpiredCert

Purpose: To verify that the DUT correctly detects and rejects signed objects using expired certificates.

Procedure and execution steps

Preconditions

- Signed object using a certificate whose validity period has expired.
- The DUT's system time is configured to a date after the certificate's expiration date.
- The DUT has access to the relevant trust anchors.

Execution steps

1) Submit the signed object to the DUT via the appropriate interface.
2) Trigger the DUT to initiate signature verification of the signed object according to Table 9-4.
3) The DUT checks the certificate's validity period against the current time.

Expected Results

- Signature verification fails due to expired certificate
- Signed object is rejected

NOTE: If the DUT is configured, based on operator policy, to allow expired certificates during deployment or instantiation, the signature verification may succeed, and the signed object may be accepted in accordance with that policy.

Expected format of evidence: Logs indicate expired certificate error.

10 ML security validation for O-RAN system

10.1 Overview

AI/ML technologies and models are adopted at the O-RAN system Non-RT RIC and Near-RT RIC to enable O-RAN use cases: traffic steering, massive MIMO optimization, radio resource allocation for UAV applications, position accuracy enhancement, beam management, and enhance CSI feedback. Other uses cases could be checked in document O-RAN Use Cases Detailed Specification [22].

10.2 ML Data Poisoning

Void

11 Security tests of O-RAN interfaces

11.1 Open FH

11.1.1 Overview

This clause contains security tests to validate the security protection mechanism of the O-RAN Open Fronthaul interface.

11.1.2 Open Fronthaul Point-to-Point LAN Segment

11.1.2.0 Introduction

IEEE 802.1X-2020 Port-based Network Access Control [11] provides the means to control network access in point-to-point LAN segments within the Open Fronthaul network. Port-based network access control in the O-RAN Alliance Open Fronthaul comprises supplicant, authenticator, and authentication of server entities described in IEEE 802.1X-2020 [11].

The security test cases in this clause cover the validation of the authenticator and supplicant functionalities of the 802.1X, affecting to all the elements acting as an O-RAN Open Fronthaul network elements, including but not limited to, O-DU, O-RU, switches, FHM, FHGW, TNE and PRTC-T/GM as defined in clause 5.2.5.5 of Security Requirements and Controls Specifications [5].

11.1.2.1 Authenticator Validation

Requirement Name: Authenticator function validation

Requirement Reference: SEC-CTL-OFHPLS-3, SEC-CTL-OFHPLS-5, REQ-SEC-OFHPLS-1, clause 5.2.5.5, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Requirements of Authenticator in the Open Fronthaul network and its interface to an Authentication Server

Threat References: T-FRHAUL-02

DUT/s: O-DU

Test Name: TC_Authenticator_Validation

Purpose: To verify and validate the authenticator requirements of the network component to serve the request from supplicant(s) using EAP TLS authentication per 802.1X-2020 [11].

Procedure and execution steps

Preconditions

DUT supports authenticator role of the 802.1X for port-based network access control.

IP enabled network interface of DUT reachable to the authentication server and 802.1X enabled for its Open Fronthaul interface.

Set up an authentication server with root, server and client certificates, and the start the authentication server.

EXAMPLE: RADIUS server (e.g. free radius on Linux®) can be a possible authentication server.

NOTE: RADIUS support is required over interface between an authenticator and authentication server in the security requirement specification, only RADIUS authentication server is called for in this security test environment setup. Diameter based authentication server could be used as an alternative.

Execution steps

Run the 802.1X test tool emulating the request(s) from the supplicant(s) towards the DUT, which is the authenticator and ensure the 802.1X authentication process runs to completion.

The following test scenarios are executed:

Table 11-1: Scenarios to be executed

Scenario ID	Configuration
1	Test tool (as supplicant) setting for 802.1X with EAPoL, correct Identity (Certificate DN) and Client Certificate (provisioned on the Radius server)
2	Test tool (as supplicant) setting for 802.1X with EAPoL, correct Identity (Certificate DN) and incorrect Client Certificate (not provisioned on the Radius server)
3	Test tool (as supplicant) setting for 802.1X with EAPoL and incorrect Identity (Certificate DN)
4	Test tool (as supplicant) setting for 802.1X with EAP non-TLS authentication

Expected results

DUT successfully completes the procedure for the 802.1X Authentication validation (being granted or denied), for each test scenario:

Table 11-2: Expected results

Scenario ID	Expected result	Reason
1	Connection established	Authentication successfully
2	Connection not established	Fail Authentication because the certificate is wrong
3	Connection not established	Fail Authentication because the Identity is wrong
4	Connection not established	Fail Authentication because the authentication type is wrong

Expected format of evidence: Log files with traffic captures including the different scenarios executed in the execution steps and with a clear identification of successful and unsuccessful authentication

11.1.2.2 Supplicant Validation

Requirement Name: Supplicant function validation

Requirement Reference: SEC-CTL-OFHPLS-2, SEC-CTL-OFHPLS-5, REQ-SEC-OFHPLS-1, clause 5.2.5.5, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Requirements of Supplicant in the Open Fronthaul network

Threat References: T-FRHAUL-02

DUT/s: O-RU, O-DU

Test Name: TC_Supplicant_Validation

Purpose: To verify the supplicant requirement of the network component for port connection request using EAP TLS authentication per 802.1X-2020 [11].

Procedure and execution steps

Preconditions

DUT supports supplicant role of the 802.1X for port-based network access control.

Set up an authentication server with root, server and client certificates, and start the authentication server.

EXAMPLE: RADIUS server (e.g. free radius on Linux) can be a possible authentication server.

Set up the 802.1X test tool host/device as the authenticator with EAP TLS authentication for 802.1X protocol and configure the preset authentication server.

NOTE: RADIUS support is required over interface between an authenticator and authentication server in the security requirement specification, only RADIUS authentication server is called for in this security test environment setup. Diameter based authentication server could be used as an alternative.

Execution steps

Start the test run as an emulated authenticator waiting for the supplicant request.

Configure and enable the DUT to start the port connection request as a supplicant towards the 802.1X test tool, which is the authenticator and verify the 802.1X authentication process runs to completion.

The following test scenarios are executed:

Table 11-3: Scenarios to be executed

Scenario ID	Configuration
1	DUT (as supplicant) setting for 802.1X with EAPoL, correct Identity (Certificate DN) and Client Certificate (provisioned on the Radius server)
2	DUT (as supplicant) setting for 802.1X with EAPoL, correct Identity (Certificate DN) and incorrect Client Certificate (un-provisioned on the Radius server)
3	DUT (as supplicant) setting for 802.1X with EAPoL and incorrect Identity (Certificate DN)
4	DUT (as supplicant) setting for 802.1X with EAP non-TLS authentication (optional)

Expected results

DUT successfully completes the procedure for the supplicant validation (being granted or denied), for each test scenario:

Table 11-4: Expected results

Scenario ID	Expected result	Reason
1	Connection established	Authentication successfully
2	Connection not established	Fail Authentication because the certificate is wrong
3	Connection not established	Fail Authentication because the Identity is wrong
4	Connection not established	Fail Authentication because the authentication type is wrong

Expected format of evidence: Log files with traffic captures including the different scenarios executed in the execution steps and with a clear identification of successful and unsuccessful authentication.

11.1.2.3 Port-Based Access Enforcement Validation

Requirement Name: Port-Based Access Enforcement Validation

Requirement Reference: SEC-CTL-OFHPLS-4, clause 5.2.5.5.3, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: An authenticator in an Open Fronthaul network element that supports port-based network access control on each point-to-point LAN segment as defined in IEEE 802.1X-2020.

Threat References: T-FRHAUL-02

DUT/s: O-DU

Test Name: TC_Port_Access_Enforcement_Validation

Purpose: To verify that the DUT, when acting as an 802.1X authenticator, enforces port-based access control by allowing only EAPoL frames prior to authentication, and blocking all other Ethernet traffic—including frames carrying IP packets—until EAP-TLS authentication is successfully established.

Procedure and execution steps

Preconditions

- DUT supports the authenticator role of IEEE 802.1X.
- The DUT is configured with 802.1X enabled on its Open FH.
- The authentication server is set up and reachable (e.g., FreeRADIUS with appropriate certificates).
- The DUT is connected to a test tool or host representing a network device attempting to access the Open FH.

Execution steps

1) **Unauthenticated traffic attempt:** Connect the test tool to the DUT port. Without initiating any 802.1X (EAPoL) authentication, attempt to send standard Ethernet traffic such as ARP requests, ICMP echo (ping), and TCP SYN packets toward the DUT.
2) **Authenticated traffic attempt:** Configure the test tool to act as a supplicant and initiate a full 802.1X authentication using valid EAP-TLS credentials. Upon successful authentication and port authorization, re-attempt sending the same Ethernet traffic.

Expected results

1) **Unauthenticated traffic attempt:** The DUT blocks all Ethernet frames. The port remains in the unauthorized state.
2) **Authenticated traffic attempt:** The DUT authorizes the port and permits all Ethernet traffic.

Expected format of evidence: Packet capture logs showing that no Ethernet traffic is permitted before authentication, and that traffic flows after successful authentication.

11.1.2.4 O-RU Authentication Lifecycle with Manufacturer and Operator Certificates

Requirement Name: O-RU Authentication Lifecycle Validation

Requirement Reference: SEC-CTL-OFHPLS-11, SEC-CTL-OFHPLS-12, clause 5.2.5.5.3.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The O-RU has a manufacturer installed X.509 Certificate. The procedure defined in IEEE 802.1X and shown in Figures 5.2.5.5.3.2-1/2 [5] is performed to authenticate and authorize a DUT within the Open FH.

Threat References: T-FRHAUL-01A

DUT/s: O-RU

Test Name: TC_ORU_Authentication_Lifecycle_Validation

Purpose: To verify that the O-RU implements the full 802.1X authentication lifecycle for secure onboarding and operational access, including:

- Presence and use of a manufacturer-installed X.509 certificate during initial EAP-TLS authentication.

- Placement into a provisioning VLAN following initial authentication as instructed by the authentication server to the authenticator.
- Enrolment into the operator's PKI and installation of an operator-issued certificate.
- Re-authentication using the operator certificate and transition to operational VLAN.

Procedure and execution steps

Preconditions

- The DUT supports IEEE 802.1X in the supplicant role.
- The DUT has not yet been enrolled in the operator's PKI.
- The DUT is equipped with a valid manufacturer-installed X.509 certificate, provisioned by a trusted manufacturer CA.
- The DUT supports a certificate enrolment mechanism to obtain an operator-issued X.509 certificate after initial authentication.
- The authentication server is reachable by the authenticator (EXAMPLE: O-DU or switch connected to the O-RU) and provisioned with trust anchors (CA certificates) for both the manufacturer and operator CAs. It is configured to:
  - Accept the manufacturer certificate for initial onboarding and instruct the authenticator to assign a provisioning VLAN.
  - Accept the operator certificate for operational access and instruct the authenticator to assign an operational VLAN.

Execution steps

1) Initial EAP-TLS authentication using manufacturer certificate

- The DUT performs EAP-TLS using its manufacturer certificate.
- Upon successful authentication, the authentication server instructs the authenticator to assign a provisioning VLAN to the port connected to the DUT.
- The authenticator transitions the DUT's port to the authorized state and applies the provisioning VLAN.

2) Certificate enrolment into operator PKI

- Within the provisioning VLAN, the DUT connects to the operator PKI.
- The DUT requests and installs an operator-issued X.509 certificate.

3) Re-authentication using operator certificate

- The DUT restarts the supplicant interface.
- A second EAP-TLS exchange is initiated using the operator certificate.
- The authentication server validates the certificate and instructs the authenticator to assign the operational VLAN to the port connected to the DUT.
- The authenticator authorizes the port and applies the updated VLAN.

Expected results

- The DUT successfully authenticates using its manufacturer certificate and is given access to the provisioning VLAN.
- The DUT successfully enrols in the operator PKI.

- The DUT re-authenticates using the operator certificate and is given access to the operational VLAN.

Expected format of evidence:

- Packet captures showing two EAP-TLS authentication flows (manufacturer and operator certificates).
- Authentication server logs confirming identity and policy assignment for each certificate.
- DUT logs confirming success of certificate enrolment and the current certificate status.

11.1.3 M-Plane

11.1.3.1 SSH-based M-Plane authentication, authorization and access control protection

11.1.3.1.0 Introduction

The test cases outlined in this clause verify M-Plane authenticity, authorization, and access control protection over the Open FH interface using SSH.

11.1.3.1.1 Secure Password-Based Authentication and Authorization in Open FH M-Plane Using SSH

Requirement Name: M-Plane authenticity protection over Open FH interface using SSH

Requirement Reference: Clause 5.4.2 in O-RAN WG4 Management Plane Specification [21]

Requirement Description: Password based Authentication and Authorization for Open FH M-Plane over SSH across the Open FH interface.

Threat References: T-O-RAN-05, T-FRHAUL-01, T-FRHAUL-02, T-MPLANE-01

DUT/s: O-RU

Test Name: TC_OFH_MPLANE_SSH-PASSWORD-BASED_AUTHENTICATION_AUTHORIZATION

Purpose: The purpose of this test is to verify the SSH password-based authentication and authorization mechanisms on the Open FH interface by the O-RU.

Procedure and execution steps

Preconditions

- The O-RU is properly configured and operational.
- Test equipment (potentially an O-DU or a dedicated SSH client simulator) is configured to establish SSH connections to the O-RU
- NACM with NETCONF is enabled and configured for authorization on the O-RU Open FH interface.
- SSH is properly implemented and configured in the O-RU as defined in [2] clause 4.1.

Execution steps

1) Execute the test on the SSH protocol as defined in clause 6.2.
2) Positive Case: Successful SSH password-based authentication and authorization.

- Test the successful SSH password-based authentication and authorization of the test equipment by the O-RU.
  - Establish an SSH connection from the test equipment (acting as SSH client) to the O-RU (acting as SSH server) using the SSH password.

EXAMPLE 1: "Command: ssh <username>@<O-RU_IP>"

- Verify that the O-RU successfully authenticates the test equipment using the SSH password.
- Validate that the test equipment is authorized to perform the requested operations on the Open FH interface after successful authentication. This operation should be within the scope of permitted actions for the authenticated entity.

EXAMPLE 2: Examples of operations: "start up" installation, software management, configuration management, performance management, fault management and file management towards the O-RU

- Monitor the responses from the O-RU to these operations.
- Record whether each operation was successfully executed, partially executed, or rejected.
- Verify the O-RU logs to confirm that the operations were authorized.

3) Negative Case: Failed SSH password-based authentication.

- Test the handling of failed SSH password-based authentication attempts of the test equipment by the O-RU in different scenarios.
  - Attempt with incorrect password
    - Attempt to establish an SSH connection from the test equipment to the O-RU using an incorrect password.

EXAMPLE 3: Command: **ssh <valid_username>@<O-RU_IP>**, using an incorrect password

- Verify that the O-RU rejects the SSH connection due to the authentication failure.

- Attempt with non-existent username
  - Attempt to establish an SSH connection using a username that does not exist in the O-RU's user database.

EXAMPLE 4: Command: ssh <invalid_username>@<O-RU_IP>

- Verify that the O-RU rejects the SSH connection, confirming that authentication does not proceed with non-existent usernames.

Expected Results

For step 1): Expected results in clause 6.2

For step 2):

- The SSH connection is successfully established using the SSH password.
- The O-RU validates the test equipment's SSH password for authentication.
- The O-RU grants the necessary authorization for the requested operations.

For step 3):

- The SSH connection attempt fails due to the incorrect password.
- The O-RU identifies the authentication failure and denies access.
- The SSH connection attempt fails due to the invalid username.
- The O-RU identifies the authentication failure and prevents access.

Expected format of evidence

For step 1): Logs and screenshots showing adherence to SSH protocol specifications as defined in [2] clause 4.1.

For step 2): Logs showing successful SSH authentication and authorization events.

For step 3): Logs or error messages indicating failed SSH password-based authentication attempts for both incorrect password and invalid username scenarios.

11.1.3.1.2 Open FH M-Plane SSH-certificate-based authentication and authorization at O-RU

Requirement Name: M-Plane authenticity protection over Open FH interface using SSH

Requirement Reference: Clause 5.4.2 in O-RAN WG4 Management Plane Specification [21]

Requirement Description: Certificate based Authentication and Authorization for Open FH M-Plane over SSH across the Open FH interface at O-RU.

Threat References: T-O-RAN-05, T-FRHAUL-01, T-FRHAUL-02, T-MPLANE-01

DUT/s: O-RU

Test Name: TC_OFH_MPLANE_SSH-CERTIFICATE-BASED_AUTHENTICATION_AUTHORIZATION_O-RU

Purpose: The purpose of this test is to verify the SSH-certificate-based authentication and authorization mechanisms on the Open FH interface at O-RU, using test equipment to simulate O-DU.

Procedure and execution steps

NOTE: Test equipment simulates the role of O-DU for the purpose of this test.

Preconditions

- The O-RU is properly configured and operational.
- Test equipment capable of simulating SSH client functionality is prepared to represent the O-DU.
- SSH keys and certificates are generated and installed on both the O-RU and the test equipment.
- NACM with NETCONF is enabled and configured for authorization on the O-RU Open FH interface.
- SSH is properly implemented and configured in the O-RU as defined in [2] clause 4.1.

Execution steps

Execute the test on the SSH protocol as defined in clause 6.2.

Authentication and authorization of O-DU by O-RU (test equipment simulates O-DU)

- Positive Case: Successful SSH-certificate-based authentication and authorization.
  - Establish an SSH connection from the O-DU to the O-RU using the SSH certificate.
  - Verify that the O-RU successfully authenticates the O-DU using the SSH certificate.
  - Validate that the O-DU is authorized to perform the requested operations on the Open FH interface.
    - Perform an operation on the Open FH interface that requires authorization. This operation should be within the scope of permitted actions for the authenticated O-DU.

EXAMPLE 1: Examples of operations: "start up" installation, software management, configuration management, performance management, fault management and file management towards the O-RU

    - Monitor the responses from the O-RU to these operations.
    - Record whether each operation was successfully executed, partially executed, or rejected.
    - Verify the O-RU logs to confirm that the operations were authorized.

- Negative Case: Failed SSH-certificate-based authentication.

- Test the handling of failed SSH-certificate-based authentication attempts by the O-RU in different scenarios.
  - Attempt with invalid certificate (Invalid due to public – private key mismatch)

    Attempt to establish an SSH connection using an incorrect or invalid SSH certificate.

EXAMPLE 2: "Command: **ssh -i <path_to_invalid_private_key> -o CertificateFile=<path_to_invalid_certificate> <valid_username>@<O-RU_IP>**"

Verify that the O-RU rejects the SSH connection due to the authentication failure.

  - Attempt with invalid username

    Attempt to establish an SSH connection using a valid SSH certificate, but with a username that does not exist in the O-RU's system.

EXAMPLE 3: "Command: **ssh -i <path_to_valid_private_key> -o CertificateFile=<path_to_valid_certificate> <invalid_username>@<O-RU_IP>**"

Verify that the O-RU rejects the SSH connection, confirming that the system does not authenticate usernames that are not registered or recognized.

Expected Results

For step 1): Expected results in clause 6.2

For Positive Case:

- The SSH connection is successfully established using the correct SSH certificate.
- The DUT (O-RU) validates the test equipment's SSH certificate for authentication.
- The O-RU grants the necessary authorization to the O-DU for the requested operations.

For Negative Case:

- The SSH connection attempt fails due to the incorrect or invalid SSH certificate.
- The DUT identifies the authentication failure and denies access accordingly.

Expected format of evidence

For step 1): Logs and screenshots showing adherence to SSH protocol specifications as defined in [2] clause 4.1.

For Positive Case: Logs showing successful SSH authentication and authorization events.

For Negative Case: Logs or error messages indicating failed SSH-certificate-based authentication attempts for both invalid certificate and non-existent username scenarios.

11.1.3.1.3 Open FH M-Plane SSH-certificate-based NACM Access Control

Requirement Name: M-Plane access control protection over Open FH interface using SSH

Requirement Reference: Clause 5.4.2 in O-RAN WG4 Management Plane Specification [21]

Requirement Description: Certificate based NACM Access control for Open FH M-Plane over SSH across the Open FH interface.

Threat References: T-O-RAN-05, T-FRHAUL-01, T-FRHAUL-02, T-MPLANE-01

DUT/s: O-RU

Test Name: TC_OFH_MPLANE_SSH-CERTIFICATE-BASED_NACM_ACCESS_CONTROL

Purpose: The purpose of this test is to verify the SSH-certificate-based NACM access control on the Open FH interface between O-RU and O-DU.

Procedure and execution steps

Preconditions

- NACM with NETCONF is enabled and configured for SSH-certificate-based authorization on the O-RU Open FH interface.
- Access control rules and permissions are defined and configured on the O-RU.
- SSH is properly implemented and configured in the O-RU as defined in [2] clause 4.1.
- Test equipment (potentially an O-DU or a dedicated SSH client simulator) is configured to establish SSH connections to the O-RU.

Execution steps

1) Execute the test on the SSH protocol as defined in clause 6.2.
2) Positive Case: Successful SSH-certificate-based NACM authorization and access control.

- Test the successful enforcement of SSH-certificate-based NACM policies on the Open FH interface.
  - Establish an SSH connection using the SSH certificate.
  - Perform an operation on the Open FH interface with the O-RU using the SSH connection.
  - Verify that the O-RU grants or denies access based on the SSH-certificate-based NACM rules and permissions.

3) Negative Case: Unauthorized access denial.

- Test the denial of access to unauthorized operations on the Open FH interface, including attempts with invalid credentials and invalid usernames.
  - Attempt with invalid certificate
    - Attempt to establish an SSH connection using an invalid certificate.
    - Confirm that the O-RU denies the SSH connection due to invalid credentials.
  - Attempt with invalid username
    - Attempt to establish an SSH connection using a valid SSH certificate but with an invalid username.
    - Verify that the O-RU denies the SSH connection attempt due to the invalid username.

Expected Results

For step 1): Expected results in clause 6.2

For step 2):

- The SSH connection is successfully established using the SSH certificate.
- The O-RU evaluates the SSH certificate-based NACM rules and permissions.
- The O-RU grants or denies access to the O-DU based on the SSH-certificate-based NACM configuration.

For step 3):

- Denial of SSH connection due to invalid certificate.
- Denial of SSH connection due to an invalid username.

Expected format of evidence

For step 1): Logs showing adherence to SSH protocol specifications as defined in [2] clause 4.1.

For step 2), Logs indicating both successful access and access denial based on SSH-certificate-based NACM. In case the logs do not show the required information, the screenshots are used.

For step 3), Logs with error messages indicating access denial for unauthorized operations for both invalid credentials (certificate) and invalid usernames. In case the logs do not show the required information, the screenshots are used.

11.1.3.1.4 Open FH M-Plane SSH-certificate-based authentication authorization at O-DU

Requirement Name: M-Plane authenticity protection over Open FH interface using SSH

Requirement Reference: Clause 5.4.2 in O-RAN WG4 Management Plane Specification [21]

Requirement Description: Certificate based Authentication and Authorization for Open FH M-Plane over SSH across the Open FH interface at O-DU.

Threat References: T-O-RAN-05, T-FRHAUL-01, T-FRHAUL-02, T-MPLANE-01

DUT/s: O-DU

Test Name: TC_OFH_MPLANE_SSH-CERTIFICATE-BASED_AUTHENTICATION_AUTHORIZATION_O-DU

Purpose: The purpose of this test is to verify the SSH-certificate-based authentication and authorization mechanisms on the Open FH interface at O-DU, using test equipment to simulate O-RU.

Procedure and execution steps

NOTE: Test equipment simulates the role of O-RU for the purpose of this test.

Preconditions

- The O-DU is properly configured and operational.
- Test equipment capable of simulating SSH server functionality is prepared to represent the O-RU.
- SSH keys and certificates are generated and installed on both the O-DU and the test equipment.
- NACM with NETCONF is enabled and configured for authorization on the Open FH interface.
- SSH is properly implemented and configured as defined in [2] clause 4.1.

Execution steps

1) Execute the test on the SSH protocol as defined in clause 6.2.

Authentication of O-DU by O-RU (test equipment simulates O-RU)

2) Positive Case: Successful SSH-certificate-based authentication:

- Establish an SSH connection from the O-DU to the O-RU using the SSH certificate.
- Verify that the O-RU successfully authenticates the O-DU using the SSH certificate.

3) Negative Case: Failed SSH-certificate-based authentication.

- Test the handling of failed SSH-certificate-based authentication attempts by the O-DU in different scenarios.
  - Attempt with invalid certificate (Invalid due to public – private key mismatch)
    - Attempt to establish an SSH connection using an incorrect or invalid SSH certificate.

EXAMPLE 1: "Command: **ssh -i <path_to_invalid_private_key> -o CertificateFile=<path_to_invalid_certificate> <valid_username>@<O-DU_IP>**"

- Verify that the O-RU rejects the SSH connection due to the authentication failure.

- Attempt with invalid username
  - Attempt to establish an SSH connection using a valid SSH certificate, but with a username that does not exist in the O-RU's system.

EXAMPLE 2: "Command: **ssh -i <path_to_valid_private_key> -o CertificateFile=<path_to_valid_certificate> <invalid_username>@<O-DU_IP>**"

- Verify that the O-RU rejects the SSH connection, confirming that the system does not authenticate usernames that are not registered or recognized.

Expected Results

For step 1): Expected results as in clause 6.2

For step 2): Positive Case:

- The SSH connection is successfully established using the correct SSH certificate.

For step 3): Negative Case:

- The SSH connection attempt fails due to the incorrect or invalid SSH certificate.
- The DUT is denied SSH connection due to authentication failure.

Expected format of evidence

For step 1): Logs and screenshots showing adherence to SSH protocol specifications as defined in [2] clause 4.1.

For step 2): Positive Case: Logs showing successful SSH authentication and authorization events.

For step 3): Negative Case: Logs or error messages indicating failed SSH-certificate-based authentication attempts for both invalid certificate and non-existent username scenarios.

11.1.3.2 SSH-based M-Plane integrity, confidentiality and replay protection

Requirement Name: M-Plane confidentiality, integrity and replay protection over Open FH M-Plane interface using SSH

Requirement Reference: Clause 5.4 in O-RAN WG4 Management Plane Specifications [21]

Requirement Description:

Threat References: T-O-RAN-05, T-FRHAUL-01, T-FRHAUL-02, T-MPLANE-01

DUT/s: O-RU, O-DU

Test Name: TC_OFH_MPLANE_SSH_CONFIDENTIALITY_INTEGRITY_REPLAY

Purpose: To verify the enforcement of security policies over the Open FH M-Plane interface, ensuring that sensitive data remains protected through confidentiality, integrity, and replay protection using SSH.

Procedure and execution steps

Preconditions

- The O-RU and O-DU devices are properly configured and operational.
- SSH keys and certificates are generated and installed on both the O-RU and O-DU devices.
- SSH configuration is enabled to enforce confidentiality, integrity and replay protection on the Open FH M-plane interface.
- SSH is properly implemented and configured as defined in [2] clause 4.1.

Execution steps

1) Confidentiality verification:

- Establish an SSH connection between the O-RU and O-DU using proper SSH keys and certificates.
- Transmit data over this connection.
- Capture and analyse the transmitted data to verify encryption, ensuring confidentiality.

2) Integrity protection verification:

- During the same SSH session, modify the transmitted packets midway.
- Attempt to deliver the modified packets to the DUT.
- Verify that the DUT detects and discards these packets.

3) Replay protection verification:

- Replay previously captured packets to the DUT within the same SSH session.
- Confirm that the DUT detects and discards replayed packets.

Expected Results

- Confidentiality: All sensitive data transmitted over the Open FH M-Plane interface is encrypted, with no data exposed in clear text.
- Integrity protection: The DUT detects and discards altered packets, ensuring the data has not been tampered with.
- Replay protection: The DUT detects and discards replayed packets, preventing replay attacks.

Expected format of evidence

- Logs or screenshots showing SSH protocol adherence, as defined in the O-RAN Security Protocols Specifications [2] clause 4.1.
- Evidence of secure communication sessions established over the Open FH M-Plane interface, including details of encryption verification.
- Logs or screenshots showing the DUT's response to replayed and integrity-compromised packets, demonstrating the effectiveness of the security mechanisms in place.

11.1.3.3 TLS-based M-Plane authentication, authorization and access control protection

11.1.3.3.0 Introduction

The test cases outlined in this clause verify TLS-based Open FH M-Plane authenticity, NACM authorization, and access control protection.

11.1.3.3.1 Open FH M-plane TLS Authentication

Requirement Name: M-Plane authenticity protection over Open FH interface using TLS

Requirement Reference: Clause 5.4.3 in O-RAN WG4 Management Plane Specification [21]

Requirement Description:

Threat References: T-O-RAN-05, T-FRHAUL-01, T-FRHAUL-02, T-MPLANE-01

DUT/s: O-RU, O-DU

Test Name: TC_OFH_MPLANE_TLS_AUTHENTICATION

Purpose: Verify the TLS-based authentication mechanism for M-Plane over the Open FH interface by testing each DUT (O-RU or O-DU) independently. This test validates the correctness of the TLS handshake, certificate verification, and authentication process.

Procedure and execution steps

Preconditions

- Valid and invalid certificates are available for the testing.
- Testing equipment acting as a client (for O-RU as DUT) or server (for O-DU a DUT).
- For positive case: The DUT and the testing equipment are configured with valid TLS certificates for mutual authentication.
- For negative case: The DUT and the testing equipment are configured with invalid TLS certificates.

Execution steps

1) Execute the test on the TLS protocol as defined in clause 6.3.
2) Positive Case: Successful authentication.

- Test the successful authentication of the DUT over the Open FH M-Plane interface using TLS.
- For O-RU as the DUT
  - The test equipment initiates a TLS handshake with the DUT (NETCONF server).
  - The DUT presents its certificate to the client and validates the client certificate.
  - Observe and validate that the DUT verifies the client's certificate.
    - Check O-RU logs or use network monitoring tools to confirm certificate verification and presentation.
- For O-DU as the DUT
  - The DUT NETCONF client initiates a TLS handshake with the testing equipment (NETCONF server).
  - The DUT presents its certificate to the server and validates the server certificate.
  - Observe and validate that the DUT correctly verifies the server certificate.
    - Check DUT logs or use network monitoring tools to confirm certificate verification and presentation.

3) Negative Case: Failed authentication.

- Test the failure of authentication over the Open FH M-Plane interface due to invalid certificates.
- For O-RU as DUT:
  - The test equipment initiates a TLS handshake with the DUR, presenting an invalid certificate.
  - Observe and validate that the DUT rejects the invalid certificate.
    - Check DUT logs or network monitoring tools for certificate validation errors.
  - Confirm that the TLS handshake fails and mutual authentication is not completed.

Expected Results

For step 1): Expected results in clause 6.3

For step 2) positive case (successful authentication): The DUT successfully authenticates against the NETCONF client/server over the TLS-based Open FH M-Plane interface. The TLS handshake completes without errors, and both sides verify each other's certificates correctly.

For step 3), negative case (failed authentication) The DUT correctly rejects the invalid certificate. The TLS handshake fails, and mutual authentication is not established.

Expected format of evidence

For step 1): Logs showing adherence to TLS protocol specifications as defined in [2] clause 4.2. In case the logs do not show the required information, screenshots are used.

For step 2), Logs indicating successful authentication. In case the logs do not show the required information, screenshots are used.

For step 3), Logs indicating failed authentication. In case the logs do not show the required information, screenshots are used.

11.1.3.3.2 Open FH M-plane NACM Authorization

Requirement Name: M-Plane authorization and access control protection over Open FH interface using NACM

Requirement Reference: Clause 6.4.2 in O-RAN WG4 Management Plane Specification [21]

Requirement Description:

Threat References: T-O-RAN-05, T-FRHAUL-01, T-FRHAUL-02, T-MPLANE-01

DUT/s: O-RU

Test Name: TC_OFH_MPLANE_NACM_AUTHORIZATION

Purpose: Verify that NACM authorization policies are correctly enforced within the NETCONF session that has already been secured using TLS. This test ensures that properly authorized operations succeed and unauthorized operations are denied according to the configured NACM rules.

Procedure and execution steps

Preconditions

- The DUT (NETCONF server) and the testing equipment (NETCONF client) are mutually authenticated using mTLS.
- For positive case: The NACM rules and policies are properly configured on the NETCONF server to enforce authorization.
- For negative case: The NACM rules and policies are misconfigured denying the requested operation.

Execution steps

1) Positive Case: Successful authorization.

- Test the successful authorization of requests initiated by the O-RU Controller over the TLS-based NACM with NETCONF on the Open FH interface.
    - The testing equipment (NETCONF client) sends a NETCONF request to the DUT to perform an authorized operation.
    - The DUT evaluates the NACM rules and policies to determine if the testing equipment is authorized to perform the requested operation.
    - The DUT executes the authorized operation and sends a response to the testing equipment.

2) Negative Case: Failed authorization.

- Test the failure of authorization for unauthorized operations initiated by the testing equipment over the TLS-based NACM with NETCONF on the Open FH interface.
  - The testing equipment sends a NETCONF request to the DUT to perform an unauthorized operation.
  - The DUT (NETCONF server) evaluates the NACM rules and policies and denies the unauthorized operation.
  - The DUT rejects the unauthorized operation and sends an error response to the testing equipment.

Expected Results

For step 1) Positive case: successful authorization:

- The NETCONF request for an authorized operation is successfully received by the DUT.
- The DUT, after evaluating the NACM rules and policies, grants permission for the authorized operation.
- The DUT successfully executes the authorized operation and sends a confirmation response to the testing equipment.

For step 2) Negative case: failed authorization:

- The NETCONF request for an unauthorized operation is received by the DUT.
- The DUT, upon evaluating the NACM rules and policies, denies the unauthorized operation.
- The DUT does not execute the unauthorized operation and sends an error response to the testing equipment, indicating the rejection.

Expected format of evidence

For step 1), Logs containing the successful authorization. In case the logs do not show the required information, screenshots are used.

For step 2), Logs containing the failed authorization and rejection of the unauthorized operation. In case the logs do not show the required information, screenshots are used.

11.1.3.4 TLS-based M-Plane integrity, confidentiality and replay protection

Requirement Name: M-Plane confidentiality, integrity, and replay protection over Open FH M-plane interface using TLS

Requirement Reference: Clause 5.4 in O-RAN WG4 Management Plane Specifications [21]

Requirement Description:

Threat References: T-O-RAN-05, T-FRHAUL-01, 02, T-MPLANE-01

DUT/s: O-RU, O-DU

Test Name: TC_OFH_MPLANE_TLS_CONFIDENTIALITY_INTEGRITY_REPLAY

Purpose: To verify the confidentiality, integrity, and replay protection of Open FH M-Plane data using TLS.

Procedure and execution steps

Preconditions

- O-RU, O-DU support TLS and are connected in a simulated or real network environment.
- The Open FH M-Plane interface is configured for testing.
- TLS is properly implemented and configured as defined in [2] clause 4.2.

Execution steps

1) Confidentiality verification:

- Establish a secure communication session over the Open FH M-Plane interface.
- Capture the network traffic during the session.
- Analyse the captured traffic to verify that all data is encrypted, ensuring confidentiality.

2) Integrity protection verification:

- Capture protected packets after the TLS handshake.
- Modify the captured packets.
- Inject the modified packets into the DUT.
- Confirm that the DUT discards the injected packets, e.g., does not deliver it to the higher layer.

3) Replay protection verification:

- Capture protected packets after the TLS handshake.
- Replay the captured packets into the DUT.
- Confirm that the DUT discards the replayed packets.

Expected results

- Confidentiality: All data transmitted over the Open FH M-Plane interface is encrypted, with no data exposed in clear text.
- Integrity protection: The DUT detects and discards altered packets, ensuring data has not been tampered with.
- Replay protection: The DUT detects and discards replayed packets, preventing replay attacks.

Expected format of evidence

The following evidence, in one or more formats as applicable, should be provided:

- Logs or screenshots showing TLS protocol adherence, as defined in the O-RAN Security Protocols Specifications [2] clause 4.2.
- Logs or screenshots of the encrypted packets delivered to each TLS endpoint on the Open FH M-Plane.
- Logs or screenshots showing the DUT's response to replayed and integrity-compromised packets, demonstrating the effectiveness of the security mechanisms in place.

11.1.4 U-Plane

11.1.4.1 U-Plane eCPRI Unexpected Input

11.1.4.1.0 Introduction

The test cases in this clause focus on the O-DU's capability to recognize, handle, and respond appropriately to such anomalies in user plane packets over the eCPRI. This includes scenarios where packets are malformed or when they present unexpected payload sizes.

11.1.4.1.1 Open FH U-Plane Malformed Packet

Requirement Name: Handling and rejection of malformed or invalid user plane packets

Requirement Reference: Clause 5.2.5.2.1 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-FRHAUL-01, T-FRHAUL-02, T-UPLANE-01

DUT/s: O-RU, O-DU

Test Name: TC_OFH_U-PLANE_MALFORMED_PACKET

Purpose: The purpose of this test is to verify the O-DU's ability to handle and reject malformed or invalid user plane packets.

Procedure and execution steps

Preconditions

A valid eCPRI connection between the O-RU and O-DU.

Execution steps

1) Generate a user plane packet with invalid or malformed data, such as incorrect headers, corrupted payload, or unsupported formats.
2) Transmit the malformed packet over the eCPRI.
3) Monitor the O-DU's response and behaviour.
4) Verify that the O-DU identifies and rejects the malformed packet.
5) Observe the impact on the O-DU, such as error messages, logging, or abnormal behaviour.

Expected Results

- The O-DU detects and rejects malformed or invalid user plane packets.
- It handles the rejection gracefully without affecting normal operation.
- Appropriate error messages or log entries are generated.

Expected Format of Evidence:

- Detailed execution logs of the performed steps.
- Logs containing the detection and rejection of the malformed packet. In case the logs do not show the required information, screenshots are used.

11.1.4.1.2 Open FH U-Plane Unexpected Payload Size

Requirement Name: Handling and rejection of malformed or invalid user plane packets

Requirement Reference: Clause 5.2.5.2.1 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-FRHAUL-01, T-FRHAUL-02, T-UPLANE-01

DUT/s: O-RU, O-DU

Test Name: TC_OFH_U-PLANE_UNEXPECTED_PAYLOAD_SIZE

Purpose: The purpose of this test is to verify the O-DU's ability to handle unexpected payload sizes in user plane packets.

Procedure and execution steps

Preconditions

A valid eCPRI connection between the O-RU and O-DU.

Execution steps

1) Generate a user plane packet with an unexpected payload size, exceeding the normal or allowed range.
2) Transmit the packet with the unexpected payload size over the eCPRI.
3) Monitor the O-DU's response and behaviour.
4) Verify that the O-DU detects the unexpected payload size and takes appropriate action.
5) Observe the impact on the O-DU, such as error handling, packet drops, or performance degradation.

Expected Results

- The O-DU detects and handles unexpected payload sizes in user plane packets.
- It either rejects the packet or handles it with appropriate error handling mechanisms.
- The O-DU maintains acceptable performance levels despite the unexpected payload size.

Expected Format of Evidence:

- Detailed execution logs of the performed steps.
- Logs containing the detection and handling of the unexpected payload size. In case the logs do not show the required information, screenshots are used.

11.1.5 S-Plane

11.1.5.0 Introduction

The tests outlined in this clause focus on PTP as a packet-based clock synchronization protocol, which is susceptible to various network-based attacks. The LLS-C1, LLS-C2, and LLS-C3 configurations are considered in this assessment, as they rely on external timeTransmitters for synchronization. In contrast, LLS-C4 features a local PRTC (Primary Reference Time Clock) embedded within the O-RU, which provides time synchronization internally to the O-RU. This local PRTC operates independently and does not rely on an external timeTransmitter, such as an O-DU or a PRTC/T-GM in the Open Fronthaul (Open FH), to maintain synchronization. Consequently, there is no external network communication path that could be exploited to disrupt synchronization in the O-RU. Additionally, while LLS-C4 may be vulnerable to GNSS-related attacks, such attacks are not considered in the current set of tests.

11.1.5.1 DoS Attack against a timeTransmitter

11.1.5.1.0 Introduction

The tests outlined in this clause evaluate the system's defence capabilities against DoS attacks targeting the timeTransmitter, especially in different LLS configurations.

11.1.5.1.1 DoS timeTransmitter LLS C1 C2 C3

Refer to the test case in clause 7.5.1 for test verification

11.1.5.1.2 DoS timeTransmitter LLS C4

NOTE: In LLS-C4, the local PRTC is embedded in the O-RU and provides time synchronization internally to the O-RU. This local PRTC does not depend on an external timeTransmitter, such as the O-DU or a PRTC/T-GM in Open FH, to maintain its synchronization. Therefore, there is no external network communication path where a DoS attack targeting a timeTransmitter could be launched.

11.1.5.2 Spoofing of timeTransmitters in the S-Plane

11.1.5.2.0 Introduction

The tests presented in this clause focus on assessing the system's defences against potential spoofing attacks on timeTransmitters. Specifically, these tests examine scenarios where attackers may try to impersonate or manipulate the timeTransmitter's communications to disrupt accurate time synchronization.

11.1.5.2.1 Impersonation timeTransmitter

Requirement Name: Spoofing Prevention for timeTransmitters in the S-Plane

Requirement Reference: REQ-SEC-OFSP-2, clause 5.2.5.3.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The S-Plane provides a means to prevent spoofing of timeTransmitters

Threat References: T-SPLANE-02, T-SPLANE-03

DUT/s: O-RU, O-DU

Test Name: TC_IMPERSONATION_TIMETRANSMITTER

Purpose: The purpose of this test is to verify the protection of the S-plane against an impersonation attack where an attacker sends fake ANNOUNCE messages to declare itself as the best clock (Grandmaster). The test evaluates whether the DUT correctly enforces vendor-implemented protection mechanisms to prevent time synchronization disruptions.

Procedure and execution steps

Preconditions

- The vendor documentation on the anti-spoofing mechanisms implemented for protecting the S-plane against timeTransmitter (EXAMPLE: 802.1X Port-based Network Access Control (PNAC), redundancy with multiple timeTransmitters, cryptographic integrity checks).
- The vendor detailed the test procedures they followed internally to validate their implementation, including the expected behavior and results of their security mechanisms under both normal and attack conditions.
- For LLS-C1: The timeTransmitter functionality is enabled on the O-DU. O-DU is acting as a timeTransmitter and directly synchronizes O-RU.
- For LLS-C2: One or more Ethernet switches are allowed in the Open Fronthaul network. O-DU acting as timeTransmitter to distribute network timing toward O-RU.
- For LLS-C3: One or more PRTC/T-GM are implemented in the Open Fronthaul network to distribute network timing toward O-DU and O-RU.
- A network monitoring tool is set up to capture and analyse network traffic.

Execution steps

1) Verify that the vendor documentation provides clear descriptions of security mechanisms for preventing timeTransmitter spoofing.
2) Use a network monitoring tool to observe normal time synchronization traffic.
3) Establish a baseline for comparison before testing.
4) Follow the vendor's test procedures to reproduce the vendor's internal tests.
5) Compare the obtained results with the documented results from the vendor to verify if the DUT behaves as expected.

6) Verify that the DUT rejects the impersonated clock and maintains the synchronization based on the legitimate timeTransmitter.

7) If gaps are found in vendor-provided tests, propose further test steps to enhance security validation.

Expected Results

- The vendor's test cases are sufficient, and security mechanisms perform as expected.
- The DUT detects and mitigates the impersonation attack.
- The synchronization status remains stable and accurate.
- The actual test results match the vendor's documented expected results.
- If insufficient tests are found:
  - The tester documents missing areas and recommends further test procedures.
  - The vendor may be required to implement additional security measures.

Expected Format of Evidence:

- Summary of the vendor's provided test procedures and security mechanisms.
- Logs showing successful execution of vendor tests.
- Statement confirming whether vendor tests are adequate or insufficient.
- Recommendations for additional security validations (if necessary).

11.1.5.2.2 Rogue PTP Instance

Requirement Name: Spoofing Prevention for timeTransmitter in the S-Plane

Requirement Reference: REQ-SEC-OFSP-2, clause 5.2.5.3.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The S-Plane provides a means to prevent spoofing of master clocks

Threat References: T-SPLANE-02, T-SPLANE-03

DUT/s: O-RU, O-DU

Test Name: TC_ROGUE_PTP_INSTANCE

Purpose: The purpose of this test is to verify the protection of the S-plane against an attacker sending manipulated or malicious ANNOUNCE messages to declare itself as the best clock (Grandmaster). The test evaluates whether the DUT correctly enforces vendor-implemented protection mechanisms to prevent time synchronization disruptions.

Procedure and execution steps

Preconditions

- The vendor documentation describes the security mechanisms implemented to prevent rogue PTP instances.
- The vendor details the test procedures they followed to validate their implementation, including the expected behaviour and results of their security mechanisms under both normal and attack conditions.
- For LLS-C1: The timeTransmitter functionality is enabled on the O-DU. O-DU is acting as a timeTransmitter and directly synchronizes O-RU.
- For LLS-C2: One or more Ethernet switches are allowed in the Open Fronthaul network. O-DU acting as timeTransmitter to distribute network timing toward O-RU.

- For LLS-C3: One or more PRTC/T-GM are implemented in the Open Fronthaul network to distribute network timing toward O-DU and O-RU.
- A network monitoring tool is set up to capture and analyse network traffic.

Execution steps

1) Verify that the vendor documentation provides clear descriptions of security mechanisms for preventing rogue PTP instances.
2) Use a network monitoring tool to observe normal time synchronization traffic.
3) Establish a baseline for comparison before testing.
4) Follow the vendor's test procedures to reproduce the vendor's internal tests.
5) Compare the obtained results with the documented results from the vendor to verify if the DUT behaves as expected.
6) Verify that the DUT detects and rejects the attacker's proposed Grandmaster candidate.
7) If gaps are found in vendor-provided tests, propose further test steps to enhance security validation.

Expected Results

- The vendor’s test cases are sufficient, and security mechanisms perform as expected.
- The DUT detects and rejects rogue PTP messages.
- The synchronization remains stable and accurate.
- The actual test results match the vendor’s documented expected results.
- If insufficient tests are found:
  - The tester documents missing areas and recommends further test procedures.
  - The vendor may be required to implement additional security measures.

Expected Format of Evidence:

- Summary of the vendor’s provided test procedures and security mechanisms.
- Logs showing successful execution of vendor tests.
- Statement confirming whether vendor tests are adequate or insufficient.
- Recommendations for additional security validations (if necessary).

11.1.5.3 Clock Accuracy Protection Against MITM Attacks

11.1.5.3.1 Selective Interception and Removal of PTP Timing Packets

Requirement Name: Clock Accuracy Protection Against MITM Attacks

Requirement Reference: REQ-SEC-OFSP-3, clause 5.2.5.3.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-SPLANE-04, T-SPLANE-05

DUT/s: O-RU, O-DU

Test Name: TC_SELECTIVE_INTERCEPTION_REMOVAL_PTP_TIMING_PACKETS

Purpose: The purpose of this test is to verify the resilience of the S-plane against an attack where PTP timing packets are selectively intercepted and removed.

Procedure and execution steps

Preconditions

- For LLS-C2: One or more Ethernet switches are allowed in the Open Fronthaul network. O-DU acting as master to distribute network timing toward O-RU.
- For LLS-C3: One or more PRTC/T-GM are implemented in the Open Fronthaul network to distribute network timing toward O-DU and O-RU.
- The network monitoring tool is set up to capture and analyse network traffic between the O-RU and O-DU.

Execution steps

1) Set up the test environment with the O-RAN O-RU, O-DU, and other relevant network components.
2) Configure the network monitoring tool to capture PTP timing packets between the O-RU and O-DU.
3) Start the network monitoring tool to capture the initial state of PTP timing packets.
4) Simulate an attack by using a packet manipulation tool to selectively intercept and remove specific PTP timing packets.

- For LLS-C2, use a network device or software with packet interception capabilities to intercept PTP timing packets between the O-RU and O-DU. Modify the intercepted packets to selectively remove PTP timing packets.
- For LLS-C3, use a network device or software capable of deep packet inspection (DPI) to intercept and analyse PTP timing packets. Modify the intercepted packets to selectively remove PTP timing packets.

5) Verify the functionality of the O-RU and O-DU during the attack simulation.
6) Observe the synchronization status and the impact on timing accuracy between the O-RU and O-DU.
7) Capture and analyse the network traffic using the network monitoring tool during the attack simulation.

NOTE: The network monitoring tool can be Wireshark or tcpdump, configured to capture packets on the interfaces between the O-RU, O-DU and to identify the intercepted and removed PTP timing packets.

8) Stop the network monitoring tool to finalize the captured traffic.

Expected Results

- Detection of missing PTP timing packets: The S-plane is able to detect the absence of specific PTP timing packets that were selectively intercepted and removed.
- Synchronization maintenance: Despite the missing PTP timing packets, the O-RU and O-DU still maintain synchronization. Any deviations from expected synchronization are minimal and within acceptable thresholds.
- Corrective actions: Upon detecting the missing PTP timing packets, the O-RU and O-DU initiate predefined corrective actions to restore synchronization and mitigate the effects of the missing packets.
- Network traffic analysis: The captured network traffic clearly shows the instances where specific PTP timing packets were intercepted and removed.
- No system failures: The system (O-RU and O-DU) doesn't experience any catastrophic failures or shutdowns due to the missing PTP timing packets.

Expected Format of Evidence:

The following evidence, in one or more formats as applicable, should be provided for each configuration (LLS-C2, LLS-C3):

- Recorded network traffic captured by the monitoring tool during the attack simulation showing selective interception and removal of PTP timing packets in LLS-C2 (with Ethernet switches) and LLS-C3 (with PRTC/T-GM).
- Observations and analysis of the impact on synchronization and timing accuracy.
- Any issues or anomalies encountered during the attack simulation.

11.1.5.3.2 Delay Attack on PTP Timing Packets

Requirement Name: Clock Accuracy Protection Against MITM Attacks

Requirement Reference: REQ-SEC-OFSP-3, clause 5.2.5.3.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-SPLANE-04, T-SPLANE-05

DUT/s: O-RU, O-DU

Test Name: TC_DELAY_ATTACK_PTP_TIMING_PACKETS

Purpose: The purpose of this test is to verify the S-plane's resilience against a delay attack on PTP timing packets.

Procedure and execution steps

Preconditions

- For LLS-C2: One or more Ethernet switches are allowed in the Open Fronthaul network. O-DU acting as master to distribute network timing toward O-RU.
- For LLS-C3: One or more PRTC/T-GM are implemented in the Open Fronthaul network to distribute network timing toward O-DU and O-RU.
- Time synchronization is established and operational within the network.

Execution steps

1) Start the network monitoring tool to capture the initial state of PTP timing packets.
2) Simulate an attack by introducing delays in PTP timing packets using a network emulation tool.

- For LLS-C2 and LLS-C3, use a custom script or tool that supports packet manipulation and delay to introduce artificial delays in PTP timing packets between the O-RU and O-DU or between PRTC/T-GM devices.

3) Verify the functionality of the O-RU and O-DU during the delay attack on PTP timing packets.
4) Observe the synchronization status and timing accuracy within the LLS configuration.

Expected Results

- The S-plane detects the delay attack on PTP timing packets and applies appropriate measures to mitigate the impact within all LLS configurations.
- The O-RU and O-DU detects the delayed PTP timing packets, compensate for the introduced delays, and maintain synchronization.

Expected Format of Evidence:

The following evidence, in one or more formats as applicable, should be provided for each configuration (LLS-C2, LLS-C3):

- Recorded network traffic captured by the monitoring tool during the attack. This includes logs showing the introduction of delays in PTP timing packets for LLS-C2 (with Ethernet switches) and LLS-C3 (with PRTC/T-GM).
- Observations and analysis of the impact on synchronization and timing accuracy within each LLS configuration.
- Any issues or anomalies encountered during the attack simulation.

11.2 Y1

11.2.1 Y1 Authenticity

Requirement Name: Y1 protection in terms of authenticity

Requirement Reference-NEAR-RT-9, clause 5.1.3.2.4, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-Y1-01, T-Y1-02, T-Y1-03

DUT/s: Near-RT RIC, Y1 consumers

Test Name: TC_Y1_AUTHENTICATION

Purpose: The purpose of this test is to verify the authenticity of the Y1 interface, ensuring that only legitimate and mutually authenticated Near-RT RIC, Y1 consumers can participate in the communication over the Y1 interface.

Procedure and execution steps

Preconditions

- Near-RT RIC & Y1 Consumers support mTLS and be connected in a simulated/real network environment.
- The test environment is set up with the Y1 interface configured.
- The tester has access to the original data transported over the Y1 interface.
- mTLS is properly implemented and configured as defined in [2] clause 4.2.

Execution steps

1) Execute the test on the mTLS protocol as defined in clause 6.3.
2) Valid Authentication Certificates (positive case):

- The tester sends a request to establish a connection with the Y1 interface using valid authentication certificates.
- The tester verifies the mutual certificate verification between Near-RT RIC and Y1 consumers.
- The tester captures and analyses the response received from the Y1 interface.

3) Invalid Authentication Certificates (negative case):

- The tester sends a request to establish a connection with the Y1 interface with invalid certificates.
- The tester captures and analyses the response received from the Y1 interface.

4) No Authentication Certificates (Negative Case):

- The tester sends a request to establish a connection without any certificates.
- The tester captures and analyses the response from the Y1 interface.

Expected results

For 1) Expected results in clause 6.3

For 2) 'Valid Authentication Certificates': The Y1 interface accepts the valid certificates and responds with a successful authentication message. The mutual certificate verification process is successful.

For 3) 'Invalid Authentication Certificates': The connection attempt is rejected, and an authentication failure message is received. The mutual certificate verification process fails due to the use of invalid certificates.

For 4) 'No Authentication Certificates': The connection attempt is rejected, and an authentication failure message is received. The mutual certificate verification process fails due to the absence of certificates.

Expected Format of Evidence:

The following evidence, in one or more formats as applicable, should be provided:

- Logs and screenshots showing adherence to mTLS protocol specifications as defined in [2] clause 4.2.
- Logs of authentication requests and responses on the Y1 interface
- Logs of the mutual certificate verification process.
- Screenshots or logs of error messages or unusual behaviours for both invalid and no certificate scenarios.

11.2.2 Y1 confidentiality, integrity, and replay protection

Requirement Name: Y1 protection in terms of confidentiality, integrity and replay

Requirement Reference: SEC-CTL-NEAR-RT-11, clause 5.1.3.2.4, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-Y1-01, T-Y1-02, T-Y1-03

DUT/s: Near-RT RIC, Y1 consumers

Test Name: TC_Y1_CONFIDENTIALITY_INTEGRITY_REPLAY

Purpose: To verify the enforcement of security policies over the Y1 interface, ensuring that sensitive data remains protected through confidentiality, integrity, and replay protection.

Procedure and execution steps

Preconditions

- Near-RT RIC and Y1 consumers support TLS and connected within simulated or real network environments.
- The Y1 interface is configured for testing.
- TLS is properly implemented and configured as defined in [2] clause 4.2.

Execution steps

1) Confidentiality verification:

- Establish a secure communication session over the Y1 interface.
- Capture the network traffic during the session.
- Analyse the captured traffic to verify that all data is encrypted, ensuring confidentiality.

2) Integrity protection verification:

- Capture protected packets after the TLS handshake.

- Modify the captured packets.
- Inject the modified packets to the DUT.
- Confirm that the DUT discards the injected packets, e.g., does not deliver it to the higher layer.

3) Replay protection verification:

- Capture protected packets after the TLS handshake.
- Replay the captured packets to the DUT.
- Confirm that the DUT discards the replayed packets.

Expected results

- Confidentiality: All sensitive data transmitted over the Y1 interface is encrypted, with no data exposed in clear text.
- Integrity protection: The DUT detects and discards altered packets, ensuring data has not been tampered with.
- Replay protection: The DUT detects and discards replayed packets, preventing replay attacks.

Expected Format of Evidence:

The following evidence, in one or more formats as applicable, should be provided:

- Logs showing TLS protocol adherence, as defined in the O-RAN Security Protocols Specifications [2] clause 4.2. In case the logs do not show the required information, screenshots are used.
- Logs with the evidence of secure communication sessions established over the Y1 interface, including details of encryption verification.
- Logs showing the DUT's response to replayed and integrity-compromised packets, demonstrating the effectiveness of the security mechanisms in place. In case the logs do not show the required information, screenshots are used.

11.2.3 Y1 Authorization

Requirement Name: Y1 protection in terms of authorization

Requirement Reference: SEC-CTL-NEAR-RT-10, clause 5.1.3.2.4, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Near-RT RIC Y1 Interface supports OAuth 2.0 based authorization.

Threat References: T-Y1-01

DUT/s: Near-RT RIC

Test Name: TC_Y1_AUTHORIZATION

Purpose: The purpose of this test is to validate that the Y1 interface enforces an authorization mechanism to prevent unauthorized access.

Procedure and execution steps

Preconditions

- Near-RT RIC Y1 Interface supports OAuth 2.0 and is the Resource Server
- Y1 Consumer may be real or simulated
- Test 6.6.2 Scenario:

- Preconditions enabled as specified in Clause 6.6.2 (to validate DUT as O-Auth2.0 supported Resource Server)
- Y1 Consumer configured with the permitted role to have access to the resource to be requested

- Not permitted role:
  - Y1 Consumer configured with a role, that does not permit access to the resource
- Invalid Access Token Scenario:
  - Y1 Consumer is configured to present an invalid access token
- No Access Token Scenario:
  - Y1 Consumer is configured to present no access token

Execution steps

1) Execute the test to verify OAuth 2.0 enabled Resource Server as defined in clause 6.6.2. This test includes the check for Y1 consumer access with Valid access tokens (Positive Case)

- Verify the validation required for valid access token as defined in clause 6.6.2

2) Not permitted role (negative case):

- The Y1 Consumer sends a request directly to the resource server to access protected resources using a valid access token, however the role does not permit access to the resource.
- Verify that Resource Server rejects access to the requested resource as Consumer does not have permitted role.

3) Invalid access tokens (negative case):

- The Y1 Consumer sends a request directly to the resource server to access protected resources using an incorrect access token.
- Verify that Resource Server rejects access to the requested resource due to incorrect access token.

4) No access tokens (negative case):

- The Y1 Consumer sends a request directly to the resource server to access protected resources without providing any access token.
- Verify that Resource Server rejects access to the requested resource without an access token.

Expected Results

For 1) Expected results as in clause 6.6.2

For 2) Not permitted role: The DUT verifies the role of the Y1 consumer for the requested resource, as not allowed/incorrect, and the access is rejected, and an access failure message is generated.

For 3) Incorrect access tokens: The DUT verifies the access token, and token being invalid, the access is rejected, and an access failure message is generated.

For 4) No access tokens: The DUT rejects the access due to the absence of tokens, and an appropriate error or unauthorized access message is generated.

Expected Format of Evidence:

The following evidence, in one or more formats as applicable, should be provided:

- Logs of the request sent to access protected resources using valid access tokens.
- Logs highlighting the successful authorization message. In case the logs do not show the required information, screenshots are used.

- Logs of the request sent to access protected resources when role is not permitted, when using invalid access tokens and no access token.
- Logs showing the rejection of the access and the access failure message when consumer role is not permitted, consumer configured to present invalid access token and consumer configured to present no access token are used. In case the logs do not show the required information, screenshots are used.

11.3 O1

11.3.1 O1 Authenticity

Requirement Name: O1 protection in terms of authenticity

Requirement Reference: SEC-CTL-O1-2, SEC-CTL-O1-5, clause 5.2.2.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-MPLANE-01, T-O-RAN-05

DUT/s: SMO, O-CU, O-DU, Near-RT RIC

Test Name: TC_O1_AUTHENTICATION

Purpose: The purpose of this test is to verify the authenticity of the O1 interface, ensuring that only legitimate and authenticated O-RAN NFs can participate in the communication over the O1 interface.

Procedure and execution steps

Preconditions

- SMO, O-CU, O-DU, Near-RT RIC support mTLS and be connected in simulated/real network environment.
- The test environment is set up with O1 interface configured.
- The tester has access to the original data transported over the O1 interface.
- mTLS is properly implemented and configured as defined in [2] clause 4.2.

Execution steps

1) Execute the test on the mTLS protocol as defined in clause 6.3.
2) Valid Authentication Certificates (positive case):

- The tester sends a request to establish a connection with the O1 interface using valid authentication certificates.
- The tester verifies the mutual certificate verification between the ORAN NFs
- The tester captures and analyses the response received from the O1 interface.

3) Invalid Authentication Certificates (negative case):

- The tester sends a request to establish a connection with the O1 interface with invalid certificates.
- The tester captures and analyses the response received from the O1 interface.

4) No Authentication Certificates (Negative Case):

- The tester sends a request to establish a connection without any certificates.
- The tester captures and analyses the response from the O1 interface.

Expected results

For 1) Expected results in clause 6.3

For 2) 'Valid Authentication Certificates': The O1 interface accepts the valid certificates and responds with a successful authentication message. The mutual certificate verification process is successful.

For 3) 'Invalid Authentication Certificates': The connection attempt is rejected, and an authentication failure message is received. The mutual certificate verification process fails due to the use of invalid certificates.

For 4) 'No Authentication Certificates': The connection attempt is rejected, and an authentication failure message is received. The mutual certificate verification process fails due to the absence of certificates.

Expected Format of Evidence:

The following evidence, in one or more formats as applicable, should be provided:

- Logs and screenshots showing adherence to mTLS protocol specifications as defined in [2] clause 4.2.
- Logs of authentication requests sent to the O1 interface.
- Logs of the mutual certificate verification process.
- Screenshots or logs of error messages or unusual behaviours for both invalid and no certificate scenarios.

11.3.2 O1 confidentiality, integrity and replay protection

Requirement Name: O1 protection in terms of confidentiality, integrity and replay

Requirement Reference:SEC-CTL-O1-1, SEC-CTL-O1-4, clause 5.2.2.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-MPLANE-01, T-O-RAN-05

DUT/s: SMO, O-CU, O-DU, Near-RT RIC

Test Name: TC_O1_CONFIDENTIALITY_INTEGRITY_REPLAY

Purpose: To verify the enforcement of security policies over the O1 interface, ensuring that sensitive data remains protected through confidentiality, integrity, and replay protection.

Procedure and execution steps

Preconditions

- SMO, O-CU, O-DU, Near-RT RIC support TLS and be connected in simulated/real network environment.
- The O1 interface is configured for testing.
- TLS is properly implemented and configured as defined in [2] clause 4.2.

Execution steps

1) Confidentiality verification:

- Establish a secure communication session over the O1 interface.
- Capture the network traffic during the session.
- Analyse the captured traffic to verify that all data is encrypted, ensuring confidentiality.

2) Integrity protection verification:

- Capture protected packets after the TLS handshake.

- Modify the captured packets.
- Inject the modified packets to the DUT.
- Confirm that the DUT discards the injected packets, e.g., does not deliver it to the higher layer.

3) Replay protection verification:

- Capture protected packets after the TLS handshake.
- Replay the captured packets to the DUT.
- Confirm that the DUT discards the replayed packets.

Expected results

- Confidentiality: All sensitive data transmitted over the O1 interface is encrypted, with no data exposed in clear text.
- Integrity protection: The DUT detects and discards altered packets, ensuring data has not been tampered with.
- Replay protection: The DUT detects and discards replayed packets, preventing replay attacks.

Expected Format of Evidence:

The following evidence, in one or more formats as applicable, should be provided:

- Logs or screenshots showing TLS protocol adherence, as defined in the O-RAN Security Protocols Specifications [2] clause 4.2.
- Evidence of secure communication sessions established over the O1 interface, including details of encryption verification.
- Logs or screenshots showing the DUT's response to replayed and integrity-compromised packets, demonstrating the effectiveness of the security mechanisms in place.

11.3.3 O1 Interface Network Configuration Access Control Model (NACM) Validation

11.3.3.0 Introduction

Following zero trust principles, O-RAN O1 interface shall enforce confidentiality, integrity and authenticity through an encrypted transport, and shall support least privilege access control using the network configuration access control model. The network configuration access control model (NACM) [14] provides the means to restrict access for users to a preconfigured subset of all available NETCONF protocol operations and content.

The security test case in this clause validates the NACM enforcement on the O-RAN architecture element O1 interface for the role-based access control.

11.3.3.1 O1 Interface NACM Validation

Requirement Name: O1 Interface security requirements

Requirement Reference: REQ-NAC-FUN-1 to REQ-NAC-FUN-10, clause 5.2.2.3, O-RAN Security Requirements and Controls Specifications [5]

Threat References: T-O-RAN-02, T-O-RAN-06

Requirement Description: Requirements of O1 Interface Confidentiality, Integrity & Authenticity protection and Least Privilege Access Control

DUT/s: Non-RT RIC, Near-RT RIC, O-CU-CP, O-CU-UP, O-DU

Test Name: TC_O1_NACM_VALIDATION

Purpose: O-RAN architecture elements managed by SMO through O1 interface shall support secured NETCONF sessions over TLS and role-based least privilege access control enforced by NACM [14]. This test validates the O1 interface security requirements of the O-RAN architecture elements with the focus on role-based NACM rule(s) set enforcement.

Procedure and execution steps

Preconditions

DUT shall be the O-RAN architecture element with:

- IP enabled O1 interface, reachable from the authentication server
- Valid certificate loaded for the server and necessary certificate authorities (CAs)
- Client's root CA required to validate NETCONF client certificate
- Valid TLS Client-to-NETCONF username mapping
- Configure the O-RAN element with the SMO details (SMO network address and port)

Execution steps

First set up a host/device with TLS client software installed, valid client certificates, keys, root CA certificate for the server (O-RAN architecture element), and all intermediate CA certificates required to validate the client certificate.

The following test steps shall be validated:

1) Initiate NETCONF call home procedure from the O-RAN element towards SMO over O1 interface.

NOTE: The O-RAN element may initiate the NETCONF call home procedure as part of its initialization automatically.

2) SMO connects with O-RAN element over O1 interface using TLS 1.2 or TLS 1.3 - if available with a user account from the O1_nacm_management group

3) Verify the session is established and mapped to the correct NETCONF user

4) Verify the global NACM enforcement control setting of

- enable-nacm = true
- read-default = permit
- write-default = deny
- exec-default = deny
- enable-external-groups = true

5) Verify the NACM rule sets for the following pre-defined groups

- O1_nacm_management
- O1_user_management
- O1_network_management
- O1_network_monitoring
- O1_software_management for only PNFs

6) Close the NETCONF session and TLS connection

Upon availability of the NETCONF operations set(s) definition per NACM group, the NACM rule set(s) enforcement by the DUT shall be validated for each of those pre-defined groups listed above.

Expected results

The O-RAN architecture element supports the NETCONF over TLS session over its O1 interface and NACM enforcement control settings.

Expected format of evidence:

Logs or screenshots showing:

- O1 interface setup.
- Valid server certificate and CA details.
- Client's root CA and intermediate CA certificates.
- TLS Client-to-NETCONF username mapping.
- O-RAN element configured with SMO details.
- Initiation of NETCONF call home procedure.
- TLS 1.2 or TLS 1.3 connection establishment.
- Correct NETCONF user session mapping.

11.4 O2

11.4.1 O2 Authenticity

Requirement Name: O2 protection in terms of authenticity

Requirement Reference: SEC-CTL-O-CLOUD-INTERFACE-3, clause 5.1.8.9.2, O-RAN Security Requirements and Controls Specifications [5]

Threat References: T-O2-01

DUT/s: SMO, O-Cloud

Test Name: TC_O2_AUTHENTICATION

Purpose: The purpose of this test is to verify the authenticity of the O2 interface, ensuring that only legitimate and authenticated O-Cloud and SMO can participate in the communication over the O2 interface.

Procedure and execution steps

Preconditions

- O-Cloud and SMO support mTLS and be connected in simulated/real network environment.
- The test environment is set up with O2 interface configured.
- The tester has access to the original data transported over the O2 interface.
- mTLS is properly implemented and configured as defined in [2] clause 4.2.

Execution steps

1) Executes the tests on the mTLS protocol as defined in clause 6.3
2) Valid Authentication Certificates (positive case):

- The tester sends a request to establish a connection with the O2 interface using valid authentication certificates.
- The tester verifies the mutual certificate verification between the ORAN NFs.
- The tester captures and analyses the response from the O2 interface.

3) Invalid Authentication Certificates (negative case):

- The tester sends a request to establish a connection with the O2 interface with invalid certificates.
- The tester captures and analyses the response from the O2 interface.

4) No Authentication Certificates (negative case):

- The tester sends a request to establish a connection without any certificates.
- The tester captures and analyses the response from the O2 interface.

Expected results

For 1) Expected results in clause 6.3

For 2) 'Valid Authentication Certificates': The O2 interface accepts the valid certificates and respond with a successful authentication message.

For 3) 'Invalid Authentication Certificates': The connection is rejected, and an authentication failure message is received. The mutual certificate verification process fails due to the use of invalid certificates.

For 4) 'No Authentication Certificates': The connection attempt is rejected, and an authentication failure message is received. The mutual certificate verification process fails due to the absence of certificates.

Expected Format of Evidence:

The following evidence, in one or more formats as applicable, should be provided:

- Logs and screenshots showing adherence to mTLS protocol specifications as defined in [2] clause 4.2.
- Logs of authentication requests and responses on the O2 interface.
- Logs of the mutual certificate verification process.
- Screenshots or logs of error messages or unusual behaviours for both invalid and no certificate scenarios.

11.4.2 O2 confidentiality, integrity and replay protection

Requirement Name: O2 protection in terms of confidentiality, integrity and replay

Requirement Reference: SEC-CTL-O-CLOUD-INTERFACE-1, clause 5.1.8.9.2, O-RAN Security Requirements and Controls Specifications [5]

Threat References: T-O2-01

DUT/s: SMO, O-Cloud

Test Name: TC_O2_CONFIDENTIALITY_INTEGRITY_REPLAY

Purpose: To verify the enforcement of security policies over the O2 interface, ensuring that sensitive data remains protected through confidentiality, integrity, and replay protection.

Procedure and execution steps

Preconditions

- O-Cloud and SMO support TLS and be connected in simulated/real network environment.

- The O2 interface is configured for testing.
- TLS is properly implemented and configured as defined in [2] clause 4.2.

Execution steps

1) Confidentiality verification:

- Establish a secure communication session over the O2 interface.
- Capture the network traffic during the session.
- Analyse the captured traffic to verify that all data is encrypted, ensuring confidentiality.

2) Integrity protection verification:

- Capture protected packets after the TLS handshake.
- Modify the captured packets.
- Inject the modified packets to the DUT.
- Confirm that the DUT discards the injected packets, e.g., does not deliver it to the higher layer.

3) Replay protection verification:

- Capture protected packets after the TLS handshake.
- Replay the captured packets to the DUT.
- Confirm that the DUT discards the replayed packets.

Expected results

- Confidentiality: All sensitive data transmitted over the O2 interface is encrypted, with no data exposed in clear text.
- Integrity protection: The DUT detects and discards altered packets, ensuring data has not been tampered with.
- Replay protection: The DUT detects and discards replayed packets, preventing replay attacks.

Expected Format of Evidence:

The following evidence, in one or more formats as applicable, should be provided:

- Logs or screenshots showing TLS protocol adherence, as defined in the O-RAN Security Protocols Specifications [2] clause 4.2.
- Evidence of secure communication sessions established over the O2 interface, including details of encryption verification.
- Logs or screenshots showing the DUT's response to replayed and integrity-compromised packets, demonstrating the effectiveness of the security mechanisms in place.

11.4.3 O2 Authorization

Requirement Name: O2 protection in terms of authorization

Requirement Reference: SEC-CTL-O-CLOUD-INTERFACE-2, clause 5.1.8.9.2, O-RAN Security Requirements and Controls Specifications [5]

Threat References: T-O2-01

DUT/s: SMO, O-Cloud

Test Name: TC_O2_AUTHORIZATION

Purpose: The purpose of this test is to validate that the O2 interface enforces an authorization mechanism to prevent unauthorized access.

Procedure and execution steps

Preconditions

- O-Cloud and SMO support OAuth 2.0 and are connected in simulated/real network environment.
- The test environment is set up with O2 interface configured.
- The tester has access to the original data transported over the O2 interface.
- OAuth 2.0 is properly implemented and configured.

Execution steps

1) Execute the tests on the OAuth 2.0 protocol as defined in clause 6.6
2) Valid access tokens (positive case):

- The tester sends a request to access protected resources using a valid access token.
- The tester captures and analyses the response from the O2 interface.

3) Invalid access tokens (negative case):

- The tester sends a request to access protected resources using an invalid or incorrect access token.
- The tester captures and analyses the response from the O2 interface.

4) No access tokens (negative case):

- The tester sends a request to access protected resources without providing any access token.
- The tester captures and analyses the response from the O2 interface.

Expected results

For 1). Expected results in clause 6.6

For 2) 'Valid access tokens': The O2 interface accepts the valid access tokens and responds with a successful authorization message.

For 3) 'Invalid access tokens': The access is rejected, and an access failure message is received.

For 4) 'No access tokens': The access is rejected due to the absence of tokens, and an appropriate error or unauthorized access message is received.

Expected Format of Evidence:

The following evidence, in one or more formats as applicable, should be provided:

- Logs of the request sent to access protected resources using valid access tokens.
- Screenshots or logs highlighting the successful authorization message.
- Logs of the request sent to access protected resources using invalid or incorrect access tokens.
- Screenshots or logs showing the rejection of the access and the access failure message.

11.5 E2

11.5.1 E2 confidentiality, integrity and replay protection

Requirement Name: E2 protection in terms of confidentiality, integrity, and replay

Requirement Reference: SEC-CTL-E2, clause 5.2.4.2, SEC-CTL-NEAR-RT-2, SEC-CTL-NEAR-RT-7, clause 5.1.3.2.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-O-RAN-05, T-NEAR-RT-01, T-NEAR-RT-02, T-NEAR-RT-03, T-NEAR-RT-04

DUT/s: O-CU, O-DU, Near-RT RIC

Test Name: TC_E2_CONFIDENTIALITY_INTEGRITY_REPLAY

Purpose: To verify the enforcement of security policies over the E2 interface, ensuring that sensitive data remains protected through confidentiality, integrity, and replay protection

Procedure and execution steps

Preconditions

- Near-RT RIC and E2 nodes support IPsec and are connected in simulated/real network environment.
- The E2 interface is configured for testing.
- The tunnel mode IPsec ESP and IKE certificate authentication is implemented.
- The tester shall base the test on the profile defined in [2] clause 4.5.

Execution steps

1) Confidentiality verification:

- Establish a secure communication session using IPsec over the E2 interface.
- Capture the network traffic during the session.
- Analyse the captured traffic to verify that all data is encrypted, ensuring confidentiality.

2) Integrity protection verification:

- Capture traffic during a secure session.
- Modify the captured packets.
- Inject the modified packets to the DUT.
- Confirm that the DUT discards the injected packets, e.g., does not deliver it to the higher layer.

3) Replay protection verification:

- Capture protected packets during a secure session and attempt to replay them.
- Replay the captured packets to the DUT.
- Confirm that the DUT discards the replayed packets.

Expected Results

- Confidentiality: All sensitive data transmitted over the E2 interface is encrypted, with no data exposed in clear text.

- Integrity protection: The DUT detects and discards altered packets, ensuring data has not been tampered with.
- Replay protection: The DUT detects and discards replayed packets, preventing replay attacks.

Expected format of evidence:

The following evidence, in one or more formats as applicable, should be provided:

- Logs or screenshots showing TLS protocol adherence, as defined in the O-RAN Security Protocols Specifications [2] clause 4.5.
- Evidence of secure communication sessions established over the E2 interface, including details of encryption verification.
- Logs or screenshots showing the DUT's response to replayed and integrity-compromised packets, demonstrating the effectiveness of the security mechanisms in place.

11.5.2 E2 Authenticity

11.5.2.1 E2 Authenticity with certificate

Requirement Name: Data Authentication over E2 interface

Requirement Reference: SEC-CTL-NEAR-RT-2, clause 5.1.3.2.1, SEC-CTL-E2-1, clause 5.2.4.2 in O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-O-RAN-05, T-NEAR-RT-01, T-NEAR-RT-02, T-NEAR-RT-03, T-NEAR-RT-04

DUT/s: O-CU, O-DU, Near-RT RIC

Test Name: TC_E2_AUTHENTICATION_CERT

Purpose: The purpose of this test is to verify the authenticity of the E2 interface with valid certificates, ensuring that only legitimate and authenticated Near-RT RIC and E2 nodes can participate in the communication over the E2 interface.

Procedure and execution steps

Preconditions

- Near-RT RIC and E2 nodes support IPsec and are connected in simulated/real network environment.
- Near-RT RIC and E2 nodes support IPsec and are configured to use certificate-based authentication.
- The test environment is set up with E2 interface configured. Communication sessions over the E2 interface are established.
- The vendor provides documentation describing how authenticity protection is achieved for the data transmission over the E2 interface.
- The tunnel mode IPsec ESP and IKE certificate authentication is implemented.
- Tester has knowledge of the security parameters of tunnel for decrypting the ESP packets.
- Tester has access to the original user data transported over the E2 interface.
- IPsec is properly implemented and configured. The tester bases the test on the profile defined in [2] clause 4.5.

Execution steps

1) Execute the tests on the IPsec protocol as defined in clause 6.5.

2) Valid Authentication Credentials:

- The tester sends a request to establish a connection with the E2 interface using valid certificates.
- The tester captures and analyses the response from the E2 interface.

3) Invalid Authentication Credentials:

- The tester sends a request to establish a connection with the E2 interface using invalid certificates.
- The tester captures and analyses the response from the E2 interface.

4) No Authentication Credentials:

- The tester sends a request to establish a connection with the E2 interface without providing any certificates.
- The tester captures and analyses the response from the E2 interface.

Expected Results

For 1) Expected results in clause 6.5.4

For 2) 'Valid Authentication Credentials': The E2 interface accepts the valid certificate and responds with a successful authentication message.

For 3) 'Invalid Authentication Credentials': The connection is rejected due to the certificate verification failure, and an authentication failure message is received.

For 4) 'No Authentication Credentials': The connection attempt fails due to the absence of certificates, and an authentication failure message is received.

Expected format of evidence:

- Logs and screenshots showing adherence to IPsec protocol specifications as defined in [2] clause 4.5.
- Screenshots or logs of request-response messages confirming authentication with valid credentials.
- Screenshots or logs capturing the rejection of requests with invalid credentials.
- Screenshots or logs documenting attempts to connect without credentials and their rejection.

11.5.2.2 E2 Authenticity with PSK

Requirement Name: Data Authentication over E2 interface

Requirement Reference: SEC-CTL-NEAR-RT-2, clause 5.1.3.2.1, SEC-CTL-E2-1, clause 5.2.4.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-O-RAN-05, T-NEAR-RT-01, T-NEAR-RT-02, T-NEAR-RT-03, T-NEAR-RT-04

DUT/s: O-CU, O-DU, Near-RT RIC

Test Name: TC_E2_AUTHENTICATION_PSK

Purpose: The purpose of this test is to verify the authenticity of the E2 interface with valid PSK, ensuring that only legitimate and authenticated Near-RT RIC and E2 nodes can participate in the communication over the E2 interface.

Procedure and execution steps

Preconditions

- Near-RT RIC and E2 nodes support IPsec and are connected in simulated/real network environment.
- Near-RT RIC and E2 nodes support IPsec and are configured to use PSK-based authentication.

- The test environment is set up with E2 interface configured. Communication sessions over the E2 interface are established.
- The vendor provides documentation describing how authenticity protection is achieved for the data transmission over the E2 interface.
- The tunnel mode IPsec ESP and IKE certificate authentication is implemented.
- Tester has knowledge of the security parameters of tunnel for decrypting the ESP packets.
- Tester has access to the original user data transported over the E2 interface.
- IPsec is properly implemented and configured. The bases the test on the profile defined in [2] clause 4.5.

Execution steps

1) Execute the tests on the IPsec protocol as defined in clause 6.5.
2) Valid Authentication Credentials:

- The tester sends a request to establish a connection with the E2 interface using valid PSKs.
- The tester captures and analyses the response from the E2 interface.

3) Invalid Authentication Credentials (Incorrect PSKs):

- The tester sends a request to establish a connection with the E2 interface with incorrect PSKs.
- The tester captures and analyses the response from the E2 interface.

4) No Authentication Credentials (No PSKs):

- The tester sends a request to establish a connection with the E2 interface without providing any PSKs.
- The tester captures and analyses the response from the E2 interface.

Expected Results

For 1) Expected results in clause 6.5.4

For 2) 'Valid Authentication Credentials': The E2 interface accepts the valid PSK and responds with a successful authentication message.

For 3) 'Invalid Authentication Credentials (Incorrect PSKs)': The connection is rejected due to PSK verification failure, and an authentication failure message is received.

For 4) 'No Authentication Credentials (No PSKs)': The connection attempt fails due to the absence of PSKs, and an authentication failure message is received.

Expected format of evidence:

- Logs and screenshots showing adherence to IPsec protocol specifications as defined in [2] clause 4.5.
- Logs or screenshots documenting request and response messages for successful authentication using valid credentials.
- Logs or screenshots capturing the request and response messages when invalid credentials are rejected.
- Logs or screenshots documenting the request and response messages for rejections of connections without PSKs.

11.5.2.3 E2 Interface data validation by Near-RT RIC

Requirement Name: Validation of the data received via E2 interface by Near-RT RIC

Requirement Reference: SEC-CTL-NEAR-RT-17, clause 5.1.3.2.5, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The Near-RT RIC shall verify data received through the E2 interface as follows:

The data values are valid.

The data is being received at or below a pre-defined rate.

The Near-RT RIC shall log security event(s) if any of the verification steps fail.

Threat References: T-NEAR-RT-01, T-xApp-01

DUT/s: Near-RT RIC

Test Name: TC_E2_Interface_data_validation_by_NearRTRIC

Purpose: To validate the E2 traffic that is received by Near-RT RIC via E2 interface. The Near-RT RIC uses E2 interface to collect near real-time information (EXAMPLE:- UE basis, Cell basis) and provide value added services. These real-time information needs to be validated when it gets received at Near-RT RIC and security events are to be logged if data validation fails. E2 interface connects the Near-Real-Time RIC with other E2 nodes like O-CU, O-DU, and O-eNB.

Procedure and execution steps

EXAMPLE 1: One of the incoming data values to the Near-RT RIC are the measurement reports (carried in E2 Indication messages) that include:

- Channel quality reports: Signal strength, signal-to-noise ratio (SNR), and modulation quality.
- Interference reports: Identifying sources of interference.
- Load reports: Current load on E2 nodes

Preconditions

Client is the test system which simulates E2 data traffic towards Near-RT RIC. This includes all the supported E2 support services (EXAMPLE: E2 RESET procedure), Measurement reports and other supported services. Test system is also capable of simulating multiple E2 connections where E2 traffic can be pushed.

- Near-RT RIC is fully operational and data value validation in Near-RT RIC is defined, and the pre-defined data threshold rate is set. By fully operational Near-RT RIC, this means the Near-RT RIC is enabled with necessary xApps and configurations at the platform level.
- Client system is logged in and the initial control connections are up with Near-RT RIC. At this point the Near-RT RIC has subscribed with E2 Nodes in the Client system and is expecting data at a predefined rate.
- Login to the DUT with authorized credentials and start data collection required for checking the data handling.

Execution steps

1) From the client system, Initiate the E2 traffic with valid data values towards Near-RT RIC over single E2 connection
2) From the client test system, Initiate the E2 traffic with valid data on multiple E2 connections simultaneously
3) From the client test system, initiate invalid E2 traffic data

EXAMPLE 2: Invalid values (or) Invalid format in the measurement reports (or) Invalid E2 Node configuration information sent in E2 setup request (or) Invalid cause in the E2 reset request

4) From the client test system, initiate the E2 data which is equal to the Near-RT RIC predefined data rate via multiple E2 connections simultaneously

5) From the client test system, initiate sudden burst of E2 data which is more than the Near-RT RIC predefined data rate via multiple E2 connections simultaneously

Expected results

After step 1, the DUT processes the data traffic received over a single E2 connection.

After step 2, the DUT processes the data traffic received simultaneously over multiple E2 connections.

After step 3, the DUT discards the data and security event is logged with the logs fields as per clause 5.3.8.8 of [5].

After step 4, the DUT receives E2 traffic and handles it because E2 data is at, or below the pre-defined rate in DUT.

After step 5, the DUT discards the spilled over data and security events are logged with the log fields are as per clause 5.3.8.8 of [5] because the E2 data rate is higher than the pre-defined rate in DUT.

Expected format of evidence: Log files, traffic captures and/or report files.

11.6 A1

11.6.1 A1 Authenticity

Requirement Name: A1 protection in terms of authenticity

Requirement Reference: SEC-CTL-A1-2, clause 5.2.1.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-A1-01

DUT/s: Non-RT RIC, Near-RT RIC

Test Name: TC_A1_Authentication

Purpose: The purpose of this test is to verify the authenticity of the A1 interface, ensuring that only legitimate and authenticated Non-RT RIC, Near-RT RIC can participate in the communication over the A1 interface.

Procedure and execution steps

Preconditions

- Non-RT RIC & Near-RT RIC support mTLS and be connected in a simulated/real network environment.
- The test environment is set up with the A1 interface configured.
- The tester has access to the original data transported over the A1 interface.
- mTLS is properly implemented and configured as defined in [2] clause 4.2.

Execution steps

1) Execute the test on the mTLS protocol as defined in clause 6.3.
2) Valid Authentication Certificates (positive case):

- The tester sends a request to establish a connection with the A1 interface using valid authentication certificates.
- The tester verifies the mutual certificate verification between Non-RT RIC and Near-RT RIC.
- The tester captures and analyses the response received from the A1 interface.

3) Invalid Authentication Certificates (negative case):

- The tester sends a request to establish a connection with the A1 interface with invalid certificates.
- The tester captures and analyses the response received from the A1 interface.

4) No Authentication Certificates (negative Case):

- The tester sends a request to establish a connection without any certificates.
- The tester captures and analyses the response from the A1 interface.

Expected results

For 1) Expected results in clause 6.3

For 2) 'Valid Authentication Certificates': The A1 interface accepts the valid certificates and responds with a successful authentication message. The mutual certificate verification process is successful.

For 3) 'Invalid Authentication Certificates': The connection attempt is rejected, and an authentication failure message is received. The mutual certificate verification process fails due to the use of invalid certificates.

For 4) 'No Authentication Certificates': The connection attempt is rejected, and an authentication failure message is received. The mutual certificate verification process fails due to the absence of certificates.

Expected Format of Evidence:

The following evidence, in one or more formats as applicable, should be provided:

- Logs and screenshots showing adherence to mTLS protocol specifications as defined in [2] clause 4.2.
- Logs of authentication requests and responses on the A1 interface.
- Logs of the mutual certificate verification process.
- Screenshots or logs of error messages or unusual behaviours for both invalid and no certificate scenarios.

11.6.2 A1 confidentiality, integrity and replay protection

Requirement Name: A1 protection in terms of confidentiality, integrity, and replay

Requirement Reference: SEC-CTL-A1, clause 5.2.1.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-A1-02, T-A1-03

DUT/s: Non-RT RIC, Near-RT RIC

Test Name: TC_A1_CONFIDENTIALITY_INTEGRITY_REPLAY

Purpose: To verify the enforcement of security policies over the A1 interface, ensuring that sensitive data remains protected through confidentiality, integrity, and replay protection.

Procedure and execution steps

Preconditions

- Non-RT RIC & Near-RT RIC support TLS and connected within simulated or real network environments.
- The A1 interface is configured for testing.
- TLS is properly implemented and configured as defined in [2] clause 4.2.

Execution steps

1) Confidentiality verification:

- Establish a secure communication session over the A1 interface.
- Capture the network traffic during the session.
- Analyse the captured traffic to verify that all data is encrypted, ensuring confidentiality.

2) Integrity protection verification:

- Capture protected packets after the TLS handshake.
- Modify the captured packets.
- Inject the modified packets to the DUT.
- Confirm that the DUT discards the injected packets, e.g., does not deliver it to the higher layer.

3) Replay protection verification:

- Capture protected packets after the TLS handshake.
- Replay the captured packets to the DUT.
- Confirm that the DUT discards the replayed packets.

Expected results

- Confidentiality: All sensitive data transmitted over the A1 interface is encrypted, with no data exposed in clear text.
- Integrity protection: The DUT detects and discards altered packets, ensuring data has not been tampered with.
- Replay protection: The DUT detects and discards replayed packets, preventing replay attacks.

Expected Format of Evidence:

The following evidence, in one or more formats as applicable, should be provided:

- Logs or screenshots showing TLS protocol adherence, as defined in the O-RAN Security Protocols Specifications [2] clause 4.2.
- Evidence of secure communication sessions established over the A1 interface, including details of encryption verification.
- Logs or screenshots showing the DUT's response to replayed and integrity-compromised packets, demonstrating the effectiveness of the security mechanisms in place.

11.6.3 A1 Authorization

Requirement Name: A1 protection in terms of authorization

Requirement Reference: SEC-CTL-A1-3, clause 5.2.1.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-A1-01

DUT/s: Non-RT RIC, Near-RT RIC

Test Name: TC_A1_Authorization

Purpose: The purpose of this test is to validate that the A1 interface enforces an authorization mechanism to prevent unauthorized access.

Procedure and execution steps

Preconditions

- Non-RT RIC and Near-RT RIC support OAuth 2.0 and are connected in simulated/real network environment.
- The test environment is set up with A1 interface configured.
- The tester has access to the original data transported over the A1 interface.
- OAuth 2.0 is properly implemented and configured.

Execution steps

1) Execute the test on the OAuth 2.0 protocol as defined in clause 6.6.
2) Valid access tokens (positive case):

- The tester sends a request to access protected resources using a valid access token.
- The tester captures and analyses the response from the A1 interface.

3) Invalid access tokens (negative case):

- The tester sends a request to access protected resources using an invalid or incorrect access token.
- The tester captures and analyses the response from the A1 interface.

4) No access tokens (negative case):

- The tester sends a request to access protected resources without providing any access token.
- The tester captures and analyses the response from the A1 interface.

Expected Results

For 1) Expected results in clause 6.6

For 2) 'Valid access tokens': The A1 interface accepts the valid access tokens and responds with a successful authorization message.

For 3) 'Invalid access tokens': The access is rejected, and an access failure message is received.

For 4) 'No access tokens': The access is rejected due to the absence of tokens, and an appropriate error or unauthorized access message is received.

Expected Format of Evidence:

The following evidence, in one or more formats as applicable, should be provided:

- Logs of the request sent to access protected resources using valid access tokens.
- Screenshots or logs highlighting the successful authorization message.
- Logs of the request sent to access protected resources using invalid or incorrect access tokens.
- Screenshots or logs showing the rejection of the access and the access failure message.

11.7 R1

11.7.1 R1 Authenticity

Requirement Name: R1 protection in terms of authenticity

Requirement Reference: SEC-CTL-R1-2, clause 5.2.6.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-R1-03

DUT/s: Non-RT RIC, rApps

Test Name: TC_R1_AUTHENTICATION

Purpose: The purpose of this test is to verify the authenticity of the R1 interface, ensuring that only legitimate and authenticated Non-RT RIC, rApps can participate in the communication over the R1 interface.

Procedure and execution steps

Preconditions

- Non-RT RIC & rApps support mTLS and be connected in a simulated/real network environment.
- The test environment is set up with the R1 interface configured.
- The tester has access to the original data transported over the R1 interface.
- mTLS is properly implemented and configured as defined in [2] clause 4.2.

Execution steps

1) Execute the test on the mTLS protocol as defined in clause 6.3.
2) Valid Authentication Certificates (positive case):

- The tester sends a request to establish a connection with the R1 interface using valid authentication certificates.
- The tester verifies the mutual certificate verification between Non-RT RIC and rApps.
- The tester captures and analyses the response received from the R1 interface.

3) Invalid Authentication Certificates (negative case):

- The tester sends a request to establish a connection with the R1 interface with invalid certificates.
- The tester captures and analyses the response received from the R1 interface.

4) No Authentication Certificates (negative Case):

- The tester sends a request to establish a connection without any certificates.
- The tester captures and analyses the response from the R1 interface.

Expected results

For 1) Expected results in clause 6.3

For 2) 'Valid Authentication Certificates': The R1 interface accepts the valid certificates and responds with a successful authentication message. The mutual certificate verification process is successful.

For 3) 'Invalid Authentication Certificates': The connection attempt is rejected, and an authentication failure message is received. The mutual certificate verification process fails due to the use of invalid certificates.

For 4) 'No Authentication Certificates': The connection attempt is rejected, and an authentication failure message is received. The mutual certificate verification process fails due to the absence of certificates.

Expected Format of Evidence:

The following evidence, in one or more formats as applicable, should be provided:

- Logs and screenshots showing adherence to mTLS protocol specifications as defined in [2] clause 4.2.
- Logs of authentication requests and responses on the R1 interface.

- Logs of the mutual certificate verification process.
- Screenshots or logs of error messages or unusual behaviours for both invalid and no certificate scenarios.

11.7.2 R1 confidentiality, integrity and replay protection

Requirement Name: R1 protection in terms of confidentiality, integrity and replay

Requirement Reference: SEC-CTL-R1-1, clause 5.2.6.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-R1-06, T-R1-07

DUT/s: Non-RT RIC, rApps

Test Name: TC_R1_CONFIDENTIALITY_INTEGRITY_REPLAY

Purpose: To verify the enforcement of security policies over the R1 interface, ensuring that sensitive data remains protected through confidentiality, integrity and replay protection.

Procedure and execution steps

Preconditions

- Non-RT RIC & rApps supporting TLS, connected within simulated or real network environments.
- The R1 interface is configured for testing.
- TLS is properly implemented and configured as defined in [2] clause 4.2.

Execution steps

1) Confidentiality verification:

- Establish a secure communication session over the R1 interface.
- Capture the network traffic during the session.
- Analyse the captured traffic to verify that all data is encrypted, ensuring confidentiality.

2) Integrity protection verification:

- Capture protected packets after the TLS handshake.
- Modify the captured packets.
- Inject the modified packets to the DUT.
- Confirm that the DUT discards the injected packets, e.g., does not deliver it to the higher layer.

3) Replay protection verification:

- Capture protected packets after the TLS handshake.
- Replay the captured packets to the DUT.
- Confirm that the DUT discards the replayed packets.

Expected results

- Confidentiality: All sensitive data transmitted over the R1 interface is encrypted, with no data exposed in clear text.
- Integrity protection: The DUT detects and discards altered packets, ensuring data has not been tampered with.

- Replay protection: The DUT detects and discards replayed packets, preventing replay attacks.

Expected Format of Evidence:

The following evidence, in one or more formats as applicable, should be provided:

- Logs or screenshots showing TLS protocol adherence, as defined in the O-RAN Security Protocols Specifications [2] clause 4.2.
- Evidence of secure communication sessions established over the R1 interface, including details of encryption verification.
- Logs or screenshots showing the DUT's response to replayed and integrity-compromised packets, demonstrating the effectiveness of the security mechanisms in place.

11.7.3 R1 Authorization

Requirement Name: R1 protection in terms of authorization

Requirement Reference: SEC-CTL-R1-3, clause 5.2.6.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-R1-01, T-R1-04, T-R1-05

DUT/s: Non-RT RIC, rApps

Test Name: TC_R1_AUTHORIZATION

Purpose: The purpose of this test is to validate that the R1 interface enforces an authorization mechanism to prevent unauthorized access.

Procedure and execution steps

Preconditions

- Non-RT RIC and rApps support OAuth 2.0 and are connected in simulated/real network environment.
- The test environment is set up with R1 interface configured.
- The tester has access to the original data transported over the R1 interface.
- OAuth 2.0 is properly implemented and configured.

Execution steps

1) Execute the test on the OAuth 2.0 protocol as defined in clause 6.6.
2) Valid access tokens (positive case):

- The tester sends a request to access protected resources using a valid access token.
- The tester captures and analyses the response from the R1 interface.

3) Invalid access tokens (negative case):

- The tester sends a request to access protected resources using an invalid or incorrect access token.
- The tester captures and analyses the response from the R1 interface.

4) No access tokens (negative case):

- The tester sends a request to access protected resources without providing any access token.

- The tester captures and analyses the response from the R1 interface.

Expected Results

For 1) Expected results in clause 6.6

For 2) 'Valid access tokens': The R1 interface accepts the valid access tokens and responds with a successful authorization message.

For 3) 'Invalid access tokens': The access is rejected, and an access failure message is received.

For 4) 'No access tokens': The access is rejected due to the absence of tokens, and an appropriate error or unauthorized access message is received.

Expected Format of Evidence:

The following evidence, in one or more formats as applicable, should be provided:

- Logs of the request sent to access protected resources using valid access tokens.
- Screenshots or logs highlighting the successful authorization message.
- Logs of the request sent to access protected resources using invalid or incorrect access tokens.
- Screenshots or logs showing the rejection of the access and the access failure message.

12 Security test of O-RU

12.1 Overview

This clause contains security tests to validate the security protection mechanism specific to O-RU.

12.2 SSH on M-Plane interface

Requirement Name: Network Security Protocol - SSH

Requirement Reference: Clause 5.4, O-RAN WG4 Management Plane Specification [21]

Requirement Description: Robust protocol implementation with adequately strong cipher suites is being required for SSH

Threat References: T-O-RAN-05

DUT/s: O-RU

Test name: TC_SSH_MPlane

Purpose: To verify implementation of the SSH protocol in O-RU along with validation of supported SSH version and robustness of cryptographic algorithms used for host key, symmetric encryption, key exchange, and MACs as specified in [21]

Procedure and execution steps

Preconditions

DUT is the O-RU with SSH service enabled as server. Client is a test equipment with SSH audit tool which is used for server-side testing.

Execution steps

Follow the test case in clause 6.2 of the present document.

Expected results

The DUT supports only SSHv2 version with no older version supported and algorithms (for host key, symmetric encryption, key exchange, and MACs) defined in clause 5.4 of [21].

Expected format of evidence: As defined in clause 6.2 of the present document.

12.3 TLS on M-Plane interface

Requirement Name: Network Security Protocol - TLS

Requirement Reference: Clause 5.4, O-RAN WG4 Management Plane Specification [21]

Requirement Description: Support TLS 1.2 and/or TLS 1.3 with protocol profiles

Threat References: T-O-RAN-05

DUT/s: O-RU

Test name: TC_TLS_MPlane

Purpose: To verify implementation of the TLS protocol in O-RU along with validation of mandated/optional TLS versions and cipher suites specified in clause 5.4 of [21]. Since NETCONF implementations support X.509v3 certificate-based authentication using TLS 1.2, mutual authentication is also be tested using both valid and invalid client certificates.

Procedure and execution steps

Preconditions

DUT is the O-RU with TLS service enabled as server equipped with CA cert for signing client certificate(s). Client is a testing equipment with TLS scanning tool with client certificate(s).

Execution steps

Follow the test case in clause 6.3 of the present document.

Expected results

The DUT supports TLS starting from version 1.2 with no older version enabled along with protocol profiles/Cipher suites defined in clause 5.4 of [21].

Expected format of evidence: As defined in clause 6.3 of the present document.

12.4 Security functional requirements and test cases

The 802.1X Supplicant Validation test cases in clause 11.2.2 of the present document apply to O-RU.

13 Security test of Near-RT RIC

13.1 Overview

This clause contains security tests to validate the security protection mechanism specific to Near-RT RIC.

13.2 Void

13.3 Transactional APIs

13.3.1 Introduction

Transactional APIs in the Near-RT RIC are APIs that are based on HTTP/TLS, i.e. APIs based on REST or gRPC.

13.3.2 TLS for transactional APIs

Requirement Name: TLS for transactional APIs

Requirement Reference: SEC-CTL-NEAR-RT-6, clause 5.1.3.2.3, O-RAN Security Requirements and Controls Specifications [5].

Requirement Description: Transactional APIs (REST and gRPC) support TLS to provide message confidentiality and integrity.

Threat References: T-NEAR-RT-01, T-NEAR-RT-02, T-NEAR-RT-03, T-NEAR-RT-04

DUT/s: xApp, Near-RT RIC

Test name: TC_TLS_APIs

Purpose: To verify the transactional APIs (REST and gRPC) support TLS to provide message confidentiality and integrity.

Procedure and execution steps

Preconditions

DUT is configured and with TLS support enabled.

The other end may be simulated or a testing equipment.

Execution steps

Follow the test case in clause 6.3 of the present document.

Expected results

The transaction APIs provides confidentiality and integrity protection for data in transit.

Expected format of evidence: Tool reports, log files, traffic captures and/or screenshots.

13.3.3 mTLS for transactional APIs

Requirement Name: mTLS for transactional APIs

Requirement Reference: SEC-CTL-NEAR-RT-1, clause 5.1.3.2.1, O-RAN Security Requirements and Controls Specifications [5].

Requirement Description: The communication between xApps and Near-RT RIC platform APIs is mutually authenticated.

Threat References: T-NEAR-RT-01, T-NEAR-RT-02, T-NEAR-RT-03, T-NEAR-RT-04

DUT/s: xApp, Near-RT RIC

Test Name: TC_mTLS_APIs

Purpose: To verify the transactional APIs (REST and gRPC) support mutual TLS (mTLS) authentication via X.509v3 certificates.

Procedure and execution steps

Preconditions

Applicability: DUTs that support mTLS as a mutual authentication mechanism.

DUT is configured and with mTLS support enabled. The other end may be simulated or a testing equipment.

Execution steps

Follow the test case in clause 6.3 of the present document.

Expected results

The transactional APIs support mutual TLS (mTLS) authentication.

Expected format of evidence: Tool reports, log files, traffic captures and/or screenshots.

13.3.4 OAuth 2.0 for transactional APIs

Requirement Name: OAuth 2.0 for transactional APIs

Requirement Reference: SEC-CTL-NEAR-RT-3, clause 5.1.3.2.2, O-RAN Security Requirements and Controls Specifications [5].

Requirement Description: Near-RT RIC architecture provides an authorization framework.

Threat References: T-NEAR-RT-01, T-NEAR-RT-02, T-NEAR-RT-03, T-NEAR-RT-04

DUT/s: xApp, Near-RT RIC

Test Name: TC_OAuth2.0_API

Purpose: To verify the transactional APIs (REST and gRPC) in the DUT support the OAuth 2.0 authorization framework.

Procedure and execution steps

Preconditions

DUT is configured and with OAuth 2.0 support enabled.

The other end may be simulated or a testing equipment.

Execution steps

Follow the test case in clause 6.6.2 of the present document.

Expected results

The transactional APIs support the use of OAuth 2.0.

Expected format of evidence: Tool reports, log files, traffic captures and/or screenshots.

13.4 Security test of Near-RT RIC OAuth 2.0 Resource Owner/Server

13.4.1 Overview

This clause contains security tests to verify OAuth2.0 implementation on Near-RT RIC as resource owner/server for A1-P.

13.4.2 Near-RT RIC OAuth 2.0 Resource Owner/Server

Requirement Name: Near-RT RIC support as OAuth2.0 resource owner/server

Requirement Reference: SEC-CTL-NEAR-RT-4, clause 5.1.3.2.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: OAuth 2.0 security controls for Near-RT RIC authorization of service requests

Threat References: T-NEAR-RT-01, T-NEAR-RT-02, T-NEAR-RT-03, T-NEAR-RT-04

DUT/s: Near-RT RIC

Test Name: TC_NearRTRIC_OAuth2.0_Server

Purpose: To validate the Near-RT RIC support as OAuth 2.0 resource owner/server for A1-P, as specified in clause 4.7, O-RAN Security Protocols Specifications [2] for service requests received from a Near-RT RIC.

Procedure and execution steps

Preconditions

DUT is acting as a resource owner/server with OAuth 2.0 support enabled. OAuth2.0 Client is the test system equipped to send the service requests over a secured TLS communication with mutual TLS authentication.

Execution steps

Follow the test case in clause 6.6.2 of the present document.

Expected results

The DUT is able to authorize/deny access to resources using OAuth 2.0.

Expected format of evidence: Log files, traffic captures and/or report files.

13.5 Security test of Near-RT RIC OAuth 2.0 client

13.5.1 Overview

This clause contains security tests to verify the implementation on Near-RT RIC as OAuth2.0 client for A1-EI.

13.5.2 Near-RT RIC OAuth 2.0 client

Requirement Name: Near-RT RIC support as OAuth2.0 client for A1-EI

Requirement Reference: SEC-CTL-NEAR-RT-5, clause 5.1.3.2.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: OAuth 2.0 security controls for Near-RT RIC authorization of service requests

Threat References: T-NEAR-RT-01, T-NEAR-RT-02, T-NEAR-RT-03, T-NEAR-RT-04

DUT/s: Near-RT RIC

Test Name: TC_NearRTRIC_OAuth2.0_Client

Purpose: To validate the Near-RT RIC support as OAuth 2.0 client for A1-EI, as specified in clause 4.7, O-RAN-Security Protocols Specifications [2]

Procedure and execution steps

Preconditions

DUT is acting as a resource client with OAuth 2.0 support enabled.

Execution steps

Follow the test case in clause 6.6.1 of the present document.

Expected results

The DUT is able to request and be permitted access to resources using OAuth2.0

Expected format of evidence: Log files, traffic captures and/or report files.

14 Security test of xApps

14.1 Overview

This clause contains security tests to validate the security protection mechanism specific to xApps deployed on Near-RT RIC.

14.2 xApp Signing and Verification

The security test cases defined in clause 9.5.2 apply to this clause.

14.3 xAppID

14.3.1 xApp ID format check

Requirement Name: xApp ID uniqueness check for the xApp instance

Requirement Reference: SEC-CTL-NEAR-RT-13, clause 5.1.3.2.5, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: To validate the format of xApp ID string and the uniqueness of the same which will be a Universally Unique Identifier (UUID) version 4 (as described in IETF RFC 9562 [30]).

Threat References: T-NEAR-RT-05

DUT/s: Near-RT RIC

Test Name: TC_xApp_ID_validation

Purpose: To validate the xApp ID format that uniquely identifies the xApp instance. In this test, we are initiating registration requests from 3 xApp instances and validating the response from Near-RT RIC. The xApp ID format is checked against Universally Unique Identifier (UUID) version 4 (as described in IETF RFC 9562 [30]).

Procedure and execution steps

Preconditions

xApp instances are pre-provisioned with initial registration credential (OAuth 2.0 token), and the xApp instance CSR message.

NOTE 1: xApp instances can be instantiated of the same or different xApp images

Execution steps

1) Initiate the first xApp instance registration procedure with DUT and get for the registration response.
2) Initiate the second xApp instance registration procedure with DUT and get for the registration response.
3) Initiate the third xApp instance registration procedure with DUT and get for the registration response.

Expected results

After each execution step, the Registration response from DUT includes the xApp certificate for the xApp instance. The field "Subject Alternative Name" in the xApp instance certificate contains URI for the xApp ID as an URN. This URI contains the xApp ID of the different xApp instances.

The xApp ID generated in SAN field of xApp instance certificate after each execution step is unique and randomly different from the others (i.e., the UUIDs do not reflect a counter-based increment).

The assigned xApp ID format is complaint with Universally Unique Identifier (UUID) version 4 (as described in IETF RFC 9562 [30]. Section 6.9 "Unguessability" in IETF RFC 9562 states "Implementations SHOULD utilize a cryptographically secure pseudorandom number generator (CSPRNG) to provide values that are both difficult to predict ("unguessable") and have a low likelihood of collision ("unique").").

NOTE 2: the certificate details could be seen with the openssl tool

Expected format of evidence: Log files, traffic captures, report files, certificates and/or screenshots.

14.3.2 xApp ID in xApp instance Certificate

Requirement Name: xApp ID presence in "Subject Alternative Name" field of the xApp instance certificate.

Requirement Reference: SEC-CTL-NEAR-RT-14, clause 5.1.3.2.5, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: "Subject Alternative Name" in the xApp instance certificate contains URI for the xApp ID as an URN. This URI contains the xApp ID of the xApp instance using the UUID format as described in IETF RFC 9562 [30].

Threat References: T-xApp-02, T-NEAR-TR-05

DUT/s: Near-RT RIC

Test Name: TC_xApp_ID_check_in_xApp_instance_certificate

Purpose: To check the xApp ID embedded in subject Alternate Name field of xApp instance certificate.

Procedure and execution steps

Preconditions

xApp image is available for instantiation.

xApp Registration procedure is successfully done and xApp instance certificate has been assigned to xApp as part of the Registration response.

Execution steps

1) Establish a TLS session to the xApp instance with authorized credentials.

EXAMPLE 1: TLS session may be established using one of the services that xApp instance provides

2) Capture the xApp instance certificate (X.509v3) on the xApp instance and open the certificate to check the details.

EXAMPLE 2: Openssl x509 -in <xApp_certificate.pem> -text -noout

3) Check the "Subject Alternative Name" field in the certificate details.

Expected results

The "Subject Alternative Name" contained in the certificate of the xApp instance, contains URI for the xApp ID as an URN. xApp ID of the xApp instance is conformant to UUID format as described in IETF RFC 9562 [30].

Expected format of evidence: Log files, traffic captures, screenshots, certificates and/or report files.

15 Security test of Non-RT RIC

15.1 Overview

This clause contains security tests to validate the security protection mechanism specific to Non-RT RIC and the R1 and A1 interfaces. Security test cases for rApps are covered in a separate sub-clause.

15.2 Non-RT RIC

15.2.1 Non-RT RIC OAuth 2.0 Resource Owner/Server

Requirement Name: Server authorization support

Requirement Reference: SEC-CTL-NonRTRIC-1, clause 5.1.2.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Non-RT RIC supports OAuth 2.0 as a Server

Threat References: T-NONRTRIC-01, T-NONRTRIC-02, T-NONRTRIC-03

DUT/s: Non-RT RIC

Test Name: TC_NonRTRIC_OAuth2.0_Server

Purpose: To verify the Non-RT RIC supports OAuth 2.0 resource owner/server for A1-EI.

Procedure and execution steps

Preconditions

The DUT is acting as a Resource Owner/Server and has OAuth 2.0 support enabled.

The rest of the elements of the setup may be real or simulated.

Execution steps

Follow the test case in clause 6.6.2 of the present document.

Expected results

The DUT is able to authorize/deny access to resources using OAuth 2.0.

Expected format of evidence: Log files, traffic captures and/or report files.

15.2.2 Non-RT RIC OAuth 2.0 Client

Requirement Name: Client authorization support

Requirement Reference: SEC-CTL-NonRTRIC-2, clause 5.1.2.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Non-RT RIC supports OAuth 2.0 as a Client

Threat References: T-NONRTRIC-01, T-NONRTRIC-02, T-NONRTRIC-03

DUT/s: Non-RT RIC

Test Name: TC_NonRTRIC_OAuth2.0_Client

Purpose: To verify the Non-RT RIC supports OAuth 2.0 client for A1-P.

Procedure and execution steps

Preconditions

The DUT is acting as a Client and has OAuth 2.0 support enabled.

The rest of the elements of the setup may be real or simulated.

Execution steps

Follow the test case in clause 6.6.1 of the present document.

Expected results

The DUT is able to request and be permitted access to resources using OAuth 2.0.

Expected format of evidence: Log files, traffic captures and/or report files.

15.2.3 Non-RT RIC Framework OAuth 2.0

Requirement Name: Framework Server authorization support

Requirement Reference: SEC-CTL-NonRTRIC-4, SEC-CTL-NonRTRIC-5, clause 5.1.2.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Non-RT RIC Framework supports OAuth 2.0 as a Server

Threat References: T-NONRTRIC-01, T-NONRTRIC-02, T-NONRTRIC-03

DUT/s: Non-RT RIC

Test Name: TC_NonRTRIC_OAuth2.0_Framework_Server

Purpose: To verify the Non-RT RIC Framework supports OAuth 2.0 as a resource owner/server.

Procedure and execution steps

Preconditions

The DUT is acting as a Resource Owner and has OAuth 2.0 support enabled.

The rest of the elements of the setup may be real or simulated.

Execution steps

Follow the test case in clause 6.6.2 of the present document.

Expected results

The DUT is able to authorize access to resources using OAuth 2.0.

Expected format of evidence: Log files, traffic captures and/or report files.

15.3 R1 interface

Void

15.4 A1 interface

Void

16 Security test of rApps

16.1 Overview

This clause contains security tests to validate the security protection mechanism specific to rApps deployed on Non-RT RIC.

16.2 rApp Signing and Verification

The security test cases defined in clause 9.5.2 apply to this clause.

16.3 rApp Authorization

16.3.1 rApp OAuth 2.0 Client

Requirement Name: rApp OAuth2.0 Client support

Requirement Reference: SEC-CTL-NonRTRIC-6, clause 5.1.2.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: rApps provide client authorization requests to the Non-RT RIC Framework.

Threat References: T-rAPP-04

DUT/s: rApps

Test Name: TC_OAuth2.0_rApp

Purpose: To verify the rApp supports OAuth 2.0 client capabilities.

Procedure and execution steps

Preconditions

The DUT is acting as an OAuth2.0 Client with OAuth 2.0 support enabled.

The rest of the elements of the setup may be real or simulated.

Execution steps

Follow the test case in clause 6.6.1 of the present document.

Expected results

The DUT is able to request access and be permitted access to resources using OAuth 2.0.

Expected format of evidence: Log files, traffic captures and/or report files.

17 Security test of SMO

17.1 Overview

This clause contains security tests to validate security protection mechanisms related to the SMO. The test cases validate the security of SMO termination of O1 interfaces, SMO, SMO Services (SMOS) Communications, SMO External Interfaces, and SMO Logging are secured to zero trust principles for confidentiality, integrity, authentication, and authorization. Definitions for the O-RAN terms SMO Service (SMOS), SMO Function (SMOF), SMO External Interfaces, and SMO External System are provided in [1].

The test cases apply to the normative security requirements specified in [5] based upon the following approved security architecture:

The SMO enforces confidentiality, integrity and authenticity through an encrypted transport for the O1 interface and supports least privilege access control using the network configuration access control model (NACM) for authorization.

The SMO supports mutual authentication and authorization of SMO Functions (SMOF) and External Interfaces.

SMO Internal Communications provide communication and services between the SMO, SMOFs, Non-RT RIC Functions, and rApps. SMO Internal Communications shall provide confidentiality and integrity protection of data in transit and shall support mutual authentication and authorization for access to services and resources.

SMO External Interfaces provide import of AI enrichment data from external data sources to the SMO. SMO External Interfaces shall provide confidentiality and integrity protection of data in transit and shall support mutual authentication and authorization for access to services and resources.

17.2 Void

17.3 SMO

17.3.1 SMO OAuth 2.0 Resource Owner/Server

Requirement Name: SMO supports OAuth 2.0 resource owner/server role

Requirement Reference: SEC-CTL-SMO-3, clause 5.1.1.2.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: OAuth 2.0 security controls for SMO to authorize service requests from SMO Functions.

Threat References: T-SMO-02, T-SMO-05

DUT/s: SMO, SMO Functions

Test Name: TC_SMO_OAuth2.0_Resource_Owner_Server

Purpose: To verify that the SMO supports OAuth 2.0 resource owner/server capabilities.

Procedure and execution steps

Preconditions

DUT is the SMO with OAuth 2.0 support enabled.

Execution steps

Follow the test case in clause 6.6.2 of the present document.

Expected results

The DUT is able to authorize/deny access requests received from SMO Functions using OAuth 2.0.

Expected format of evidence: Log entries, packet captures, and screenshots.

17.3.2 SMO OAuth 2.0 Client

Requirement Name: SMO supports OAuth 2.0 client functionality

Requirement Reference: SEC-CTL-SMO-4, clause 5.1.1.2.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: OAuth 2.0 security controls for SMO to support client functionality for service requests to other SMO Functions

Threat References: T-SMO-02, T-SMO-05

DUT/s: SMO

Test Name: TC_SMO_OAuth2.0_Client

Purpose: To verify the SMO supports OAuth 2.0 client capabilities.

Procedure and execution steps

Preconditions

DUT is the SMO with OAuth 2.0 support enabled.

Execution steps

Follow the test case in clause 6.6.1 of the present document.

Expected results

The DUT is able to request and be permitted/denied access to resources using OAuth 2.0.

Expected format of evidence: Log entries, packet captures, and screenshots.

17.3.3 SMO mTLS for mutual authentication

Requirement Name: SMO support for mutual authentication of SMO Functions using mTLS

Requirement Reference: SEC-CTL-SMO-5, clause 5.1.1.2.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: SMO support mTLS for mutual authentication with SMO Functions.

Threat References: T-SMO-01, T-SMO-04

DUT/s: SMO

Test Name: TC_SMO_mTLS

Purpose: To verify the SMO supports mutual authentication with SMO Functions using mTLS, with PKI and X.509 certificates.

Procedure and execution steps

Preconditions

DUT is the SMO with mTLS support enabled. An external OAuth 2.0 Authorization Server is available and configured.

Execution steps

Follow the test case in clause 6.3 of the present document.

Expected results

The DUT supports mutual authentication of SMO Functions using mTLS.

Expected format of evidence: Log entries, packet captures, and screenshots.

17.4 SMO Internal Communications

17.4.1 TLS for SMO Internal Communications

Requirement Name: TLS support for protection at transport layer between SMO Functions

Requirement Reference: SEC-CTL-SMO-Internal-1, clause 5.1.1.2.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Data in transit protection with TLS for SMO Internal Communications

Threat References: T-SMO-09

DUT/s: SMO, Non-RT RIC

Test Name: TC_SMO_TLS_Internal_Communications

Purpose: To verify the SMO supports TLS on SMO Internal Communications.

Procedure and execution steps

Preconditions

DUT is the SMO with TLS support enabled.

Execution steps

Follow the test case in clause 6.3 of the present document.

Expected results

The SMO Internal Communications provide confidentiality and integrity protection using TLS for data in transit.

Expected format of evidence: Log entries, packet captures, and screenshots.

17.4.2 mTLS for SMO Internal Communications – SMO Functions

Requirement Name: mTLS support for mutual authentication between SMO Functions

Requirement Reference: SEC-CTL-SMO-Internal-2, clause 5.1.1.2.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Mutual authentication with mTLS for SMO Internal Communications

Threat References: T-SMO-01, T-SMO-04

DUT/s: SMO

Test Name: TC_SMO_mTLS_Internal_Communications

Purpose: To verify SMO Functions support mutual authentication using mTLS, with PKI and X.509 certificates, for SMO Internal Communications.

Procedure and execution steps

Preconditions

DUT is the SMO Function with mTLS support enabled.

Execution steps

Follow the test case in clause 6.3 of the present document.

Expected results

The DUT supports mutual authentication using mTLS.

Expected format of evidence: Log entries, packet captures, and screenshots.

17.5 SMO External Interfaces

17.5.1 TLS for SMO External Interfaces

Requirement Name: SMO External Interfaces support for TLS

Requirement Reference: SEC-CTL-SMO-External-1, clause 5.1.1.2.3, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Data in transit protection with TLS on SMO External Interfaces

Threat References: T-SMO-09

DUT/s: SMO

Test Name: TC_SMO_TLS_External_Interfaces

Purpose: To verify the SMO supports TLS on SMO External Interface.

Procedure and execution steps

Preconditions

DUT is the SMO with TLS support enabled.

Execution steps

Follow the test case in clause 6.3 of the present document.

Expected results

The DUT's External Interface provides confidentiality and integrity protection using TLS for data in transit.

Expected format of evidence: Log entries, packet captures, and screenshots.

17.5.2 mTLS for SMO External Interfaces

Requirement Name: SMO External Interfaces support for mTLS

Requirement Reference: SEC-CTL-SMO-External-2, clause 5.1.1.2.3, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Mutual authentication with mTLS on SMO External Interfaces

Threat References: T-SMO-01, T-SMO-04

DUT/s: SMO

Test Name: TC_SMO_mTLS_External_Interfaces

Purpose: To verify the SMO supports mutual authentication using mTLS, with PKI and X.509 certificates for SMO External Interfaces.

Procedure and execution steps

Preconditions

DUT is the SMO with mTLS support enabled.

Execution steps

Follow the test case in clause 6.3 of the present document.

Expected results

The DUT supports mutual authentication of SMO Functions using mTLS for SMO External Interfaces.

Expected format of evidence: Log entries, packet captures, and screenshots.

17.5.3 SMO Framework OAuth 2.0 Resource Owner/Server for External Interface

Requirement Name: SMO External Interfaces support for OAuth 2.0 resource owner/server role

Requirement Reference: SEC-CTL-SMO-External-3, clause 5.1.1.2.3, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: SMO supports OAuth 2.0 resource owner/server role to authorize service requests from external systems

Threat References: T-SMO-02, T-SMO-05

DUT/s: SMO

Test Name: TC_SMO_OAuth2.0_Resource_Owner_Server_External_Interface

Purpose: To verify the SMO supports OAuth 2.0 resource owner/server capabilities for SMO External Interfaces.

Procedure and execution steps

Preconditions

DUT is the SMO with OAuth 2.0 support enabled. An external OAuth 2.0 Authorization Server is available and configured.

Execution steps

Follow the test case in clause 6.6.2 of the present document.

Expected results

The DUT is able to authorize/deny access requests received from an external system using OAuth 2.0.

Expected format of evidence: Log entries, packet captures, and screenshots.

17.5.4 SMO Functions OAuth 2.0 Client

Requirement Name: SMO External Interfaces support for OAuth 2.0 client functionality

Requirement Reference: SEC-CTL-SMO-External-4, clause 5.1.1.2.3, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: SMO support as OAuth 2.0 client for service requests to external systems

Threat References: T-SMO-02, T-SMO-05

DUT/S: SMO

Test Name: TC_SMO_OAuth2.0_Client_External_Interface

Purpose: To verify if the SMO supports OAuth 2.0 client capabilities for External Interfaces.

Procedure and execution steps

Preconditions

DUT is the SMO with OAuth 2.0 support enabled. An external OAuth 2.0 Authorization Server is available and configured.

Execution steps

Follow the test case in clause 6.6.1 of the present document.

Expected results

The DUT is be able to request and be permitted/denied access to external resources using OAuth 2.0.

Expected format of evidence: Log entries, packet captures, and screenshots.

17.6 SMO Logging

17.6.1 TLS for SMO Logging Export

Requirement Name: SMO log export support for TLS via SMO External Interface

Requirement Reference: SEC-CTL-SMO-Log-1, clause 5.1.1.2.4, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: TLS support for SMO Logging Export

Threat References: T-SMO-16

DUT/s: SMO

Test Name: TC_SMO_TLS_Logging_Export

Purpose: To verify the SMO supports TLS for SMO logging export.

Procedure and execution steps

Preconditions

DUT is the SMO with TLS support enabled.

Execution steps

Follow the test case in clause 6.3 of the present document.

Expected results

The DUT provides confidentiality and integrity protection for logging export.

Expected format of evidence: Log entries, packet captures, and screenshots.

17.6.2 mTLS for SMO Logging Export

Requirement Name: mTLS support on SMO logging export

Requirement Reference: SEC-CTL-SMO-Log-3, clause 5.1.1.2.4, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: SMO log export support for mutual authentication using mTLS with public key infrastructure (PKI) with X.509v3 certificates

Threat References: T-SMO-01

DUT/s: SMO

Test Name: TC_SMO_mTLS_Logging_Export

Purpose: To verify the SMO supports mutual authentication using mTLS, with PKI and X.509 certificates, for SMO logging export.

Procedure and execution steps

Preconditions

DUT is the SMO with mTLS support enabled.

Execution steps

Follow the test case in clause 6.3 of the present document.

Expected results

The DUT supports mutual authentication using mTLS for SMO logging export.

Expected format of evidence: Log entries, packet captures, and screenshots.

18 Security test of O-Cloud

18.1 Overview

This clause contains security tests to validate the security protection mechanism specific to O-Cloud hosting the O-RAN architecture elements/system.

18.2 Void

18.3 O-Cloud virtualization layer

18.3.1 Secure authentication (positive case)

Requirement Name: Secure authentication to O-Cloud APIs

Requirement Reference: REQ-SEC-OCLOUD-1, clause 5.1.8.1.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Authentication for accessing management interfaces of the O-Cloud Platform

Threat References: T-VM-C-01, T-VM-C-02, T-VM-C-03, T-VM-C-04, T-VM-C-05, T-VM-C-06

DUT/s: O-Cloud

Test Name: TC_OCloud_Secure_Authentication_Positive

Purpose: The purpose of this test is to ensure secure authentication to O-Cloud APIs.

Procedure and execution steps

Preconditions

- O-Cloud authentication mechanism is enabled.
- Valid credentials are available for authentication.

Execution steps

Attempt to access O-Cloud APIs with valid authentication credentials:

- Send an API request with valid authentication credentials.

  EXAMPLE: Send an API request by executing **curl** command or using a Kubernetes client using the valid API key or access token for authentication (e.g., valid kubeconfig file or service account token).

- Capture the response received, including the status code and response body.
- Verify that the API response returns a success status code.

Expected results

The API response returns a success status code.

Expected Format of Evidence:

Logs containing the requests to the DUT and the responses from the DUT.

18.3.2 Secure authentication (negative case)

Requirement Name: Secure authentication to O-Cloud APIs

Requirement Reference: REQ-SEC-OCLOUD-1, clause 5.1.8.1.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Authentication for accessing management interfaces of the O-Cloud Platform

Threat References: T-VM-C-01, T-VM-C-02, T-VM-C-03, T-VM-C-04, T-VM-C-05, T-VM-C-06

DUT/s: O-Cloud

Test Name: TC_OCloud_Secure_Authentication_Negative

Purpose: The purpose of this test is to intentionally validate the behaviour of the authentication mechanism when encountering invalid or unauthorized authentication credentials.

Procedure and execution steps

Preconditions

O-Cloud authentication mechanism is enabled.

Execution steps

Attempt to access O-Cloud APIs with invalid authentication credentials:

- Send an API request with invalid authentication credentials.

  EXAMPLE: Send an API request by executing **curl** command or using a Kubernetes client using the invalid or expired API key or access token for authentication (e.g., invalid kubeconfig file, expired service account token).

- Capture the response received, including the status code and response body.
- Verify that the API response returns an authentication failure status code.

Expected results

The API response returns an authentication failure status code.

Expected Format of Evidence:

Logs containing the requests to the DUT and the responses from the DUT.

18.3.3 Secure authorization (positive case)

Requirement Name: Secure authorization for accessing O-Cloud APIs

Requirement Reference: REQ-SEC-OCLOUD-2, clause 5.1.8.1.1, SEC-CTL-OCLOUD-1, clause 5.1.8.1.2, SEC-CTL-OCLOUD-ISO-3, clause 5.1.8.4.3, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: O-Cloud ensures authorized management access and enforce least privilege

Threat References: T-VM-C-01, T-VM-C-02, T-VM-C-03, T-VM-C-04, T-VM-C-05, T-VM-C-06

DUT/s: O-Cloud

Test Name: TC_OCloud_Secure_Authorization_Positive

Purpose: The purpose of this test is to verify that the authorization mechanism for accessing O-Cloud APIs is functioning correctly, ensuring that entities have appropriate permissions to perform specific actions on O-Cloud resources.

Procedure and execution steps

NOTE: Entities include Applications, SMO and O-Cloud software components.

Preconditions

- Valid authentication credentials.
- O-Cloud access control system is enabled containing different levels of permissions assigned to entities.

EXAMPLE 1: Access control system such as Role-Based Access Control (RBAC), Attribute-Based Access Control (ABAC).

Execution steps

1) Authenticate with valid credentials:

- Use valid authentication credentials to establish a connection with the O-Cloud API.

2) Send an API request with authorized permissions:

- Construct a valid API request to perform a specific action,

EXAMPLE 2: specific action includes creating a pod, updating a deployment, or deleting a service.

- Ensure that the requested action aligns with the entity's assigned permissions.
- Send the request to the O-Cloud API endpoint.

3) Validate the response:

- Verify that the API response returns a success status code indicating the action was successfully executed.

Expected results

The API response returns a success status code, confirming that the requested action was authorized and executed successfully.

Expected Format of Evidence:

Logs containing the requests to the DUT and the responses from the DUT.

18.3.4 Secure authorization (negative case)

Requirement Name: Secure authorization for accessing O-Cloud APIs

Requirement Reference: REQ-SEC-OCLOUD-2, clause 5.1.8.1.1, SEC-CTL-OCLOUD-1, clause 5.1.8.1.2, SEC-CTL-OCLOUD-ISO-3, clause 5.1.8.4.3, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: O-Cloud ensures authorized management access and enforce least privilege

Threat References: T-VM-C-01, T-VM-C-02, T-VM-C-03, T-VM-C-04, T-VM-C-05, T-VM-C-06

DUT/s: O-Cloud

Test Name: TC_OCloud_Secure_Authorization_Negative

Purpose: The purpose of this test is to intentionally validate the behaviour of the authorization mechanism when encountering unauthorized or invalid access attempts.

Procedure and execution steps

Preconditions

- Valid authentication credentials.
- O-Cloud access control system is enabled containing different levels of permissions assigned to entities.

EXAMPLE 1: Access control system such as Role-Based Access Control (RBAC), Attribute-Based Access Control (ABAC).

Execution steps

1) Authenticate with valid credentials:

- Use valid authentication credentials to establish a connection with the O-Cloud API.

2) Send an API request with unauthorized permissions:

- Construct a valid API request to perform a specific action that exceeds the entity's assigned permissions,

EXAMPLE 2: specific action includes creating a pod, updating a deployment, or deleting a service.

- Send the request to the O-Cloud API endpoint.

3) Validate the response:

- Verify that the API response returns a failure status code indicating the action was unauthorized.

Expected results

The API response returns a failure status code, indicating that the requested action was unauthorized.

Expected Format of Evidence:

Logs containing the requests to the DUT and the responses from the DUT.

18.3.5 Validate network connections allowed by network policies

Requirement Name: Isolation & secure communication between Applications

Requirement Reference: REQ-SEC-OCLOUD-ISO-6, clause 5.1.8.4.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: O-Cloud enforces network policies that allow permitted connections.

Threat References: T-VM-C-01, T-VM-C-02, T-VM-C-03, T-VM-C-04, T-VM-C-05, T-VM-C-06

DUT/s: O-Cloud

Test Name: TC_OCloud_Connection_Allowed_Policies

Purpose: The purpose of this test is to ensure that network connections between VMs/Containers allowed by network policies are successfully established.

Procedure and execution steps

Preconditions

O-Cloud with network policies is configured to allow specific VMs/Containers to VMs/Containers communication.

Execution steps

1) Deploy two VMs/Containers A and B, in different zones or with different environment.

EXAMPLE 1: Zones such as namespaces in Kubernetes, environment such as labels in Kubernetes

2) Define network policies that explicitly allow communication between the two VMs/Containers.

3) Attempt to establish a network connection from VM/Container A to VM/Container B using tools.

EXAMPLE 2: tools such as curl or ping in Kubernetes

4) Capture the response or output received.

Expected results

The network connection from VM/Container A to VM/Container B is successfully established, indicating that the network policies allow the communication between the VMs/Containers.

Expected Format of Evidence:

Logs containing the requests to the DUT and the responses from the DUT.

18.3.6 Validate network connections not allowed by network policies

Requirement Name: Isolation & secure communication between Applications

Requirement Reference: REQ-SEC-OCLOUD-ISO-6, clause 5.1.8.4.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: O-Cloud enforces network policies that allow permitted connections.

Threat References: T-VM-C-01, T-VM-C-02, T-VM-C-03, T-VM-C-04, T-VM-C-05, T-VM-C-06

DUT/s: O-Cloud

Test Name: TC_OCloud_Connection_Not_Allowed_Policies

Purpose: The purpose of this test is to ensure that network connections between VMs/Containers not allowed by network policies are blocked.

Procedure and execution steps

Preconditions

O-Cloud with network policies is configured to deny specific VM/Container to VM/Container communication.

Execution steps

1) Deploy two VMs/Containers A and B, in different zones or with different environment.

EXAMPLE 1: Zones such as namespaces in Kubernetes, environment such as labels in Kubernetes

2) Define network policies that explicitly deny communication between the two VMs/Containers.

3) Attempt to establish a network connection from VM/Container A to VM/Container B using tools.

EXAMPLE 2: tools such as curl or ping in Kubernetes

4) Capture the response or output received.

Expected results

The network connection from VM/Container A to VM/Container B is blocked, indicating that the network policies correctly deny the communication between the VMs/Containers.

Expected Format of Evidence:

Logs containing the requests to the DUT and the responses from the DUT.

18.3.7 Validate network connections from outside the allowed network ranges

Requirement Name: Isolation & secure communication between Applications

Requirement Reference: REQ-SEC-OCLOUD-ISO-6, clause 5.1.8.4.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: O-Cloud enforces network policies that allow permitted connections.

Threat References: T-VM-C-01, T-VM-C-02, T-VM-C-03, T-VM-C-04, T-VM-C-05, T-VM-C-06

DUT/s: O-Cloud

Test Name: TC_OCloud_Connection_Allowed_Outside

Purpose: The purpose of this test is to ensure that network connections from IP addresses outside the allowed network ranges are denied.

Procedure and execution steps

Preconditions

O-Cloud with network policies is configured to restrict access based on IP ranges.

Execution steps

1) Define network policies that restrict access to certain IP ranges.

2) Attempt to access services or VMs/Containers from IP addresses outside the allowed ranges, either through direct IP access or using service names.

3) Capture the response or output received.

EXAMPLE: In this test case, the service name refers to the Kubernetes service object's name. The service acts as a load balancer and provides a stable DNS name that can be used to access the pods associated with it. For example, consider a service named **my-service** that is linked with a set of pods. The test case involves attempting to access my-service from IP addresses outside the allowed ranges. This can be done using tools like curl or by making HTTP requests to *http://my-service*.

Expected results

Access attempts from outside the allowed IP ranges is denied, and the response or output indicates a connection failure.

Expected Format of Evidence:

Logs containing the requests to the DUT and the responses from the DUT.

18.3.8 Exploitation of O-Cloud component vulnerabilities

Requirement Name: O-Cloud hardening and secure configuration.

Requirement Reference: REQ-SEC-SYS-1, clause 5.3.6.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Known vulnerabilities in the software of an O-RAN architecture element are identified

Threat References: T-O-RAN-02, T-VM-C-01, T-VM-C-05

DUT/s: O-Cloud

Test Name: TC_OCloud_Vulnerability_Scanning

Purpose: The purpose of this test is to identify and assess the presence of vulnerabilities in O-Cloud components and evaluate the effectiveness of their mitigation measures.

Procedure and execution steps

Preconditions

- O-Cloud with various O-Cloud components deployed.

EXAMPLE: in the context of Kubernetes, components include etcd, kubelet

- O-Cloud with security best practices is implemented.

Execution steps

1) Identify known vulnerabilities specific to the versions of used O-Cloud components using vulnerability scanning tools.
2) If known vulnerabilities exist, follow publicly available exploit scenarios or utilize penetration testing tools to attempt exploitation.
3) Monitor the O-Cloud and capture any signs of successful exploitation or vulnerabilities being triggered.

Expected results

For step 1), no known vulnerabilities exist in the O-Cloud

For step 2), mitigation measures, such as applying security patches or configuration changes are implemented to address known vulnerabilities.

For step 3), Exploit attempts fails to compromise the O-Cloud.

Expected Format of Evidence:

Logs containing the requests to the DUT and the responses from the DUT.

18.3.9 Identification and remediation of insecure configuration settings

Requirement Name: O-Cloud hardening and secure configuration

Requirement Reference: REQ-SEC-O-CLOUD-ISO-7, Clause 5.1.8.4.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The O-Cloud Platform does not permit configuration change of any component without proper authorization

Threat References: T-O-RAN-02, T-VM-C-01, T-VM-C-05

DUT/s: O-Cloud

Test Name: TC_OCloud_Insecure_Configuration

Purpose: The purpose of this test is to identify insecure configuration settings in the O-Cloud and verify the effectiveness of remediation measures.

Procedure and execution steps

Preconditions

O-Cloud with a configuration review and hardening process in place.

Execution steps

1) Review the O-Cloud configuration for common security misconfigurations, such as weak authentication settings, insecure defaults, or unencrypted communication.
2) Identify and simulate scenarios where insecure configurations can be exploited.
3) Monitor the O-Cloud and capture any signs of insecure configurations being successfully exploited.

Expected results

- The O-Cloud configuration is hardened and securely configured to mitigate common security misconfigurations.
- Insecure scenarios are identified and remediated, ensuring a hardened O-Cloud. If insecure scenarios are rectified, testing has to be repeated.

Expected Format of Evidence:

Logs containing the requests to the DUT and the responses from the DUT.

18.3.10 Validation of logging and monitoring for security incidents

Requirement Name: logging and monitoring for security incidents

Requirement Reference: REQ-SEC-OCLOUD-O2dms-4, clause 5.1.8.9.1.1, REQ-SEC-OCLOUD-O2ims-4, clause 5.1.8.9.1.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: O-Cloud DMS and IMS log SMO's management operations for auditing.

Threat References: T-GEN-01, T-GEN-02, T-GEN-03, T-GEN-04, T-GEN-05, T-GEN-06

DUT/s: O-Cloud

Test Name: TC_OCloud_Security_Logs

Purpose: The purpose of this test is to validate logging and monitoring for security incidents.

Procedure and execution steps

Preconditions

- An O-Cloud is deployed and operational
- A logging and monitoring system is implemented and configured to capture and analyse security events.

Execution steps

1) Simulate security incidents such as unauthorized access attempts or Application compromise:

- Attempt to perform unauthorized API requests or access O-Cloud resources without appropriate permissions.
- Mimic a compromised Application by running malicious code or attempting privilege escalation.
- Monitor the O-Cloud and capture any signs of security incidents being logged or detected.

2) Monitor the O-Cloud for detection and alerting of security events:

- Configure the logging and monitoring systems to capture relevant security events, such as failed authentication attempts, privilege escalation, or anomalous Application behaviour.

- Monitor the O-Cloud in real-time or periodically to detect the simulated security incidents.
- Verify that the monitoring system generates alerts or notifications for detected security events.

Expected results

- For the first step, unauthorized access attempts and Application compromise attempts are captured as security events in the logs.
- For the second step, the monitoring system detects and generates alerts for the simulated security incidents.

Expected Format of Evidence:

Logs containing the requests to the DUT and the responses from the DUT.

18.3.11 O-Cloud Privilege Escalation Prevention

Requirement Name: O-Cloud privilege escalation prevention

Requirement Reference: REQ-SEC-OCLOUD-ISO-1, REQ-SEC-OCLOUD-ISO-3, clause 5.1.8.4.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: O-Cloud prevents unauthorized privilege escalation

Threat References: T-VM-C-01

DUT/s: O-Cloud

Test Name: TC_OCloud_Privilege_Escalation_Prevention

Purpose: To verify that privilege escalation is effectively prevented in O-Cloud by enforcing security policies.

EXAMPLE 1: PodSecurity admission (PSA).

Procedure and execution steps

Preconditions:

O-Cloud with security policies (EXAMPLE: Kubernetes cluster with PodSecurity admission (PSA)) configured and enforced.

Execution steps:

1) Attempt to create a VM or Container that attempts to escalate privileges

EXAMPLE 2: in Kubernetes by specifying the **hostPID: true or hostNetwork: true** field in the pod's security context.

2) Monitor the API server response and logs

Expected results:

For step 1: The VM or Container creation request is denied by the O-Cloud API server.

For step 2: The O-Cloud API server logs should show a message indicating a violation of the security policies.

Expected format of evidence:

- Screenshot: Displaying the API server's response to the VM or Container creation attempt.
- Executed Commands: Details of the VM or Container creation parameters and security context used.
- API Server Logs: Messages indicating a violation of security policies.
- Conclusion Logs: Indicating whether the test passed or failed based on expected results.

18.3.12 O-Cloud mutual authentication

Requirement Name: O-Cloud mutual authentication between applications

Requirement Reference: SEC-CTL-O-CLOUD-ISO-1, clause 5.1.8.4.3, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: O-Cloud ensures mTLS authentication is enforced

Threat References: T-GEN-04

DUT/s: O-Cloud

Test Name: TC_OCLOUD_MUTUAL_AUTHENTICATION

Purpose: To verify that communication between different applications running on the O-Cloud is secured through mutual TLS (mTLS) authentication.

Procedure and execution steps

Preconditions:

- Environment: An O-Cloud is set up with two or more deployed applications

EXAMPLE: A cluster with two applications, each running in separate pods.

- mTLS configuration: Deployed applications in the O-Cloud are configured with mTLS as defined in [2] clause 4.2.
- Tools setup: Network sniffers, packet capture and TLS inspection tools are deployed to monitor and verify TLS handshake process.
- Valid, expired, and revoked certificates are prepared for testing. Ensure that the infrastructure for checking revoked certificates (CRL/OCSP server) is operational and accessible to the applications.

Execution steps:

- Initiate mTLS-secured sessions between applications and capture the TLS handshake process.
- Validate the exchange and authentication of certificates using TLS inspection tools.

1) Attempt connections using valid certificates and record the outcomes.
2) Attempt connections using expired certificates and record the outcomes.
3) Attempt connections, confirm that applications recognize the certificates as revoked (evidenced by querying the CRL or OCSP server), and record outcomes.
4) Attempt to establish an unauthenticated session (no certificate presented) and record the outcome.

Expected results:

- mTLS sessions are successfully established only with valid certificates.
- mTLS session establishment with expired certificates fails.
- mTLS session establishment with revoked certificates fails.
- Any attempt to initiate an unauthenticated session (without presenting a certificate) is rejected.

Expected format of evidence:

Logs from network sniffers, packet captures and TLS inspection tools showing:

- Successful mTLS handshakes with valid certificates.

- Rejections due to expired certificates, ensuring the application appropriately identifies and handles certificates beyond their validity period.
- Rejections due to revoked certificates, with specific emphasis on the application's process for recognizing revoked certificates through mechanisms such as CRL (Certificate Revocation List) and OCSP (Online Certificate Status Protocol) queries.
- Rejection of unauthenticated sessions, demonstrating the system's enforcement of mTLS authentication by not allowing sessions without certificate authentication.

18.3.13 O-Cloud authorization

Requirement Name: O-Cloud authorization

Requirement Reference: REQ-SEC-OCLOUD-2, clause 5.1.8.1.1, SEC-CTL-OCLOUD-1, clause 5.1.8.1.2, SEC-CTL-O-CLOUD-ISO-3, clause 5.1.8.4.3, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: O-Cloud enforces access control, ensuring only authorized actions per least privilege

Threat References: T-GEN-04

DUT/s: O-Cloud

Test Name: TC_OCLOUD_AUTHORIZATION

Purpose: To verify that authorization policies are correctly enforced according to the least privilege principle.

Procedure and execution steps

Preconditions:

- Environment: An O-Cloud is set up with two or more applications

EXAMPLE 1: A cluster with two applications, each running in separate pods.

- Access control configuration: Access control policies are defined and applied to applications, ensuring permissions are scoped to the minimum necessary privileges.
- Tools setup: auditing tools are deployed to monitor and verify access control policies.

EXAMPLE 2: Kubernetes audit logs for access control verification.

Execution steps:

- Map out the actions each application can perform on another according to the access control policies.

EXAMPLE 3: using ‘kubectl describe role’ and ‘kubectl describe rolebinding’ to detail the actions each application is permitted to perform on another under the access control policies.

- Perform an action that is allowed by the access control policy and record the outcomes. Validate that permitted actions align with the mapped policies.

EXAMPLE 4: using ‘kubectl auth can-i’ to validate that the action is permitted.

- Attempt actions that are not permitted by the access control policies and record the outcomes.

EXAMPLE 5: using ‘kubectl auth can-i’ to confirm that actions beyond the scope of granted permissions are denied.

Expected results:

- All actions that are explicitly granted by the access control policies are successfully performed without errors. Audit logs reflect the correct enforcement of these policies.

- Any attempts to perform actions outside the scope of granted permissions are denied, with audit logs accurately recording these access denials in accordance with the access control policies.

EXAMPLE 6: Monitor Kubernetes audit logs to capture policy decisions, noting both allowed and denied actions.

Expected format of evidence:

Detailed logs capturing:

- Allowed actions, correlating with the defined access control policies.
- Denied actions, specifically those attempted outside the granted permissions, highlighting the effective enforcement of access control policies.

18.4 Application deployment by O-Cloud

18.4.1 Verification of Application artifacts with valid signature by O-Cloud during deployment

The security test cases defined in clause 9.5.2 apply to this clause.

18.4.2 Verification of Application artifacts with incorrect signature by O-Cloud during deployment

The security test cases defined in clause 9.5.2 apply to this clause.

18.5 Resource Management and enforcement in O-Cloud

18.5.1 O-Cloud Resource Consumption Limit Enforcement

Requirement Name: Resource Management and enforcement in O-Cloud

Requirement Reference: REQ-SEC-LCM-SD-5 to REQ-SEC-LCM-SD-6, clause 5.3.2.3.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The NFO compares an Application's resource consumption with the defined quotas from the Application descriptor and enforces limits. If the resource consumption exceeds the defined quotas, the NFO generates an alarm to notify the SMO.

Threat References: T-VM-C-05, T-AppLCM-04, T-AppLCM-05

DUT/s: SMO (NFO)

Test Name: TC_OCloud_Resource_Consumption_Limit_Enforcement

Purpose: To verify the DUT is able to ensure that resources (CPU, memory, etc.) consumed by VMs or Containers are within the defined limits, preventing any single application from monopolizing the system's resources.

Procedure and execution steps

Preconditions:

- O-Cloud environment with resource quotas and limits enforced.
- A configured SMO to set and enforce resource quotas and limits.

Execution steps:

1) Set up resource quotas and limit ranges:

- Create a dedicated isolated environment for testing.
- Define a resource quota for the environment, specifying the maximum allowed CPU and memory.
- Define a limit range to set default request and limit values for resources.

2) Attempt to deploy a VM or Container that requests resources beyond the defined limits:

- Create a VM or Container configuration that requests resources exceeding the set limits.
- Try to deploy the VM or Container in the test environment.

3) Monitor the deployment status and logs:

- Check the deployment status of the VM or Container.
- Check NFO logs for an alarm when a resource limit is exceeded.

Expected results:

For step 1: Confirmation that a dedicated isolated environment for testing has been setup and both resource quota and limit range have been established.

For step 2: The deployment request for the VM or Container is denied or remains in a "Pending" or equivalent state.

For step 3: Logs or descriptions should show a message indicating a violation of the resource quotas or limits.

Expected format of evidence:

- Configuration Details: Information on the set resource quotas and limit ranges, including the maximum allowed CPU and memory.
- Executed Commands: Details of the VM or Container creation parameters, specifically the requested resources.
- NFO Logs: Messages indicating any violations of the resource quotas or limits during the deployment attempt.
- Deployment Status: Logs or screenshots showing the status of the VM or Container deployment, especially if it’s denied or remains in a "Pending" state due to resource constraints.

EXAMPLE: using Kubernetes:

1) Set up resource quotas and limit ranges in Kubernetes:

- Create a namespace: kubectl create namespace test-limits
- Apply a ResourceQuota and LimitRange as previously detailed.

2) Attempt to deploy a Pod in Kubernetes:

- Create a pod configuration (resource-hog-pod.yaml) that requests excessive resources.
- Deploy using: kubectl apply -f resource-hog-pod.yaml

3) Monitor the deployment status and logs in Kubernetes:

- Check pod status: kubectl get pods -n test-limits
- Describe the pod for details: kubectl describe pod resource-hog -n test-limits

END of EXAMPLE

18.5.2 O-Cloud Storage Volume Limit Enforcement

Requirement Name: Resource Management and enforcement in O-Cloud

Requirement Reference: REQ-SEC-LCM-SD-5, clause 5.3.2.3.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The NFO compares an Application's resource consumption with the defined quotas from the Application descriptor and enforces limits. If the resource consumption exceeds the defined quotas, the NFO generates an alarm to notify the SMO.

Threat References: T-VM-C-05, T-AppLCM-04, T-AppLCM-05

DUT/s: SMO (NFO)

Test Name: TC_OCloud_Storage_Volume_Limit_Enforcement

Purpose: To verify the DUT is able to limit the storage volume allocations for applications predefined in a O-Cloud environment.

Procedure and execution steps:

Preconditions:

- O-Cloud environment with storage volume configurations.
- A configured SMO to set and enforce resource quotas for storage.

Execution steps:

1) Set up Storage Volume Quotas:

- Create a dedicated isolated environment for testing.
- Define a storage volume quota for the environment, specifying the maximum allowed storage volume size.

2) Attempt to allocate a storage volume beyond the defined limits:

- Create a configuration that requests a storage volume size exceeding the set limits.
- Deploy the configuration in the test environment.

3) Monitor the storage allocation status and logs:

- Check the status of the storage allocation.
- Monitor the NFO logs for an alarm being generated when a storage quota violation occurs.

Expected results:

For step 1: Confirmation that a dedicated isolated environment for testing has been setup and storage volume quota has been defined.

For step 2: The storage volume allocation request is denied.

For step 3: Logs or descriptions should show a message indicating a violation of the storage quotas.

Expected format of evidence:

- Configuration Details: Information on the set storage volume quotas, including the maximum allowed storage volume size.
- Executed Commands: Details of the storage volume allocation parameters, specifically the requested storage size.
- NFO Logs: Messages indicating any violations of the storage volume quotas during the allocation attempt.
- Allocation Status: Logs or screenshots showing the status of the storage volume allocation, especially if it is denied due to exceeding the set limits.

EXAMPLE: using Kubernetes:

- Create a namespace: kubectl create namespace test-storage
- Apply a ResourceQuota for storage:
  apiVersion: v1
  kind: ResourceQuota
  metadata:
  name: storage-quota
  namespace: test-storage
  spec:
  hard:
  requests.storage: 10Gi
- Apply the ResourceQuota: kubectl apply -f storage-quota.yaml
- Create and deploy a PersistentVolumeClaim (PVC) requesting 15Gi.
- Monitor the PVC status and logs.

END of EXAMPLE

18.5.3 O-Cloud CPU Overcommit Prevention

Requirement Name: Resource Management and enforcement in O-Cloud

Requirement Reference: REQ-SEC-LCM-SD-5 to REQ-SEC-LCM-SD-6, clause 5.3.2.3.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The NFO compares an Application's resource consumption with the defined quotas from the Application descriptor and enforces limits. If the resource consumption exceeds the defined quotas, the NFO generates an alarm to notify the SMO.

Threat References: T-VM-C-05, T-AppLCM-04, T-AppLCM-05

DUT/s: SMO (NFO)

Test Name: TC_OCloud_CPU_Overcommit_Prevention

Purpose: To verify that the O-Cloud does not overcommit CPU resources, leading to performance degradation or system instability.

Procedure and execution steps

Preconditions:

- O-Cloud with CPU allocation settings.
- A configured SMO to manage CPU overcommitment.

Execution steps:

1) Set CPU Overcommit Ratios:

- Create a dedicated isolated environment for testing.
- Define CPU overcommit ratios.

2) Attempt to deploy multiple applications:

- Sequentially deploy applications until the CPU limits are reached based on the overcommit ratios.
- Monitor the CPU utilization of each deployed application.

3) Monitor the deployment status and CPU utilization metrics:

- Check the deployment status of the applications.
- Monitor CPU utilization metrics.
- Check NFO logs for an alarm when an application exceeds CPU limits.

Expected results:

For step 1: Confirmation that a dedicated isolated environment for testing has been setup and CPU overcommit ratios has been defined.

For step 2: Applications should not be deployed beyond the capacity determined by the CPU overcommit ratios.

For step 3: CPU utilization metrics should remain stable and within acceptable thresholds. NFO generates an alarm when CPU limits are exceeded.

Expected format of evidence:

- Configuration Details: Information on the set CPU overcommit ratios.
- Executed Commands: Details of the application deployments and their respective CPU utilization.
- NFO Logs: Messages indicating any violations of the CPU overcommit ratios during application deployments.
- Deployment Status: Logs or screenshots showing the status of the application deployments, especially if any are denied due to reaching CPU limits.
- CPU Utilization Metrics: Graphs or logs showing the CPU utilization of each deployed application, ensuring they remain within acceptable thresholds.

EXAMPLE: using Kubernetes:

1) Set CPU Overcommit Ratios:

- Manage CPU overcommitment in Kubernetes by setting CPU requests and limits on Pods.

2) Attempt to deploy multiple applications:

- Deploy a Pod with a CPU request of '500m' (half a CPU core) and a limit of '1' (one full CPU core).

3) Monitor the deployment status and CPU utilization metrics:

- Use 'kubectl describe node <NODE_NAME>' to view CPU allocation and utilization.
- Monitor CPU metrics using tools like Prometheus.

END of EXAMPLE

18.5.4 O-Cloud Memory Overcommit Prevention

Requirement Name: Resource Management and enforcement in O-Cloud

Requirement Reference: REQ-SEC-LCM-SD-5 to REQ-SEC-LCM-SD-6, clause 5.3.2.3.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The NFO compares an Application's resource consumption with the defined quotas from the Application descriptor and enforces limits. If the resource consumption exceeds the defined quotas, the NFO generates an alarm to notify the SMO.

Threat References: T-VM-C-05, T-AppLCM-04, T-AppLCM-05

DUT/s: SMO (NFO)

Test Name: TC_OCloud_Memory_Overcommit_Prevention

Purpose: To verify that the O-Cloud does not overcommit memory resources.

Procedure and execution steps

Preconditions:

- O-Cloud with memory allocation settings.
- A configured SMO to manage memory overcommitment

Execution steps:

1) Set Memory Overcommit Ratios:

- Create a dedicated isolated environment for testing.
- Define memory overcommit ratios.

2) Attempt to deploy applications:

- Sequentially deploy applications until memory limits are reached based on the overcommit ratios.
- Monitor memory utilization of each deployed application.

3) Monitor deployment status and memory utilization metrics:

- Check deployment status of the applications.
- Monitor memory utilization metrics.
- Check NFO logs for an alarm when an application exceeds memory limits

Expected results:

For step 1: Confirmation that a dedicated isolated environment for testing has been setup and memory overcommit ratios has been defined.

For step 2: Applications should not be deployed beyond the capacity determined by the memory overcommit ratios.

For step 3: Memory utilization should remain stable and within acceptable thresholds. NFO generates an alarm when memory limits are exceeded.

Expected format of evidence:

- Configuration Details: Information on the set memory overcommit ratios.
- Executed Commands: Details of the application deployments and their respective memory use.
- NFO Logs: Messages indicating any violations of the memory overcommit ratios during application deployments.
- Deployment Status: Logs or screenshots showing the status of the application deployments, especially if any are denied due to reaching memory limits.
- Memory Use Metrics: Graphs or logs showing the memory utilization of each deployed application, ensuring they remain within acceptable thresholds.

EXAMPLE: using Kubernetes:

1) Set Memory Overcommit Ratios:

- Manage memory overcommitment in Kubernetes by setting memory requests and limits on Pods.

2) Attempt to deploy applications:

- Deploy a Pod with a memory request of '256Mi' and a limit of '512Mi'.

3) Monitor deployment status and memory utilization metrics:

- Use kubectl describe node <NODE_NAME> to view memory allocation and utilization.
- Monitor memory metrics using tools like Prometheus.

END of EXAMPLE

18.5.5 O-Cloud Network Overcommit Prevention

Requirement Name: Resource Management and enforcement in O-Cloud

Requirement Reference: REQ-SEC-OCLOUD-ISO-6, clause 5.1.8.4.2 REQ-SEC-LCM-SD-5 to REQ-SEC-LCM-SD-6, clause 5.3.2.3.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-VM-C-05, T-AppLCM-04, T-AppLCM-05

DUT/s: O-Cloud, SMO (NFO)

Test Name: TC_OCloud_Network_Overcommit_Prevention

Purpose: To verify that the O-Cloud environment does not overcommit network bandwidth.

Procedure and execution steps

Preconditions:

- O-Cloud environment with network configurations.
- Tools to manage network overcommitment.

Execution steps:

1) Set Network Overcommit Ratios:

- Define network bandwidth overcommit ratios.

2) Attempt to utilize network bandwidth:

- Deploy applications designed to generate high network traffic.
- Monitor network traffic.

3) Monitor network traffic metrics:

- Check network traffic metrics for the applications.
- Check NFO logs for an alarm when an application exceeds network bandwidth limits.

Expected results:

For step 1: Confirmation that network bandwidth overcommit ratios has been defined.

For steps 2 and 3: Applications' network traffic should be throttled or limited once the bandwidth determined by the overcommit ratios is reached. NFO generates an alarm when network limits are exceeded.

Expected format of evidence:

- Configuration Details: Information on the set network bandwidth overcommit ratios.
- Executed Commands: Details of the application deployments and their respective network traffic generation.

- O-Cloud and NFO Logs: Messages indicating any violations of the network overcommit ratios during high network traffic.
- Deployment Status: Logs or screenshots showing the status of the application deployments, especially if network traffic is throttled or limited.
- Network Traffic Metrics: Graphs or logs showing the network traffic of each deployed application, ensuring they remain within the set bandwidth limits.

EXAMPLE: using Kubernetes:

1) Set Network Overcommit Ratios:

- Native Kubernetes doesn't offer direct network bandwidth controls. However, third-party plugins like ‘Calico’ or ‘Cilium’ can be used to set network policies that limit bandwidth.

2) Attempt to utilize network bandwidth:

- Deploy a Pod and apply a network policy that limits its bandwidth.

3) Monitor network traffic metrics:

- Use monitoring tools integrated with the network plugin (e.g., ‘calicoctl’ for Calico) to observe the network traffic metrics.

END of EXAMPLE

18.5.6 O-Cloud Storage Overcommit Prevention

Requirement Name: Resource Management and enforcement in O-Cloud

Requirement Reference: REQ-SEC-LCM-SD-5 to REQ-SEC-LCM-SD-6, clause 5.3.2.3.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The NFO compares an Application’s resource consumption with the defined quotas from the Application descriptor and enforces limits. If the resource consumption exceeds the defined quotas, the NFO generates an alarm to notify the SMO.

Threat References: T-VM-C-05, T-AppLCM-04, T-AppLCM-05

DUT/s: SMO (NFO)

Test Name: TC_OCloud_Storage_Overcommit_Prevention

Purpose: To verify that the O-Cloud does not overcommit storage resources.

Procedure and execution steps

Preconditions:

- O-Cloud environment with storage configurations.
- A configured SMO to manage Storage overcommitment.

Execution steps:

1) Set Storage Overcommit Ratios:

- Define storage overcommit ratios.

2) Attempt to allocate storage beyond defined limits:

- Deploy applications that request storage space.
- Monitor storage allocation and utilization.

3) Monitor storage allocation and utilization metrics:

- Check storage metrics for the applications.
- Check NFO logs for an alarm when an application exceeds storage limits.

Expected results:

For step 1: Confirmation that storage overcommit ratios has been defined.

For step 2: Storage allocations do not exceed the capacity determined by the storage overcommit ratios.

For step 3: Storage usage remains stable and within acceptable thresholds. NFO generates an alarm when storage limits are exceeded.

Expected format of evidence:

- Configuration Details: Information on the set storage overcommit ratios.
- Executed Commands: Details of the application deployments and their respective storage requests.
- NFO Logs: Messages indicating any violations of the storage overcommit ratios during storage allocation.
- Deployment Status: Logs or screenshots showing the status of the application deployments, especially if storage allocations are denied or limited.
- Storage usage Metrics: Graphs or logs showing the storage utilization of each deployed application, ensuring they remain within the set storage limits.

EXAMPLE: using Kubernetes:

1) Set Storage Overcommit Ratios:

- Use PersistentVolumeClaims (PVCs) with specific storage requests. Overcommitment can occur if the total storage requested by PVCs exceeds the actual available storage.

2) Attempt to allocate storage beyond defined limits:

- Deploy a Pod that uses a PVC requesting more storage than available on the PersistentVolume (PV).

3) Monitor storage allocation and utilization metrics:

- Use ‘kubectl get pvc’ and ‘kubectl get pv’ to monitor storage allocations.
- Monitor storage metrics using tools like Prometheus.

END of EXAMPLE

18.6 Secure Update

18.6.1 O-Cloud Infrastructure Software Package Integrity - Positive

Requirement Name: O-Cloud software images authenticity and Integrity

Requirement Reference: REQ-SEC-ALM-PKG-2 to REQ-SEC-ALM-PKG-6, REQ-SEC-ALM-PKG-16, clause 5.3.2.1.1, SEC-CTL-ALM-PKG-1, SEC-CTL-ALM-PKG-1B, clause 5.3.2.1.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Ensure Authenticity and Integrity of O-Cloud Software Images

Threat References: T-GEN-01

DUT/s: O-Cloud, SMO

Test Name: TC_OCloud_Software_Package_Integrity

Purpose: To verify the O-Cloud software image authenticity and integrity.

Procedure and execution steps

Preconditions

- Signed O-Cloud software package as per Clause 5 of O-RAN Security Protocols Specifications [2]
- All necessary artifacts of the O-Cloud software image (public key, digitally signed certificates, signature (Signed hash) encryption key if any for security-sensitive artifacts) are provided.

EXAMPLE: O-Cloud software includes AAL drivers, IMS, DMS, Host OS, Hypervisor, Container Engine.

Execution steps

1) The tester is properly authenticated and have the required access privileges to perform the test activity.
2) The tester shall verify the authenticity and integrity of the list of images. The O-Cloud software package shall be verified with the provided X.509 certificate and signature provided by the O-Cloud Software Provider. The cryptographic hash of the software image is calculated and verified against the hash in the signature by the Software Provider.
3) On successful validation of O-Cloud software images in Step 2, the Service Provider shall sign the verified O-cloud software image with its private key and onboard it to the SMO.
4) The newly signed O-Cloud images shall be onboarded to the O-cloud Image Repository.
5) The tester shall verify the digital signature of the O-Cloud software image bundle provided by the Software and Service Provider before deployment.
6) Monitor the SMO logs for signature verification events related to the upgrade.
7) Monitor the O-Cloud logs for any signature verification events related to the upgrade.

Expected results

Logs show that the software package integrity check has been executed for the O-Cloud software at each stage

The signature validation for the O-Cloud software image during onboarding are checked and is successful.

Expected format of evidence:

Snapshots captured in SMO logs regarding the Signature verification success.

Logs from SMO and O-Cloud (O2ims logs) to indicate the successful signature verification from the Software Provider.

18.6.2 O-Cloud Infrastructure Software Package Integrity Failure – Negative

Requirement Name: O-Cloud software images authenticity and Integrity

Requirement Reference: REQ-SEC-ALM-PKG-2 to REQ-SEC-ALM-PKG-6, REQ-SEC-ALM-PKG-16, clause 5.3.2.1.1, SEC-CTL-ALM-PKG-1, SEC-CTL-ALM-PKG-1B, clause 5.3.2.1.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Ensure Authenticity and Integrity of O-Cloud Software Images

Threat References: T-GEN-01

DUT/s: O-Cloud, SMO

Test Name: TC_OCloud_Software_Package_Integrity_Failure

Purpose: To verify the O-Cloud software image authenticity and integrity validation failure for invalid O-cloud software image.

Procedure and execution steps

Preconditions

- O-Cloud software package obtained from the Software Provider.
- All necessary artifacts of the O-Cloud software image (public key, digitally signed certificates, Signature (signed hash) encryption key if any for security-sensitive artifacts) are provided.

EXAMPLE: O-Cloud software includes AAL drivers, IMS, DMS, Host OS, Hypervisor, Container Engine.

Execution steps

1) The tester is properly authenticated and has the required access privileges to perform the upgrade activity.
2) Attempt to validate the O-Cloud Software with the wrong public key.
3) Verify that the SMO detects the incorrect cryptographic signature and does not allow onboarding of the software package.
4) Monitor the SMO logs for any signature verification events related to the software integrity check.

Expected results

Logs show that the software package integrity check has failed.

The O-cloud software image shall not be onboarded due to the software integrity failure.

Expected format of evidence:

Snapshots captured in SMO regarding the signature verification failure.

SMO Logs: Onboarding failure logs to indicate that integrity failure for the O-Cloud software Package.

18.6.3 Secure Update procedure for O-Cloud Platform – Positive

Requirement Name: Secure update of O-Cloud software at the infrastructure level layer.

Requirement Reference: REQ-SEC-OCLOUD-SU-1, REQ-SEC-OCLOUD-SU-5, REQ-SEC-OCLOUD-SU-7, clause 5.1.8.5.1, SEC-CTL-OCLOUD-SU-4, SEC-CTL-OCLOUD-SU-7, clause 5.1.8.5.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Ensure secure update of O-Cloud Software Images at the Infrastructure level.

Threat References: T-GEN-01

DUT/s: O-Cloud

Test Name: TC_SECURE_UPDATE_OF_O-CLOUD_PLATFORM

Purpose: To verify the secure update procedure for the O-Cloud Infrastructure using verified O-cloud software image.

Procedure and execution steps

Preconditions

- Verified O-Cloud software package obtained from Service Provider.
- All necessary artifacts of the O-Cloud software image (public key, digitally signed certificates, encryption key if any for security-sensitive artifacts) shall be provided.
- All necessary documents related to the Upgrade procedure of the O-Cloud components shall be available.

- All necessary dependencies for O-Cloud software packages are considered prior to update.
- All documents related to backward compatibility are made available by the O-Cloud Software provider.

EXAMPLE 1: O-Cloud software includes AAL drivers, IMS, DMS, Host OS, Hypervisor, Container Engine.

Execution steps

1) The tester is properly authenticated and has the required access privileges to perform the upgrade activity.
2) The O-Cloud Platform to ensure image verification.
3) The tester performs all the necessary pre-upgrade steps on the O-Cloud Platform to ensure successful update.

EXAMPLE 2: Back up (using Snapshots, Clones) any important components, such as app-level state stored in a database, or state of critical nodes.

4) As per the Upgrade documentation, the tester shall perform the upgrade of the O-Cloud Platform components.

EXAMPLE 3: Phased upgrades for service availability.

EXAMPLE 4: Stage the Upgrade procedure: upgrade control plane nodes and upgrade the worker nodes.

5) Monitor the O-Cloud logs (see EXAMPLE 5) for the update steps performed on the platform.

EXAMPLE 5: O2ims logs

6) Perform a Post-Update Audit to verify the status of the O-Cloud Platform.
7) Verify in the SMO that the software version for the O-Cloud platform components is updated to the required version.

Expected results

The version of the O-Cloud software components is updated to the required version.

EXAMPLE 6: AAL driver version, IMS version, DMS version, Host OS version, Hypervisor, Container Engine.

Expected format of evidence:

O-Cloud logs: Log captures indicating the Steps performed during the Update.

Snapshot: Executed command on CLI, GUI, API server

SMO Log: Notification on the successful upgrade of the O-Cloud components.

18.6.4 Secure Update failure and rollback

Requirement Name: Rollback to the previous version on the unsuccessful update of the O-Cloud Platform.

Requirement Reference: REQ-SEC-OCLOUD-SU-5, REQ-SEC-OCLOUD-SU-6, clause 5.1.8.5.1, SEC-CTL-OCLOUD-SU-6, SEC-CTL-OCLOUD-SU-7, clause 5.1.8.5.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The O-Cloud platform maintains its last verified state if updates fail or incidents occur during update.

Threat References: T-GEN-01

DUT/s: O-Cloud

Test Name: TC_SECURE_UPDATE_FAILURE_OF_O-CLOUD_PLATFORM

Purpose: To verify on failure of secure update procedure for the O-Cloud platform, it rolls back to its last verified state.

Procedure and execution steps

Preconditions

- A valid O-Cloud platform software package is available. The version of the package is greater that the stable version running in the O-Cloud platform.
- The O-Cloud platform is running a stable version of its software.

EXAMPLE 1: O-Cloud software includes AAL drivers, IMS, DMS, Host OS, Hypervisor, Container Engine.

Execution steps

1) The O-Cloud Platform verifies the signature of the new software package.
2) The tester performs all the pre-upgrade steps on the O-Cloud Platform.

EXAMPLE 2: Back up any important components, such as app-level state stored in a database, state of critical nodes.

3) Begin the update according to vendor documentation (e.g., phased upgrades).
4) Simulate an upgrade failure scenario.

EXAMPLE 3: Unexpected upgrade termination, Abrupt Power failure, Network disruption.

5) Monitor the O-Cloud logs (see EXAMPLE 4) for the upgrade steps performed on the platform. Verify that platform detects the failure and initiates an automatic rollback.

EXAMPLE 4: O2ims logs

6) Confirm that the O-Cloud platform has reverted to the same version of its software used prior to performing step 1 and confirm that the software execution is stable and operates as expected on the O-Cloud platform.

Expected results

The O-Cloud platform automatically rolls back to its previous version and last verified state.

Expected format of evidence:

O-Cloud logs indicating secure update failure detection and rollback.

18.6.5 Unauthorised Rollback Prevention

Requirement Name: Unauthorized rollback prevention to an earlier vulnerable version

Requirement Reference: REQ-SEC-OCLOUD-SU-6, clause 5.1.8.5.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The O-Cloud platform prevents the unauthorized rollback of its software to an earlier vulnerable version.

Threat Reference: T-GEN-01

DUT/s: O-Cloud

Test Name: TC_UPDATE_ROLLBACK_PREVENTION_O-CLOUD_PLATFORM

Purpose: To verify that the O-Cloud platform prevents unauthorized rollback to an earlier vulnerable version of its software, even if the rollback package is correctly signed, while allowing authorized rollback under controlled conditions.

Procedure and execution steps

Preconditions

- The O-Cloud platform has a newer software version installed and operational.

- A correctly signed older vulnerable software version is available.
- An incorrectly signed older non-vulnerable software version is also available.

Execution steps

1) Unauthorized rollback attempt to a vulnerable version:

- Initiate a rollback using the correctly signed older vulnerable version of the software.
- Observe if the O-Cloud platform detects and rejects the rollback attempt.

2) Authorized rollback scenario (if required for performance recovery):

- Initiate an authorized rollback using the correctly signed older version of the software following the approved rollback process.
- Verify that the O-Cloud platform allows rollback in this scenario.

Expected results

For step 1. the platform rejects unauthorized rollback attempts.

For step 2, the platform successfully executes the authorized rollback.

Expected format of evidence

Logs showing:

- Rejection events for unauthorized rollback attempts.
- Authorized rollback initiation and completion logs.

Screenshot showing the version status and error messages.

18.7 Secure Storage

18.7.1 Sensitive data protection in O-Cloud

Requirement Name: Sensitive data protection in O-Cloud

Requirement Reference: REQ-SEC-ALM-PKG-13, clause 5.3.2.1.1, REQ-SEC-OCLOUD-SS-1, clause 5.1.8.6.1, SEC-CTL-OCLOUD-SS-1, clause 5.1.8.6.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The O-Cloud Platform ensures the confidentiality and integrity of sensitive data at rest, in use, and in transit.

Threat References: T-GEN-05

DUT/s: O-Cloud

Test Name: TC_DATA_PROTECTION_OCLOUD

Purpose: To validate that the O-Cloud ensures the integrity and confidentiality of sensitive data at rest, in use, and in transit, using state-of-the-art encryption and security practices.

Procedure and execution steps

Preconditions:

O-Cloud operational with a simulated deployment of workloads.

Execution steps:

1) Data at rest:

- Store simulated sensitive data (e.g., secrets) in O-Cloud storage.
- Verify encryption using tools designed to check for industry-standard encryption mechanisms, focusing on confirming that data is encrypted to current security standards.

2) Data in use:

- Process sensitive data using an application.
- Identify write operations involving sensitive data
  - Employ process monitoring tools to monitor file I/O operations and capture all write activities.
  - Look for write operations to temporary file paths, cache directories or outside of the application secure storage.
  - Analyse the context of write operations – are they occurring during a process that handles sensitive data?
- Ensure that the application does not log sensitive information
  - Review the application's logging configuration and log output.
  - Verify that sensitive data is either not logged or appropriately anonymized/encrypted before being logged.
- Check for the use of secure enclaves, if applicable.

3) Data in Transit:

- Initiate data transfer between O-Cloud services.
- Use packet-sniffing tools to capture the data packets.
- Analyse the TLS encrypted data, ensuring TLS is used as specified in O-RAN Security Protocols Specifications [2], clause 4.2.

Expected Results:

- Sensitive data in O-Cloud storage is encrypted according to current industry standards. Unauthorized access attempts are logged and denied.
- Data processing is secure, with no plaintext data exposure in logs or disk. Secure enclaves are used where relevant.
- All data transfers employ TLS as specified in O-RAN Security Protocols Specifications [2], clause 4.2.

Expected Format of Evidence:

- Screenshots and logs showing encryption validation and the response to unauthorized access attempts.
- Logs from process monitoring tools demonstrating the handling of sensitive data during processing.
- Packet capture files confirming the data encryption in transit using TLS as specified in O-RAN Security Protocols Specifications [2], clause 4.2.

18.7.2 Secure data deletion in O-Cloud

Requirement Name: Secure data deletion in O-Cloud

Requirement Reference: REQ-SEC-OCLOUD-SS-2, clause 5.1.8.6.1, SEC-CTL-OCLOUD-SS-2, SEC-CTL-OCLOUD-SS-3, clause 5.1.8.6.2 O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The O-Cloud Platform supports secure deletion of data from storage and memory, ensuring that unused or deallocated data is irrecoverable by applying media-specific sanitization techniques and overwriting addressable memory locations with secure patterns.

Threat References: T-GEN-05

DUT/s: O-Cloud

Test Name: TC_DATA_DELETION_OCLOUD

Purpose: To ensure that the O-Cloud platform securely deletes data from addressable memory locations that are no longer in use, by overwriting them with specific binary patterns.

Procedure and execution steps

Preconditions:

- O-Cloud platform operational with workloads.
- Tools for memory analysis are available.

EXAMPLE 1: **dd** for Unix/Linux or **sdelete** for Windows environments

- File recovery tools for testing data recoverability after deletion.

EXAMPLE 2: **TestDisk**

Execution steps:

1) Data preparation:

- Create files with identifiable data patterns.

EXAMPLE 3: a file filled with a repeating pattern of '1234'

- Store these files in the O-Cloud platform's memory or storage system.

2) Data deletion:

- Delete the files using the O-Cloud platform's standard deletion process, which should invoke secure deletion process.

3) Verification of secure deletion:

- Inspect the memory or storage locations where the files were stored to confirm that the data has been overwritten.
- Search for both the original data patterns and the specific overwriting patterns (zeroes, ones, random).

4) Data recovery attempt:

- Use data recovery tools to attempt to retrieve the deleted files or any part of them.
- Assess if any of the original data or identifiable patterns can be recovered.

Expected Results:

- Deleted data locations are overwritten with the specified binary patterns.
- File recovery attempts are not able to reconstruct any meaningful data from these locations.

Expected Format of Evidence:

- Logs or screenshots from memory analysis tools showing the overwriting patterns.
- Reports from file recovery tools indicating the failure to recover any meaningful data.

18.7.3 Data isolation in VM/Container reallocation

Requirement Name: Data isolation in VM/Container reallocation

Requirement Reference: REQ-SEC-OCLOUD-SS-3, clause 5.1.8.6.1, SEC-CTL-OCLOUD-SS-2, clause 5.1.8.6.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The O-Cloud Platform ensures that any data contained in a resource is not accessible when it is de-allocated from one VM/Container and reallocated to another by supporting secure deletion of data in addressable memory locations before reuse.

Threat References: T-GEN-05

DUT/s: O-Cloud

Test Name: TC_DATA_ISOLATION_VM_CONTAINER_OCLOUD

Purpose: To verify that the O-Cloud effectively prevents data contained in a resource (like memory or storage) from being accessible after it is de-allocated from one VM/Container and reallocated to another.

Procedure and execution steps:

Preconditions:

- Set up multiple VMs/Containers within the O-Cloud.
- Tools for analyzing memory and storage content

EXAMPLE: **hexdump, dd**, memory inspection tools

Execution steps:

1) Resource allocation and data storage:

- Allocate a dedicated resource (like a disk volume or memory segment) to a VM/Container.
- Store known test data in this resource.

2) Resource de-allocation & re-allocation:

- De-allocate the resource from the first VM/Container.
- Re-allocate the same resource to a different VM/Container.

3) Data accessibility check:

- Within the new VM/Container, attempt to access any residual data from the previous allocation.
- Use data analysis tools to inspect the resource for traces of the previous data.

4) Verification of data isolation:

- Confirm that no data from the first VM/Container is accessible or present in the resource after re-allocation.

Expected Results:

- No trace of the test data is found in the reallocated resource.
- The new VM/Container doesn't have access to any residual data from the previous allocation.

Expected Format of Evidence:

Logs or screenshots showing the absence of the test data in the re-allocated resource.

18.8 Chain of trust

18.8.1 Chain of Trust verification in static O-Cloud SW

Requirement Name: Support of root of trust and integrity verification of static O-Cloud SW (Firmware and BIOS/UEFI, Bootloader, OS kernel)

Requirement Reference: REQ-SEC-OCLOUD-COT-1, clause 5.1.8.7.1, SEC-CTL-OCLOUD-COT-1, SEC-CTL-OCLOUD-COT-2, clause 5.1.8.7.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The O-Cloud Platform supports a Chain of Trust (CoT) by verifying the integrity of all relevant components using a hardware root of trust (RoT) or, where hardware is unavailable, a software root of trust to ensure platform security.

Threat References: T-VL-02

DUT/s: O-Cloud

Test Name: TC_OCLOUD_CHAIN_OF_TRUST_STATIC_SW

Purpose: To confirm the presence and proper functioning of a secure boot process and integrity verification for O-Cloud static SW (Firmware and BIOS/UEFI, Bootloader, OS kernel), using hardware-based or software-based roots of trust mechanisms.

Procedure and execution steps

Preconditions:

- Ensure the O-Cloud is set up with all components, including hardware, operating system, virtualization layer, and any applications or services running on the O-Cloud.
- Use tools capable of interfacing with the O-Cloud's root of trust mechanism.

EXAMPLE 1: TPM management tools for hardware-based roots of trust, Keylime or any equivalent integrity verification system for automated integrity verification.

- Access requirements: Tester needs administrative access to the O-Cloud platform to execute integrity verification commands and to collect integrity verification reports.

Execution steps:

Hardware RoT verification:

Confirm the functionality of the root of trust mechanism in each O-Cloud node, whether it is hardware-based or an equivalent software-based solution.

EXAMPLE 2: Use a Kubernetes DaemonSet to run tpm2_pcrread on all nodes, which verifies TPM presence and functionality by reading the PCR (Platform Configuration Registers) values. The DaemonSet collects outputs and send them for verification.

Integrity check:

Verify the integrity measurements against known good baselines. These measurements ensure the boot process and static O-Cloud SW integrity.

EXAMPLE 3: Use a securely stored baseline to obtain the expected PCR values for a known secure state of the O-Cloud static SWs. This could involve securely storing PCR values following a clean installation or using manufacturer-provided values.

EXAMPLE 4: Schedule Kubernetes CronJobs to use Keylime for periodic integrity verification. Keylime agents on nodes interact with the TPM to attest the integrity measurements, comparing them against known good values stored in Keylime's verifier.

Report collection and analysis:

Collect integrity reports and analyse them for discrepancies or signs of tampering.

EXAMPLE 5: Use Keylime's centralized reporting and alerting features to collect and analyse attestation data. Integrate Keylime with an ELK stack deployed within the Kubernetes cluster for enhanced log analysis and visualization of attestation outcomes.

Expected Results:

- All O-Cloud nodes demonstrate the presence of RoT.
- Integrity measurements align with known good baselines.

Expected Format of Evidence:

- Logs confirming RoT presence on each node.
- Logs indicating successful integrity verification against known good baselines.

18.8.2 Chain of Trust verification of dynamic O-Cloud SW

Requirement Name: Support of root of trust and integrity verification of dynamic O-Cloud SW (virtualization layer and workloads)

Requirement Reference: REQ-SEC-OCLOUD-COT-1, REQ-SEC-OCLOUD-COT-2, clause 5.1.8.7.1, SEC-CTL-OCLOUD-COT-2, SEC-CTL-OCLOUD-COT-3, clause 5.1.8.7.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The O-Cloud Platform supports a Chain of Trust (CoT) by verifying the integrity of all relevant components from the hardware layer to the application layer, using a hardware or software root of trust. It also supports remote attestation services to continuously monitor integrity and detect unauthorized changes.

Threat References: T-VL-02

DUT/s: O-Cloud

Test Name: TC_OCLOUD_INTEGRITY_VERIFICATION_DYNAMIC_SW

Purpose: To ensure the integrity of dynamic software in the O-Cloud through continuous verification.

Procedure and execution steps

Preconditions:

- Ensure the O-Cloud is set up with all components, including hardware, operating system, virtualization layer, and any applications or services running on the O-Cloud.
- Use tools capable of interfacing with the O-Cloud's root of trust mechanism.

EXAMPLE 1: TPM management tools for hardware-based roots of trust, Keylime to automate integrity measurements and attestation of dynamic software components, integrating with IMA for capturing runtime integrity measurements.

- Access requirements: Tester needs administrative access to the O-Cloud platform to execute integrity verification commands and to collect integrity verification reports.

Execution steps:

Dynamic software verification:

Initiate continuous integrity verification of the dynamic O-Cloud SW, including executable binaries and configuration files used by the container engine and workloads.

EXAMPLE 2: Implement an IMA policy to measure container images and runtime configurations upon execution. Configure Keylime to monitor the integrity of container runtime environments and deployed containers on Kubernetes nodes. This involves setting up Keylime agents within the cluster that automatically update integrity measurements for dynamic software components and verify them against expected values.

Attestation:

Perform attestation of the container engine configurations and active workloads to detect any unauthorized changes or potential integrity breaches.

EXAMPLE 3: Deploy Keylime agents on each O-Cloud node to periodically attest the integrity of dynamic SW components based on IMA measurements. Use Keylime to trigger attestation procedures that verify IMA logs against expected integrity measurements for containerized applications.

Report collection and anomaly detection:

Collect attestation reports and analyse them for any discrepancies, unauthorized changes, or signs of tampering in the container engine and workloads.

EXAMPLE 4: Use Keylime's web interface or API integrated with an ELK stack for logging and monitoring attestation results. Set up alerts for any attestation failures or integrity breaches detected in dynamic software.

Expected Results:

- Dynamic software components are measured upon execution, with their integrity measurements securely recorded.
- Integrity measurements are successfully captured and verified against known good baselines, indicating no unauthorized modifications.

Expected Format of Evidence:

- Logs attesting the integrity of dynamic software components, including any alerts generated for integrity failures.
- Reports detailing the comparison of runtime integrity measurements against known good baselines, demonstrating continuous integrity verification of dynamic O-Cloud dynamic software.

18.9 Secure time synchronization for O-Cloud

Requirement Name: Secure time synchronization for O-Cloud

Requirement Reference: REQ-SEC-OCLOUD-TS-1, clause 5.1.8.12.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: It ensures all O-Cloud nodes exclusively connect to an authenticated time synchronization server for Time of Day (ToD) synchronization.

Threat Reference: T-TS-01

DUT/s: O-Cloud

Test Name: TC_OCLOUD_SECURE_TIME_SYN

Purpose: To verify that all O-Cloud nodes are configured to exclusively connect to an authenticated time synchronization server for Time of Day (ToD) synchronization.

Procedure and execution steps

Preconditions:

- A list of allowed secure time synchronization servers

- Configuration of O-Cloud nodes

Execution steps:

1) Access the configuration of each O-Cloud node to review the time synchronization settings.
2) Check that each O-Cloud node is configured to only connect to one or more time synchronization servers from the list of allowed secure time synchronization servers.
3) Ensure that no other time synchronization servers are configured.
4) Confirm that the cryptographic keying material for verifying the server's authenticity is present in the configuration.

Expected results:

Each O-Cloud node is configured to connect only to the allowed secure time synchronization server(s), and the cryptographic keying material for verifying each server's authenticity is present in the configuration.

Expected format of evidence:

Screenshots showing the O-Cloud node configuration settings with the allowed time synchronization server(s) details.

19 Security test of VNF/CNF

19.1 Overview

This clause contains security tests to validate the security protection mechanism specific to O-RAN architecture elements which are virtualized/containerized and deployed on the O-Cloud/SMO.

19.2 Executive environment protection

Requirement Name: secure executive environment provision

Requirement Reference: REQ-SEC-LCM-SD-5, REQ-SEC-LCM-SD-6, REQ-SEC-LCM-SD-7, clause 5.3.2.3.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: It ensures all O-RAN architecture elements only consume the resources from its defined resource quotas.

Threat References: T-AppLCM-04, T-AppLCM-05

DUT/s: SMO

Test Name: TC_SECURE_EXECUTIVE_ENV_PROVISION

Purpose: To test whether the NFO compares the VNF/CNF owned resource state with the defined resource quotas from the Application descriptor.

To test whether the NFO sends an alarm to the SMO if the two resource states are inconsistent.

Procedure and execution steps

Preconditions

There are: VNF/CNF, O-Cloud, SMO, NFO, Application Descriptor, (or simulated O-Cloud, SMO) on the test environment.

Execution steps

1) The tester utilizes the O-Cloud to change the resource state of VNF/CNF (e.g. change vCPU size of the VNF/CNF).

2) The tester uses the NFO to query the parsed resource state from the Application descriptor.

3) The tester checks whether the NFO sends an alarm to the SMO when the NFO receives the parsed resource state from the Application descriptor and finds that the owned resource state and the parsed resource state are inconsistent.

Expected Results

The NFO sends an alarm to the SMO when the NFO detects an inconsistency between the parsed resource state from the Application descriptor and the owned resource state.

Expected format of evidence:

Screenshots of logs containing the alarm on the SMO.

19.3 Signature validation during App image onboarding

The security test cases defined in clause 9.5.2 apply to this clause.

19.4 Application image deployment security

Requirement Name: Application image deployment security

Requirement Reference: REQ-SEC-ALM-PKG-12, clause 5.3.2.1.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-IMG-04, T-AppLCM-02

DUT/s: O-CU, O-DU, Near-RT RIC, xApps, rApps

Test Name: TC_APP_IMAGE_VULNERABILITY_CHECK_ON_DEPLOY

Purpose: The purpose of this test is to verify that an Application image is free from known vulnerabilities.

Procedure and execution steps

Preconditions

O-Cloud with Application image scanning tools integrated.

Execution steps

1) Deploy an Application image known to have vulnerabilities:

- Select an Application image with known vulnerabilities, such as an image with outdated software or documented security issues.
- Attempt to deploy the image to an O-Cloud using the appropriate deployment configuration.
- Monitor the deployment process and capture any error messages or logs.

2) Deploy an Application image with outdated or unapproved software libraries:

- Create a custom Application image that includes outdated or unapproved software libraries.
- Attempt to deploy the custom image to an O-Cloud using the appropriate deployment configuration.
- Monitor the deployment process and capture any error messages or logs.

Expected Results

For the first step, the container image with known vulnerabilities is rejected or flagged as insecure, preventing its deployment.

For the second step, the container image with outdated or unapproved software is blocked from deployment, ensuring compliance with security policies.

Expected format of evidence:

- Vulnerability scan reports generated by the Application image scanning tool, indicating the detected vulnerabilities and their severity.
- Rejection logs or error messages from the Application image registry or O-Cloud, indicating the rejection or blocking of insecure images.

20 Security tests of Common Application Lifecycle Management

20.1 Overview

This clause contains security tests to validate the security protection relevant to Common App LCM.

20.2 Application package

20.2.1 Application package signature verification

The security test cases defined in clause 9.5.2 apply to this clause.

20.2.2 Minimum Requirements

Requirement Name: Application package includes minimal artifacts.

Requirement Reference: REQ-SEC-ALM-PKG-3, clause 5.3.2.1.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: The application package includes the signing certificate and signature(s) of Solution Provider.

Threat References: T-O-RAN-09

DUT/s: O-RU, O-DU, O-CU, Near-RT RIC, xApp, rApp

Test Name: TC_App_Pkg_Min_Artifacts

Purpose: The purpose of this test is to verify that an Application package includes minimal artifacts according to REQ-SEC-ALM-PKG-3.

Procedure and execution steps

Preconditions

Application package available for access.

Execution steps:

1) Access the Application package contents.

2) Verify the Application package includes minimally the following artifacts:

   a) Signing certificate

   b) Solution Provider signature(s)

Expected Results:

Application package includes the minimal artifacts required.

Expected format of evidence:

Screenshots or logs providing evidence that the application package contains the signing certificate and signatures of the Solution Provider.

20.2.3 App Package Change Log

Requirement Name: Application package shall have change logs.

Requirement Reference: REQ-SEC-ALM-PKG-15, clause 5.3.2.1.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Application packages have a Change Log.

Threat References: T-O-RAN-09

DUT/s: O-RU, O-DU, O-CU, Near-RT RIC, xApp, rApp

Test Name: TC_App_Pkg_Change_Log

Purpose: The purpose of this test is to verify that the Application packages contains a change log according to REQ-SEC-ALM-PKG-15.

Procedure and execution steps

Preconditions

Application package available for access.

Execution steps:

- Access the Application package and external artifacts if present.
- Verify the Application package or external artifacts includes change log.
- Verify latest version noted in change log matches the current Application version.

Expected Results:

Change log is included in the Application package.

Expected format of evidence:

Screenshot or logs providing evidence of the change log is either provided in the application package or provided as an external artifact.

20.3 Secure Decommissioning

20.3.1 Post-Decommission Report

Requirement Name: A complete post-decommission report shall be generated.

Requirement Reference: REQ-SEC-ALM-DECOM-1, clause 5.3, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description:

Threat References: T-AppLCM-06

DUT/s: O-RU, O-DU, O-CU, Near-RT RIC, xApp, rApp

Test Name: TC_App_Decomm_Report

Purpose: The purpose of this test is to ensure a decommissioning report is generated for a decommissioned Application.

Procedure and execution steps

Preconditions

Ensure Application subject to decommissioning has no running instance(s) on O-RAN system.

Execution steps:

1) Execute decommissioning of Application
2) Generate report of decommissioning whether through manual or automated means

Expected Results:

Decommissioning report documenting Application decommissioning is generated.

Expected format of evidence:

A report detailing:

- Decommissioned Application name and version
- Date and time of Application decommissioning
- Tasks performed during decommissioning
- Other pertinent details

20.3.2 Trust Artifact Revocation

Requirement Name: Trust artifacts revoked during Application decommissioning.

Requirement Reference: REQ-SEC-ALM-DECOM-3, clause 5.3.2.5.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: All trust artifacts associated with an application are revoked at the time of decommissioning.

Threat References: T-AppLCM-06

DUT/s: O-RU, O-DU, O-CU, Near-RT RIC, xApp, rApp

Test Name: TC_App_Trust_Artifact_Revocation

Purpose: The purpose of this test is to verify that all trust artifacts associated with an Application are revoked at the time of Application decommissioning.

Procedure and execution steps

Preconditions

- Locate and prepare Application trust artifacts for revocation.

- Ensure Application subject to decommissioning has no running instance(s) on O-RAN system.

Execution steps:

1) Revoke trust artifacts from Application subject to decommissioning.
2) Perform a scan and verify that all trust artifacts associated with Application have been revoked.

EXAMPLE: If trust artifact is a certificate, verify that certificate is in certification revocation list.

3) Execute decommissioning of Application.
4) Attempt to instantiate Application and verify that it cannot be re-instantiated without the trust artifacts.

Expected Results:

All trust artifacts are removed from the Application and the decommissioned Application cannot be re-instantiated.

Expected format of evidence:

Screenshots, logs, or report providing evidence that the trust artifacts associated with a decommissioned application are revoked.

21 Security test of O-CU-CP

21.1 Overview

The present clause contains 3GPP security test cases applicable to O-CU-CP and O-RAN specific O-CU-CP test cases.

21.2 O-CU-CP 3GPP specific security functional requirements and test cases

Requirement Name: 3GPP specific O-CU-CP security

Requirement Reference: REQ-SEC-OCU-1, clause 5.1.4.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: “O-CU-CP and O-CU-UP shall meet the security requirements for gNB-CU-CP and gNB-CU-UP respectively”, as specified in TS 33.501 [25]

DUT/s: O-CU-CP

Purpose: To verify the O-CU-CP meet the security requirements for gNB-CU-CP

Test Name: TC_O_CU_CP_3GPP_33_523_Cl_5_2_2 (As defined in clause 5.2.2 of TS 33.523 [23])

gNB-CU-CP specific security functional requirements and test cases specified in clause 5.2.2 of TS 33.523 [23] apply to O-CU-CP.

21.3 O-RAN specific security functional requirements and test cases

The TLS test cases in clause 6.3 of the present document apply to the O1 interface of O-CU-CP.

The IPsec test cases in clause 6.4 of the present document apply to the E2 interface of O-CU-CP.

22 Security test of O-CU-UP

22.1 Overview

The present clause contains 3GPP security test cases applicable to O-CU-UP and O-RAN specific O-CU-UP test cases.

22.2 O-CU-UP 3GPP specific security functional requirements and test cases

Requirement Name: 3GPP specific O-CU-UP security

Requirement Reference: REQ-SEC-OCU-1, clause 5.1.4.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: “O-CU-CP and O-CU-UP shall meet the security requirements for gNB-CU-CP and gNB-CU-UP respectively”, as specified in TS 33.501 [25]

DUT/s: O-CU-UP

Test Name: TC_O_CU_UP_3GPP_33_523_Cl_6_2_2 (As defined in clause 6.2.2 of TS 33.523 [23])

Purpose: To verify the O-CU-CP meet the security requirements for gNB-CU-UP

gNB-CU-UP specific security functional requirements and test cases specified in clause 6.2.2 of TS 33.523 [23] apply to O-CU-UP.

22.3 O-RAN specific security functional requirements and test cases

The TLS test cases in clause 6.3 of the present document apply to the O1 interface of O-CU-UP.

The IPsec test cases in clause 6.4 of the present document apply to the E2 interface of O-CU-UP.

23 Security test of O-DU

23.1 Overview

The present clause contains 3GPP security test cases applicable to O-DU and O-RAN specific O-DU test cases.

23.2 O-DU 3GPP specific security functional requirements and test cases

Requirement Name: 3GPP specific O-DU security

Requirement Reference: REQ-SEC-ODU-1, clause 5.1.5.1, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: “O-DU shall meet the security requirements for gNB-DU” as specified in TS 33.501 [25].

DUT/s: O-DU

Test Name: TC_O_DU_3GPP_33_523_Cl_7_2_2 (As defined in clause 7.2.2 of TS 33.523 [23])

gNB-DU specific security functional requirements and test cases specified in clause 7.2.2 of TS 33.523 [23] apply to O-DU.

23.3 O-RAN specific security functional requirements and test cases

The 802.1X Authenticator Validation test cases in clause 11.2.1 applies to O-DU for the network configuration where O-DU acts as an 802.1X authenticator.

The 802.1X Supplicant Validation test cases in clause 11.2.2 apply to O-DU.

The TLS test cases in clause 6.3 of the present document apply to the O1 interface and M-Plane of O-DU.

The IPsec test cases in clause 6.4 of the present document apply to the E2 interface of O-DU.

The SSH Server & Client test cases in clause 6.2 of the present document apply to the M-Plane of O-DU.

24 End-to-End security test cases

24.0 Overview

This clause describes E2E tests evaluating and assessing the security aspects of an O-RAN conformant radio access network.

The O-RU, O-DU, O-CU-CP and O-CU-UP as defined in O-RAN Architecture Description [1] is the System under Test (SUT) and can be viewed as an integrated black box in the context of the E2E security testing.

24.1 3GPP Security Assurance Specification (SCAS)

For NR technology, **Error! Reference source not found.** applies. The test cases referenced in this table are from 3GPP TS 33.511 [8], which are applied to the O-RAN system. The table also indicates the applicable technology, specifying whether each test case pertains to NR NSA (Non-Standalone) and/or NR SA (Standalone).

For LTE technology, **Error! Reference source not found.** applies. The test cases referenced in this table are from 3GPP TS 33.216 [9], which are applied to the O-RAN system.

The tables also contain the information relative to the 3GPP releases affected for each test case.

Table 24-1: List of SCAS Test Cases for NR and applicable technology from Clause 4.2.2 of 3GPP TS 33.511

Test Case (O-RAN Ref. #)	Test Case (3GPP clause number and title)	Applicable Technology	Applicable 3GPP Releases
TC_SCAS_NR_E2E_24.1.1	4.2.2.1.1 Integrity protection of RRC-signalling	NR NSA (Options 3 and 4) NR SA	16 17 18
TC_SCAS_NR_E2E_24.1.2	4.2.2.1.2 Integrity protection of user data	NR NSA (Options 4 and 7) NR SA	16 17 18
TC_SCAS_NR_E2E_24.1.3	4.2.2.1.4 RRC integrity check failure	NR NSA NR SA	16 17 18
TC_SCAS_NR_E2E_24.1.4	4.2.2.1.5 UP integrity check failure	NR NSA NR SA	16 17 18
TC_SCAS_NR_E2E_24.1.5	4.2.2.1.6 Ciphering of RRC-signalling	NR NSA NR SA	16 17 18
TC_SCAS_NR_E2E_24.1.6	4.2.2.1.7 Ciphering of user data	NR NSA NR SA	16 17 18
TC_SCAS_NR_E2E_24.1.7	4.2.2.1.8 Replay protection of user data	NR NSA NR SA	16 17 18
TC_SCAS_NR_E2E_24.1.8	4.2.2.1.9 Replay protection of RRC-signalling	NR NSA NR SA	16 17 18
TC_SCAS_NR_E2E_24.1.9	4.2.2.1.10 Ciphering of user data based on the security policy sent by the SMF	NR NSA (Options 4 and 7) NR SA	16 17 18
TC_SCAS_NR_E2E_24.1.10	4.2.2.1.11 Integrity of user data based on the security policy sent by the SMF	NR NSA (Options 4 and 7) NR SA	16 17 18
TC_SCAS_NR_E2E_24.1.11	4.2.2.1.12 AS algorithms selection	NR NSA NR SA	16 17 18
TC_SCAS_NR_E2E_24.1.12	4.2.2.1.13 Key refresh	NR NSA NR SA	16 17 18
TC_SCAS_NR_E2E_24.1.13	4.2.2.1.14 Bidding down prevention in Xn-handovers	NR NSA (Options 4 and 7) NR SA	16 17 18
TC_SCAS_NR_E2E_24.1.14	4.2.2.1.15 AS protection algorithm selection	NR NSA NR SA	16 17 18

TC_SCAS_NR_E2E_24.1.15	4.2.2.1.16 Control plane data confidentiality protection over N2/Xn interface	NR NSA (Options 4 and 7) NR SA	16 17 18
TC_SCAS_NR_E2E_24.1.16	4.2.2.1.17 Control plane data integrity protection over S1/NG/Xn interface	NR NSA NR SA	16 17 18
TC_SCAS_NR_E2E_24.1.17	4.2.2.1.18 Key update on dual connectivity	NR NSA NR SA	16 17 18
TC_SCAS_NR_E2E_24.1.18	4.2.2.1.19 User plane security activation in Inactive scenario	NR NSA (Options 4 and 7) NR SA	16 17 18
TC_SCAS_NR_E2E_24.1.19	4.2.2.1.20 User plane data confidentiality protection over N3/Xn interface	NR NSA (Options 4 and 7) NR SA	18
TC_SCAS_NR_E2E_24.1.20	4.2.2.1.21 User plane data integrity protection over N3/Xn interface	NR NSA (Options 4 and 7) NR SA	18

Table 24-2: List of SCAS Test Cases for LTE and applicable technology from Clause 4.2.2 of 3GPP TS 33.216

Test Case (O-RAN Ref. #)	Test Case (3GPP clause number and title)	Applicable Technology	Applicable 3GPP Releases
TC_SCAS_LTE_E2E_24.1.1	4.2.2.1.1 Control plane data confidentiality protection over S1/X2	LTE	16 17 18
TC_SCAS_LTE_E2E_24.1.2	4.2.2.1.2 Control plane data integrity protection over S1/X2	LTE	16 17 18
TC_SCAS_LTE_E2E_24.1.3	4.2.2.1.3 User plane data ciphering	LTE	16 17 18
TC_SCAS_LTE_E2E_24.1.4	4.2.2.1.4 User plane data integrity protection	LTE	16 17 18
TC_SCAS_LTE_E2E_24.1.5	4.2.2.1.5 AS algorithms selection	LTE	16 17 18
TC_SCAS_LTE_E2E_24.1.6	4.2.2.1.6 RRC integrity protection	LTE	16 17 18
TC_SCAS_LTE_E2E_24.1.7	4.2.2.1.7 Selection of EIA0	LTE	16 17 18
TC_SCAS_LTE_E2E_24.1.8	4.2.2.1.8 (1) Key refresh (PDCP Count)	LTE	16 17 18
TC_SCAS_LTE_E2E_24.1.9	4.2.2.1.8 (2) Key refresh (DRB ID)	LTE	16 17 18
TC_SCAS_LTE_E2E_24.1.10	4.2.2.1.9 AS integrity algorithm selection	LTE	16 17 18
TC_SCAS_LTE_E2E_24.1.11	4.2.2.1.10 Bidding down prevention in X2-handovers	LTE	16 17 18
TC_SCAS_LTE_E2E_24.1.12	4.2.2.1.11 AS protection algorithm selection	LTE	16 17 18
TC_SCAS_LTE_E2E_24.1.13	4.2.2.1.12 RRC and UP downlink ciphering	LTE	16 17 18
TC_SCAS_LTE_E2E_24.1.14	4.2.2.1.13 Map a UE NR security capability	LTE	16 17 18

TC_SCAS_LTE_E2E_24.1.15	4.2.2.1.14 UE NR security capability	LTE	16 17 18
TC_SCAS_LTE_E2E_24.1.16	4.2.2.1.15 Bidding down prevention in X2-handovers	LTE	16 17 18
TC_SCAS_LTE_E2E_24.1.17	4.2.2.1.16 Integrity protection of user data	LTE	18
TC_SCAS_LTE_E2E_24.1.18	4.2.2.1.17 Select the right UP integrity protection policy	LTE	18
TC_SCAS_LTE_E2E_24.1.19	4.2.2.1.18 Select the right UP IP policy	LTE	18
TC_SCAS_LTE_E2E_24.1.20	4.2.2.1.19 Select the right UP IP policy in S1 handover	LTE	18
TC_SCAS_LTE_E2E_24.1.21	4.2.2.1.20 Bidding down prevention for UP IP Policy	LTE	18

24.2 DoS, fuzzing and blind exploitation test

Due to the open and disaggregated nature of the O-RAN system (SUT), the attack surfaces associated with some of its transport protocols and interfaces are easy targets for attackers. Cyberattacks like DoS, fuzzing and blind exploitation are easy to launch, require little information about the target system, and can cause significant performance degradation or service interruption if not properly mitigated.

The duration of test TRAFFIC GENERATION specified in this clause shall have a 3 minute minimum.

Table 24-3 summarizes the test cases and the applicable technology.

Table 24-3: End-to-end test cases and applicable technology

Test case		Applicable technology		
		LTE	NSA	SA
Test ID	Name			
24.2.1.1	TC_E2E_ODU_SPlane_DoS	N/A	Y	Y
24.2.1.2	TC_E2E_ODU_SPlane_Robustness	N/A	Y	Y
24.2.2.1	TC_E2E_ODU_CPlane_eCPRI_DoS	N/A	Y	Y
24.2.2.2	TC_E2E_ODU_CPlane_eCPRI_Robustness	N/A	Y	Y
24.2.2.3	TC_E2E_ORU_CPlane_eCPRI_DoS	N/A	Y	Y
24.2.2.4	TC_E2E_ORU_CPlane_eCPRI_Robustness	N/A	Y	Y
24.2.3.1	TC_E2E_NearRTRIC_A1_DoS	N/A	Y	Y
24.2.3.2	TC_E2E_NearRTRIC_A1_Robustness	N/A	Y	Y
24.2.3.3	TC_E2E_NearRTRIC_A1_Vulnerabilities	N/A	Y	Y
24.2.4.1	TC_E2E_OCloud_SideChannel_DoS	N/A	Y	Y

24.2.1 S-Plane

24.2.1.1 S-Plane PTP DoS Attack

Requirement Name: O-DU S-Plane DoS Attack

Requirement Reference: REQ-SEC-DOS-1, clause 5.3.5.1, O-RAN Security Requirements and Control Specification [5]

Requirement Description: An O-RAN architecture element with a network interface withstands network transport protocol based volumetric DDoS attack without system crash and returning to service level after the attack.

Threat References: T-O-RAN-09

SUT/s: O-RAN system

Test Name: TC_E2E_ODU_SPlane_DoS

Purpose: To verify that a predefined volumetric DoS attack against O-DU via the Open FH S-Plane will not crash the SUT and that after the attack ends, the SUT will return to the pre-attack service level.

Procedure and execution steps

Preconditions

- The test emulator has the MAC address of the O-DU's Open FH interface and the L2 connectivity to the O-DU
- Normal UE procedures are in place and the SUT correctly handles user plane traffic.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Figure 24-1 shows test setup and configuration.

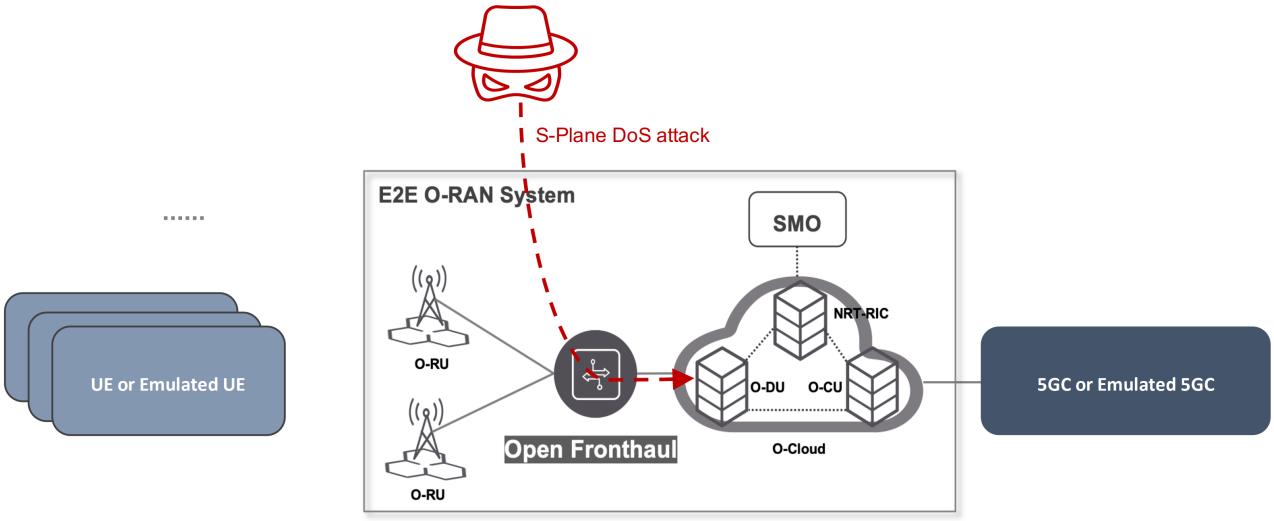

Figure 24-1: S-Plane O-DU Test setup

Execution steps

The tester uses a test tool to generate different types of volumetric DoS Open FH S-Plane attacks against the O-DU.

1) Perform test at volumetric tiers of 10Mbps, 100Mbps, 1Gbps.

2) Send DoS traffic streams to the O-DU over the Open FH S-Plane that are a random mix of generic Ethernet frames and PTP announce/sync messages with random MAC source addresses and the MAC address of the T-GM/T-BC or T-TC known to the O-DU.

Expected results

- During the test, the SUT maintains an operational level.
- After the execution of the test, the degradation of service availability and performance of the SUT is not noticeable.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Expected format of evidence: Traffic captures and/or report files

24.2.1.2 S-Plane PTP Unexpected Input

Requirement Name: O-DU S-Plane Robustness

Requirement Reference: REQ-SEC-OFSP-4, clause 5.2.5.3.2, O-RAN Security Requirements and Control Specification [5]

Requirement Description: The O-DU can detect and defend against application level attacks across the Open FH S-Plane interface, due to misbehaviour or malicious intent.

Threat References: T-O-RAN-09

SUT/s: O-RAN system

Test Name: TC_E2E_ODU_SPlane_Robustness

Purpose: To verify that an input not conforming to the Open FH S-Plane specification sent to the O-DU will not compromise the security of the SUT.

Procedure and execution steps

Preconditions

- The test emulator has the MAC address of the O-DU's Open FH interface and L2 connectivity to the O-DU.
- Normal UE procedures are in place and the SUT correctly handles user plane traffic.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Figure 24-2 shows the test setup and configuration.

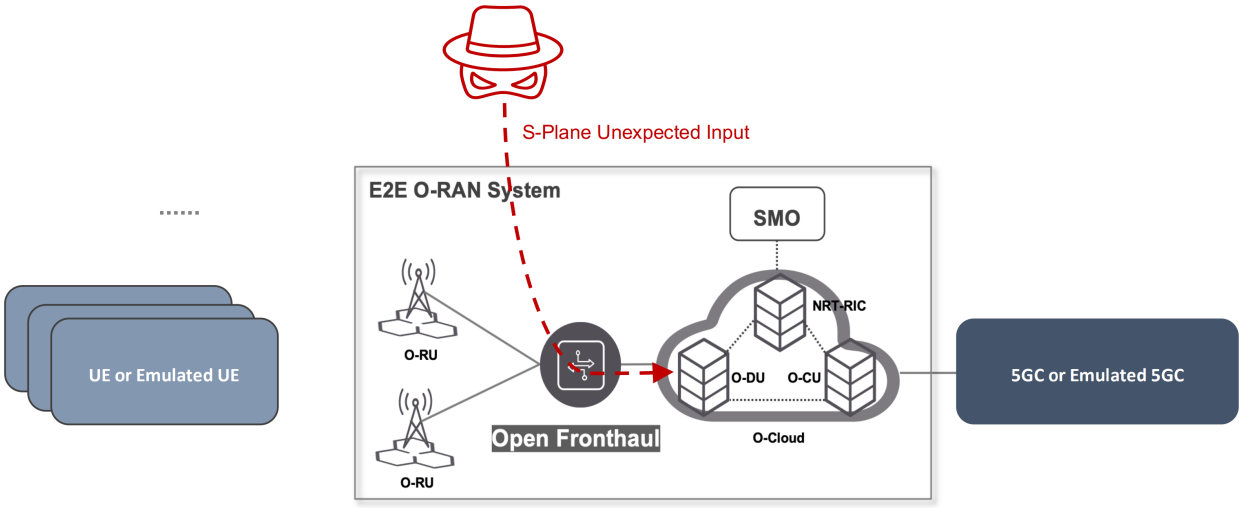

Figure 24-2: S-Plane PTP Unexpected Input Test Setup

Execution steps

1) The tester uses a packet capture tool to capture sample of legitimate PTP messages sent to the O-DU over Open FH S-Plane.
2) The tester uses a fuzzing tool to replay each captured PTP message while mutating its content and keeping the original source/destination MAC address.
3) The tester sends at least 250,000 iterations of mutated PTP message based on a random seed

Expected results

During the execution of the test, the degradation of service availability and performance of the SUT is not noticeable.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Expected format of evidence: Log files, traffic captures and/or reports

24.2.2 C-Plane

24.2.2.1 C-Plane eCPRI DoS Attack

Requirement Name: O-DU C-Plane eCPRI DoS Attack

Requirement Reference: REQ-SEC-DOS-1, clause 5.3.5.1, O-RAN Security Requirements and Control Specification [5]

Requirement Description: An O-RAN architecture element with a network interface withstands network transport protocol based volumetric DDoS attack without system crash and returning to service level after the attack.

Threat References: T-CPLANE-O2, T-O-RAN-09

SUT/s: O-RAN system

Test Name: TC_E2E_ODU_CPlane_eCPRI_DoS

Purpose: To verify that a predefined volumetric DoS attack against the O-DU over Open FH C-Plane will not crash the SUT and that after the attack ends, the SUT will return to the pre-attack service level.

Procedure and execution steps

Preconditions

- The test emulator has the MAC address of the O-DU's Open FH interface and L2 connectivity to the O-DU.
- Normal UE procedures are in place and the SUT correctly handles user plane traffic.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Figure 24-3 shows the test setup and configuration.

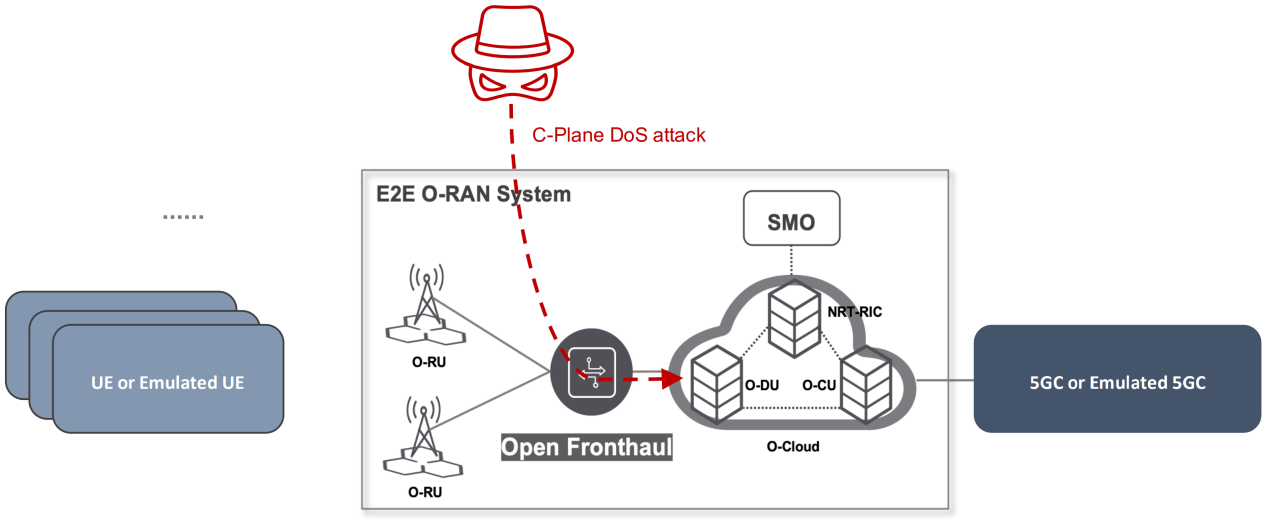

Figure 24-3: C-Plane eCPRI DoS Attack Test Setup

Execution steps

Use test tool to generate several types of volumetric DoS Open FH C-Plane attacks against the O-DU.

1) Perform test at volumetric tiers of 10Mbps, 100Mbps, 1Gbps

2) Send DoS traffic to the O-DU over the Open FH S-Plane that is a random mix of eCPRI real-time Open FH C-Plane messages from arbitrary MAC addresses and the MAC address of a T-GM/T_BC or T-TC known to the O-DU of the following types (i) LAA LBT status and response messages, (ii) Ack/Nack messages, and (iii) Wake-up Ready indication messages. Refer to Figure 4.2-1 Lower layer fronthaul data flows in [26].

Expected results

- During the test, the SUT maintains an operational level.

- After the execution of the test, the degradation of service availability and performance of the SUT is not noticeable.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Expected format of evidence: Traffic captures and/or report files

24.2.2.2 C-Plane eCPRI Unexpected Input

Requirement Name: O-DU C-Plane Robustness

Requirement Reference: REQ-SEC-OFCP-2, clause 5.2.5.1.2, O-RAN Security Requirements and Control Specification [5]

Requirement Description: The O-DU can detect and defend against application level attacks across the Open FH C-Plane messages with O-RUs, due to misbehaviour or malicious intent.

Threat References: T-O-RAN-09

SUT/s: O-RAN system

Test Name: TC_E2E_ODU_CPlane_eCPRI_Robustness

Purpose: To verify that input not conforming to the Open FH C-Plane specification will not compromise the security of the SUT.

Procedure and execution steps

Preconditions

- The test emulator has the MAC address of the O-DU's Open FH interface and L2 connectivity to the O-DU.
- Normal UE procedures are in place and the SUT correctly handles user plane traffic.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Figure 24-4 shows the test setup and configuration.

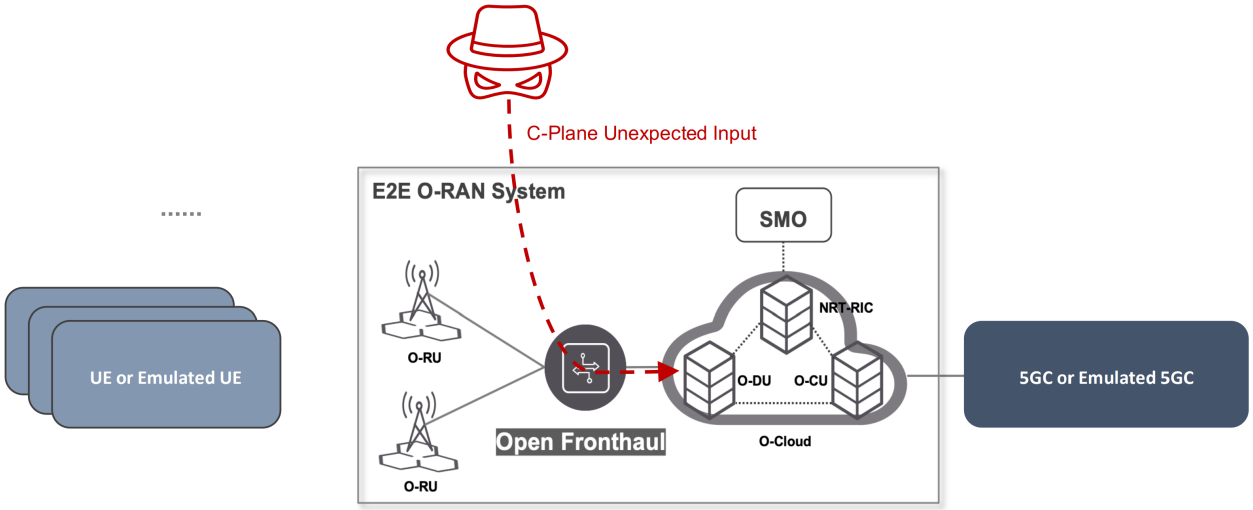

Figure 24-4: C-Plane eCPRI Unexpected Input Test Setup

Execution steps

1) The tester uses a packet capture tool to capture a sample of legitimate eCPRI messages sent over the Open FH C-Plane to the O-DU.

2) The tester uses a fuzzing tool to replay each captured eCPRI message while mutating its content (message type and/or payload) and keeping original source/destination MAC address.

3) The tester sends at least 250,000 iterations of mutated eCPRI message based on a random seed

Expected results

During the execution of the test, the degradation of service availability and performance of the SUT is not noticeable.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Expected format of evidence: Traffic captures and/or report files

24.2.2.3 C-Plane eCPRI DoS Attack on O-RU

Requirement Name: O-RU C-Plane eCPRI DoS Attack

Requirement Reference: REQ-SEC-DOS-1, clause 5.3.5.1, O-RAN Security Requirements and Control Specification [5]

Requirement Description: An O-RAN architecture element with a network interface withstands network transport protocol based volumetric DDoS attack without system crash and return to service level after the attack.

Threat References: T-O-RAN-09

SUT/s: O-RAN system

Test Name: TC_E2E_ORU_CPlane_eCPRI_DoS

Procedure and execution steps

Purpose: To verify that a predefined volumetric DoS attack against O-RU over the Open FH C-Plane will not crash the SUT and that after the attack ends, the SUT will return to the pre-attack service level.

Preconditions

- The test emulator has the MAC address of the O-RU's Open FH interface and L2 connectivity to the O-RU.
- Normal UE procedures are in place and the SUT correctly handles user plane traffic.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Figure 24-5 shows the test setup and configuration.

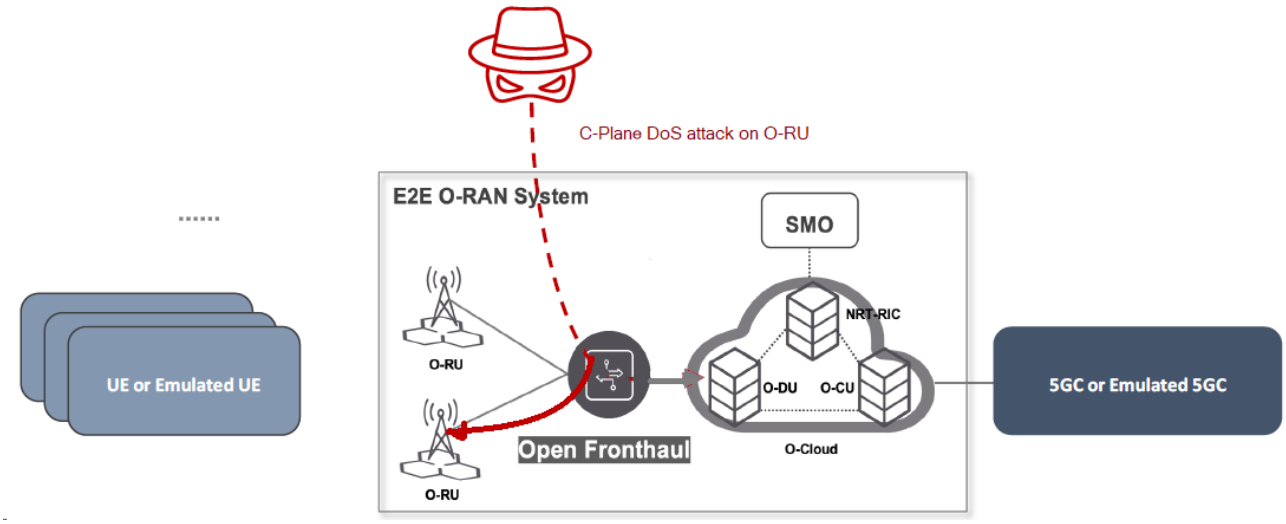

Figure 24-5: C-Plane eCPRI DoS Attack on O-RU Test Setup

Execution steps

The tester uses a test tool to generate different types of volumetric DoS Open FH C-Plane attacks against the O-RU.

1) Perform test at volumetric tiers of 10Mbps, 100Mbps, 1Gbps.

2) Send DoS traffic stream to the O-RU over the Open FH C-Plane that is a random mix of Open FH C-Plane Scheduling commands (DL & UL), Beamforming commands, LAA LBT configuration commands/requests, and UE Channel information.

Expected results

- During the test, the SUT maintains an operational level
- After the execution of the test, the degradation of service availability and performance of the SUT is not noticeable.

Expected format of evidence: Traffic captures and/or report files

24.2.2.4 C-Plane eCPRI Unexpected Input on O-RU

Requirement Name: O-RU C-Plane Robustness

Requirement Reference: REQ-SEC-OFCP-2, clause 5.2.5.1.2, O-RAN Security Requirements and Control Specification [5]

Requirement Description: The O-RU can detect and defend against application level attacks across the Open FH C-Plane messages with O-DUs, due to misbehaviour or malicious intent.

Threat References: T-O-RAN-09

SUT/s: O-RAN system

Test Name: TC_E2E_ORU_CPlane_eCPRI_Robustness

Purpose: To verify that input not conforming to the Open FH C-Plane specification will not compromise the security of the SUT.

Procedure and execution steps

Preconditions

- The test emulator has the MAC address of the O-RU's Open FH interface and L2 connectivity to the O-DU.

- Normal UE procedures are in place and the SUT correctly handles user plane traffic.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Figure 24-6 shows the test setup and configuration.

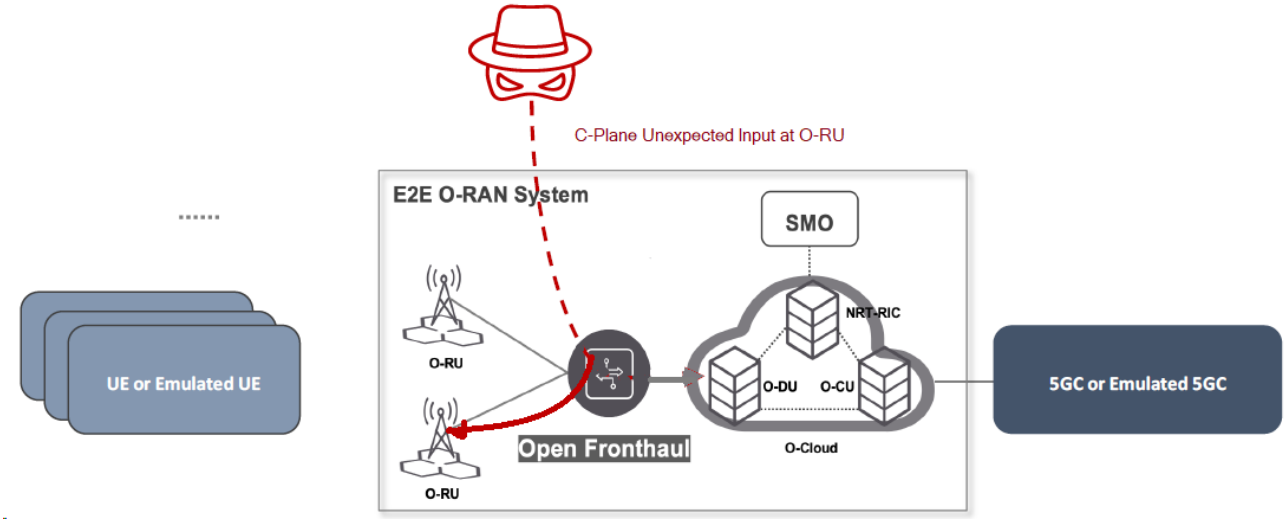

Figure 24-6: C-Plane eCPRI Unexpected Input on O-RU Test Setup

Execution steps

1) The tester uses a packet capture tool to capture a sample of legitimate eCPRI messages sent over the Open FH C-Plane to the O-RU.
2) The tester uses a fuzzing tool to replay each captured eCPRI message while mutating its content (message type and/or payload) and keeping original source/destination MAC address.
3) The tester sends at least 250,000 iterations of mutated eCPRI message based on a random seed.

Expected results

- After the execution of the test, the degradation of service availability and performance of the SUT is not noticeable.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Expected format of evidence: Traffic captures and/or report files

24.2.3 A1 interface

24.2.3.1 Near-RT RIC A1 Interface DoS Attack

Requirement Name: Near-RT RIC A1 interface DoS recover

Requirement Reference: REQ-SEC-NEAR-RT-6, clause 5.1.3.1, O-RAN Security Requirements and Control Specification [5]

Requirement Description: The Near-RT RIC recovers, without catastrophic failure, from a volumetric DDoS attack across the A1 interface, due to misbehaviour or malicious intent.

Threat References: T-O-RAN-09

SUT/s: O-RAN system

Test Name: TC_E2E_NearRTRIC_A1_DoS

Purpose: To verify that a predefined volumetric DoS attack against the Near-RT RIC over the A1 interface will not crash the SUT and that after the attack ends, the SUT will return to the pre-attack service level.

Procedure and execution steps

Preconditions

- The test emulator has the IP address information of the Near-RT RIC's A1 interface and a routable path to the target.
- Normal UE procedures are in place and the SUT correctly handles user plane traffic.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Figure 24-7 shows the test setup and configuration.

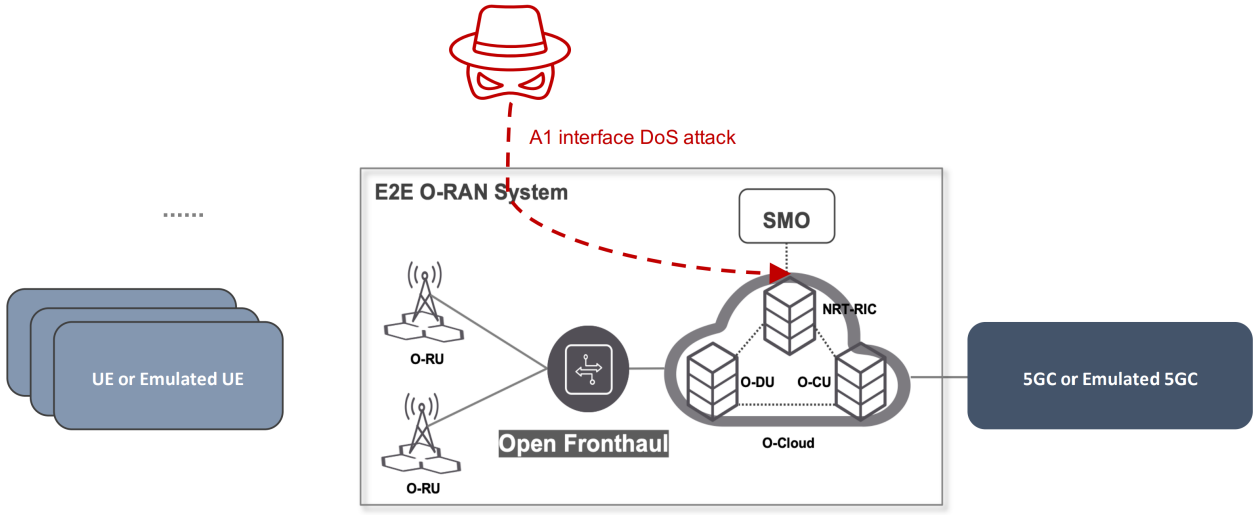

Figure 24-7: Near-RT RIC A1 Interface DoS Attack Test Setup

Execution steps

1) The tester uses a test tool to generate several types of volumetric DoS attack against the IP address of the Near-RT RIC A1 interface:
2) Perform test at volumetric tiers of 10Mbps, 100Mbps, 1Gbps
3) Send DoS traffic to the Near-RT RIC over the A1 interface that is a random mix of generic UDP packets and HTTP/HTTPS REST API calls with source addresses of Non-RT RIC, other IPs and broadcast IPs.

Expected results

- During the test, the SUT maintains an operational level.
- After the execution of the test, the degradation of service availability and performance of the SUT is not noticeable.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Expected format of evidence: Traffic captures and/or report files

24.2.3.2 Near-RT RIC A1 Interface Unexpected Input

Requirement Name: Near-RT RIC A1 Robustness

Requirement Reference: REQ-SEC-NEAR-RT-7, clause 5.1.3.1, O-RAN Security Requirements and Control Specification [5]

Requirement Description: The Near-RT RIC detects and defends against content-related attacks across the A1 interface, due to misbehaviour or malicious intent.

Threat References: T-O-RAN-09

SUT/s: O-RAN system

Test Name: TC_E2E_NearRTRIC_A1_Robustness

Purpose: To verify that input not conforming to the AI interface specification will not compromise the security of the SUT.

Procedure and execution steps

Preconditions

- The test emulator has the IP address information of the Near-RT RIC's A1 interface and a routable path to the target.
- Normal UE procedures are in place and the SUT correctly handles user plane traffic.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Figure 24-8 shows the test setup and configuration.

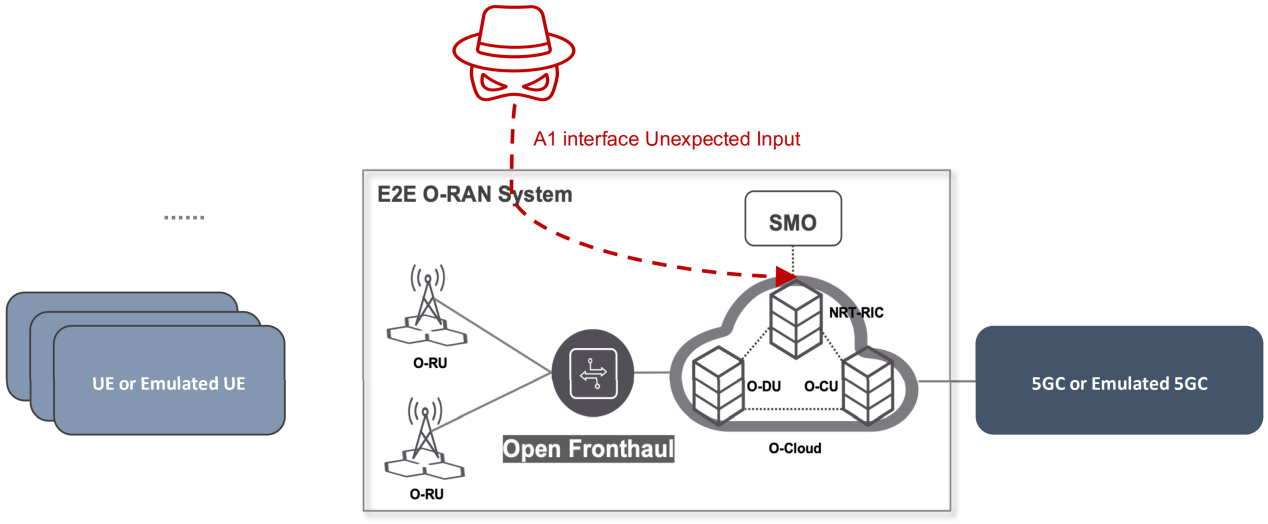

Figure 24-8: Near-RT RIC A1 Interface Unexpected Input Test Setup

Execution steps

1) The tester uses a packet capture tool to capture samples of legitimate HTTP/HTTPs REST API messages sent over the A1 interface to the Near-RT RIC.

2) The tester uses a fuzzing tool to replay each captured HTTP/HTTPs REST API message while mutating its content and keeping original source/destination IP/port.

3) The tester sends at least 250,000 iterations of mutated HTTP/HTTPs REST API message based on a random seed

Expected results

During the execution of the test, the degradation of service availability and performance of the SUT is not noticeable.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Expected format of evidence: Traffic captures and/or report files

24.2.3.3 Near-RT RIC A1 Vulnerability Assessment

Requirement Name: Near-RT RIC A1 Vulnerability Assessment

Requirement Reference: REQ-SEC-SYS-1, clause 5.3.6, O-RAN Security Requirements and Control Specification [5]

Requirement Description: Known vulnerabilities in the OS and applications of an O-RAN architecture element clearly identified.

Threat References: T-OPENSRC-01, T-OPENSRC-02

SUT/s: O-RAN system

Test Name: TC_E2E_NearRTRIC_A1_Vulnerabilities

Purpose: To verify that exploitation attempts of known vulnerabilities in the Near-RT RIC will not compromise the security of the SUT.

Procedure and execution steps

Preconditions

- The test emulator has the IP address information of the Near-RT RIC's A1 interface and a routable path to the target.
- Normal UE procedures are in place and the SUT correctly handles user plane traffic.
- The test requires the vulnerability scanning tool to have an up-to-date database of well-known vulnerabilities (signatures/plugins) based on Common Vulnerabilities and Exposures (CVE).

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Figure 24-9 shows the test setup and configuration.

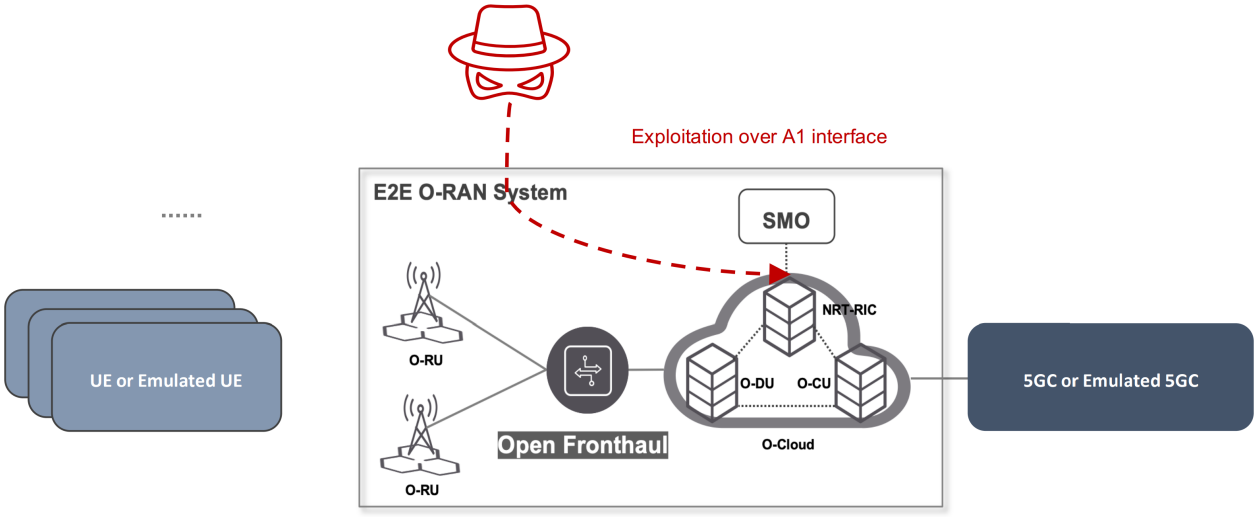

Figure 24-9: Near-RT RIC A1 Vulnerability Assessment Test Setup

Execution steps

The tester uses a vulnerability scanning tool to scan the IP address of the Near-RT RIC A1 interface with the following parameters defined.

- TCP Ports: Port scanner scans all TCP ports in range 0-65535 on the IP interface of SUT. TCP SYN/ACK response by SUT is interpreted as open port.
- UDP Ports: All UDP ports documented in vendor-provided list. Other UDP ports may be considered as open for the purpose of service detection.
- Safe Checks: Disabled (to make sure that exploitation attempts of the vulnerabilities will be performed)

NOTE: Due to the nature of UDP protocol, there is no simple method of open port detection similar to TCP/SCTP methods based on analysis of response message type (TCP: SYN/ACK, SCTP: INIT-ACK). In case of UDP, open port detection relies on service detection which is discussed in step 2 of this test procedure. In practice, port scans of entire UDP port range 0-65535 are impractical and time consuming. Thus, service detection is generally performed only for subset of UDP ports. UDP port subset selection is arbitrary and not standardized. Service detection in this test procedure is required for UDP ports from vendor-provided list and is optional for other UDP ports.

Expected results

During the execution of the test, the degradation of service availability and performance of the SUT is not noticeable.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Expected format of evidence: Tool testing report and traffic captures

24.2.4 O-Cloud

24.2.4.1 O-Cloud side-channel DoS attack

Requirement Name: O-Cloud DoS Attack

Requirement Reference: REQ-SEC-DOS-1, clause 5.3.5.1, O-RAN Security Requirements and Control Specification [5]

Requirement Description: An O-RAN architecture element with a network interface withstands network transport protocol based volumetric DDoS attack without system crash and returning to service level after the attack.

Threat References: T-O-RAN-09

SUT/s: O-RAN system

Test Name: TC_E2E_OCloud_SideChannel_DoS

Purpose: To verify that a noisy neighbour DoS attack against O-Cloud for resource starvation will not crash the SUT and that after the attack ends, the SUT will return to the pre-attack service level.

Procedure and execution steps

Preconditions

- The test emulator has access to the O-Cloud platform hosting the network slice(s) of the O-RAN system.
- Normal UE procedures are in place and the SUT correctly handles user plane traffic.
- Logging and alerts in the O-cloud are enabled. Network monitoring tools may be used.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Figure 24-10 shows the test setup and configuration.

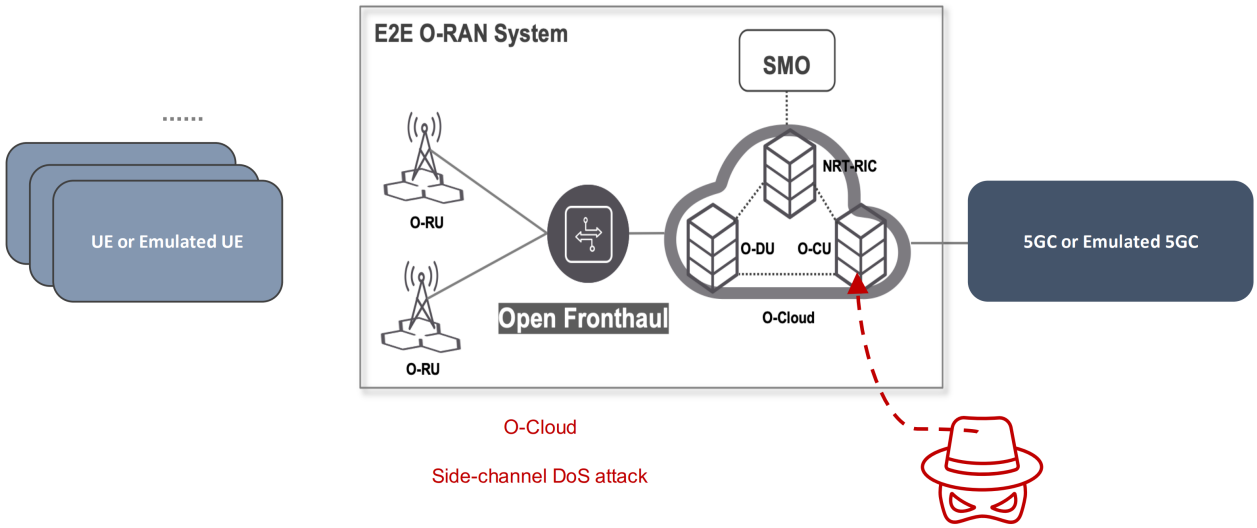

Figure 24-10: O-Cloud side-channel DoS attack Test Setup

Noisy neighbour VNF(s) can be deployed into an existing slice or a new slice of the shared resources with the existing slice under test.

Execution steps

1) The tester uses a test tool through O-Cloud MANO to instantiate noisy neighbour VNFs.

- Noisy neighbour tenant: existing slice or a new slice of the shared resources with the existing slice.
- The Noisy Neighbour tenants utilize shared resources that include CPU, memory, storage, and network. The Noisy Neighbour tenants exhaust all the remaining shared resources. The duration of the test is at least 3 minutes or long enough to cover the benchmarking tests.

2) Run the benchmark test again with the noisy neighbours.

3) Check the logs and alerts that are associated with the test.

Expected results

- After the execution of the test, the degradation of service availability and performance of the SUT is not noticeable.
- The Noisy Neighbour attack is properly logged and alerted by the O-Cloud.

RECOMMENDATION: Use Bidirectional throughput in different radio conditions and Data Services tests (clauses 6.0 and 7.1 from ORAN TIFG End-to-End Test Specifications [4]) as benchmarks for indicating the correct behaviour of the SUT.

Expected format of evidence: Logs, results, screenshots, report

25 Security test of Shared O-RU

25.1 Overview

This clause contains security tests to validate security controls related to the Shared O-RU and the Shared O-RU architecture.

25.2 Shared O-RU test cases

25.2.1 mTLS for mutual authentication

Requirement Name: Shared O-RU support for mTLS 1.2, or higher, for mutual authentication on the Open FH M-Plane interface with an O-RU Controller

Requirement Reference: SEC-CTL-SharedORU-1, clause 5.1.9.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: mTLS support on Shared O-RU

Threat References: T-SharedORU-06

DUT/s: Shared O-RU

Test Name: TC_SharedORU_mTLS

Purpose: To verify the Shared O-RU is able to mutually authenticate with an O-RU Controller using mTLS, with PKI-based X.509 certificates.

Procedure and execution steps

Preconditions

DUT is the Shared O-RU with mTLS 1.2, or 1.3, support enabled.

Execution steps

Follow the test case in clause 6.3 of the present document.

Expected results

The DUT supports mutual authentication with an O-RU Controller using mTLS.

Expected format of evidence: Log entries and packet captures.

25.2.2 NACM Authorization

Requirement Name: Shared O-RU support for NACM for permitting or denying access to an SRO

Requirement Reference: SEC-CTL-SharedORU-3, clause 5.1.9.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: NACM support for Shared O-RU

Threat References: T-SharedORU-22, T-SharedORU-23

DUT/s: Shared O-RU

Test Name: TC_SharedORU_NACM_Authorization

Purpose: To verify the Shared O-RU is able to enforce role-based least privilege access control on the Open Fronthaul by using NACM [14].

Procedure and execution steps

Preconditions

DUT is the Shared O-RU with:

- IP enabled Open Fronthaul M-Plane interface, reachable from the authentication server
- Valid certificate loaded for the server and necessary certificate authorities (CAs)
- Client's root CA required to validate NETCONF client certificate
- Valid TLS Client-to-NETCONF username mapping

Execution steps

First set up a host/device with TLS client software installed, valid client certificates, keys, root CA certificate for the server (Shared O-RU), and all intermediate CA certificates required to validate the client certificate.

The following test steps are validated:

1) Start the NETCONF-over-TLS session using OpenSSL s_client command to connect with DUT using TLS 1.2 or TLS 1.3
2) Verify the session is established and mapped to the correct NETCONF user
3) Verify the global NACM enforcement control setting of

- enable-nacm = true
- read-default = permit
- write-default = deny
- exec-default = deny
- enable-external-groups = true

4) Verify the NACM rule sets for the pre-defined groups
5) Close the NETCONF session and TLS connection

Upon availability of the NETCONF operations set(s) definition per NACM group, the NACM rule set(s) enforcement by the DUT is validated for each of those pre-defined groups listed above.

Expected results

The DUT supports NETCONF over TLS session over its Open Fronthaul M-Plane interface and NACM enforcement control settings.

Expected format of evidence: Log entries and packet captures.

25.2.3 TLS across Open Fronthaul

Requirement Name: Shared O-RU support for TLS 1.2, or higher, on the Open FH M-Plane interface with an O-RU controller

Requirement Reference: SEC-CTL-SharedORU-4, clause 5.1.9.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: TLS on Shared O-RU

Threat References: T-SharedORU-27, T-SharedORU-28

DUT/s: Shared O-RU

Test Name: TC_SharedORU_TLS

Purpose: To verify whether Shared O-RU establishes a TLS session with an O-RU Controller and provides confidentiality and integrity protection for messages exchanged with the O-RU Controller.

Procedure and execution steps

Preconditions

DUT is the Shared O-RU with TLS support enabled.

Execution steps

Follow the test case in clause 6.3 of the present document.

Expected results

The DUT provides confidentiality and integrity protection for data in transit on the Open Fronthaul M-Plane interface.

Expected format of evidence: Log entries and packet captures.

25.2.4 Reject Password-based authentication

Requirement Name: Denial of password-based authentication with an O-RU Controller by Shared O-RU

Requirement Reference: SEC-CTL-SharedORU-2, clause 5.1.9.2, O-RAN Security Requirements and Controls Specifications [5]

Requirement Description: Shared O-RU is able to reject password-based authentication on the Open Fronthaul M-Plane

Threat References: T-SharedORU-04

DUT/s: Shared O-RU

Test Name: TC_SharedORU_SSH_Password_Based_Authentication

Purpose: To verify the Shared O-RU can reject a SSH session using password-based authentication on the Open Fronthaul.

Procedure and execution steps

Preconditions

DUT is the Shared O-RU with password-based authentication for SSH on the Open Fronthaul disabled.

Execution steps

1) Enable SSH on the Open Fronthaul for the Shared O-RU. Ensure password-based authentication is not enabled on the Shared O-RU for SSH on the Open Fronthaul.
2) Configure the O-RU Controller as the SSH client with password-based authentication on the Open Fronthaul M-Plane.
3) Attempt to establish the Open Fronthaul M-Plane session between the Shared O-RU and O-RU Controller.

Expected results

The DUT rejects the Open Fronthaul M-Plane session with the O-RU Controller.

Expected format of evidence: Log entries, packet captures, and/or screenshots.

Annex A (informative): Example of Security Testing Tools / Toolset

Table Annex A-1: List of sample open source security testing tools/toolset

Testing Tool	Example(s)
DTLS scanning tool	open source "pySSLScan": https://github.com/DinoTools/pysslscan
IPsec IKE scanning tool	open source "ike-scan": https://github.com/royhills/ike-scan
Port scanner	open source "Nmap": https://nmap.org/
SSH audit tool	open source "ssh-audit": https://github.com/jtesta/ssh-audit
TLS scanning tool	open source "sslyze": https://github.com/nabla-c0d3/sslyze
Software image signing tool	open source "Sigstore": https://github.com/sigstore

Annex B (informative): Template of test report

A. GENERAL INFORMATION

A1 Name of test campaign	
A2 Version of the report – reference ID	A3 Date(s) of testing
A4 Contact person (tester) – incl. Name, Organization, E-mail address	
A4 Test location (lab) – incl. the address	
A5 Description of test campaign, summary of test results, conclusions	

List of tests - details of each test can be found in Section E.

Test No.	Test name	Test status [PASS / FAIL / -]
01		
02		
03		

B. TEST AND MEASUREMENT EQUIPMENT AND TOOLS

#	Equipment or tool	Type	Manufacture	Version (HW/SW)	Notes*
01					
02					
03					

* Specific details such as the sub-module version (such as vulnerability database version)

C. SYSTEM UNDER TEST

C1 Total number of DUTs included in SUT	C2 Deployment architecture
C3 Description of SUT – connection/block diagram	

DUT 1*

C3 Type	C4 Serial Number	C5 Supplier (manufacture)
C6 SW version	C7 HW version (if applicable)	
C8 Interface/IOT profile(s) if applied		
C9 Description incl. parameters, setting/configuration		

* If SUT contains more DUTs, please copy the table

D. TEST CONFIGURATION

D1 Function(s) and Service(s) setting	D2 Network setting

E. TEST RESULTS

E1 Test No.	E2 Test name
E3 Date(s) of test execution	E4 Reference to test specification
E5 Utilized test and measurement equipment and tools, incl. the specific setting/configuration – reference to Section B	
E6 Test setup – connection/block diagram – deployment scenario	
E7 Execution steps – describe differences in comparison with the execution steps defined in test spec. – limitations	
E8 Test results –including outputs of the test properties and the attachment of log file(s) and/or screenshots	
E9 Notes, including observed issues with the solutions	
E10 Conclusions – pass/fail – assessment of test results in comparison with the expected results – gap analysis	

Annex (informative): Change History

Date	Revision	Description
2021.11.10	01.00	Final initial version 01.00
2022.03.23	02.00.07	Updated clauses: 7.4 Network Protocol Fuzzing 9.4 Software Bill of Materials (SBOM) 11.2 Open Fronthaul Point-to-Point LAN Segment 17.2 O1 Interface Network Configuration Access Control Model (NACM) Validation
2022.07.19	03.00.01	Applied the latest O-RAN technical specifications template Updated clauses: 2.1 Normative references 3.2 Abbreviations 6.3 TLS 6.6 OAuth 2.0 9.5 Software Image Signing and Verification 13.2 Testing of IPSec on E2 14.2 Testing of TLS on A1
2022.07.25	03.00.02	Updated cross-reference clause numbers of the test cases to align with latest WG11 specs Updated table of contents
2023.03.09	04.00.00	Updated clauses: • 14 Security Test of xApps • 15 Security test of Non-RT RIC • 16 Security test of rApps Added content to: • 14.2 xApp Signing and Verification • 16.2 rApp Signing and Verification Change wording in many places to align document with ETSI PAS

2023.07.10	05.00.00	Added content to: • 9.4.3 SBOM Format • 9.4.4 SBOM Depth • 9.5.2 Software Signature Verification • 12.4 O-RU Security functional requirement and test cases • 13.2 IPSec on E2 interface • 13.3 Transactional APIs • 18.2 O2 Interface • 18.3 O-Cloud virtualization layer • 20 Security tests of Common Application Lifecycle Management • 21 Security test of O-CU-CP • 22 Security test of O-CU-UP • 23 Security test of O-DU Alignment for ETSI PAS
2024.03.20	07.00.00	• OAuth2.0 in Near-RT RIC • OCloud tests • Reorganization of interfaces testing (A1, R1) • Update of SCTP test cases, removing not related with security • TIFG E2E test cases adoption into clause 24, and needed update • SBOM and Package testing increased • E2 data validation tests for Near-RT RIC

2024.07.17	08.00.00	Added content to: • 6.9 X.509 • 7.5 Denial of Service/Message flooding • 7.6 Input validation and error handling • 7.7 Secure configuration enforcement • 11 Security tests of O-RAN interfaces • 18.4 Application instantiation by O-Cloud • 18.9 Secure time synchronization for O-Cloud • 20.2 Application Package Removed content from: • 6.2 SSH • 6.3 TLS • 6.4 DTLS • 6.7 NACM • 6.8 802.1X • 7.5 Denial of Service/Message flooding • 7.7 Secure configuration enforcement • 9.4 SBOM • 11 Security tests of O-RAN interfaces

2024.11.27	09.00.00	Changed content in: • 6 Security Protocol & APIs Validation • 7 Common Network Security Tests for O-RAN architecture elements • 8 System security evaluation for O-RAN architecture element • 9 Software security evaluation for O-RAN architecture elements • 11.1 FH • 14 Security test of xApps • 18.4 Application instantiation deployment by O-Cloud • 19.2 Executive environment protection • 24 End-to-End security test cases Removed content from: • 6.10.1 eCPRI Session Management • 6.10.4 eCPRI Access Control • 6.10.6 eCPRI Timeout Error Handling • 8.4.9 Storage • 9.4.5 SBOM completeness check • 9.4.10 SBOM OSC Components • 9.5.1 Software Image/Application Package Signing
2025.03.23	10.00.00	Changed content in clauses 6, 7, 8, 11, 12, 13, 15, 16, 17, 18, 19, 20, 24, 25 Added content: • 6.6.1 API Consumer • 6.6.2 Resource Server • 18.6.5 Unauthorised Rollback Prevention Removed content from: • 11.1.5.1.2 DoS timeTransmitter LLS C4

2025.07.23	11.00.00	Editorial updates applied Added content: 6.3.1 TLS Support 6.3.2 TLS Version Negotiation 6.3.3 TLS Deprecated Versions 6.12.1 Transactional API Authentication - service producer role 6.12.3. Transactional API Input Validation and Sanitization 6.13 MACsec 9.5.1 Signature Verification Matrix per DUT and Phase 9.5.2 Signature Verification Test Cases 11.1.2.3 Port-Based Access Enforcement Validation 11.1.2.4 O-RU Authentication Lifecycle with Manufacturer and Operator Certificates 11.1.3.1.4 Open FH M-Plane SSH-certificate-based authentication authorization at O-DU 11.2.3. Y1 Authorization 14.3.1. xApp ID format check Removed content from: 11.1.5.3.1. Selective Interception and Removal of PTP Timing Packets 11.1.5.3.2 Delay Attack on PTP Timing Packets 14.2 xApp Signing and Verification 16.2 rApp Signing and Verification 18.4.1 Verification of Application artifacts with valid signature by O-Cloud during deployment 18.4.2 Verification of Application artifacts with incorrect signature by O-Cloud during deployment 19.3 Signature validation during App image onboarding 20.2.1 Application package signature verification

